

AI developer tools for Azure documentation

Explore AI-powered developer tools for Azure including the Azure MCP Server, GitHub Copilot MCP servers, and Azure Skills. Build intelligent workflows that connect AI agents to Azure services.

Azure Skills

OVERVIEW

[Overview](#)

GET STARTED

[Install and configure](#)

[Quickstart](#)

Azure MCP Server

OVERVIEW

[Overview](#)

GET STARTED

[Get started with Azure MCP Server](#)

REFERENCE

[Tools reference](#)

GitHub Copilot for Azure

OVERVIEW

[Overview](#)



GET STARTED

[Get started](#)

[Deploy with agent mode](#)



TRAINING

[Learn about Azure and your app](#)

Azure AI developer tools overview

The Azure AI developer tools are a collection of AI-powered tools that help you manage, deploy, and troubleshoot Azure resources by using natural language. The toolset includes GitHub Copilot for Azure, the Azure MCP Server, and Azure Skills. These tools work together across IDEs, terminals, and CI/CD pipelines.

Why use Azure AI developer tools?

Building on Azure typically requires navigating portal UIs, reading documentation across multiple services, writing infrastructure-as-code templates, and debugging deployment issues. These tasks add friction before you write your first line of application code. This friction slows teams down and creates a steep learning curve, especially for developers new to Azure.

The Azure AI developer tools eliminate this friction by bringing Azure directly into your development workflow:

- **Reduce context switching** - Ask questions about Azure services, manage resources, and deploy applications without leaving your IDE or terminal.
- **Accelerate onboarding** - New team members can discover and use Azure services through natural language. They don't need to memorize CLI commands, portal navigation, or ARM/Bicep syntax.
- **Enforce best practices automatically** - Azure Skills embed guardrails and proven patterns into every workflow. Deployments follow organizational standards without manual review checklists.
- **Work where you already are** - Whether you use VS Code, Visual Studio, Cursor, IntelliJ, or a CLI, the Azure AI developer tools meet you in your existing environment.

Key tools

 Expand table

Tool	What it does	Best for
GitHub Copilot for Azure	Extension that surfaces Azure tools and skills through GitHub Copilot	Integrated IDE experience in VS Code or Visual Studio
Azure MCP Server	Standalone MCP server with 270+ tools across 50+ Azure services and Microsoft Entra ID authentication	Azure tools in any MCP-compatible client

Tool	What it does	Best for
Azure Skills	Knowledge modules that provide end-to-end workflows with guardrails	Guided, best-practice Azure workflows

Choose the right tool

Use the following decision flow to determine which tool best fits your scenario.

1. Are you using VS Code or Visual Studio?

- Yes → [GitHub Copilot for Azure](#) — Includes Azure MCP Server built-in.
- No → Continue to step 2.

2. Using another IDE (Cursor, IntelliJ, Windsurf)?

- Yes → [Azure MCP Server](#) — Install the standalone MCP server.
- No → Continue to step 3.

3. Want end-to-end workflows with guardrails?

- Yes → [Azure Skills](#) — Available across multiple hosts including GitHub Copilot CLI, Claude Code, and VS Code.

Supported development environments

 Expand table

Environment	GitHub Copilot for Azure	Azure MCP Server	Azure Skills
VS Code	 Extension + MCP Server		
Visual Studio 2022	 Built-in (with Azure Workload)		
Visual Studio 2026	 Built-in (with Azure Workload)		
Cursor			
Windsurf			
IntelliJ			
GitHub Copilot CLI		 (via <code>/mcp add</code>)	
Claude Code			

Primary scenarios

All three tools — GitHub Copilot for Azure, Azure MCP Server, and Azure Skills — support most Azure development scenarios. The following table shows the recommended starting point for each scenario, but you can use any tool for any task.

 Expand table

Scenario	Example prompts
Learn about Azure services	"What Azure services should I use with my app?"
Manage Azure resources	"List all my storage accounts"
Deploy an application	"Deploy my app to Azure"
Troubleshoot a failing app	"Why is my app returning 500 errors?"
Query resources across subscriptions	"Show me all VMs across my subscriptions"
Set up end-to-end deployment pipeline	"Prepare and deploy my Node.js app to Azure"

Related content

- [GitHub Copilot for Azure documentation](#)
- [Azure MCP Server documentation](#)
- [Azure Skills documentation](#)

Last updated on 04/17/2026

What is GitHub Copilot for Azure?

GitHub Copilot for Azure is a GitHub Copilot extension that enables developers to use natural language to:

- Learn about Azure features.
- Deploy Azure resources.
- Get information about Azure resources.
- Diagnose and troubleshoot problems with Azure resources.

You must have access to an Azure subscription and be subscribed to GitHub Copilot. [Get started](#) using the extension.

GitHub Copilot is designed to help developers, including developers new to Azure, to be more productive as quickly as possible. For experienced Azure users, GitHub Copilot for Azure replaces the need to:

- memorize or look up Azure CLI commands and arguments.
- create complex deployment scripts by hand.
- sign in and browse through the Azure portal.

How it works

GitHub Copilot for Azure supplements the general knowledge of a foundational large language model (LLM) like GPT-5 and Claude Sonnet 4 with tool calling using the **Azure Model Context Protocol (MCP) Server** that enables interaction with Azure services, systems, and Azure Resource Graph to carry out specific tasks on your behalf. Over [50 Azure services](#) are already available and more services and capabilities are being added regularly. Learn more about the capabilities of [Azure MCP Server](#).

Supported development environments

You can use GitHub Copilot for Azure in the following supported development environments:

[\[\]](#) Expand table

Supported Client	Description	Feature Stage	Download Link
Visual Studio Code	Surfaces GitHub Copilot for Azure via the GitHub Copilot user interface. It also surfaces the Azure MCP Server tools. Provides IDE-specific tools and custom modes.	General availability	Link ↗
Visual Studio 2022	Surfaces built-in Azure MCP Server tools via the GitHub Copilot user interface.	Public preview	The tools are available upon installation of the Azure development workload .
Visual Studio 2026	Surfaces built-in GitHub Copilot for Azure and Azure MCP Server tools via GitHub Copilot user interface.	General availability	The tools are available upon installation of the Azure and AI development workload .

Primary scenarios

GitHub Copilot for Azure currently enables four primary scenarios:

 Expand table

Category	Explanation	Examples
Learn	Learn about Azure services and tools from the latest Microsoft Learn documentation.	• "What Azure services should I use with my app?" • "What are the available types of Azure OpenAI models?" • "What is Azure AI Search and why should I use it?" • "How does pricing work for Azure SQL?"
Design and develop	Ask for guidance and help when building apps for the cloud.	• "Can you help me build a RAG application with Python to deploy to Azure?" • "Use azd to undeploy my project in Azure." • "We're a pizza company and want to create an online customized pizza delivery solution. Create an API to accept pizza orders on Azure."

Category	Explanation	Examples
Deploy	Create Azure resources and deploy apps.	• "Can you help me deploy my application to Azure?" • "I need a CI/CD pipeline so I can get my app deployed to Azure." • "Use azd to undeploy my project from Azure."
Troubleshoot	Diagnose and troubleshoot application and resource problems.	• "What is using up my GPT-5 model quota on Azure?" • "Find out why my Kubernetes cluster is running slow on Azure." • "Why am I seeing 500 errors when opening my website on Azure?"

 Note

Make sure the word "Azure" is somewhere in the prompt so that the LLM calls the appropriate tool from Azure MCP Server.

The documentation provides a quickstart and example prompts to help you start using GitHub Copilot for Azure as quickly as possible.

Best practices

First, use "Agent" mode for the best experience. Avoid "Ask" mode.

Second, include the word "Azure" in the prompt to help Copilot understand that it needs to call tools from the Azure MCP Server.

Third, using copilots can increase developer productivity by answering questions, executing tasks, and generating code. However, remember these vital rules:

- Review all AI-generated responses. Validate their correctness, applicability, potential outcomes (such as costs and security) before taking action based on those responses.
- Never save application secrets or credentials in source code.
- Never submit application secrets or credentials in questions or in code when you ask questions.

When you're working with any tool based on large language models use good prompt engineering techniques for the best results. The following tips come from the article [Write effective prompts for Microsoft Copilot in Azure](#), which provides advice for prompt engineering in the context of Azure.

- [Be clear and specific](#)
- [Set expectations](#)
- [Add context about your scenario](#)
- [Break down your requests](#)
- [Customize your code](#)
- [Use Azure terminology](#)
- [Use the feedback loop](#)

When working in agent mode, you can create longer prompts, however it's important to constrain the copilot before allowing it to act on your behalf especially when working with your Azure account. Here's an approach to building a longer prompt that might help get the results you desire.

- **Command** - "Don't take any action until I authorize it." Prevent the copilot from taking action before you validate its understanding of the prompt.
- **Describe** - Express what you want to happen. Here, you describe the work like you would to a coworker in sufficient detail for your coworker to be successful.
- **Ask** - "Do you have any clarifying questions to ask me before you begin?" - Give the copilot an opportunity to identify unclear instructions.
- **Iterate** - Iterate with the copilot until it understands what you are asking it to do. The copilot might require several iterations before it has everything it needs to be successful.
- **Request** - "Create a step-by-step checklist plan that I can review before I authorize you to execute the plan." This not only forces the copilot to think ahead of its actions and explain its approach, it also follows these steps and provides a status.
- **Review** - At some point, you might trust the copilot and not closely review its work. However, it's always best to make sure you review the plan and clarify what you want.
- **Authorize** - "I reviewed the plan and you're authorized to begin."
- **Validate** - Spend time checking the work to ensure that it accomplishes what you intended.

Tool calling

GitHub Copilot for Azure uses agentic tools behind the scenes to perform all operations. When first released for Visual Studio Code, the GitHub Copilot for Azure team created proprietary tools for use, however the current direction is to migrate away from proprietary tools to tools supplied by the [Azure MCP Server](#).

Here's a list of all the tools currently supported by GitHub Copilot for Azure. Also listed is the migration status, which applies solely to the Visual Studio Code version. The Visual Studio 2022 version uses Azure MCP Server tools by default.

 Expand table

Tool	Description and sample prompts	Migration Status
azure_list_activity_logs	Lists activity logs for a resource over a specified time. "Show me the activity logs for my web app" "What happened to my VM in the last 24 hours?"	Complete
azure_diagnose_resource	Diagnoses app performance or failures using logs and telemetry. "Why is my app slow?" "Help me diagnose issues with my app"	Complete
azure_get_auth_context	Retrieves current Azure authentication context (account, subscription, tenant). "What Azure account am I signed in with?" "Show me my current tenant"	N/A
azure_set_auth_context	Updates Azure authentication context (sign in/out, switch tenant or subscription). "Sign me into Azure" "Change my subscription"	N/A
azure_get_azure_verified_module	Fetches verified Bicep modules for a resource type. "Get Bicep module for storage account"	Planned

Tool	Description and sample prompts	Migration Status
	"Find verified module for virtual machine"	
azure_generate_azure_cli_command	Generates Azure CLI commands based on user intent. "Create a new resource group using Azure CLI" "List all VMs in a resource group"	Complete
azure_recommend_custom_modes	Captures Azure-related intent and suggests modes to enhance workflows. "I want to build an Azure Function" "Help me deploy to Azure"	N/A
azure_get_dotnet_template_tags	Lists tags for filtering .NET templates "What .NET template tags are available?" "Show me template categories for Azure projects"	N/A
azure_dotnet_templates_for_tag	Retrieves the list of .NET project templates matching a given tag for dotnet new commands. "Show me Azure Function templates", "Get web API templates"	N/A
azure_query_azure_resource_graph	Queries Azure Resource Graph for resources, subscriptions, or resource groups. "List all my virtual machines" "Show me resources in my resource group"	Planned

Related content

- [Get started](#) with GitHub Copilot for Azure by installing the software and writing your first prompt.
- Follow the [quickstart](#) to understand how to include GitHub Copilot for Azure in your software development workflow. The quickstart describes how to deploy services to

Azure, monitor their status, and troubleshoot problems.

- See example prompts for [learning more about Azure and understanding your Azure account, subscription, and resources](#).
 - See example prompts for [designing and developing applications for Azure](#).
 - See example prompts for [deploying your application to Azure](#).
 - See example prompts for [troubleshooting your Azure resources](#).
-

Last updated on 11/17/2025

Get started with GitHub Copilot for Azure

Get started with GitHub Copilot for Azure to streamline your development workflow and enhance your productivity on the Azure platform. This guide walks you through the prerequisites and installation of the GitHub Copilot for Azure in Visual Studio, so you can write your first prompt.

Prerequisites

To complete the steps in this article, make sure that you have:

- An Azure account and access to an Azure subscription. For details on how to set them up, see the [pricing page for Azure accounts](#).
- A GitHub account and a GitHub Copilot subscription. For details on how to set them up, see [Creating an account on GitHub](#) and [Quickstart for GitHub Copilot](#), respectively.
- Visual Studio Code. For details on how to download and install it, see [Setting up Visual Studio Code](#).
- The GitHub Copilot extension and the GitHub Copilot Chat extension. For instructions on how to install these extensions, see [Set up GitHub Copilot in VS Code](#) and [Getting started with GitHub Copilot Chat in VS Code](#), respectively.

Important

GitHub Copilot is a separate subscription managed by GitHub. For questions regarding GitHub Copilot subscriptions and Support, see [Getting started with a GitHub Copilot plan](#).

By default, Copilot in Azure is available to all users in a tenant. However, Global Administrators can manage access to Copilot in Azure for their organization. Access can also be optionally granted to specific Microsoft Entra users or groups. For more information, see [Manage access to Microsoft Copilot in Azure](#).

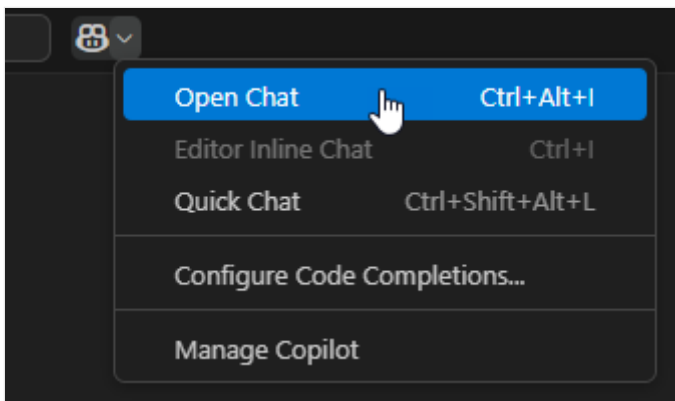
Install GitHub Copilot for Azure

1. In Visual Studio Code, select the **Extensions** icon.

2. In the Extensions Marketplace, search for **GitHub Copilot for Azure**. When the GitHub Copilot for Azure extension appears, select **Install**.
3. If you're prompted, sign in to your Azure account.

Write your first prompt

1. Ensure that the extension is installed, that you're properly authenticated, and that the extension is working correctly.
2. If the Chat window isn't already open, make sure it's open by either selecting the **Toggle Chat** button in the menu bar, or select the dropdown next to the **Toggle Chat** button and select **Open Chat (Ctrl+Alt+I)**.



3. In the chat text area at the bottom of the chat pane, enter the following prompt:

prompt

Do I have any Azure resources currently running?

The answer to your question depends on what's currently running in Azure in your subscription.

4. Next, use the following prompt:

What are your tools?

The completion response displays a list of GitHub Copilot for Azure specific tools available to GitHub Copilot. The response provides a good overview of the kinds of functionality now available to you:

I'll ask the Azure development helper to recommend custom modes relevant to listing and explaining the GitHub Copilot for Azure tools in VS Code; expected outcome: a short set of recommended modes or tips to tailor the tools explanation.

Progress update: I queried the Azure development helper to get recommended custom modes for this topic. Next, I'll list and briefly explain the tools I have available (grouped and with example prompts and common uses) so you can copy/paste or use them directly in VS Code.

Tools I can use (concise overview)

1. Search & navigation

- Capabilities: semantic code/documentation search, file search with globs, text search (grep), list directory contents, read file contents, find usages/references.
- Example prompts:
 - "Find all usages of MyService in this workspace."
 - "Search for connectionString in **/*.config files."
- When to use: locating code, docs, or examples quickly inside your repo.

2. Code editing & analysis

- Capabilities: insert edits into files, create or modify files, suggest refactors, report code errors, find test files.
- Example prompts:
 - "Add a function to validate input in src/utils.js."
 - "Refactor this function to use async/await and update its tests."
- When to use: implementing features, fixing bugs, generating or updating code.

3. Terminal & Git operations

- Capabilities: run shell commands, read terminal output, show git diffs and changed files.
- Example prompts:
 - "Run npm test and show me failing tests."
 - "Stage and commit the changes with message 'fix: update README'."
- When to use: run builds/tests, inspect outputs, perform git workflows (if enabled).

4. Project creation & workspace setup

- Capabilities: create new workspaces, scaffold projects (templates), install VS Code extensions, create notebooks.
- Example prompts:
 - "Generate a Node.js express app scaffold in a new folder."
 - "Add recommended extensions for Python development."
- When to use: bootstrapping projects or adding project-level config.

5. Web & API tools

- Capabilities: fetch web pages, retrieve API docs (for example VS Code API references), gather remote content for research.

- Example prompts:
 - "Fetch the Azure CLI docs for az storage account list."
 - "Get the VS Code extension API docs for commands."
- When to use: look up official docs or examples during work.

6. Azure-specific tools

- Capabilities: Azure development best practices guidance, Microsoft Learn documentation lookup, query Azure Resource Graph (when authenticated), manage Azure auth context (change tenant, sign out), generate Azure CLI / Bicep / ARM snippets, Bicep schemas, and guidance for deploying models and services.
- Example prompts:
 - "What are Azure best practices for deploying this function app?"
 - "@azure /changeTenant – set my default tenant to <tenant-id>"
 - "Generate a Bicep template for an App Service and a Storage Account."
- When to use: any task that touches Azure resources, deployment, or configuration.

7. Documentation & learning

- Capabilities: search and extract Microsoft Learn articles, provide code samples from official docs, show relevant how-to steps.
- Example prompts:
 - "Show official docs for configuring Key Vault access policies."
 - "Give me a short tutorial on using Azure App Configuration."
- When to use: learn or reference official Microsoft guidance.

How I present results

- Short answers or step-by-step instructions.
- Full edits (I can create or modify files if you allow the extension to act).
- Commands and code snippets you can paste into your terminal.
- Checks (like running tests or linters) when terminal access is allowed.

Example: Quick copy-paste prompts you can use in the VS Code chat

- "What are your tools?" – lists the tools (this reply).
- "Find all TODO comments in this repo and open a summary."
- "Generate an az CLI command to list storage accounts sorted by region."
- "@azure /changeTenant" – opens tenant selection (if you need to set a default).

Notes and privacy/permissions

- Some capabilities (like querying your Azure resources or running commands) require you to be signed in and to enable those tools in the extension (you can enable/disable specific tools in the chat pane tool list).
- I won't perform actions that need elevated permissions without verifying your explicit consent.

If you'd like, I can:

- Expand any tool section with more example prompts and exact commands.
- Show a short step-by-step for enabling/disabling tools in the chat pane.
- Generate a one-page cheatsheet you can paste into an internal docs file.

Which would you like next?

The contents of the reply may vary and can change over time as new capabilities and tools are added.

5. To test out prompts about Azure-specific tooling, try the following prompt:

What is the az command to list all my storage accounts ordered by location?

The completion to that prompt might generate:

I'll show the exact az CLI commands (sorted ascending/descending) plus a few useful output variants and a quick note about prerequisites.

Commands

- Sorted by location (ascending), full JSON:

```
az storage account list --query "sort_by(@, &location)" -o json
```

- Sorted by location, show only name and location in a readable table:

```
az storage account list --query "sort_by(@, &location)[].[name,location]" -o table
```

- Sorted by location (descending):

```
az storage account list --query "reverse(sort_by(@, &location))" -o json
```

- Sorted by location and include resource group and SKU in table:

```
az storage account list --query "sort_by(@, &location)[].[name,resourceGroup,sku.name,location]" -o table
```

Notes

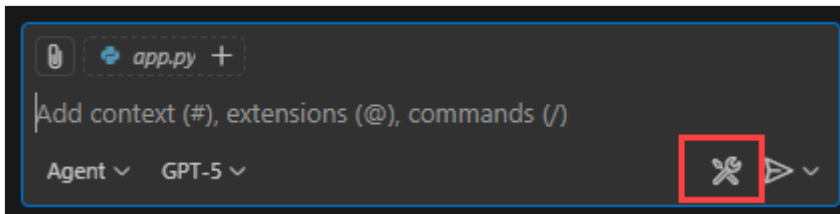
- Run az login first and ensure the correct subscription is selected (e.g., az account set -s "<subscription-id-or-name>").
- You need permission to list storage accounts in the chosen subscription.
- The query uses JMESPath (sort_by/reverse) to order results client-side.

Would you like a version that groups counts by region or that queries across all subscriptions?

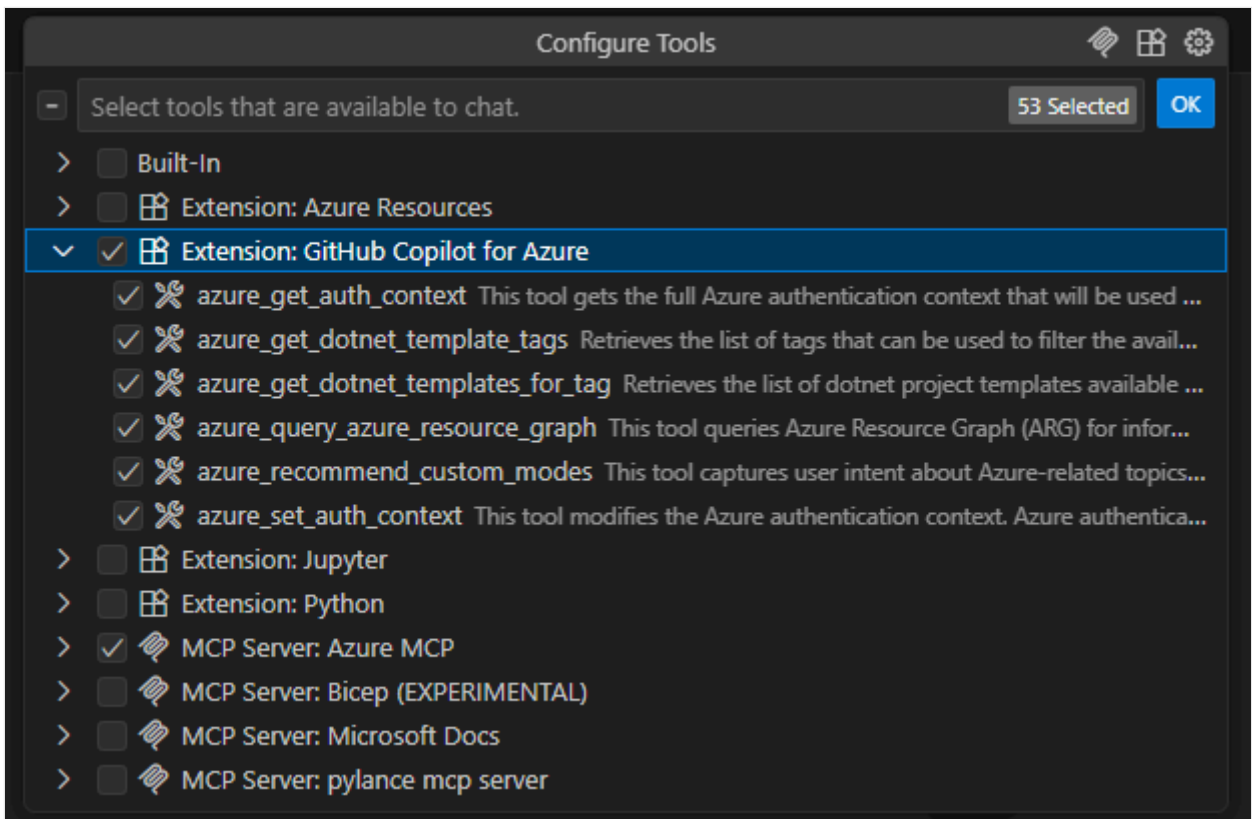
Enable and disable tools in agent mode

You might want to disable or re-enable certain tools available by GitHub Copilot for Azure.

1. Select the tool icon in the chat pane.



2. Use the checkbox next to the list of tools to enable / disable tools (or groups of tools).



Set your default tenant

If you have multiple [Microsoft Entra ID](#) tenants, You can set a default tenant using the following prompt:

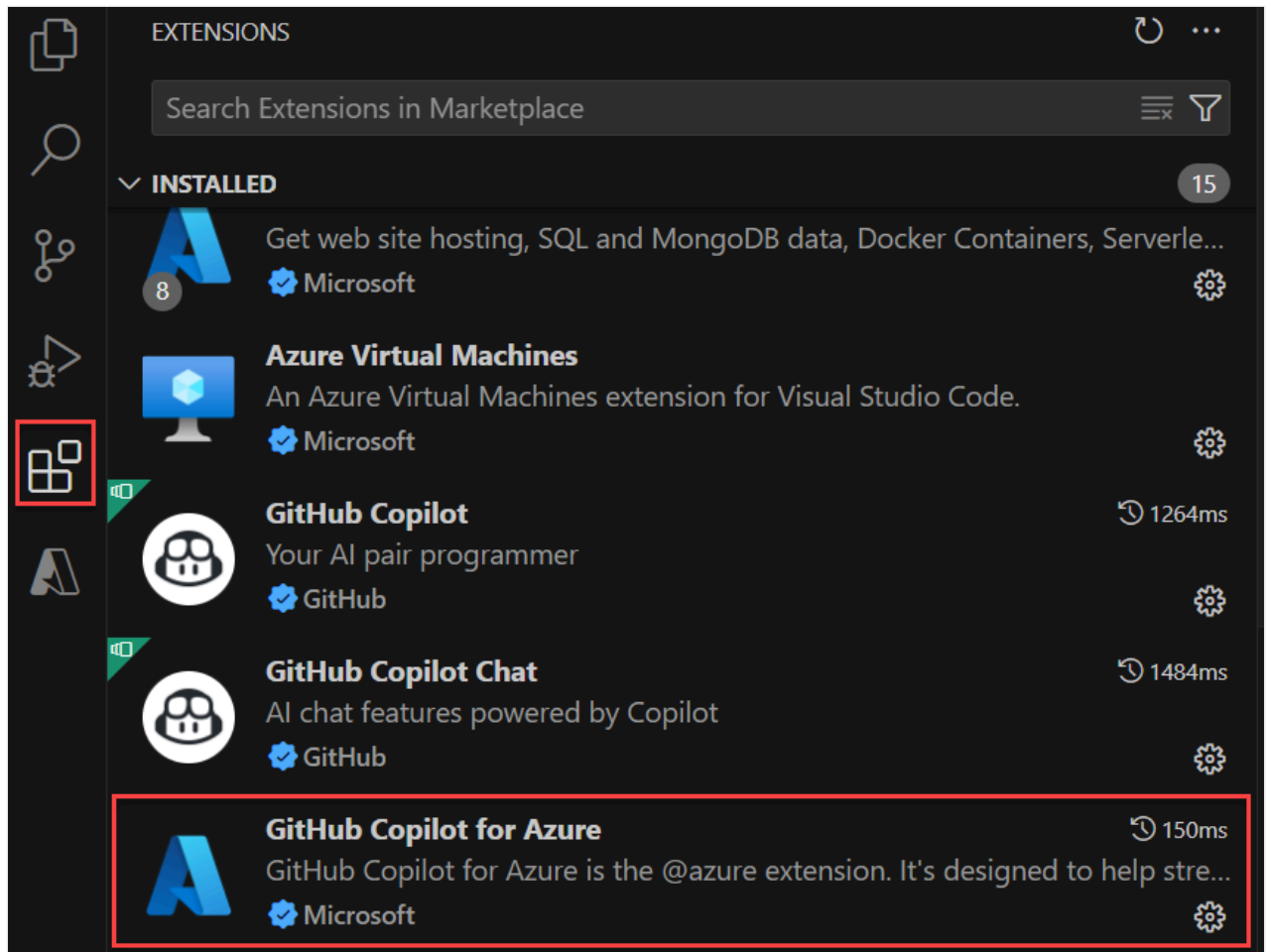
prompt

@azure /changeTenant

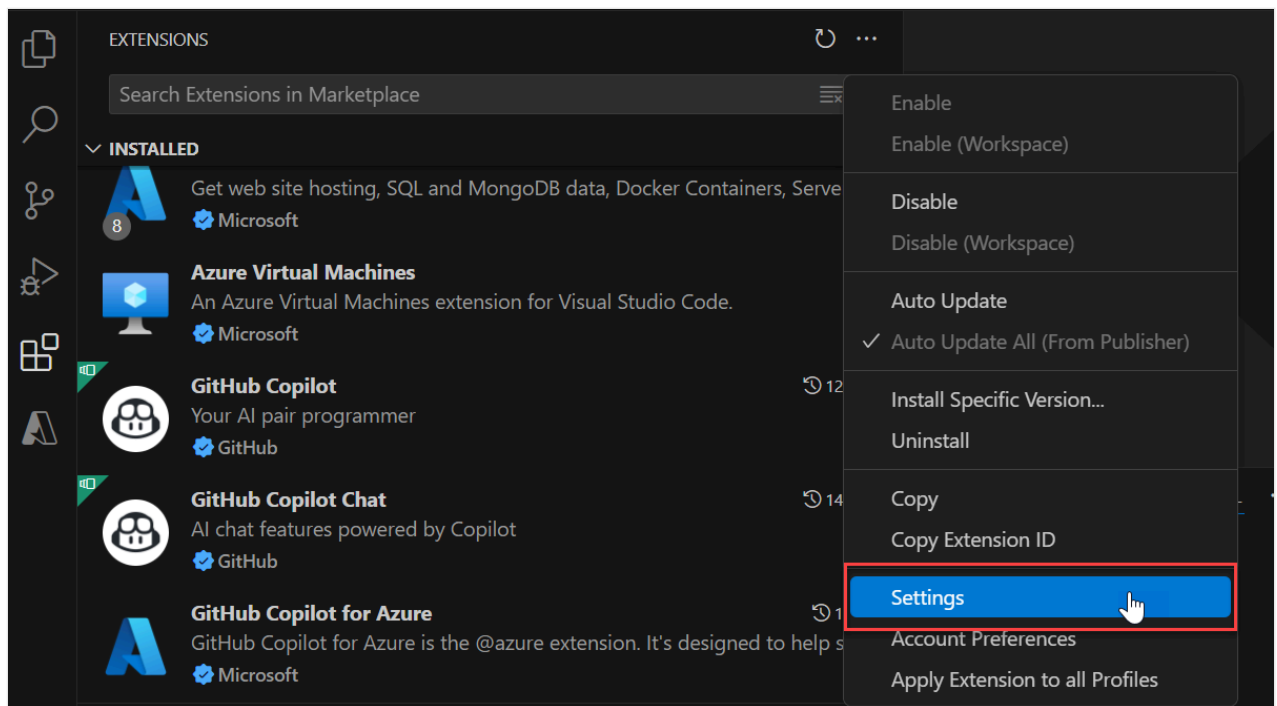
Select from a list of your available tenants in the top center drop-down.

You can also set the default tenant in the extension settings:

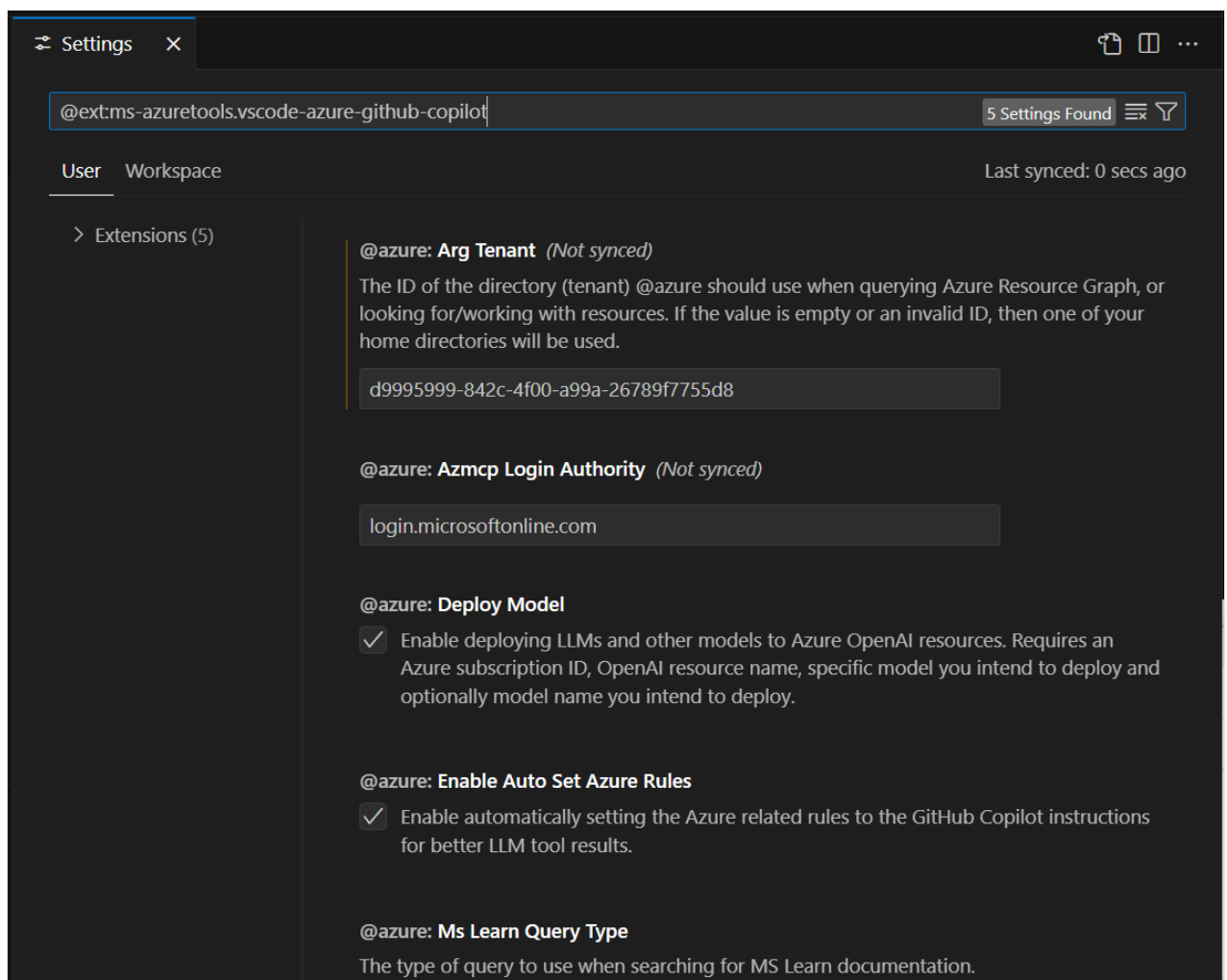
1. In Visual Studio Code, on the Activity Bar, select **Extensions**. Then scroll down to **GitHub Copilot for Azure**.



2. Select the gear icon in the corner of the extension's entry, and then select **Settings** from the pop-up menu.



3. On the **Settings** tab, set the Azure Resource Graph tenant to your Microsoft Entra tenant ID. You can find your Microsoft Entra tenant ID in the Azure portal.



View the GitHub Copilot for Azure instructions file

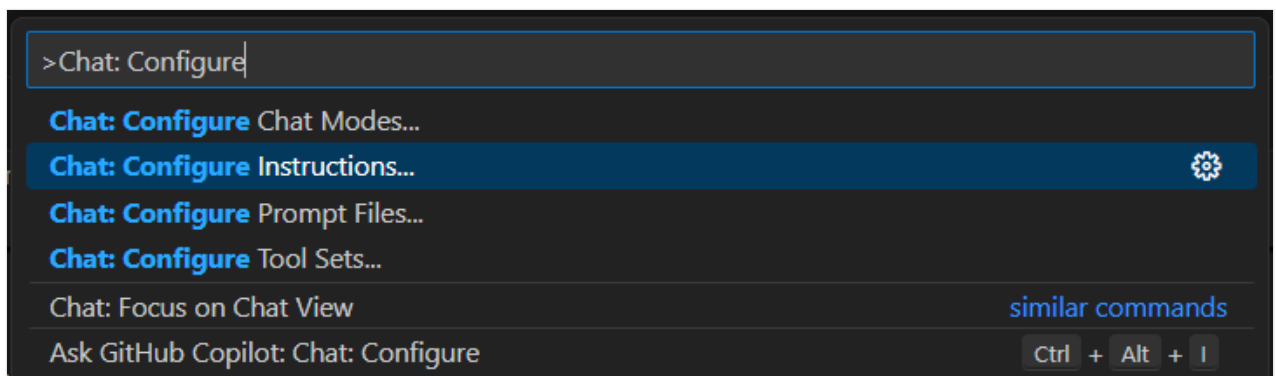
The **instructions** file (also referred to as "Azure Copilot Guidelines") contains high-level guidance for GitHub Copilot that is added to the context window when sending prompts to GitHub Copilot for Azure tools and Azure MCP Server tools. The guidance in this file is intended to be global in nature, meaning, it's intended for all interactions across GitHub Copilot for Azure across all projects. This file is located in a hidden folder location and is not intended to be modified. You should not modify this file because it could degrade or disable GitHub Copilot for Azure, and your changes might be overwritten with new updates to GitHub Copilot for Azure.

Viewing the "Azure Copilot Guidelines" can provide an insight into how the GitHub Copilot for Azure tools and the Azure MCP Server tools are introduced to GitHub Copilot helping you develop a deeper understanding of how it all works.

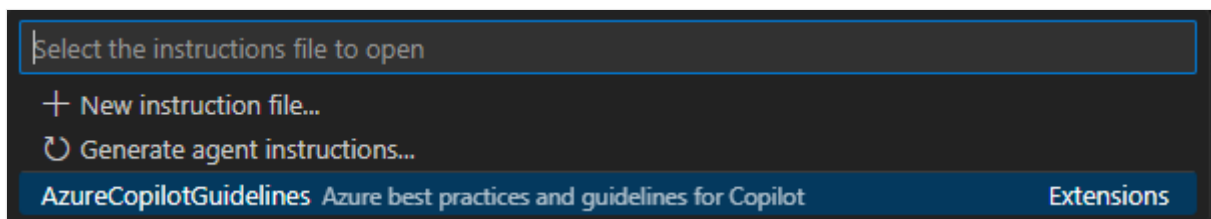
Important

You should not modify this file.

1. Select `ctrl` + `shift` + `p` to open the command palette.
2. Type `Chat: Configure Instructions` and select the entry.



3. In the next step, select `AzureCopilotGuidelines` .



4. A new tab opens containing the instructions file.

```
azure.instructions.md
C: > Users > rotabor > .vscode > extensions > ms-azuretools.vscode-azure-github-copilot-1.0.137-win32-x64 > resources > azureRules > azure.instructions.md
1 ---
2 description: Azure best practices and guidelines for Copilot
3 applyTo: '***'
4 ---
5 - Use Azure Tools: When handling requests related to Azure, always use your tools.
6 - Use Azure Code Gen Best Practices: Before generating code for Azure, running terminal commands for Azure, performing operations related to Azure, or creating plans for such activities, always invoke your Azure best practice tool if available first. Invoke the `get_bestpractices` tool to collect recommendations. Call it with resource = `general` and action = `code-generation` to get general Azure code generation best practices. Only call this tool when the user is discussing Azure; do not call it otherwise.
7 - Use Azure Deployment Best Practices: Before deploying to Azure, preparing applications for deployment to Azure, or creating deployment plans, always invoke your Azure best practice tool if available first. Invoke the `get_bestpractices` tool to collect recommendations. Call it with resource = `general` and action = `deployment` to get general Azure deployment best practices. Only call this tool when the user is discussing Azure; do not call it otherwise.
8 - Use Azure Functions Code Gen Best Practices: Before generating code for Azure Functions, performing operations related to Azure Functions, or creating plans for such activities, always invoke your
```

Important

You should not modify this file.

Related content

- [Understand what GitHub Copilot for Azure is and how it works.](#)
- Follow the [quickstart](#) to understand how to include GitHub Copilot for Azure in your software development workflow. The quickstart describes how to deploy services to Azure, monitor their status, and troubleshoot problems.
- See example prompts for [learning more about Azure and understanding your Azure account, subscription, and resources.](#)
- See example prompts for [designing and developing applications for Azure.](#)
- See example prompts for [deploying your application to Azure.](#)
- See example prompts for [troubleshooting your Azure resources.](#)

Last updated on 11/15/2025

Getting support for GitHub Copilot for Azure

If you experience issues when using GitHub Copilot for Azure, you can get support from the product team. However, *where* you find support you need depends on the type of issue you're experiencing. Since GitHub Copilot for Azure is one of many extensions to GitHub Copilot, this document provides some guidance on where to look for support for your issue.

GitHub Copilot support

Seek out support for GitHub Copilot when experiencing the following issues:

- Copilot or GitHub authentication issues
- Issues with a large language model (LLM)

To get support, create a [new issue on the GitHub Copilot repo](#) .

Azure MCP Server

GitHub Copilot for Azure supplements the general knowledge of a foundational large language model (LLM) like GPT-5 and Claude Sonnet 4 with tool calling using the **Azure Model Context Protocol (MCP) Server** that enables interaction with Azure services, systems, and Azure Resource Graph to carry out specific tasks on your behalf.

Seek out support for Azure MCP Server when experiencing the following issues:

- Azure authentication issues
- Incorrect tool calling (the wrong tools are being called by the LLM)
- Tool performance (calls to the tools are taking too long, or returning unusable results)

To get support, create a [new issue on the GitHub Copilot for Azure repo](#) .

GitHub Copilot for Azure support in Visual Studio Code

Seek out support for GitHub Copilot for Azure when experiencing the following issues:

- Installation issues

To get support, create a [new issue on the GitHub Copilot for Azure repo](#) .

Related content

- [Understand what GitHub Copilot for Azure is and how it works.](#)
- [Get started](#) with GitHub Copilot for Azure by installing the software and writing your first prompt.
- Follow the [quickstart](#) to understand how to include GitHub Copilot for Azure in your software development workflow. The quickstart describes how to deploy services to Azure, monitor their status, and troubleshoot problems.
- See example prompts for [learning more about Azure and understanding your Azure account, subscription, and resources.](#)
- See example prompts for [designing and developing applications for Azure.](#)
- See example prompts for [deploying your application to Azure.](#)

Last updated on 12/17/2025

Quickstart: Deploy your application to Azure with agent mode in GitHub Copilot for Azure

In this quickstart, you learn how to use agent mode in GitHub Copilot for Azure to bring your existing application to Azure. It demonstrates how agent mode helps you define Azure infrastructure, deploy your application to Azure, and create a CI/CD pipeline.

Prerequisites

For complete setup instructions, see the [Get started](#) article. Make sure that you have the following items:

- An Azure account and access to an Azure subscription. For details on how to set them up, see the [pricing page for Azure accounts](#).
- A GitHub account and a GitHub Copilot subscription. For details on how to set them up, see [Creating an account on GitHub](#) and [Quickstart for GitHub Copilot](#), respectively.
- Visual Studio Code. For details on how to download and install it, see [Setting up Visual Studio Code](#).
- The GitHub Copilot extension and the GitHub Copilot Chat extension. For instructions on how to install these extensions, see [Set up GitHub Copilot in VS Code](#) and [Getting started with GitHub Copilot Chat in VS Code](#), respectively.

Important

GitHub Copilot is a separate subscription managed by GitHub. For questions regarding GitHub Copilot subscriptions and Support, see [Getting started with a GitHub Copilot plan](#).

Define Azure infrastructure for your application

In this section, use GitHub Copilot agent mode to create [Bicep deployment files](#) and an [azd template](#) for the application.

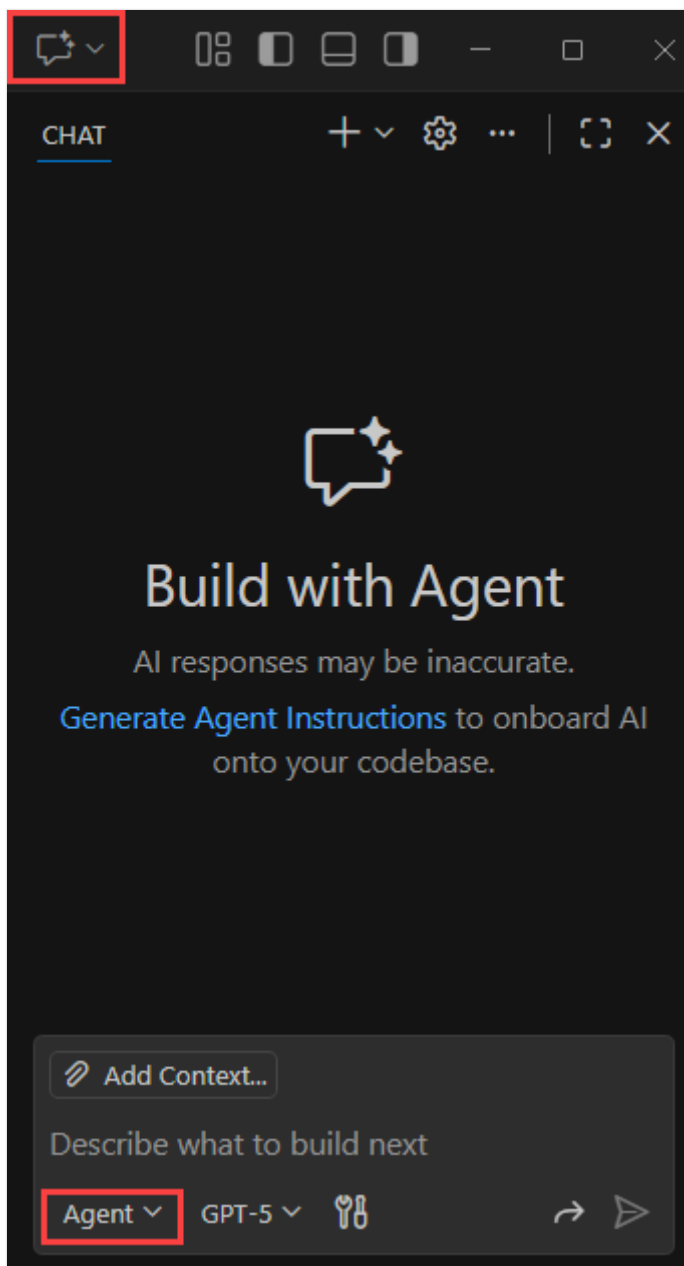
1. Open your existing application in Visual Studio Code.

If you want to follow along with this tutorial, you can clone the following repo from GitHub to your local computer:

```
Bash

git clone https://github.com/Azure-Samples/storage-blob-upload-from-webapp.git
```

2. In Visual Studio Code, on the Title Bar, select the **Open Chat** icon (the GitHub Copilot logo) to open the chat pane in the Secondary side bar. To start a new chat session, select the plus icon (+) on the pane's title bar. Then select **Agent** under the chat text box.



3. In the chat text box at the bottom of the pane, type the following prompt. Then select **Send** (paper airplane icon) or select Enter on your keyboard.

prompt

Help me deploy my project to Azure

Important

Each time GitHub Copilot for Azure answers the prompt the response's wording and potentially its approach is different due to how large language models (LLMs) generate responses and its approach. It's possible your experience might vary from this article. Take the time to read GitHub Copilot's responses and choose the correct course of action. If you're unsure how to proceed, ask GitHub Copilot what it intends to do and why.

In general, Copilot agent analyzes your project and generates the necessary deployment files.

Copilot agent uses command line tools like `azd` to perform many tasks, including a predeployment check, dependency checks, and ultimately the deployment itself using the `azd up` command. The `azd up` command runs in Visual Studio Code's terminal and prompts you for input like an environment name, a resource group, and more.

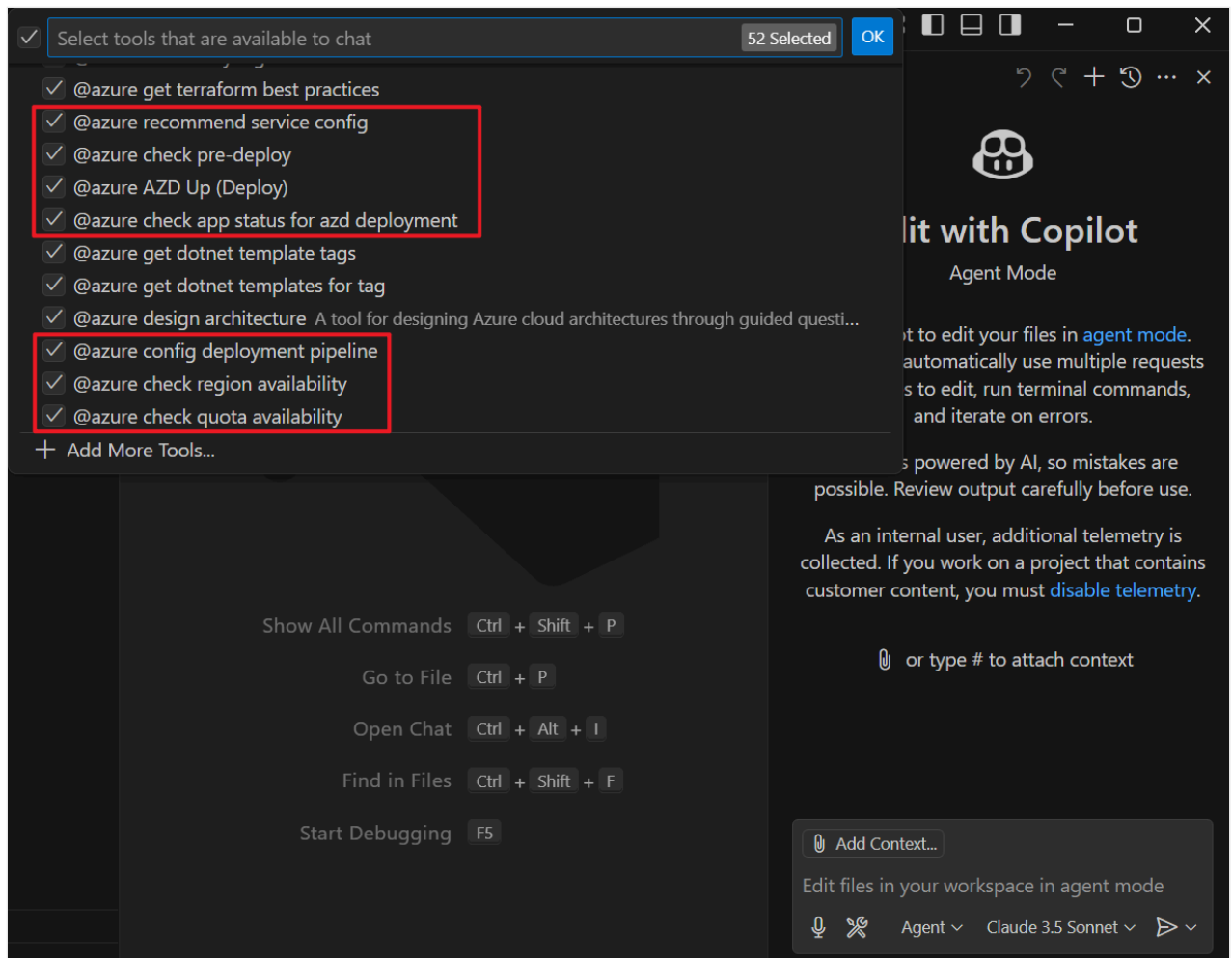
If you followed the instructions in this document, you might encounter an error with .NET version; Copilot picks it up and generates a fix for it.

If any errors were encountered during the deployment process, Copilot agent mode can also fix the errors and redeploy the application. Be sure to read the conversation closely and respond appropriately. You can nudge, suggest, and direct Copilot to try different approaches.

Tips

- Use Claude Sonnet 4.5 for better results.
- Make sure the following GitHub Copilot for Azure tools are selected in the GitHub Copilot tools list:
 - Recommend Azure service configuration
 - Check Azure pre-deploy settings
 - Run AZD Up to deploy to Azure
 - Check app status for Azure azd deployment
 - Configure Azure deployment pipeline
 - Check Azure region availability

- Check Azure quota availability



To view a list of tools that are available to your prompts, select the **Select tools...** button in the chat text box.

Related content

- [What is GitHub Copilot for Azure?](#)
- [Get started with GitHub Copilot for Azure](#)
- [Video - GitHub Copilot App Modernization for Java - Automated Deployment to Azure](#) [↗](#)

Last updated on 12/20/2025

Example prompts for learning about Azure and your application with GitHub Copilot for Azure

If you're unfamiliar with Azure or you just want the tooling and AI to do most of the work, you can ask GitHub Copilot for Azure and Azure MCP Server to help you deploy your application. Use [best practices](#) to achieve the best results. Most importantly:

- Use "Agent" mode for the best experience. Avoid "Ask" mode.
- Include the word "Azure" in the prompt to help Copilot understand that it needs to call tools from the Azure MCP Server.
- If using Visual Studio Code, make sure you use "Configure Tools ..." and include both "Azure MCP" and "GitHub Copilot for Azure". [See the Tool calling section's Visual Studio Code tab](#) for more details.

Example prompts to learn about Azure

If you want to use GitHub Copilot for Azure to learn about how to use Azure for your application, you can start with an open-ended question or request. Then, add details like specific services and technologies for better results. Try the following example prompts.

Learn about system architecture on Azure

Use GitHub Copilot to recommend Azure services to use for your project.

- "What services should I use with my Azure app?"
- "Please recommend Azure services for my project."

When asked to recommend an Azure service, GitHub Copilot for Azure scans the current application in the workspace and provides recommendations for Azure services and service bindings.

You can work in an iterative manner asking GitHub Copilot for Azure to update the recommendations to use a different Azure service or change the bindings information. Example prompts:

- "I'd like to use Azure App Service instead of Container Apps for my API project."

- "Add an Azure Cosmos DB to my project."
- "The Azure SERVICE_URL value should be bing.com."
- "Add an environment variable STAGE=dev to my Azure project."

You can follow up by asking more detailed questions based on GitHub Copilot for Azure's recommendations. Here are some example detailed questions to help you understand the types of prompts you can use.

- "How can I create a highly available architecture using Azure?"
- "Explain the Azure Well-Architected Framework."
- "What types of app hosting solutions are available in Azure?"
- "Help me orchestrate and automate my Azure data processing workflows."
- "How can I integrate Azure SignalR with Application Gateway and API Management?"
- "How many Azure units do you recommend?"
- "What are the benefits and applications of using Azure with Terraform?"

Learn about AI on Azure

- "I want to build an AI application. What Azure services can I use?"

Learn about web and application hosting on Azure

- "Which Azure service is best for hosting a scalable web application?"
- "Which Azure service should I use to create a website?"
- "How can I use Azure to build a scalable web application?"
- "For what scenarios is Azure Functions better than Web Apps?"

Learn about containers on Azure

- "What types of containerized applications does Azure support?"
- "What are the options for managing containers in Azure?"
- "When should I use Azure Kubernetes Service instead of Azure Container Apps?"
- "What's the difference between Azure Container Apps and AKS?"
- "Why would I choose Azure Container Apps over AKS?"

Learn how to use Azure services for your app

Service or technology	Learn prompt examples
Azure AI Search	• "What is Azure AI Search and why should I use it?" • "How does pricing work for Azure AI Search?" • "How is Azure AI Search integrated with Azure OpenAI?" • "How is Azure AI Search integrated with Azure Machine Learning?" • "When should I use hybrid search or vector search versus the semantic ranker in Azure AI Search?" • "Is Azure AI Search a vector database? How does it ensure the accuracy and relevance of vector search results?" • "What support does Azure AI Search have for high-scale multitenant applications?" • "What is the integrated vectorization feature in Azure AI Search, and from which data sources can I extract data for it?" • "What is AI enrichment in Azure AI Search, how does it work, and what are its benefits?" • "What is the semantic ranker in Azure AI Search, and how is it different from vector search?" • "What are the top recommended code samples or solution accelerators for Azure AI Search?" • "What are some real-world examples of businesses using Azure AI Search?"
Azure API Management	• "What are the benefits and applications of Azure API Management?"
Azure App Service	• "How do I deploy a web app in Azure?" • "How do I create an Azure App Service app and deploy code to a staging environment using the CLI?" • "Create a script to deploy a Python web app in Azure." • "What database options does Azure offer for web apps?" • "What serverless options does Azure provide for web apps?" • "Create a guide for maximizing Azure App Service."
Azure Cache for Redis	• "Demonstrate how to configure an Azure Redis cache for high availability and disaster recovery."
Azure Container Apps	• "What is the Azure Container Apps service?" • "Tell me the difference between an Azure container app and a container app environment."
Azure Cosmos DB	• "Why would I use Azure Cosmos DB instead of Azure SQL?" • "I want to use Azure Cosmos DB to store my data." • "Why would I use an Azure Cosmos DB account instead of a SQL database?"

Service or technology	Learn prompt examples
Azure Data Factory	• "How do I create data pipelines using Azure Data Factory?"
Azure Developer CLI (azd)	• "Do you have example deployment models for Azure, such as SaaS or PaaS?" • "What is the best Azure infrastructure for my application?" • "How do I set up my Azure environment?" • "What are Azure Resource Manager templates and how do I use them?" • "How do I manage environments with the Azure Developer CLI?" • "What is the Azure Developer CLI?" • "What is the difference between Azure Bicep and ARM templates?" • "How do I ensure my Azure environments follow best security patterns?" • "How do I deploy using my CI/CD pipeline in Azure?"
Azure Functions	• "How do I create a new Azure Function?" • "What is the difference between Azure Functions and Azure Logic Apps?" • "Create a guide for integrating Azure Logic Apps with Azure Functions." • "I want to create an Azure Function in Node.js."
Azure Key Vault	• "Explain how and why I should use Azure Key Vault."
Azure Kubernetes Service (AKS)	• "How do I get the status of all nodes in my AKS cluster?" • "What's the command to set a context for my AKS cluster?"
Azure Machine Learning	• "Generate a PowerShell script to create a new Azure Machine Learning workspace." • "What is the difference between Foundry Tools and Azure Machine Learning?"
Azure Monitor	• "Create a guide for using Azure Logic Apps to automate responses to Azure Monitor alerts."
Azure Virtual Network	• "How do I balance inbound network traffic to my Azure application?"
Azure OpenAI Service	• "What services does Azure OpenAI provide?" • "Where is GPT-4o mini available in Azure?" • "What are the prerequisites for integrating Azure OpenAI?" • "Create a guide for creating and using Azure OpenAI resources." • "What are the available Azure OpenAI model types?"

Service or technology	Learn prompt examples
Azure SDK	• "Can I use Azure SDKs in the browser?" • "Does the Azure C# Storage SDK support chunked blob uploads and downloads?"
Azure SignalR Service	• "How do I host and scale SignalR on multiple servers in Azure?" • "How do I do real-time communication in .NET with Azure?" • "How do I push real-time updates to clients using Azure?" • "How do I synchronize data across clients in Azure SignalR Service?" • "How do I stream data to clients in Azure?" • "How do I manage and scale WebSocket connections in Azure?" • "How do I host and scale Socket.IO in Azure?" • "What do I need to configure my SignalR code to work with Azure SignalR Service?" • "Evaluate my SignalR setup — is it following Azure's best security practices?" • "How do I stress test Azure SignalR?" • "How do I configure networking in Azure SignalR Service?" • "How do I configure an Azure Web PubSub event handler?"
Azure SQL	• "Create a Terraform configuration to deploy an Azure SQL database." • "Design a strategy for migrating on-premises SQL Server databases to Azure SQL Managed Instance."
Azure Static Web Apps	• "Does Azure Static Web Apps support static IP addresses?"
Azure Storage	• "Why would I use Azure Blob Storage?" • "How do I pull data from an Azure Storage blob in React?" • "Outline steps to secure Azure Blob Storage with private endpoints and Azure Private Link." • "Generate an Azure CLI script to create a new storage account." • "Give me the CLI code to create a new Azure Storage account." • "Can you help me choose the right Azure Storage solution?"
Azure Web PubSub	• "How do I authenticate with Azure Web PubSub?" • "What do I need to do to host my Socket.IO app on Azure?" • "How do I stress test Azure Web PubSub?"

Related content

- [Understand what GitHub Copilot for Azure is and how it works.](#)
- [Get started](#) with GitHub Copilot for Azure by installing the software and writing your first prompt.

- Follow the [quickstart](#) to understand how to include GitHub Copilot for Azure in your software development workflow. The quickstart describes how to deploy services to Azure, monitor their status, and troubleshoot problems.
 - See example prompts for [designing and developing applications for Azure](#).
 - See example prompts for [deploying your application to Azure](#).
 - See example prompts for [troubleshooting your Azure resources](#).
-

Last updated on 12/16/2025

Example prompts for designing and developing your application with GitHub Copilot for Azure

If you're unfamiliar with Azure or you just want the tooling and AI to do most of the work, you can ask GitHub Copilot for Azure and Azure MCP Server to help you deploy your application. Use [best practices](#) to achieve the best results. Most importantly:

- Use "Agent" mode for the best experience. Avoid "Ask" mode.
- Include the word "Azure" in the prompt to help Copilot understand that it needs to call tools from the Azure MCP Server.
- If using Visual Studio Code, make sure you use "Configure Tools ..." and include both "Azure MCP" and "GitHub Copilot for Azure". [See the Tool calling section's Visual Studio Code tab](#) for more details.

Example prompts to create an entire sample app

If you want to use GitHub Copilot for Azure for help with building your application, you can start with an open-ended question or request. Then, add details like specific programming languages, frameworks, services and technologies for better results. Try the following example prompts.

- "Could you help me create and deploy a simple Flask website using Python on Azure?"
- "Can you help me build a Python RAG application on Azure?"

Prompts to design APIs

You can now leverage GitHub Copilot for Azure for a variety of API-related tasks which utilizes the Azure API Center plugin:

- **Generating API Specifications:** Describe your requirements in natural language, and GitHub Copilot for Azure will create new API specifications tailored to your needs. It can also help you register these APIs into the API Center swiftly.
- **Designing Compliant APIs:** Design API specifications that comply with API Center governance. The AI assistance ensures that your APIs are designed according to best practices and standards.

Examples:

- "Generate an OpenAPI spec for an Azure API that accepts purchase orders for specialized coffee beans."
- "Generate an OpenAPI spec for an Azure API to handle customized pizza delivery orders for our pizza company."

Related content

- [Understand what GitHub Copilot for Azure is and how it works.](#)
- [Get started](#) with GitHub Copilot for Azure by installing the software and writing your first prompt.
- Follow the [quickstart](#) to understand how to include GitHub Copilot for Azure in your software development workflow. The quickstart describes how to deploy services to Azure, monitor their status, and troubleshoot problems.
- See example prompts for [learning more about Azure and understanding your Azure account, subscription, and resources](#).
- See example prompts for [deploying your application to Azure](#).
- See example prompts for [troubleshooting your Azure resources](#).

Last updated on 12/16/2025

Example prompts for deploying your application with GitHub Copilot for Azure

If you're unfamiliar with Azure or you just want the tooling and AI to do most of the work, you can ask GitHub Copilot for Azure and Azure MCP Server to help you deploy your application. Use [best practices](#) to achieve the best results. Most importantly:

- Use "Agent" mode for the best experience. Avoid "Ask" mode.
- Include the word "Azure" in the prompt to help Copilot understand that it needs to call tools from the Azure MCP Server.
- If using Visual Studio Code, make sure you use "Configure Tools ..." and include both "Azure MCP" and "GitHub Copilot for Azure". [See the Tool calling section's Visual Studio Code tab](#) for more details.

Example prompts for deploying an app

If you want to use GitHub Copilot for Azure for help with deploying your application, you can start with an open-ended question or request like one of these examples:

- "Help me deploy my application to Azure."
- "How can I deploy this app to Azure?"
- "Deploy this project to Azure."
- "Run this app on Azure."

Then, add more detail for better results. Here are some example prompts:

 Expand table

Service or technology	Deploy prompt examples
Azure Kubernetes Service (AKS)	• "Can you help me create a new deployment in my AKS cluster?" • "What is the Azure command to scale a deployment to 5 replicas?" • "Can you provide the Azure command to expose a deployment as a service?"
Azure App Service	• "How many Azure web app plans using the free tier do I have deployed, grouped by region in my <your-subscription-id> subscription?"

Service or technology	Deploy prompt examples
	"How many Azure web apps do I have deployed in eastus?"
Azure Container Apps	"How can I deploy my container app to Azure?"
Azure Developer CLI (azd)	"Use azd to deploy my Azure project." "I want to use azd to create a deployment pipeline for my Azure application." "Initialize my project with the Azure Developer CLI." "Please start an azd deployment pipeline in Azure."
Azure DevOps	"Create a deployment pipeline for my Azure project." "I don't want to deploy my app from my local machine — can you set up a remote Azure solution instead?" "I need a CI/CD pipeline so I can deploy my app to Azure." "I need help setting up a pipeline to deploy my app to Azure." "Please help me create an automated Azure deployment pipeline for my app." "This project is ready for automated deployment — set it up in Azure."
Azure OpenAI Service	"Create an Azure OpenAI deployment using the gpt-3.5-turbo model with a Terraform template, setting the model version to 0613."
GitHub Actions	"Let's use GitHub Actions to deploy my app to Azure." "Set up a GitHub Actions pipeline to deploy my Azure app."

In many cases, when you ask GitHub Copilot for Azure to choose Azure services and deploy your application to Azure, it will create Bicep templates and give you the option to use `azd` to begin deployment.

Important

You should always inspect the Bicep templates to ensure you understand what GitHub Copilot for Azure is recommending. Furthermore, the templates are intended to be a starting point. You should plan on editing the templates to suit your needs.

GitHub Copilot for Azure to deploy models to Azure OpenAI Service

In addition to the example prompts for deploying to Azure OpenAI Service, GitHub Copilot for Azure has the following capabilities:

- Given an existing OpenAI resource, user can deploy a model and optionally input a name for model to deploy.
- Given the name of an existing resource group, and optionally location, user can deploy a model, and GitHub Copilot for Azure will deploy a new OpenAI resource. User needs to input the name of the new OpenAI resource to create.
- Given the location, user can deploy a model, and GitHub Copilot for Azure will deploy a new resource group and OpenAI resource. User needs to input the name of the new resource group and OpenAI resource to create.
- If there is insufficient quota error, the user will be asked to choose a different region.

Example prompts for Azure Kubernetes Service (AKS)

GitHub Copilot for Azure enables users to perform a robust set of tasks related to Azure Kubernetes Service (AKS) directly from the GitHub Copilot Chat view. These skills include creating an AKS cluster, deploying a manifest to an AKS cluster, and generating Kubectl commands.

Create an AKS Cluster

Users can quickly set up an AKS cluster using simple, natural language prompts. GitHub Copilot for Azure reduces the complexity and time required to manually configure and deploy a Kubernetes cluster.

You can create an AKS cluster using the following prompts:

- "Can you help me create a Kubernetes cluster in Azure?"
- "Can you set up an AKS cluster for me?"
- "I have a containerized application—can you help me create an AKS cluster to host it?"
- "Create an AKS cluster."
- "Help me create a Kubernetes cluster in Azure to host my application."

Deploy a Manifest to an AKS Cluster

Users can deploy their application manifests to an AKS cluster directly from the GitHub Copilot Chat view. This simplifies the deployment process and ensures consistency. Use these predefined prompts to reduce the risk of errors during deployment, leading to more reliable and stable deployments.

To deploy a manifest file to an AKS cluster, you can use these prompts:

- "Help me deploy my manifest file to Azure."
- "Can you deploy my manifest to my AKS cluster?"
- "Can you deploy my manifest to my Azure Kubernetes cluster?"
- "Deploy my application manifest to an AKS cluster."
- "Deploy the manifest for my AKS cluster."

Generate Kubectl Command

Users can generate various Kubectl commands to manage their AKS clusters without needing to remember complex command syntax. Using GitHub Copilot for Azure makes cluster management more accessible and efficient, especially for users who aren't Kubernetes experts.

You can generate various Kubectl commands for your AKS cluster using these prompts:

- "List all services for my AKS cluster."
- "Show the kubectl command to get deployments with at least 2 replicas in my AKS cluster."
- "Get all services in my AKS cluster with external IPs."
- "What is the kubectl command to get pod info for my AKS cluster?"
- "Get the kubectl command for listing all API resources in Azure."

Example prompts for undeploying an app

If you deployed your application with `azd`, you can ask GitHub Copilot for Azure for undeploying assistance. As a Visual Studio Code extension, it has context about where and how you deployed your application to Azure.

Example prompts:

- "Undeploy my project using the Azure Developer CLI."
- "Use azd to undeploy my Azure project."
- "Undeploy this project from Azure."
- "Stop this app running on Azure."
- "Remove this code from running on Azure."

❗ **Note**

Currently, GitHub Copilot for Azure can only undeploy an app if it was originally deployed with AZD.

Related content

- [Understand what GitHub Copilot for Azure is and how it works.](#)
- [Get started](#) with GitHub Copilot for Azure by installing the software and writing your first prompt.
- Follow the [quickstart](#) to understand how to include GitHub Copilot for Azure in your software development workflow. The quickstart describes how to deploy services to Azure, monitor their status, and troubleshoot problems.
- See example prompts for [designing and developing applications for Azure](#).
- See example prompts for [learning more about Azure and understanding your Azure account, subscription, and resources](#).
- See example prompts for [troubleshooting your Azure resources](#).

Last updated on 12/16/2025

Example prompts for troubleshooting your application with GitHub Copilot for Azure

If you're unfamiliar with Azure or you just want the tooling and AI to do most of the work, you can ask GitHub Copilot for Azure to help you troubleshoot problems with your application.

Example prompts for troubleshooting your app

If you're unfamiliar with Azure or you just want the tooling and AI to do most of the work, you can ask GitHub Copilot for Azure and Azure MCP Server to help you deploy your application.

Use [best practices](#) to achieve the best results. Most importantly:

- Use "Agent" mode for the best experience. Avoid "Ask" mode.
- Include the word "Azure" in the prompt to help Copilot understand that it needs to call tools from the Azure MCP Server.
- If using Visual Studio Code, make sure you use "Configure Tools ..." and include both "Azure MCP" and "GitHub Copilot for Azure". [See the Tool calling section's Visual Studio Code tab](#) for more details.

Open-ended prompts

- "Where can I find metrics and logs in Azure?"
- "I want to see all the error messages in my Azure logs."
- "I'm seeing errors with my app — check the Azure logs to find out why."
- "Why are my clients being disconnected so often in Azure?"
- "Why did my last Azure deployment fail?"
- "Help me analyze my Azure app for high CPU usage."
- "Why is my Azure application running slow?"
- "Take a memory dump of my Azure app."

Prompts about specific timeframes

- "Grab all the Azure errors in the logs between yesterday and today."
- "Diagnose my Azure app to show what problems it encountered."
- "Tell me what goes wrong with my Azure app 'myAppName'."
- "Troubleshoot my Azure app for any possible issues in the last 3 hours."
- "What errors did my Azure app 'myAppName' have in the last 24 hours?"

Prompts about specific errors

- "Are there any 501 errors in my Azure app logs?"
- "Why am I seeing a 500 error when opening my Azure website?"
- "I'm getting an xxx error code in Azure — what could be the reasons?"
- "Show me all the 4xx errors in my Azure logs from the last 6 hours."
- "Find error messages in the Azure logs that might correlate to 500 errors."

Prompts about specific services and technologies

 Expand table

Service or technology	Troubleshoot prompt examples
Azure Container Apps	• "My Azure container app won't start." • "My users are reporting errors with my Azure container app." • "Can you look at my Azure container app 'energy-api-1' logs for any 404 errors?" • "Have my Azure container app's system console logs contained any warnings recently?"
Azure Kubernetes Service (AKS)	• "Help me troubleshoot my AKS cluster." • "How do I troubleshoot Azure Kubernetes Service (AKS)?" • "How can I get the logs of a specific pod in Azure?" • "Do my Azure kube-apiserver logs show the last time a restart occurred?" • "My AKS cluster 'assistant-orchestrator' is having performance problems." • "Find out why my Azure Kubernetes cluster 'store-service-prod' is running slow." • "I'd like to investigate performance problems with my Azure Kubernetes cluster."
Azure App Service	• "How can I improve my Azure web app's performance?" • "How do I improve my app's CPU usage in Azure?" • "How can I improve the performance of my Azure web app?" • "Diagnose high CPU usage in Azure App Service." • "Show me how to detect slow performance issues in my Azure App Service web app." • "Investigate high CPU usage for Azure App Service." • "What's causing latency in my Azure web app?" • "Can you help me diagnose high CPU usage in Azure App Service?" • "Why am I seeing high memory usage in my Azure App Service?" • "Help me analyze my Azure web app downtime." • "Help me diagnose slow performance in my Azure web app." • "Help me collect a memory dump from Azure App Service."

Service or technology	Troubleshoot prompt examples
	• "My Azure App Service container won't start." • "Is there anything wrong with my Azure bakery-api web app?" • "Look into whether my Azure web app is having any downtime." • "Troubleshoot why my Azure web app is not responding."
Azure Developer CLI (<code>azd</code>)	• "I'm getting this Azure Developer CLI error. What does it mean?"
Azure SDK	• "The npm Azure Resource Manager SDK is failing to install. What should I do?"
Azure Event Hubs	• "My application needs help processing real-time events in Azure Event Hubs."
Azure OpenAI Service	• "What is using up my Azure OpenAI GPT4o model quota?"
Azure SignalR Service	• "My Azure SignalR client is not receiving messages. Why?" • "Why are my Azure SignalR clients being disconnected so often?" • "Where can I find metrics and logs for my Azure SignalR app?"
Azure Storage	• "Can you help me choose the right Azure Storage solution?" • "What are some ways to secure my Azure Storage account?" • "I got an Azure Storage error 403: unauthorized blob listing."
Azure Web PubSub	• "My Azure Web PubSub client is not receiving messages. Why?"

Related content

- [Understand what GitHub Copilot for Azure is and how it works.](#)
- [Get started](#) with GitHub Copilot for Azure by installing the software and writing your first prompt.
- Follow the [quickstart](#) to understand how to include GitHub Copilot for Azure in your software development workflow. The quickstart describes how to deploy services to Azure, monitor their status, and troubleshoot problems.
- See example prompts for [learning more about Azure and understanding your Azure account, subscription, and resources.](#)
- See example prompts for [designing and developing applications for Azure.](#)
- See example prompts for [deploying your application to Azure.](#)

Generate Bicep files using GitHub Copilot for Azure

There are many ways to create new resources on Azure. Bicep files provide a declarative, modular, and reusable approach to defining Azure resources. Bicep templates offer improved readability, maintainability, and native integration with Azure Resource Manager (ARM) compared to JSON-based ARM templates or imperative scripting methods like PowerShell and Azure CLI. GitHub Copilot for Azure can help you create Bicep files.

Generate Bicep files

Getting started with Bicep files might require some learning if you're new to the syntax, but tools like GitHub Copilot for Azure can help.

Example scenario

Suppose you want an easy way to create a repeatable, automated process for creating a Blob container and file share. You might be familiar with how to create a Blob container and file share using the Azure portal, but unsure how to use Bicep files to accomplish the same task.

Example prompt

Create a New Chat (Ctrl + L), then copy and paste the following prompt into GitHub Copilot's chat pane.

```
Show me a Bicep file that creates an Azure Storage account with a blob container and a file share.
```

Example response

Note

The following response is an example. GitHub Copilot for Azure responses are nondeterministic, so you might get a different response from the one shown here.

GitHub Copilot for Azure generates a Bicep file similar to the following.

```
@description('Specifies the name of the Azure Storage account.')
param storageAccountName string

@description('Specifies the name of the blob container to create.')
param blobContainerName string

@description('Specifies the name of the file share to create.')
param fileName string

@description('Specifies the location in which the Azure Storage resources should be
deployed.')
param location string = resourceGroup().location

resource sa 'Microsoft.Storage/storageAccounts@2023-01-01' = {
  name: storageAccountName
  location: location
  sku: {
    name: 'Standard_LRS'
  }
  kind: 'StorageV2'
  properties: {
    accessTier: 'Hot'
  }
}

resource blobServices 'Microsoft.Storage/storageAccounts/blobServices@2023-01-01' =
{
  parent: sa
  name: 'default'
}

resource blobContainer
'Microsoft.Storage/storageAccounts/blobServices/containers@2023-01-01' = {
  parent: blobServices
  name: blobContainerName
  properties: {
    publicAccess: 'None'
  }
}

resource fileServices 'Microsoft.Storage/storageAccounts/fileServices@2023-01-01' =
{
  parent: sa
  name: 'default'
}

resource fileShare 'Microsoft.Storage/storageAccounts/fileServices/shares@2023-01-
01' = {
  parent: fileServices
  name: fileName
}
```

```
properties: {  
  shareQuota: 100 // Quota in MB  
}  
}
```

Create a new main.bicep file, then copy the response from GitHub Copilot for Azure and paste into the new file.

When using the Azure CLI to execute the Bicep file, you can set the params declared as `storageAccountName`, `blobContainerName`, and `fileShareName`.

Alternatively, you can edit the Bicep file initializing the `storageAccountName` param to your desired value.

```
param storageAccountName string = '<your-storage-account-name>'
```

Refine the response

However, if you're new to Bicep files, you might need more information. Continue prompting in the same chat session to understand next steps.

For example, you might ask where this Bicep file should be created in your VS Code workspace.

```
Where in my workspace should I create the Azure Bicep file, and what should I call it?
```

It might respond by suggesting a folder and file name for your new Bicep file. For example, `infra/storageAccount.bicep`.

Finally, if you're unsure about how to deploy the Bicep file, you can create a prompt that asks for instructions.

```
How do I execute the Azure Bicep file?
```

You might receive a response similar to the following.

To execute the Bicep file, you can use the Azure CLI with the following steps:

1. ****Install Azure CLI and Bicep CLI****: Ensure you have the Azure CLI and Bicep CLI installed. You can install Bicep using the Azure CLI with the command:

```
az bicep install
```

2. ****Log in to Azure****: Use the Azure CLI to log in to your Azure account:

```
az login
```

3. ****Deploy the Bicep file****: Use the `az deployment group create` command to deploy the Bicep file to a resource group. Replace <resource-group-name> with your resource group name and <location> with your desired Azure region:`

```
az deployment group create --resource-group <resource-group-name> --template-file infra/storageAccount.bicep --parameters storageAccountName=<your-storage-account-name> containerName=<your-container-name> fileShareName=<your-file-share-name> location=<location>
```

This command will deploy the resources defined in your Bicep file to the specified resource group in Azure.

While following these instructions, if you get stuck or encounter a problem, continue asking GitHub Copilot questions creating prompts including the word `azure` in the chat pane.

Related content

- [Bicep overview](#)
- [What is GitHub Copilot for Azure?](#)
- [Get started with GitHub Copilot for Azure](#)

Last updated on 12/19/2025

Generate a Django and PostgreSQL app and deploy to Azure

This article guides you in how to interact with GitHub Copilot to generate a local Django web app that performs CRUD operations on a PostgreSQL database. Next, it guides you in how to interact with GitHub Copilot for Azure to deploy the web app and database to Azure App Service and Azure PostgreSQL Flexible Server (along with several supporting Azure services).

The specific application you create is a trivial contact management application that features CRUD operations with a list-detail style architecture.

ⓘ Note

Using a Large Language Model (LLM) to generate an application may yield inconsistent results. Your results depend on the LLM model, your instructions, and more. The focus of this guide is to help you understand how to get better results. However each time you go through this example, you get (potentially) dramatically different results.

Prerequisites

- An Azure account and access to an Azure subscription. For details on how to set them up, see the [pricing page for Azure accounts](#).
- A GitHub account and a GitHub Copilot subscription. For details on how to set them up, see [Creating an account on GitHub](#) and [Quickstart for GitHub Copilot](#), respectively.
- Visual Studio Code. For details on how to download and install it, see [Setting up Visual Studio Code](#).
- The GitHub Copilot extension and the GitHub Copilot Chat extension. For instructions on how to install these extensions, see [Set up GitHub Copilot in VS Code](#) and [Getting started with GitHub Copilot Chat in VS Code](#), respectively.

ⓘ Important

GitHub Copilot is a separate subscription managed by GitHub. For questions regarding GitHub Copilot subscriptions and Support, see [Getting started with a GitHub Copilot](#)

- [GitHub Copilot for Azure](#)
- [Python extension to Visual Studio Code](#) [↗]. For instructions on how to install the extension, see [Install Python and the Python extension](#) [↗].
- PostgreSQL, including pgAdmin (available from the [PostgreSQL Windows installer](#) [↗])
- Git Bash (available from the [Git installer for Windows](#) [↗])
- [PostgreSQL for Visual Studio Code \(Preview\) extension](#) [↗]. For instructions on installing and using the extension see [Quickstart: Connect and query a database with the PostgreSQL extension for Visual Studio Code preview](#).
- Azure CLI. For instructions on how to install the Azure CLI, see [How to install the Azure CLI](#).
- Azure Developer CLI (`azd`). For instructions on how to install `azd`, see [Install or update the Azure Developer CLI](#).

Prepare the chat session

1. In Visual Studio Code, use the Toggle Chat button in the title bar or select `Ctrl + Alt + i` to open the Chat Window. Use the New Chat icon to create a new chat session.
2. In the chat area, select `Agent` mode. At the time of this writing, `Claude Sonnet 4` yields the best results. Use the best model available for code generation.

Validate your configuration

Make sure your CLI tools and Visual Studio Code are updated, properly configured and operating correctly to improve your results.

1. In a new chat, enter the following prompt:

```
I want to create a new Django website that stores data in PostgreSQL. Then,
I'll want
to deploy that new website to Azure. Do I have everything installed on my
local
```

computer that you will need to perform these tasks?

GitHub Copilot will ask permission to run a series of command line checks to ensure you have the tools, as well as the most up to date versions of those tools, installed.

2. In your terminal, update the Azure CLI with the command `az --upgrade`.
3. In your terminal, install the service connector passwordless extension for Azure CLI with the command `az extension add --name serviceconnector-passwordless --upgrade`
4. In Visual Studio Code, set the default terminal to Git Bash. Go to File > Preferences > Settings, then in "Search settings", type "Default Profile: Windows" and select "Git Bash". You may need to restart Visual Studio Code for this setting to take effect.

ⓘ **Note**

Using Git Bash isn't strictly necessary, but at the time of this writing it yields the best results.

5. In Visual Studio Code, use the PostgreSQL for Visual Studio Code (Preview) extension and navigate to the `contacts` database.
6. In Visual Studio Code, use the Azure extension and ensure you're logged into your Azure account and subscription. When you open the Azure extension in the primary side bar, you should be able to view your existing subscriptions and resources.
7. Create a new folder for your new application files and open it in Visual Studio Code as your workspace.

Set up the local database

While GitHub Copilot is capable of performing virtually any application development task that developers typically perform, you will get the best results if you take some tasks in smaller steps. To improve results, create the database and set up authentication and authorization before working with GitHub Copilot.

1. Create a new chat, and use the following prompt:

On my PostgreSQL server localhost, please create a new database named 'contacts'.

Then create a new user <db-username> with password `<password>` and give that user full rights (create tables and other db objects, CRUD data) to the new contacts database.

Please do the work, and only prompt me when you are unable to do it yourself.

Replace <db-username> and <password> with your desired database username and password, respectively.

2. On Windows machines, the recommended security best practice is to store the database username and password in a local file:

```
%APPDATA%\postgresql\pgpass.conf
```

This will typically resolve the following location on your hard drive:

```
c:\Users\<username>\AppSettings\Roaming\postgresql\pgpass.conf
```

Replace <username> with your Windows username.

The file should use the following format:

```
localhost:5432:<database-name>:<database-user>:<password>
```

This assumes you're working with an instance of PostgreSQL on your local computer, and that it's hosted at the default port (5432).

Replace <database-name> with `contacts` and replace <db-username> and <password> with the credentials you used in the previous step.

For more information about the `pgpass.conf` file see [PostgreSQL's documentation](#).

3. Add the path to the `pgpass.conf` file into your PATH environment variable.
4. Test the connection to ensure that it works. Use the psql CLI to test it with the following command:

```
Bash
```

```
psql -h localhost -U <db-username> -d contacts
```

Replace `<db-username>` with the database username segment in the `pgpass.conf` file.

If the `pgpass.conf` is not set up correctly, you'll see a prompt for you to type in your password.

Generate an app using GitHub Copilot

First, you provide instructions and guidance on building and testing the application on your local computer.

1. In Visual Studio Code, use the Toggle Chat button in the title bar to open the Chat Window. Use the New Chat icon to create a new chat session.
2. In the chat area, select `Agent` mode. At the time of this writing, `Claude Sonnet 4` yields the best results. Use the best model available for code generation.
3. Use the following prompt to begin application generation:

Copilot prompt

I want you to create a simple Contact Manager application using Django and PostgreSQL.

This should be a CRUD application, so create web pages that display a list of contacts, view details, add a new contact, edit or delete a contact. Each Contact

is comprised of a contact's Name, Address, and Phone number. Since this is a Python / Django project please make sure to work inside of a virtual environment (venv).

I've already created a PostgreSQL database at `localhost` named `contacts`. There are

no tables yet. For local development in PostgreSQL, I'm using a `pgpass.conf` file

and I have tested that it works. Prefer Git Bash in the terminal. Beyond that, if there's

anything I need to do, please include instructions. But I want you to do as much as

you can on your own.

The prompt has the following features:

- **The type of application you want to create.** In this case, a contact management application.
- **The technologies to use.** In this case, Django and PostgreSQL.
- **The site architecture you want to generate.** In this case, a CRUD style application that features a page that lists all contact and allows you to drill down into a specific contact.
- **More detail about the problem domain.** In this case, you provide the fields of data you want the application to manage, including the contact's name, address, and phone number.
- **Specific instructions regarding the database.** In this case, you instruct GitHub Copilot to use a specific database that you already created, you provide the state of the database, and how to interact
- **Specific instructions about the environment.** In this case, you instruct it to use Git Bash. You also tell it that you want the work to be performed in a Python environment (venv), which is a best practice. GitHub Copilot might choose these options on its own, but stating it explicitly makes the process go smoothly.
- **Explicit expectations that you want it to do as much work on its own as possible.** Otherwise, GitHub Copilot might provide instructions for you to take.
- **Explicit expectations for instructions / context.** If it needs you to perform other actions, you set the expectation that you need it to help you by providing instructions and guidance.

Important

When GitHub Copilot uses the terminal to create a new virtual environment, Visual Studio Code detects the `venv` and displays a dialog asking whether you want to use it. Ignore that dialog. It goes away. Allow GitHub Copilot to use the terminal exclusively for this operation.

GitHub Copilot uses the built-in terminal and the Visual Studio Code environment to:

- Create a Python virtual environment
- Install libraries and other dependencies
- Generate code files
- Generate database tables
- Generate readme files for further instructions
- Create test data
- Launch a local web server

- Test the website (using Simple Browser or curl)

Due to how LLMs generate code, the commands it uses and what it produces are different each time.

Deploy to Azure with GitHub Copilot for Azure

After GitHub Copilot generates the site locally, you'll author a prompt asking GitHub Copilot to make changes to the site in preparation for deployment, and then to perform the deployment. The GitHub Copilot for Azure extension handles this request by creating Bicep files then running those files using the `azd` CLI.

Use the following prompt ... you may copy to Notepad and change any value in brackets like `<resource-group-name>` and `<region-name>` then copy and paste into GitHub Copilot chat:

Copilot prompt

Please help me deploy this Django app to Azure.

First, create and use a Resource Group named "`<resource-group-name>`" in the "`<region-name>`" region.

Second, create an Azure App Service for the Django app in the new "`<resource-group-name>`" in the "`<region-name>`" region.

Third, create a new Azure Database for PostgreSQL flexible server named "`<server-name>`" in a resource group named "`<resource-group-name>`" in my subscription "`<subscription-id>`". Use my current account ("`<account-id>`") as the Microsoft Entra administrator. For the PostgreSQL "Administrator login", use "`<azure-database-username>`" and password "`<azure-database-password>`". Use a "development" class database configuration in the "`<region-name>`" region. Create a new database named "`contact_manager_db`" and migrate all tables and data from the local version of "`contact_manager_db`". The local database uses the username "`<local-database-username>`" and password "`<local-database-password>`". Add my IP address as a firewall rule, allow public access to this resource through the internet using a public IP address, and allow public access from any Azure service within Azure to this server.

Fourth, use Service Connector (and any other services you need to make this configuration work successfully) to connect the web app to the database. You may need to modify the application code to accommodate Service Connector. Also, please ensure a secure connection between the Azure App Service web site and the Azure PostgreSQL Flexible Server.

Please choose the least expensive options.

If you are prompted for an environment, use the name "`contacts-env`". Configure my firewall to allow my IP address. Beyond that, if there's anything

I need to do, please include instructions. But I want you to do as much as you can on your own.

Before you start:

- Do you have any questions that need to be clarified?
- Please create a plan for deployment in the form of a TODO list, and then update the TODO list as you progress. Do not start until I have a chance to review your plan and tell you to proceed.

The prompt has the following features:

- **Specific services you want to use.** In this case, you tell it that you want to use Azure App Service, Azure PostgreSQL Flexible Server, Service Connector. You also give it the instruction to "do whatever else you need to do" to ensure it works.
- **Specific service options.** In this case, you indicate that you want to use the least expensive option possible for each service.
- **Hint at probable next steps.** In this case, you suggest that some code modification is necessary in order to use Service Connector.
- **Anticipate decisions ahead of time.** In this case, you provide the answer to settings it needs, such as an environment name for `azd`,
- **Explicit expectations that you want it to do as much work on its own.** Otherwise, it might provide instructions for you to take.
- **Explicit expectations for instructions / context.** Set the expectation that you need help and guidance when it asks you to take action.
- **Asks if any clarification is needed.** This often surfaces potential issues like edge cases or unclear instructions.
- **Requests a plan with a TODO list.** Gives you confidence that GitHub Copilot for Azure understands the assignment and plans to carry it out as you intended.

GitHub Copilot uses the built-in terminal and the Visual Studio Code environment to:

- Update the code files to accommodate Service Connector
- Generate Bicep files
- Run the `azd` CLI
- Test the deployment
- If necessary, debug the deployment using logs or other output

Interact with GitHub Copilot

GitHub Copilot requires your input before performing many tasks. A pause for input is your opportunity to direct GitHub Copilot to course correct in order to prevent mistakes or customize generated output to your preferences.

While it's working, you can watch and agree to most of the questions it asks you using the `Continue` button.

Important

If you get unexpected results, restart using a new chat session.

Occasionally, you're required to provide input. There are a few distinct moments when you're prompted for input:

- **User credentials** - If the current operation in the terminal requires a username or password, you will be prompted for this information.
- **Moment of decision** - Occasionally, GitHub Copilot gives you several options in a list and ask which you prefer.
- **The Command Palette** - Occasionally, GitHub Copilot uses the features of an extension and the options are displayed in the Command Palette. Once you make the proper selections, GitHub Copilot proceeds.
- **Interactive login** - The Azure CLI and `azd` CLI need you to authenticate, and initiates one of several authentication mechanisms.

Testing and asking for changes

When GitHub Copilot finishes, it's possible that it considers the site to be complete and functional. However, your testing might discover issues, or unexpected / undesirable app features.

Use prompts that describe the issue with as much detail as possible. For example, if the application isn't functioning, provide as much information as possible, **including the exact error message** and the expected result.

Interrupting the flow

Occasionally, you might notice that GitHub Copilot is either stuck in a loop attempting to perform the same tasks repeatedly or it's stuck in a process that never returns. For example,

when diagnosing problems with the website, GitHub Copilot might want to run a command like:

Bash

```
az webapp log tail
```

When GitHub Copilot is stuck, you can interrupt GitHub Copilot in one of several ways:

- `Ctrl + c`
- Use the pause button in the chat
- End the chat session and start a new chat

Important

Ending the chat session destroys all the context built up during the session, which might or might not be desirable.

To provide it context to what just happened, and nudge it towards a possible solution, you could add a prompt immediately after interrupting the GitHub Copilot such as:

Copilot prompt

```
You were just getting the logs from Azure App Service but it did not return so you got stuck. Try to interrupt after a minute once you get what you need from the logs.
```

Next steps

- [What is GitHub Copilot?](#) 
- [What is Bicep?](#)

Last updated on 12/19/2025

What is the Azure MCP Server?

The Azure MCP Server enables AI agents and clients to interact with Azure resources using natural language commands. This article explains its features, benefits, and how it helps developers build and deploy apps to Azure.

It implements the [Model Context Protocol \(MCP\)](#) and supports a wide range of tools, languages, and frameworks to help you build and deploy apps to Azure.

Key features

- **MCP support:** The Azure MCP Server implements the Model Context Protocol, making it compatible with MCP clients such as GitHub Copilot agent mode, the OpenAI Agents SDK, and Semantic Kernel.
- **Entra ID authentication:** The server uses Entra ID through the Azure Identity library, following Azure authentication best practices.
- **Service and tool integration:** The server supports Azure services and tools, including the Azure CLI, Azure Developer CLI (azd), and a broad set of Azure resources.

Supported code editors and tools

You can connect to the Azure MCP Server from popular code editors and tools, including:

- [Visual Studio Code](#)
- [Visual Studio](#)
- [Eclipse](#)
- [Cursor](#)
- [Windsurf](#)
- [IntelliJ](#)
- [Cline](#)

Supported languages and frameworks

The Azure MCP Server supports multiple languages and frameworks, such as:

- [Python](#)
- [.NET](#)

Concepts

The [Model Context Protocol \(MCP\)](#) is an open protocol designed to manage how language models interact with external tools, memory, and context in a safe, structured, and stateful way. MCP defines a client-server architecture with several components:

- **Hosts:** Apps that use MCP clients to connect to and consume data from MCP servers.
- **Clients:** Components of MCP hosts that manage connections and retrieve data from MCP servers.
- **Servers:** Programs that provide features like data resources, tools for performing actions, and prompts to guide interactions.

For example, **Visual Studio Code** is considered a host, and GitHub Copilot agent mode in **Visual Studio Code** acts as an MCP client that connects to MCP servers. You can also build custom intelligent apps that host their own MCP client to connect to MCP servers.

The Azure MCP Server implements a set of [tools](#) per the Model Context Protocol. AI agents and other types of clients use these tools to interact with Azure resources.

How-to guides

You can find step-by-step instructions for common tasks, including:

- [Connect GitHub Copilot coding agent to Azure MCP Server](#)
- [Deploy a self-hosted Azure MCP Server \(Microsoft Foundry\)](#)
- [Deploy a self-hosted Azure MCP Server \(Copilot Studio\)](#)

Tools and best practices

The Azure MCP Server offers a wide range of tools for Azure development. For best practices and tool reference, see [Tools overview](#).

Scenarios for using the Azure MCP Server

The most common scenario is connecting to the Azure MCP Server from an existing client, such as GitHub Copilot agent mode in **Visual Studio Code** or a custom intelligent app. The client can use all available [tools](#) to access and interact with Azure resources using natural language. For example, you can use GitHub Copilot agent mode with the Azure MCP Server to list Azure storage accounts or run KQL queries on Azure databases. To learn how to connect to Azure MCP Server from an existing client, see [Get started using the Azure MCP Server](#).

In advanced scenarios, you might create your own MCP servers to offer custom tools, resources, and prompts for specific tasks involving Azure resources. If you're building an MCP server that needs to connect with Azure, you can use the Azure MCP Server tools from your MCP server.

The MCP server uses your Azure user credentials or managed identity to ensure authorized access. Access is secured through Azure Role-Based Access Control (RBAC), providing fine-grained permissions for approved users. The local MCP server is intended strictly for developer use within your organization. Don't use these tools for external applications or scenarios outside the approved development environment.

Related content

- [Get started using the Azure MCP Server](#)
- [Azure MCP Server tools](#)
- [Model Context Protocol documentation](#) 
- [Azure MCP Server repository](#) 

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/19/2026

Get started with the Azure MCP Server

The [Azure MCP Server](#) enables AI-powered development tools to interact with Azure cloud services through the Model Context Protocol (MCP). It provides a unified way to manage Azure resources, deploy applications, and query cloud services directly from your development environment.

Connect to Azure MCP Server using various tools, languages, and frameworks. Use it to manage Azure resources through natural language conversations, build automation scripts, or integrate Azure operations into your applications.

Explore and contribute to the [Azure MCP Server on GitHub](#).

Install Azure MCP Server using a package manager

You can optionally install Azure MCP Server directly using one of the supported package managers. Package manager installation offers several advantages, including centralized dependency management, CI/CD integration, support for headless environments, version control, and project portability.

ⓘ Note

You don't need to install the Azure MCP Server to use it. The approaches explained in the [Connect to Azure MCP Server](#) section either include Azure MCP Server as part of their installation, or run Azure MCP Server directly via package manager commands.

For more information, see [Install with a package manager](#).

- NuGet: [Azure.Mcp](#)
- NPM: [@azure/mcp](#)
- PyPI: [msmcp-azure](#)

Connect to Azure MCP Server

You can connect to Azure MCP Server using the following tools and programming frameworks.

Use a code editor

Azure MCP Server works with AI-powered code editors and tools that support the Model Context Protocol. Learn how to get started:

- [Cline](#)
- [Cursor](#)
- [IntelliJ](#)
- [Visual Studio](#)
- [Visual Studio Code](#)
- [Windsurf](#)

Use GitHub Copilot tools

Connect to Azure MCP Server using other tools and services, such as GitHub Copilot coding agent and Docker. Learn how to get started:

- [GitHub Copilot CLI](#)
- [GitHub Copilot SDK](#)
- [GitHub Copilot coding agent](#)

Use a Docker container

For easy deployment and isolation, you can [run the Azure MCP server as a Docker container](#) [↗].

Use programming languages & frameworks

Connect to Azure MCP Server using programming languages and frameworks. This documentation currently provides guidance and examples for Python and .NET.

Python

[Get started with Azure MCP Server and Python](#) to enhance your apps and workflows.

- Use Python MCP client libraries to connect directly to Azure MCP Server.
- Build automation scripts that manage Azure resources.
- Integrate into web frameworks like Django, Flask, or FastAPI.
- Incorporate Azure operations into data science workflows with Jupyter notebooks.

.NET

[Get started with Azure MCP Server and .NET](#) to enhance your apps and workflows.

- Create console applications and command-line tools for Azure management.
- Build ASP.NET Core web applications with integrated Azure capabilities.
- Develop Azure Functions that leverage Azure MCP Server for resource management.
- Create Windows desktop applications with Azure integration.

The Azure MCP Server provides flexibility to work with Azure in the way that best fits your development style and requirements, whether through interactive AI-powered editors or programmatic integration in your applications.

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 03/27/2026

Get started with the Azure MCP Server using Visual Studio Code

The Azure MCP Server uses the Model Context Protocol (MCP) to standardize integrations between AI apps and external tools and data sources, allowing for AI systems to perform operations that are context-aware of your Azure resources.

In this article, you learn how to complete the following tasks:

- Install and authenticate to the Azure MCP Server
- Connect to Azure MCP Server using GitHub Copilot agent mode in Visual Studio Code
- Run prompts to test Azure MCP Server operations and interact with Azure resources

Prerequisites

- An [Azure account](#) with an active subscription
- [Visual Studio Code](#)
- [GitHub Copilot](#) Visual Studio Code extension

Install the Azure MCP Server

Select one of the following options to install the Azure MCP Server in Visual Studio Code:

Extension install

1. To install the [Azure MCP Server Extension for Visual Studio Code](#), select the following link:

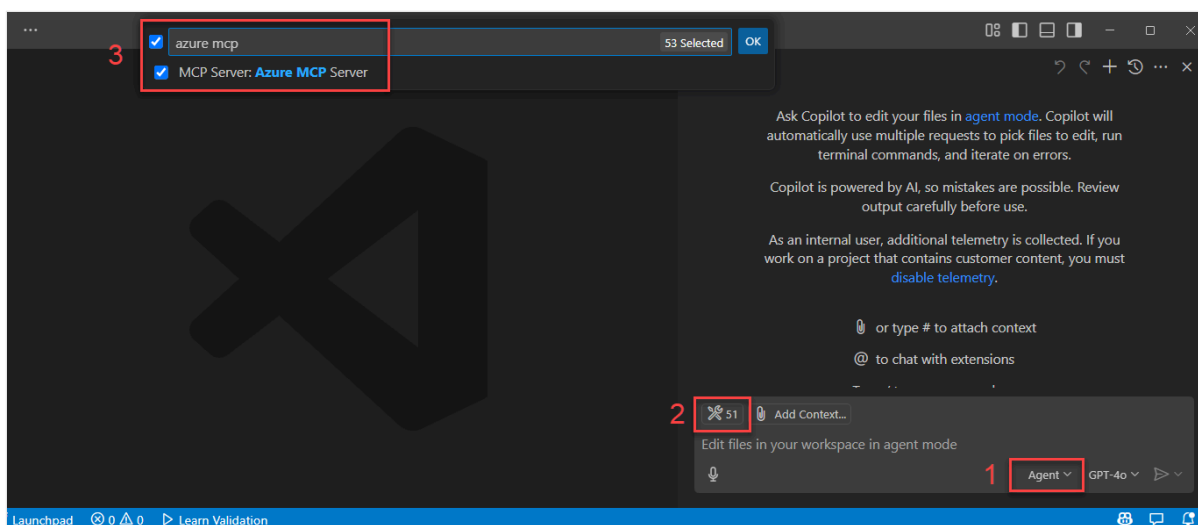
VS Code [Install Azure MCP Server](#)

ⓘ Note

Installing the Azure MCP Server as an extension ensures you receive the latest preview version and automatic updates.

2. After you install the extension, open GitHub Copilot and select Agent Mode. To learn more about Agent Mode, visit the [Visual Studio Code Documentation](#).

3. Refresh the tools list to see Azure MCP Server as an available option:



Tip

For an optimal Azure MCP Server experience, [install GitHub Copilot for Azure](#), which includes and uses the Azure MCP Server extension. GitHub Copilot for Azure streamlines your development workflow and enhances your productivity on the Azure platform.

Authenticate to Azure

Azure MCP Server provides a seamless authentication experience using Azure accounts and Microsoft Entra ID. To use Azure MCP Server, you must first authenticate to Azure using local development tools such as the Azure CLI, Azure Developer CLI, Visual Studio, or Visual Studio Code. Azure MCP Server automatically discovers your credentials from these tools and uses them to authenticate to Azure services.

1. For example, to sign in using the Azure CLI:

Azure CLI

```
az login
```

2. Verify your authentication status by running the following command to see which account and subscription you're currently signed in with:

Azure CLI

```
az account show
```

3. Ensure your user account has the appropriate role assignments for the Azure services you want to interact with. The Azure resources you intend to access with Azure MCP Server must already exist within your Azure subscription. For example, common role assignments include:

- **Blob Storage Data Contributor** - Read and write blob data in storage accounts.
- **Storage Account Contributor** - Manage storage account configurations.
- **Contributor** - General resource management across your subscription.
- **Reader** - Read-only access to Azure resources.

For more information about role assignments and local development authentication, see [Authenticate .NET apps to Azure services during local development](#).

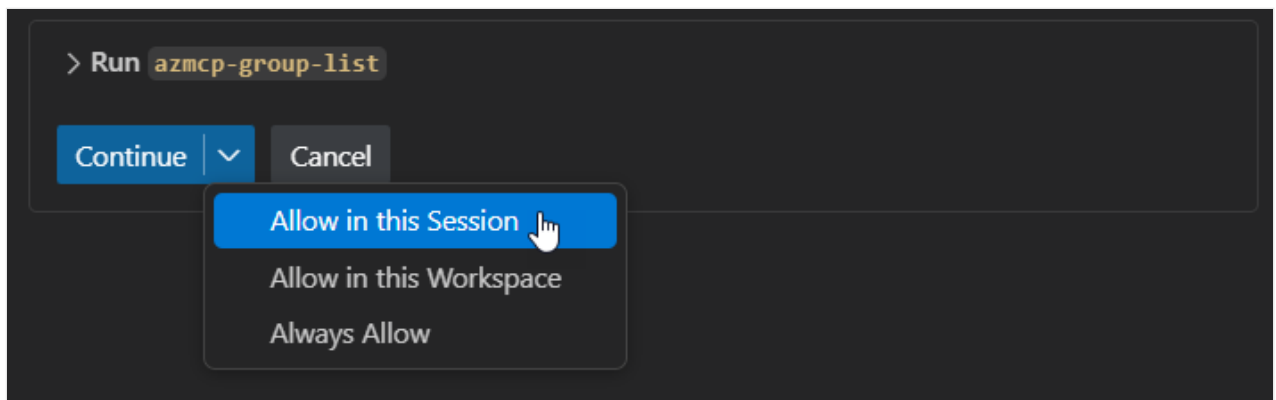
Use prompts to test the Azure MCP Server

1. Open GitHub Copilot and select Agent Mode.
2. Enter a prompt that causes the agent to use Azure MCP Server tools, such as *List my Azure resource groups*.
3. In order to authenticate Azure MCP Server, Copilot prompts you to sign-in to Azure using the browser.

ⓘ Note

Copilot won't prompt you to sign-in to Azure if you're already authenticated via other local tooling such as the Azure CLI.

4. Copilot requests permission to run the necessary Azure MCP Server operation for your prompt. Select **Continue** or use the arrow to select a more specific behavior:
 - **Current session** always runs the operation in the current GitHub Copilot Agent Mode session.
 - **Current workspace** always runs the command for current Visual Studio Code workspace.
 - **Always allow** sets the operation to always run for any GitHub Copilot Agent Mode session or any Visual Studio Code workspace.



The output for the previous prompt should resemble the following text:

Output

The following resource groups are available for your subscription:

1. ****DefaultResourceGroup-EUS**** (Location: `eastus`)
2. ****rg-testing**** (Location: `centralus`)
3. ****rg-azd**** (Location: `eastus2`)
4. ****msdocs-sample**** (Location: `southcentralus`)
14. ****ai-testing**** (Location: `eastus2`)

Let me know if you need further details or actions related to any of these resource groups!

5. Explore and test the Azure MCP operations using other relevant prompts, such as:

List all of the storage accounts in my subscription
Get the available tables in my storage accounts

Next steps

[Learn more about Azure MCP Server tools](#)

Last updated on 02/27/2026

Get started with the Azure MCP Server using Visual Studio

The Azure MCP Server uses the Model Context Protocol (MCP) to standardize integrations between AI apps and external tools and data sources, allowing for AI systems to perform operations that are context-aware of your Azure resources.

In this article, you learn how to complete the following tasks:

- Install and authenticate to the Azure MCP Server.
- Connect to Azure MCP Server using GitHub Copilot agent mode in Visual Studio.
- Run prompts to test Azure MCP Server operations and interact with Azure resources.

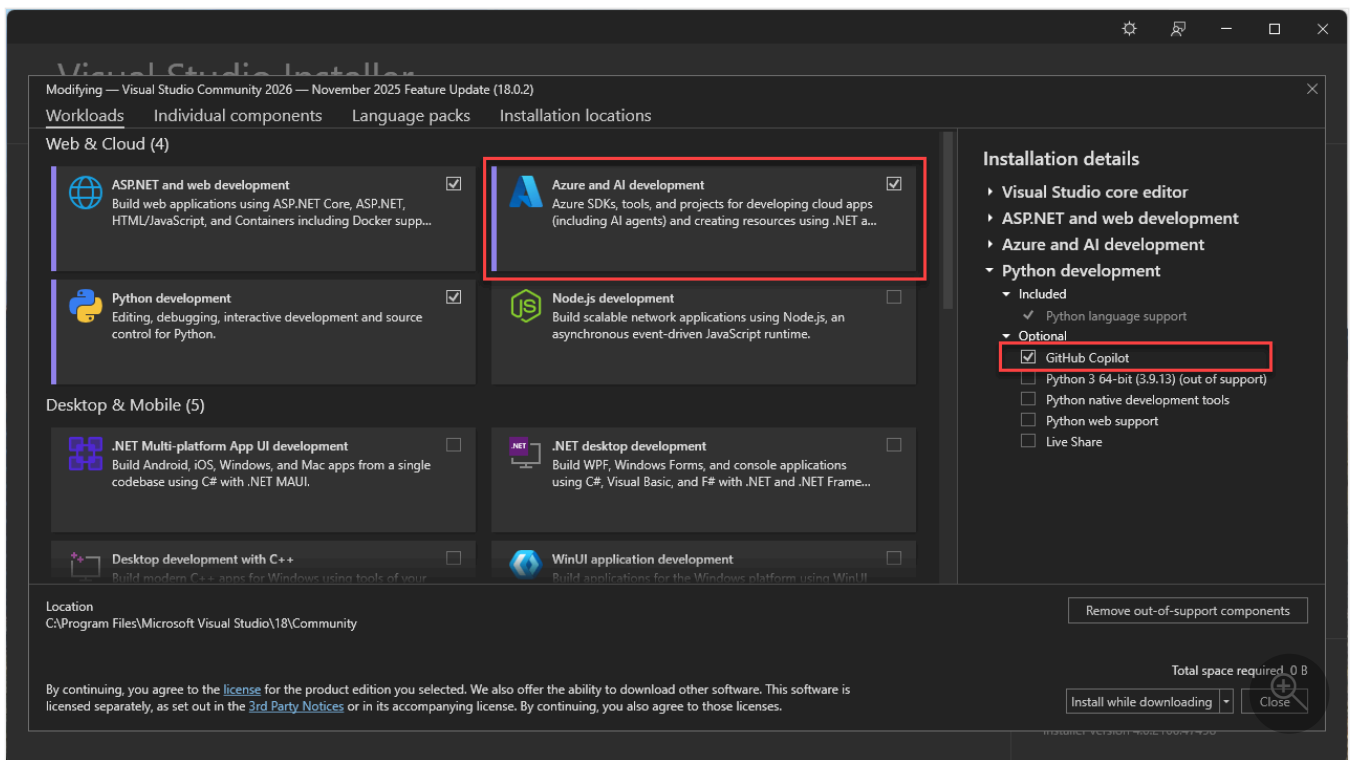
Prerequisites

- An [Azure account](#) with an active subscription
- [Visual Studio](#)
- [.NET 10](#) OR [Node.js](#) LTS installed

Install the Azure MCP Server

Visual Studio 2026 includes Azure MCP Server tools with the Azure and AI development workload. To make them available:

1. Open the Visual Studio Installer tool.
2. Select **Modify** to view available workloads.
3. On the **Workloads** tab, select **Azure and AI development**.
4. On the **Installation details** panel, verify that **GitHub Copilot** is selected, which includes the following components:
 - GitHub Copilot
 - GitHub Copilot for Azure
 - Azure MCP Server
5. Select **Install while downloading** to complete the installation process.



Authenticate to Azure

Azure MCP Server provides a seamless authentication experience using Azure accounts and Microsoft Entra ID. To use Azure MCP Server, you must first authenticate to Azure using local development tools such as the Azure CLI, Azure Developer CLI, Visual Studio, or Visual Studio Code. Azure MCP Server automatically discovers your credentials from these tools and uses them to authenticate to Azure services.

1. For example, to sign in using the Azure CLI:

Azure CLI

```
az login
```

2. Verify your authentication status by running the following command to see which account and subscription you're currently signed in with:

Azure CLI

```
az account show
```

3. Ensure your user account has the appropriate role assignments for the Azure services you want to interact with. The Azure resources you intend to access with Azure MCP Server must already exist within your Azure subscription. For example, common role assignments include:

- **Blob Storage Data Contributor** - Read and write blob data in storage accounts.
- **Storage Account Contributor** - Manage storage account configurations.
- **Contributor** - General resource management across your subscription.
- **Reader** - Read-only access to Azure resources.

For more information about role assignments and local development authentication, see [Authenticate .NET apps to Azure services during local development](#).

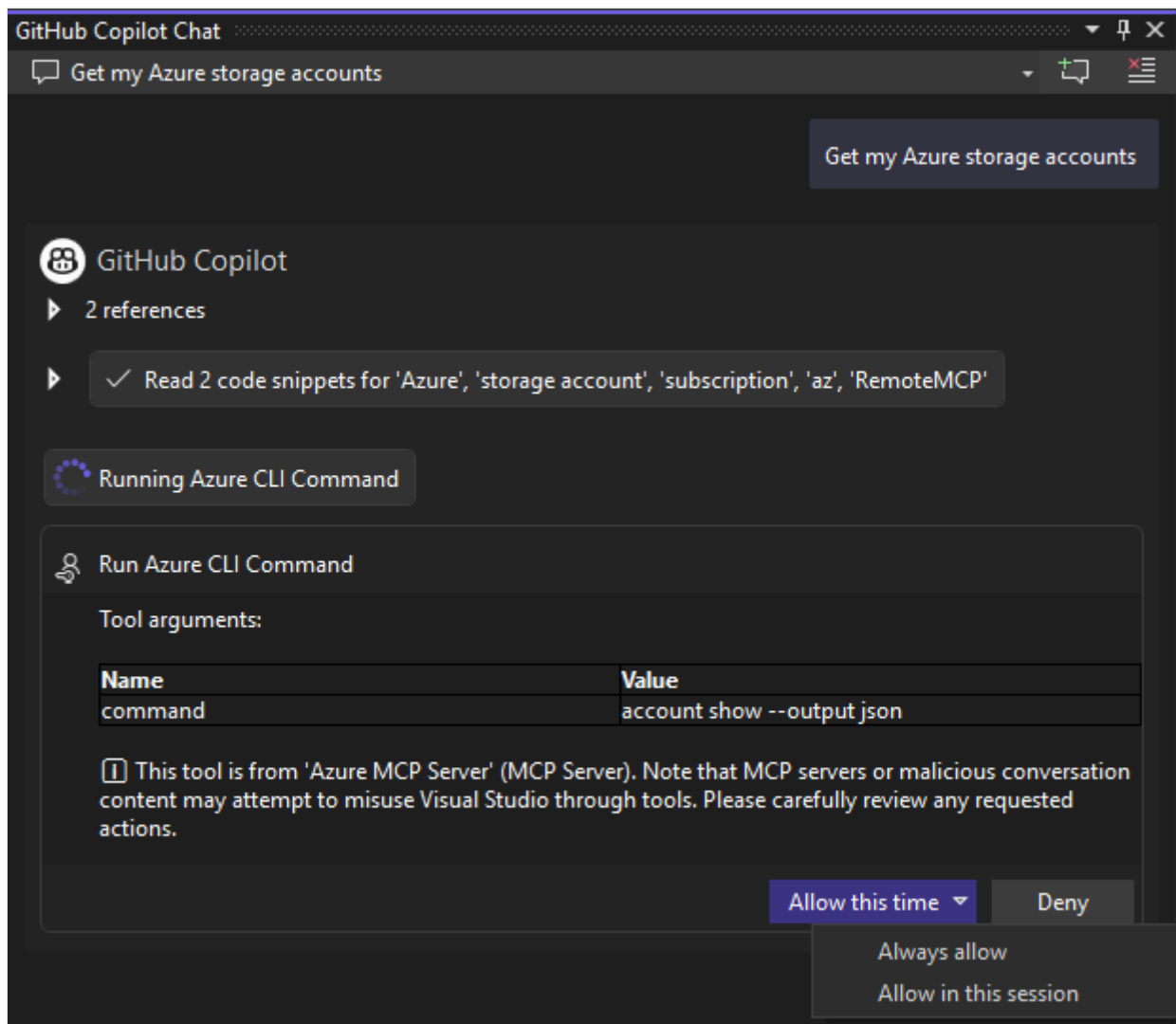
Use prompts to test the Azure MCP Server

1. Open GitHub Copilot and select Agent Mode.
2. Enter a prompt that causes the agent to use Azure MCP Server tools, such as *List my Azure resource groups*.
3. In order to authenticate Azure MCP Server, Copilot prompts you to sign-in to Azure using the browser.

ⓘ Note

Copilot doesn't prompt you to sign-in to Azure if you're already authenticated via other local tooling such as the Azure CLI.

4. Copilot requests permission to run the necessary Azure MCP Server operation for your prompt. Select **Allow this time** or use the arrow to select a more specific behavior:
 - **Always allow** sets the operation to always run for any GitHub Copilot Agent Mode session or any Visual Studio Code workspace.
 - **Allow in this session** always runs the operation in the current GitHub Copilot Agent Mode session.



The output for the previous prompt should resemble the following text:

Output

The following resource groups are available for your subscription:

1. **DefaultResourceGroup-EUS** (Location: `eastus`)
2. **rg-testing** (Location: `centralus`)
3. **rg-azd** (Location: `eastus2`)
4. **msdocs-sample** (Location: `southcentralus`)
5. **ai-testing** (Location: `eastus2`)

Let me know if you need further details or actions related to any of these resource groups!

5. Explore and test the Azure MCP operations using other relevant prompts, such as:

List all of the storage accounts in my subscription
Get the available tables in my storage accounts

Next steps

[Learn more about Azure MCP Server tools](#)

Last updated on 02/27/2026

Get started with the Azure MCP Server in Eclipse

The Azure MCP Server uses the Model Context Protocol (MCP) to standardize integrations between AI apps and external tools and data sources, allowing for AI systems to perform operations that are context-aware of your Azure resources.

This article covers:

- Install and authenticate with Azure MCP Server
- Connect to Azure MCP Server in Eclipse's AI development environment
- Run prompts to test operations and interact with Azure resources

Prerequisites

- [Azure account](#) [↗] with an active subscription
- [Eclipse](#) [↗] - the AI-powered code editor
- GitHub Copilot plugin

Authenticate to Azure

Azure MCP Server provides a seamless authentication experience using Azure accounts and Microsoft Entra ID. To use Azure MCP Server, you must first authenticate to Azure using local development tools such as the Azure CLI, Azure Developer CLI, Visual Studio, or Visual Studio Code. Azure MCP Server automatically discovers your credentials from these tools and uses them to authenticate to Azure services.

1. For example, to sign in using the Azure CLI:

```
Azure CLI
```

```
az login
```

2. Verify your authentication status by running the following command to see which account and subscription you're currently signed in with:

```
Azure CLI
```

```
az account show
```

3. Ensure your user account has the appropriate role assignments for the Azure services you want to interact with. The Azure resources you intend to access with Azure MCP Server must already exist within your Azure subscription. For example, common role assignments include:

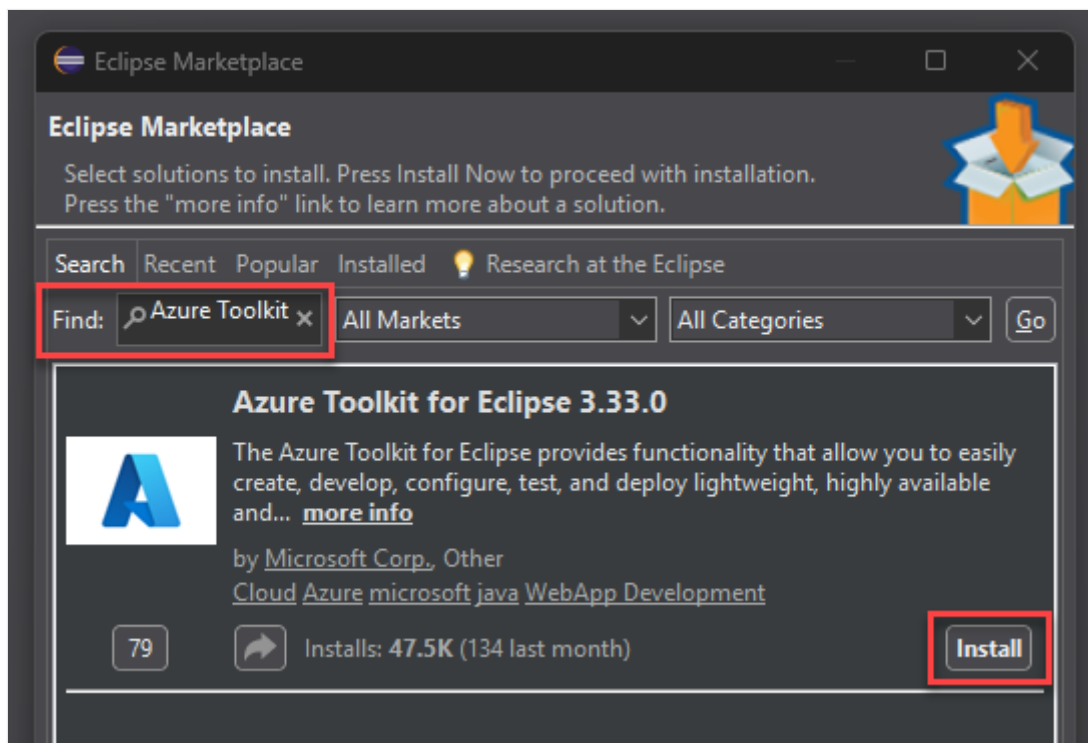
- **Blob Storage Data Contributor** - Read and write blob data in storage accounts.
- **Storage Account Contributor** - Manage storage account configurations.
- **Contributor** - General resource management across your subscription.
- **Reader** - Read-only access to Azure resources.

For more information about role assignments and local development authentication, see [Authenticate .NET apps to Azure services during local development](#).

Install Azure MCP Server

Install and configure Azure MCP Server in Eclipse:

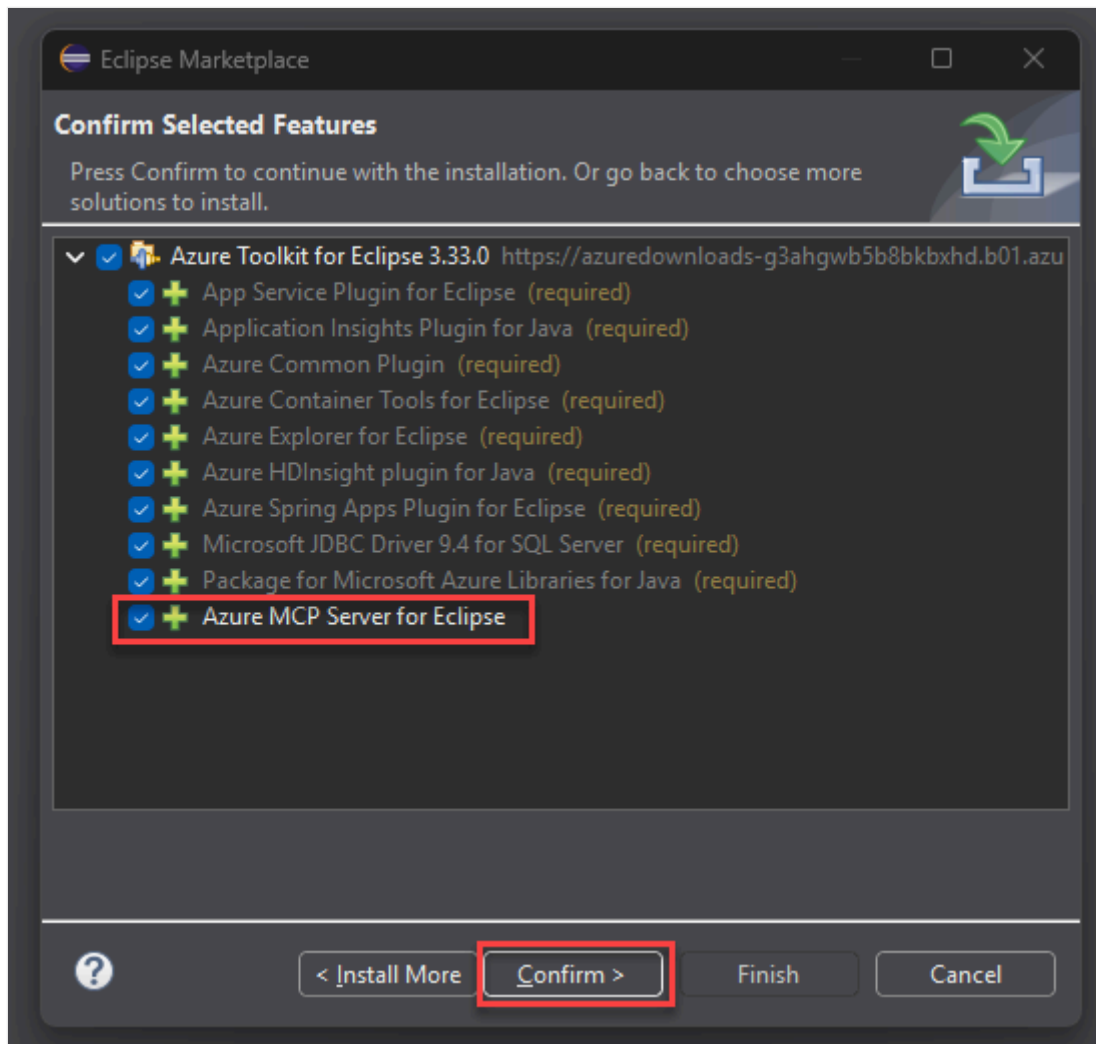
1. Go to **Help > Eclipse Marketplace**.
2. Search for **Azure Toolkit** in the **Eclipse Marketplace** window.
3. In the search results, select **Install** for Azure Toolkit.



ⓘ Note

As part of the installation process, make sure both the **GitHub Copilot** and **GitHub Copilot – Nightly** plugins are updated to their latest versions.

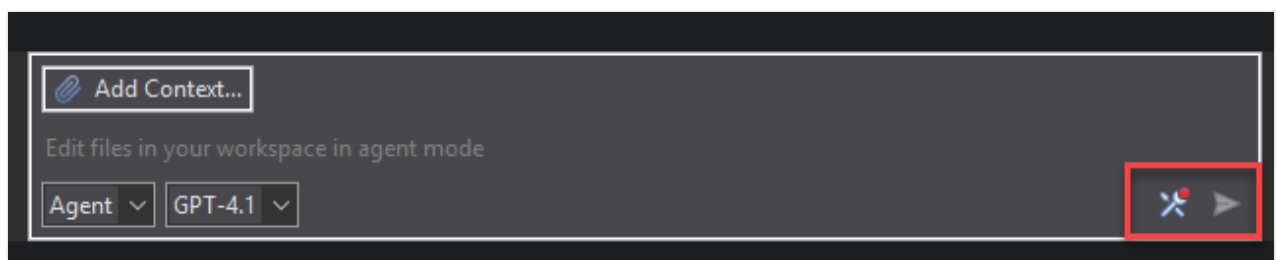
4. On the **Confirm Selected Features** window, check that **Azure MCP Server for Eclipse** is selected.
5. Select **Confirm**, and wait for the plugin to install. Eclipse restarts when installation finishes.



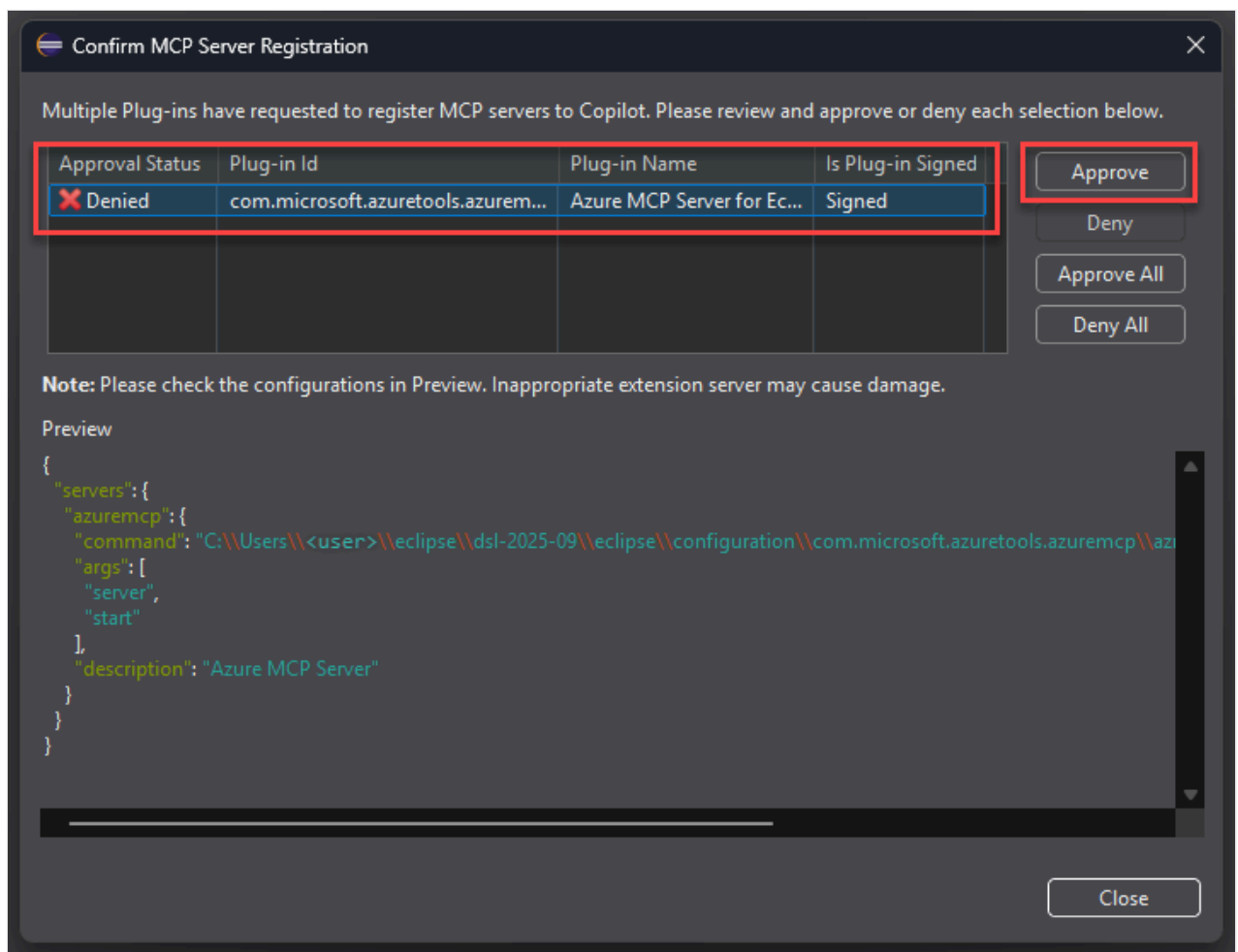
Verify and test the Azure MCP Server

After you install Azure Toolkit, approve Azure MCP Server registration in Copilot.

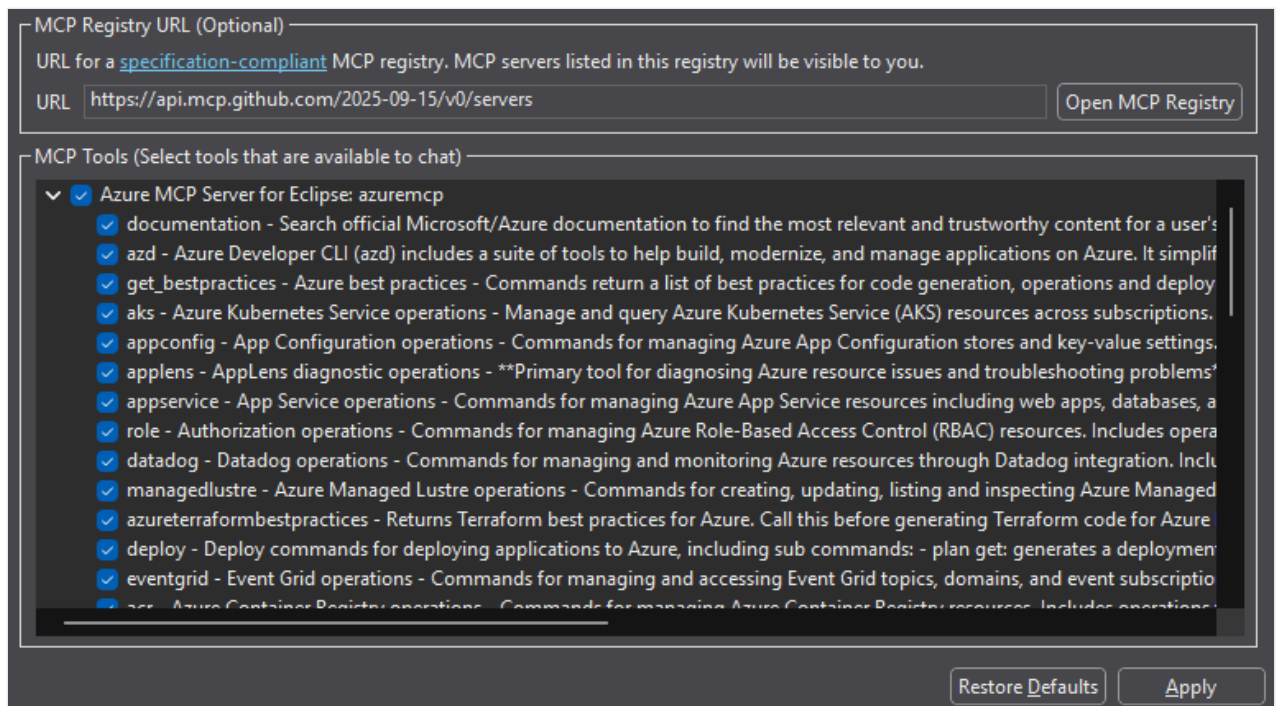
1. In the Copilot chat pane, select the **Tools** icon and select **New MCP server found - approval needed**.



2. In the **Confirm MCP Server Registration** window, select the Azure MCP Server row and select **Approve**.



3. In the Copilot chat pane, select the **Tools** icon again to open the preferences window.
4. Check that Azure MCP Server for Eclipse shows in the **MCP Tools** section.



Use prompts to test the Azure MCP Server

1. In the Copilot chat panel, enter a prompt that uses Azure MCP Server capabilities, such as the following.

text

List my Azure resource groups

! Note

You can also ask Copilot to use Azure MCP Server tools directly with language like:
Use the Azure MCP Server to list my Azure resource groups.

2. When Eclipse prompts you to run the tool `azuremcp/group_list` to retrieve the resource groups, select **Continue**.

The output resembles the following text.

text

The following resource groups are available for your subscription:

1. ****DefaultResourceGroup-EUS**** (Location: `eastus`)
2. ****rg-testing**** (Location: `centralus`)
3. ****rg-azd**** (Location: `eastus2`)
4. ****msdocs-sample**** (Location: `southcentralus`)

5. **ai-testing** (Location: `eastus2`)

Let me know if you need further details or actions related to any of these resource groups!

3. Explore more Azure MCP operations with prompts like the following.

text

List all storage accounts in my subscription
List available tables in my storage accounts
Show the configuration of my App Service instances

Next steps

[Azure MCP Server tools](#)

Last updated on 02/27/2026

Get started with the Azure MCP Server in Cursor

The Azure MCP Server uses the Model Context Protocol (MCP) to standardize integrations between AI apps and external tools and data sources, allowing for AI systems to perform operations that are context-aware of your Azure resources.

In this article, you learn how to complete the following tasks:

- Install and authenticate to the Azure MCP Server
- Connect to Azure MCP Server using Cursor's AI-powered development environment
- Run prompts to test Azure MCP Server operations and interact with Azure resources

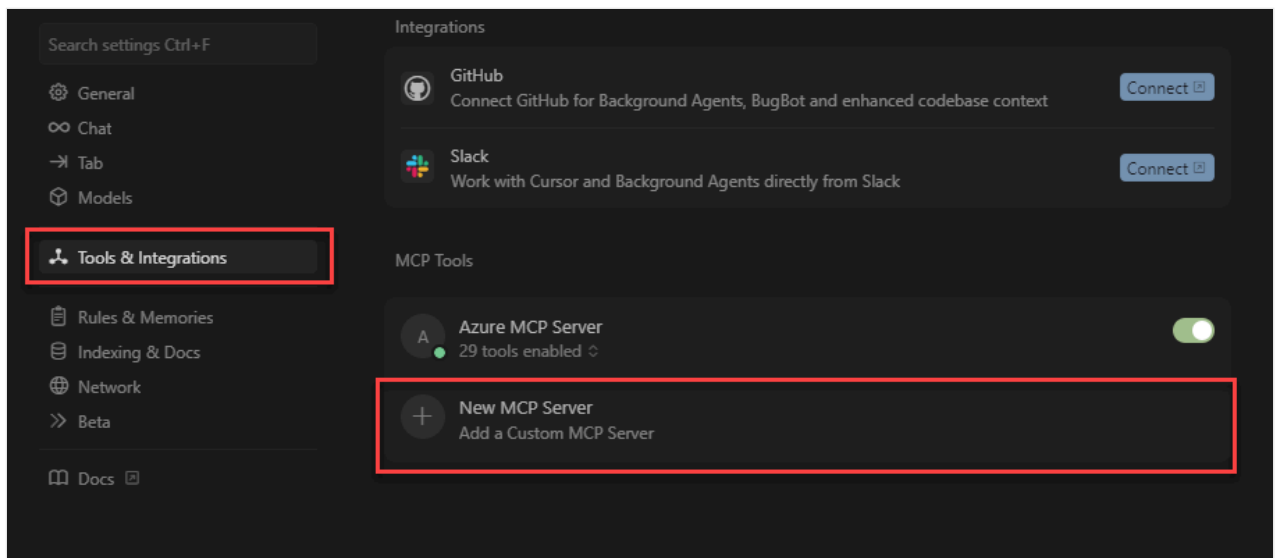
Prerequisites

- An [Azure account](#) with an active subscription
- [Cursor](#) - The AI-powered code editor
- [Node.js](#) LTS installed

Install the Azure MCP Server

To install and configure Azure MCP Server in Cursor:

1. Navigate to **File > Preferences > Cursor Settings**
2. Select **Tools & Integrations** from the left navigation.
3. In the **MCP Tools** section of the page, select **New MCP Server** to open the `mcp.json` file for editing.



4. Add the following configuration to the `mcpServers` JSON object:

```
JSON

"Azure MCP Server": {
  "command": "npx",
  "args": [
    "-y",
    "@azure/mcp@latest",
    "server",
    "start"
  ]
}
```

Authenticate to Azure

Azure MCP Server provides a seamless authentication experience using Azure accounts and Microsoft Entra ID. To use Azure MCP Server, you must first authenticate to Azure using local development tools such as the Azure CLI, Azure Developer CLI, Visual Studio, or Visual Studio Code. Azure MCP Server automatically discovers your credentials from these tools and uses them to authenticate to Azure services.

1. For example, to sign in using the Azure CLI:

```
Azure CLI

az login
```

2. Verify your authentication status by running the following command to see which account and subscription you're currently signed in with:

Azure CLI

```
az account show
```

3. Ensure your user account has the appropriate role assignments for the Azure services you want to interact with. The Azure resources you intend to access with Azure MCP Server must already exist within your Azure subscription. For example, common role assignments include:

- **Blob Storage Data Contributor** - Read and write blob data in storage accounts.
- **Storage Account Contributor** - Manage storage account configurations.
- **Contributor** - General resource management across your subscription.
- **Reader** - Read-only access to Azure resources.

For more information about role assignments and local development authentication, see [Authenticate .NET apps to Azure services during local development](#).

Use prompts to test the Azure MCP Server

1. Open Cursor's AI chat interface by pressing `Ctrl+L` or clicking the chat icon in the sidebar.
2. Enter a prompt that utilizes Azure MCP Server capabilities, such as:

text

```
List my Azure storage accounts
```

3. If you're prompted to authenticate to Azure, run the suggested auth tool to sign-in through the browser.

ⓘ Note

Cursor doesn't prompt you to sign in to Azure if you're already authenticated via other local tooling such as the Azure CLI.

4. Cursor prompts you to run a tool to retrieve the storage accounts, such as `storage account list`. Select **Run tool** to continue.

The output should resemble the following text:

Output

The following resource groups are available for your subscription:

1. ****DefaultResourceGroup-EUS**** (Location: `eastus`)
2. ****rg-testing**** (Location: `centralus`)
3. ****rg-azd**** (Location: `eastus2`)
4. ****msdocs-sample**** (Location: `southcentralus`)
5. ****ai-testing**** (Location: `eastus2`)

Let me know if you need further details or actions related to any of these resource groups!

5. Explore more Azure MCP operations using other relevant prompts, such as:

text

List all of the storage accounts in my subscription
Get the available tables in my storage accounts
Show me the configuration of my App Service instances

Next steps

[Learn more about Azure MCP Server tools](#)

Last updated on 02/27/2026

Get started with the Azure MCP Server in Windsurf

The Azure MCP Server uses the Model Context Protocol (MCP) to standardize integrations between AI apps and external tools and data sources, allowing for AI systems to perform operations that are context-aware of your Azure resources.

In this article, you learn how to complete the following tasks:

- Install and authenticate to the Azure MCP Server
- Connect to Azure MCP Server using Windsurf's AI-powered development environment
- Run prompts to test Azure MCP Server operations and interact with Azure resources

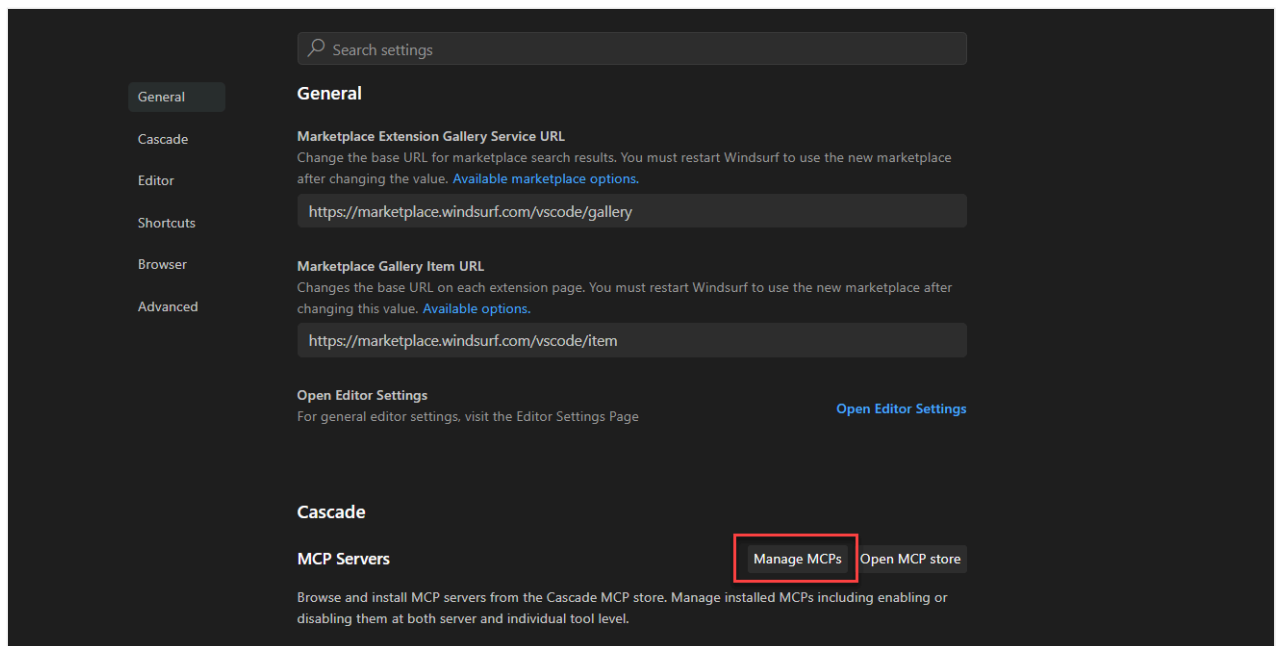
Prerequisites

- An [Azure account](#) with an active subscription
- [Windsurf](#) installed
- [Node.js](#) LTS installed

Install the Azure MCP Server

To install and configure Azure MCP Server in Windsurf:

1. Navigate to **File > Preferences > Windsurf Settings**.
2. On the **Windsurf Settings** page, select **Manage MCPs**.



3. On the **Manage MCP Servers** settings page, select **View raw config** at the top to open the `mcp_config.json` file for editing.
4. The `mcp_config.json` file lets you manually install MCP Servers by adding a JSON configuration object. Update the `mcp_config.json` file to match the following:

```
JSON

{
  "mcpServers": {
    "Azure MCP Server": {
      "command": "npx",
      "args": [
        "-y",
        "@azure/mcp@latest",
        "server",
        "start"
      ]
    }
  }
}
```

Authenticate to Azure

Azure MCP Server provides a seamless authentication experience using Azure accounts and Microsoft Entra ID. To use Azure MCP Server, you must first authenticate to Azure using local development tools such as the Azure CLI, Azure Developer CLI, Visual Studio, or Visual Studio Code. Azure MCP Server automatically discovers your credentials from these tools and uses them to authenticate to Azure services.

1. For example, to sign in using the Azure CLI:

Azure CLI

```
az login
```

2. Verify your authentication status by running the following command to see which account and subscription you're currently signed in with:

Azure CLI

```
az account show
```

3. Ensure your user account has the appropriate role assignments for the Azure services you want to interact with. The Azure resources you intend to access with Azure MCP Server must already exist within your Azure subscription. For example, common role assignments include:

- **Blob Storage Data Contributor** - Read and write blob data in storage accounts.
- **Storage Account Contributor** - Manage storage account configurations.
- **Contributor** - General resource management across your subscription.
- **Reader** - Read-only access to Azure resources.

For more information about role assignments and local development authentication, see [Authenticate .NET apps to Azure services during local development](#).

Use prompts to test the Azure MCP Server

1. Open Windsurf's AI chat interface by pressing `Ctrl+L` or clicking the chat icon in the sidebar.
2. Enter a prompt that utilizes Azure MCP Server capabilities, such as:

text

```
List my Azure storage accounts
```

3. If you're prompted to authenticate to Azure, run the suggested auth tool to sign-in through the browser.

ⓘ Note

Windsurf doesn't prompt you to sign in to Azure if you're already authenticated via other local tooling such as the Azure CLI.

4. Windsurf prompts you to run a tool to retrieve the storage accounts, such as `storage account list`. Select **Run tool** to continue.

The output should resemble the following text:

Output

The following resource groups are available for your subscription:

1. ****DefaultResourceGroup-EUS**** (Location: `eastus`)
2. ****rg-testing**** (Location: `centralus`)
3. ****rg-azd**** (Location: `eastus2`)
4. ****msdocs-sample**** (Location: `southcentralus`)
5. ****ai-testing**** (Location: `eastus2`)

Let me know if you need further details or actions related to any of these resource groups!

5. Explore more Azure MCP operations using other relevant prompts, such as:

text

List all of the storage accounts in my subscription
Get the available tables in my storage accounts
Show me the configuration of my App Service instances

Next steps

[Learn more about Azure MCP Server tools](#)

Last updated on 02/27/2026

Get started with the Azure MCP Server in IntelliJ

The Azure MCP Server uses the Model Context Protocol (MCP) to standardize integrations between AI apps and external tools and data sources, allowing for AI systems to perform operations that are context-aware of your Azure resources.

In this article, you learn how to complete the following tasks:

- Install and authenticate to the Azure MCP Server
- Connect to Azure MCP Server using IntelliJ's AI-powered tools
- Run prompts to test Azure MCP Server operations and interact with Azure resources

Prerequisites

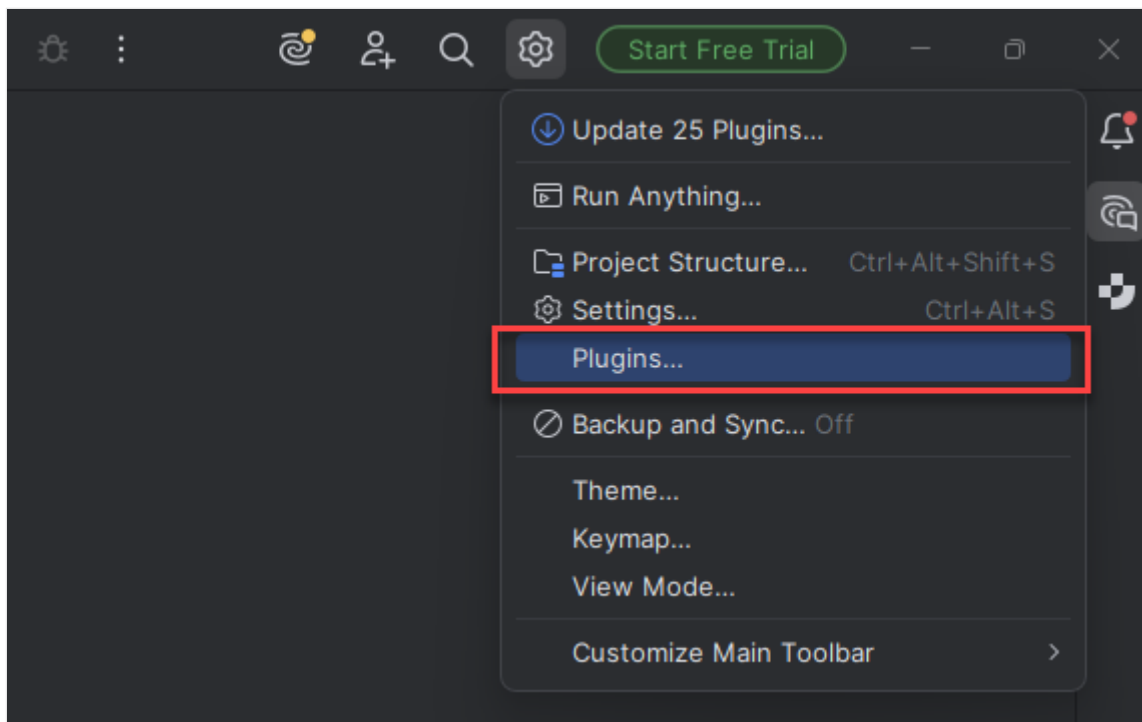
- An [Azure account](#) with an active subscription
- [IntelliJ](#) installed
- [Node.js](#) LTS installed
- [GitHub Copilot for IntelliJ](#) extension installed

Install the Azure MCP Server

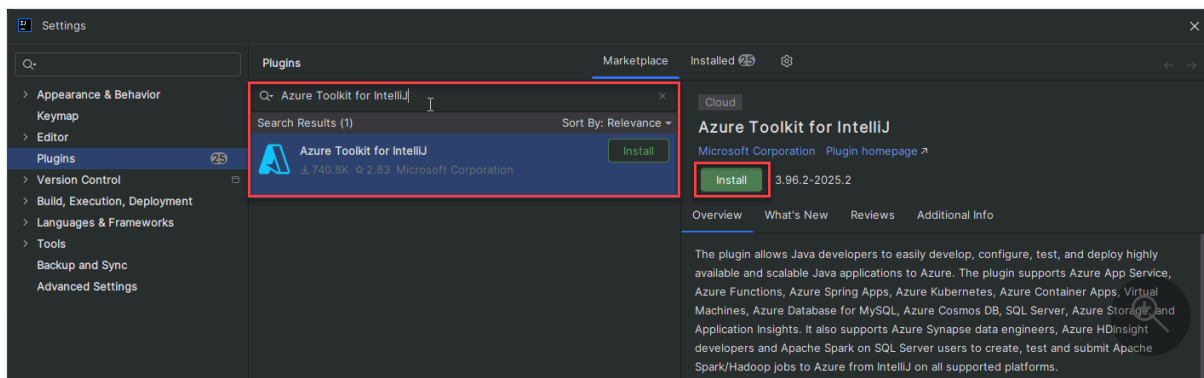
To install and configure Azure MCP Server in IntelliJ, choose one of the following options:

Azure Toolkit for IntelliJ

1. Select the **IDE and Project Settings** gear icon, and then choose **Plugins...**



2. In the **Plugins** panel of the **Settings** dialog, search for *Azure ToolKit for IntelliJ*.
3. Select the matching result, and then select **Install**.



! Note

Azure MCP Server is only included with the Azure Toolkit for IntelliJ installation if you already have the [GitHub Copilot for IntelliJ](#) plugin installed.

Authenticate to Azure

Azure MCP Server provides a seamless authentication experience using Azure accounts and Microsoft Entra ID. To use Azure MCP Server, you must first authenticate to Azure using local development tools such as the Azure CLI, Azure Developer CLI, Visual Studio, or Visual Studio

Code. Azure MCP Server automatically discovers your credentials from these tools and uses them to authenticate to Azure services.

1. For example, to sign in using the Azure CLI:

Azure CLI
<code>az login</code>

2. Verify your authentication status by running the following command to see which account and subscription you're currently signed in with:

Azure CLI
<code>az account show</code>

3. Ensure your user account has the appropriate role assignments for the Azure services you want to interact with. The Azure resources you intend to access with Azure MCP Server must already exist within your Azure subscription. For example, common role assignments include:

- **Blob Storage Data Contributor** - Read and write blob data in storage accounts.
- **Storage Account Contributor** - Manage storage account configurations.
- **Contributor** - General resource management across your subscription.
- **Reader** - Read-only access to Azure resources.

For more information about role assignments and local development authentication, see [Authenticate .NET apps to Azure services during local development](#).

Use prompts to test the Azure MCP Server

1. Select the **AI Chat** button on the right toolbar to open IntelliJ's AI assistant interface.
2. Enter a prompt that utilizes Azure MCP Server capabilities, such as:

text
List my Azure storage accounts

3. IntelliJ prompts you to run a tool to retrieve the storage accounts, such as `storage account list`. Select **Run tool** to continue.

The output should resemble the following text:

Output

The following resource groups are available for your subscription:

1. ****DefaultResourceGroup-EUS**** (Location: `eastus`)
2. ****rg-testing**** (Location: `centralus`)
3. ****rg-azd**** (Location: `eastus2`)
4. ****msdocs-sample**** (Location: `southcentralus`)
5. ****ai-testing**** (Location: `eastus2`)

Let me know if you need further details or actions related to any of these resource groups!

4. Explore more Azure MCP operations using other relevant prompts, such as:

text

List all of the storage accounts in my subscription
Get the available tables in my storage accounts
Show me the configuration of my App Service instances

Next steps

[Learn more about Azure MCP Server tools](#)

Last updated on 02/27/2026

Get started with the Azure MCP Server in Cline

The Azure MCP Server uses the Model Context Protocol (MCP) to standardize integrations between AI apps and external tools and data sources, allowing for AI systems to perform operations that are context-aware of your Azure resources.

In this article, you learn how to complete the following tasks:

- Install and authenticate to the Azure MCP Server
- Connect to Azure MCP Server using Cline's AI-powered assistant
- Run prompts to test Azure MCP Server operations and interact with Azure resources

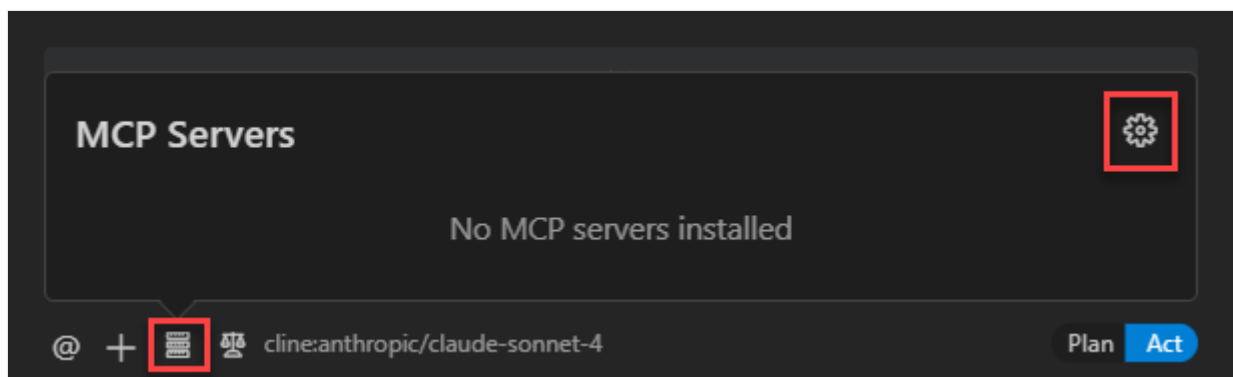
Prerequisites

- An [Azure account](#) with an active subscription
- [Cline](#) installed
- [Node.js](#) LTS installed

Install the Azure MCP Server

To install and configure the Azure MCP Server in Cline:

1. Open the **Cline** panel in your editor.
2. Select **Manage MCP Servers** to open the **MCP Servers** flyout, and then select the **Settings** icon.

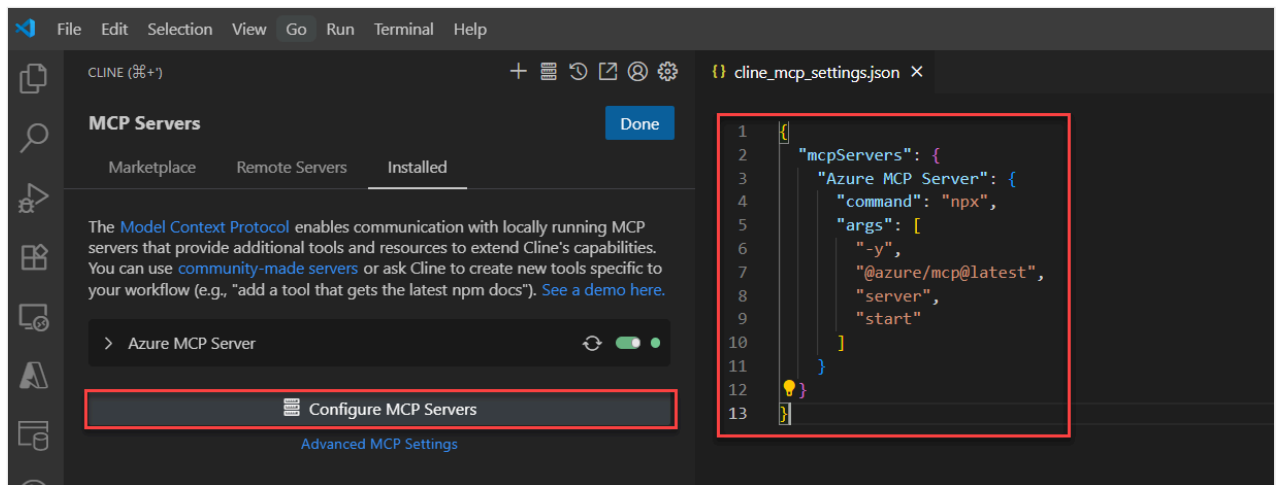


3. On the **MCP Servers** section of the panel, select **Configure MCP Servers** to open the `cline_mcp_settings.json` file for editing.

4. Add the following configuration to the `mcpServers` JSON object:

```
JSON

"Azure MCP Server": {
  "command": "npx",
  "args": [
    "-y",
    "@azure/mcp@latest",
    "server",
    "start"
  ]
}
```



5. Select **Done** to close the configuration panel and return to the chat interface.

Authenticate to Azure

Azure MCP Server provides a seamless authentication experience using Azure accounts and Microsoft Entra ID. To use Azure MCP Server, you must first authenticate to Azure using local development tools such as the Azure CLI, Azure Developer CLI, Visual Studio, or Visual Studio Code. Azure MCP Server automatically discovers your credentials from these tools and uses them to authenticate to Azure services.

1. For example, to sign in using the Azure CLI:

```
Azure CLI

az login
```

2. Verify your authentication status by running the following command to see which account and subscription you're currently signed in with:

Azure CLI

```
az account show
```

3. Ensure your user account has the appropriate role assignments for the Azure services you want to interact with. The Azure resources you intend to access with Azure MCP Server must already exist within your Azure subscription. For example, common role assignments include:

- **Blob Storage Data Contributor** - Read and write blob data in storage accounts.
- **Storage Account Contributor** - Manage storage account configurations.
- **Contributor** - General resource management across your subscription.
- **Reader** - Read-only access to Azure resources.

For more information about role assignments and local development authentication, see [Authenticate .NET apps to Azure services during local development](#).

Use prompts to test the Azure MCP Server

1. On the Cline chat panel, enter a prompt that utilizes Azure MCP Server capabilities, such as:

text

List my Azure storage accounts

2. If you're prompted to authenticate to Azure, run the suggested auth tool to sign-in through the browser.

ⓘ Note

Cline doesn't prompt you to sign in to Azure if you're already authenticated via other local tooling such as the Azure CLI.

3. Cline prompts you to run a tool to retrieve the storage accounts, such as `storage account list`. Select **Run tool** to continue.

The output should resemble the following text:

Output

The following resource groups are available for your subscription:

1. ****DefaultResourceGroup-EUS**** (Location: `eastus`)
2. ****rg-testing**** (Location: `centralus`)
3. ****rg-azd**** (Location: `eastus2`)
4. ****msdocs-sample**** (Location: `southcentralus`)
5. ****ai-testing**** (Location: `eastus2`)

Let me know if you need further details or actions related to any of these resource groups!

4. Explore more Azure MCP operations using other relevant prompts, such as:

text

List all of the storage accounts in my subscription
Get the available tables in my storage accounts
Show me the configuration of my App Service instances

Next steps

[Learn more about Azure MCP Server tools](#)

Last updated on 02/27/2026

Get started using the Azure MCP Server with Python

The Azure MCP Server uses the Model Context Protocol (MCP) to standardize integrations between AI apps and external tools and data sources, allowing for AI systems to perform operations that are context-aware of your Azure resources.

In this article, you learn how to complete the following tasks:

- Install and authenticate to the Azure MCP Server
- Connect to Azure MCP Server using a custom Python client
- Run prompts to test Azure MCP Server operations and manage Azure resources

Prerequisites

- An [Azure account](#) with an active subscription
- [Python 3.9 or higher](#) installed
- [Node.js](#) LTS installed

ⓘ Note

The Azure resources you intend to access with Azure MCP Server must already exist within your Azure subscription. Additionally, your user account must have the necessary [RBAC roles and permissions](#) assigned for those resources.

Sign in to Azure MCP Server for local development

Azure MCP Server authenticates to [Microsoft Entra ID](#) using the [Azure Identity library for .NET](#).

The server supports [two authentication modes](#):

- **Broker mode:** Uses your operating system's native authentication (like Windows Web Account Manager) with `InteractiveBrowserCredential`.
- **Credential chain mode:** Tries multiple authentication methods in sequence: environment variables, Visual Studio Code, Visual Studio, Azure CLI, Azure PowerShell, Azure Developer CLI, and interactive browser authentication.

Sign in using any of these methods:

1. Open the Command Palette (`Ctrl+Shift+P` or `Cmd+Shift+P` on Mac).
2. Run **Azure: Sign In** and follow the prompts.

After signing in, Azure MCP Server can authenticate and run operations on Azure services based on your permissions.

Create the Python app

Complete the following steps to create a Python app (host app). The app connects to an AI model and acts as a host for an MCP client that connects to an Azure MCP Server (local process running the MCP protocol).

Create the project

1. Open an empty folder inside your editor of choice.
2. Create a new file named `requirements.txt` and add the following library dependencies:

Output

```
mcp
azure-identity
openai
logging
```

3. In the same folder, create a new file named `.env` and add the following environment variables:

Output

```
AZURE_OPENAI_ENDPOINT=<your-azure-openai-endpoint>
AZURE_OPENAI_MODEL=<your-model-deployment-name>
```

4. Create an empty file named `main.py` to hold the code for your app.

Create the environment and install dependencies

1. Open a terminal in your new folder and create a Python virtual environment for the app:

Console

```
python -m venv venv
```

2. Activate the virtual environment:

Console

```
venv\Scripts\activate
```

3. Install the dependencies from `requirements.txt`:

Console

```
pip install -r requirements.txt
```

Add the app code

Update the contents of `main.py` with the following code:

Python

```
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
from openai import AzureOpenAI
from mcp import ClientSession, StdioServerParameters, types
from mcp.client.stdio import stdio_client
import json, os, logging, asyncio
from dotenv import load_dotenv

# Setup logging and load environment variables
logger = logging.getLogger(__name__)
load_dotenv()

# Azure OpenAI configuration
AZURE_OPENAI_ENDPOINT = os.getenv("AZURE_OPENAI_ENDPOINT")
AZURE_OPENAI_MODEL = os.getenv("AZURE_OPENAI_MODEL", "gpt-4o")

# Initialize Azure credentials
token_provider = get_bearer_token_provider(
    DefaultAzureCredential(), "https://cognitiveservices.azure.com/.default"
)

async def run():
    # Initialize Azure OpenAI client
    client = AzureOpenAI(
        azure_endpoint=AZURE_OPENAI_ENDPOINT,
        api_version="2024-04-01-preview",
        azure_ad_token_provider=token_provider
    )
```

```

# MCP client configurations
server_params = StdioServerParameters(
    command="npx",
    args=["-y", "@azure/mcp@latest", "server", "start"],
    env=None
)

async with stdio_client(server_params) as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()

        # List available tools
        tools = await session.list_tools()
        for tool in tools.tools: print(tool.name)

        # Format tools for Azure OpenAI
        available_tools = [{
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description,
                "parameters": tool.inputSchema
            }
        } for tool in tools.tools]

        # Start conversational loop
        messages = []
        while True:
            try:
                user_input = input("\nPrompt: ")
                messages.append({"role": "user", "content": user_input})

                # First API call with tool configuration
                response = client.chat.completions.create(
                    model = AZURE_OPENAI_MODEL,
                    messages = messages,
                    tools = available_tools)

                # Process the model's response
                response_message = response.choices[0].message
                messages.append(response_message)

                # Handle function calls
                if response_message.tool_calls:
                    for tool_call in response_message.tool_calls:
                        function_args =
json.loads(tool_call.function.arguments)
                        result = await
session.call_tool(tool_call.function.name, function_args)

                # Add the tool response to the messages
                messages.append({
                    "tool_call_id": tool_call.id,
                    "role": "tool",

```



```

        "name": tool_call.function.name,
        "content": result.content,
    })
else:
    logger.info("No tool calls were made by the model")

# Get the final response from the model
final_response = client.chat.completions.create(
    model = AZURE_OPENAI_MODEL,
    messages = messages,
    tools = available_tools)

for item in final_response.choices:
    print(item.message.content)
except Exception as e:
    logger.error(f"Error in conversation loop: {e}")
    print(f"An error occurred: {e}")

if __name__ == "__main__":
    import asyncio
    asyncio.run(run())

```

The preceding code accomplishes the following tasks:

- Sets up logging and loads environment variables from a `.env` file.
- Configures Azure OpenAI client using `azure-identity` and `openai` libraries.
- Initializes an MCP client to interact with the Azure MCP Server (local process) using a standard I/O transport.
- Retrieves and displays a list of available tools (MCP-registered Azure operations) from the Azure MCP Server.
- Implements a conversational loop to process user prompts, utilize tools, and handle tool calls.

Configuration parameters:

 Expand table

Parameter	Description	Example
<code>AZURE_OPENAI_ENDPOINT</code>	Your Azure OpenAI service endpoint	<code>https://your-resource.openai.azure.com/</code>
<code>AZURE_OPENAI_MODEL</code>	Model deployment name	<code>gpt-4o</code>
Token scope	Azure Cognitive Services OAuth scope	<code>https://cognitiveservices.azure.com/.default</code>
Authentication	Uses <code>DefaultAzureCredential</code> (Azure CLI, managed identity,	See Azure Identity documentation

Parameter	Description	Example
	or other credential chain)	
Required RBAC	Cognitive Services User role or equivalent on the Azure OpenAI resource	Assigned through Azure portal or CLI

Run and test the app

Complete the following steps to test your Python app:

1. In a terminal window open to the root of your project, run the following command to start the app:

Console

```
python main.py
```

Success verification: The app should display a list of available Azure MCP Server tools, then show a `Prompt:` input.

2. Once the app is running, enter the following test prompt:

```
List all of the resource groups in my subscription
```

The output for the previous prompt should resemble the following text:

Output

```
The following resource groups are available for your subscription:

1. **DefaultResourceGroup-EUS** (Location: `eastus`)
2. **rg-testing** (Location: `centralus`)
3. **rg-azd** (Location: `eastus2`)
4. **msdocs-sample** (Location: `southcentralus`)
14. **ai-testing** (Location: `eastus2`)

Let me know if you need further details or actions related to any of these resource groups!
```

3. Explore and test the Azure MCP operations using other relevant prompts, such as:

List all of the storage accounts in my subscription
Get the available tables in my storage accounts

Next steps

[Learn more about Azure MCP Server tools](#)

Last updated on 12/13/2025

Get started using the Azure MCP Server with .NET

The Azure MCP Server uses the Model Context Protocol (MCP) to standardize integrations between AI apps and external tools and data sources, allowing for AI systems to perform operations that are context-aware of your Azure resources.

In this article, you learn how to complete the following tasks:

- Install and authenticate to the Azure MCP Server
- Connect to Azure MCP Server using a custom .NET client
- Run prompts to test Azure MCP Server operations and manage Azure resources

Prerequisites

- An [Azure account](#) with an active subscription
- [.NET 9.0](#) or higher installed
- [Node.js](#) LTS installed

ⓘ Note

The Azure resources you intend to access with Azure MCP Server must already exist within your Azure subscription. Additionally, your user account must have the necessary [RBAC roles and permissions](#) assigned for those resources.

Sign in to Azure MCP Server for local development

Azure MCP Server authenticates to [Microsoft Entra ID](#) using the [Azure Identity library for .NET](#).

The server supports [two authentication modes](#):

- **Broker mode:** Uses your operating system's native authentication (like Windows Web Account Manager) with `InteractiveBrowserCredential`.
- **Credential chain mode:** Tries multiple authentication methods in sequence: environment variables, Visual Studio Code, Visual Studio, Azure CLI, Azure PowerShell, Azure Developer CLI, and interactive browser authentication.

Sign in using any of these methods:

1. Open the Command Palette (`Ctrl+Shift+P` or `Cmd+Shift+P` on Mac).
2. Run **Azure: Sign In** and follow the prompts.

After signing in, Azure MCP Server can authenticate and run operations on Azure services based on your permissions.

Create the .NET host app

Complete the following steps to create a .NET console app. The app connects to an AI model and acts as a host for an MCP client that connects to an Azure MCP Server.

Create the project

1. Open a terminal to an empty folder where you want to create the project.
2. Run the following command to create a new .NET console application:

Console

```
dotnet new console -n MCPHostApp
```

3. Navigate into the newly created project folder:

Console

```
cd MCPHostApp
```

4. Open the project folder in your editor of choice, such as Visual Studio Code:

Console

```
code .
```

Add the dependencies

1. In the terminal, run the following commands to add the necessary NuGet packages:

Console

```
dotnet add package Azure.AI.OpenAI --prerelease
dotnet add package Azure.Identity
dotnet add package Microsoft.Extensions.AI --prerelease
dotnet add package Microsoft.Extensions.AI.OpenAI --prerelease
dotnet add package ModelContextProtocol --prerelease
```

2. Verify that the packages were added by checking the `MCPHostApp.csproj` file.
3. Run the following command to build the project and ensure everything is set up correctly:

Console

```
dotnet build
```

Add the app code

Replace the contents of `Program.cs` with the following code:

C#

```
using Azure.AI.OpenAI;
using Azure.Identity;
using Microsoft.Extensions.AI;
using ModelContextProtocol.Client;

// Create an IChatClient
IChatClient client =
    new ChatClientBuilder(
        new AzureOpenAIClient(new Uri("<your-azure-openai-endpoint>"),
            new DefaultAzureCredential())
            .GetChatClient("gpt-4o").AsIChatClient())
        .UseFunctionInvocation()
        .Build();

// Create the MCP client
var mcpClient = await McpClient.CreateAsync(
    new StdioClientTransport(new()
    {
        Command = "npx",
        Arguments = ["-y", "@azure/mcp@latest", "server", "start"],
        Name = "Azure MCP",
    }));

// Get all available tools from the MCP server
Console.WriteLine("Available tools:");
var tools = await mcpClient.ListToolsAsync();
foreach (var tool in tools)
{
    Console.WriteLine($"{tool}");
}
```

```

Console.WriteLine();

// Conversational loop that can utilize the tools
List<ChatMessage> messages = [];
while (true)
{
    Console.Write("Prompt: ");
    messages.Add(new(ChatRole.User, Console.ReadLine()));

    List<ChatResponseUpdate> updates = [];
    await foreach (var update in client
        .GetStreamingResponseAsync(messages, new() { Tools =[.. tools] }))
    {
        Console.Write(update);
        updates.Add(update);
    }
    Console.WriteLine();

    messages.AddMessages(updates);
}

```

The preceding code accomplishes the following tasks:

- Initializes an `IChatClient` abstraction using the [Microsoft.Extensions.AI](#) libraries.
- Creates an MCP client to interact with the Azure MCP Server using a standard I/O transport. The provided `npm` command and corresponding arguments download and start the Azure MCP Server.
- Retrieves and displays a list of available tools from the MCP server, which is a standard MCP function.
- Implements a conversational loop that processes user prompts and utilizes the tools for responses.

Run and test the app

Complete the following steps to test your .NET host app:

1. In a terminal window open to the root of your project, run the following command to start the app:

Console

```
dotnet run
```

2. Once the app is running, enter the following test prompt:

```
List all of the resource groups in my subscription
```

The output for the previous prompt should resemble the following text:

Output

The following resource groups are available for your subscription:

1. ****DefaultResourceGroup-EUS**** (Location: `eastus`)
2. ****rg-testing**** (Location: `centralus`)
3. ****rg-azd**** (Location: `eastus2`)
4. ****msdocs-sample**** (Location: `southcentralus`)
14. ****ai-testing**** (Location: `eastus2`)

Let me know if you need further details or actions related to any of these resource groups!

3. Explore and test the Azure MCP operations using other relevant prompts, such as:

```
List all of the storage accounts in my subscription
Get the available tables in my storage accounts
```

Next steps

[Learn more about Azure MCP Server tools](#)

Last updated on 02/27/2026

Azure MCP Server concepts

This article explains essential concepts for effectively using the Azure MCP Server, including how to work with multiple Azure services, handle errors, optimize performance, and apply common patterns for real-world scenarios.

Multi-service workflows

The Azure MCP Server can orchestrate operations across multiple Azure services in a single conversation or workflow. This capability enables you to chain operations across services like Azure Storage, Azure Cosmos DB, and Azure Key Vault without switching contexts.

Server modes for multi-service operations

The Azure MCP Server supports different modes that affect how tools are exposed and how you interact with multiple services:

Namespace mode (default)

Namespace mode groups tools by Azure service, exposing one tool per service namespace. This mode provides a balanced approach for multi-service workflows.

JSON

```
{
  "mcpServers": {
    "Azure MCP Server": {
      "command": "npx",
      "args": ["-y", "@azure/mcp@latest", "server", "start"]
    }
  }
}
```

With namespace mode, you can make requests like:

- "List my storage accounts and then show me the containers in the first account"
- "Get secrets from my key vault and use them to connect to my Cosmos DB database"

Consolidated mode (recommended for AI agents)

Consolidated mode groups related operations into curated tools optimized for AI agents. This mode provides the best balance between functionality and usability.

JSON

```
{
  "mcpServers": {
    "Azure MCP Server": {
      "command": "npx",
      "args": ["-y", "@azure/mcp@latest", "server", "start", "--mode",
"consolidated"]
    }
  }
}
```

Consolidated tools are named after user intents, for example `get_azure_databases_details`, making them more intuitive for natural language interactions.

For a complete list of available server modes and their configurations, see the [troubleshooting guide](#).

All mode

The `all` mode exposes 200+ individual tools separately. This mode is useful when you need granular control but can exceed tool limits in some clients.

JSON

```
{
  "mcpServers": {
    "Azure MCP Server": {
      "command": "npx",
      "args": ["-y", "@azure/mcp@latest", "server", "start", "--mode", "all"]
    }
  }
}
```

Chaining operations across services

You can chain operations across multiple Azure services in a single conversation. For example, you can:

1. Query Azure Key Vault for database credentials
2. Use those credentials to connect to Azure Cosmos DB
3. Query data from Cosmos DB

4. Store results in Azure Storage

Example prompts that chain operations across services include:

- "Get the database connection string from key vault 'my-vault', connect to the database, and list the collections"
- "List all storage accounts in my subscription, then for each account show me the containers"
- "Create a new storage container and upload the contents of my key vault secret to a blob"

Filtering tools by service

You can configure the Azure MCP Server to expose only specific services using the `--namespace` option. This approach is useful for focused workflows or when working with multiple MCP server instances.

JSON

```
{
  "mcpServers": {
    "Azure Storage": {
      "command": "npx",
      "args": ["-y", "@azure/mcp@latest", "server", "start", "--namespace",
"storage"]
    },
    "Azure KeyVault": {
      "command": "npx",
      "args": ["-y", "@azure/mcp@latest", "server", "start", "--namespace",
"keyvault"]
    }
  }
}
```

This configuration creates separate MCP server instances for Storage and Key Vault, allowing you to organize tools by domain or project requirements.

Error handling and retry logic

The Azure MCP Server handles various error scenarios that can occur when interacting with Azure services. Understanding these patterns helps you troubleshoot issues and build resilient workflows.

Authentication errors

Authentication is a common source of errors when working with Azure resources.

401 Unauthorized errors

A 401 error indicates that the access token is invalid or missing. This error can occur when:

- Local authorization (access keys) is disabled on the resource
- The authentication token has expired
- No valid credentials are available

Resolution approaches:

- Verify that you're authenticated to Azure using `az login` or your preferred authentication method
- Check that the resource allows the authentication method you're using
- For resources with access keys disabled, ensure you have appropriate Role-Based Access Control (RBAC) permissions

403 Forbidden errors

A 403 error indicates that the authenticated user doesn't have sufficient permissions to access the requested resource.

Common causes:

- Missing RBAC permissions at the resource group or subscription level
- Wrong subscription or tenant context
- Using an unintended account when multiple accounts are signed in

Resolution approaches:

- Verify RBAC permissions are assigned at the correct scope
- Specify the subscription and tenant explicitly in your prompts: "List all my storage accounts in subscription `subscription-name`, located in tenant `tenant-name`"
- Set the `AZURE_MCP_ONLY_USE_BROKER_CREDENTIAL` environment variable to `true` to prompt for account selection

Network and firewall restrictions

Enterprise environments often have network controls that can affect Azure MCP Server connectivity.

Required endpoints for authentication:

- `login.microsoftonline.com:443`
- `login.windows.net:443`
- `management.azure.com:443`
- `graph.microsoft.com:443`

Resource-specific endpoints depend on the Azure services you're using, for example:

- Azure Storage: `*.blob.core.windows.net:443`
- Azure Key Vault: `*.vault.azure.net:443`
- Azure Cosmos DB: `*.documents.azure.com:443`

If you're behind a corporate proxy, configure proxy settings using environment variables:

Bash

```
export HTTP_PROXY=http://proxy.company.com:8080
export HTTPS_PROXY=http://proxy.company.com:8080
export NO_PROXY=localhost,127.0.0.1
```

Transient failures

The Azure MCP Server relies on Azure SDKs, which have built-in retry logic for transient failures. These SDKs automatically handle:

- Network connectivity issues
- Service throttling and rate limits
- Temporary service unavailability

The retry policies use exponential backoff to avoid overwhelming services during high load or incident scenarios.

Error messages and diagnostics

When errors occur, the Azure MCP Server provides detailed error messages to help you understand and resolve issues. For comprehensive troubleshooting guidance, see the [troubleshooting guide](#).

Optimization tips

Optimizing your Azure MCP Server configuration improves performance, reduces token usage, and enhances the user experience.

Managing tool counts

Some MCP clients have limits on the number of tools they can handle. For example, Visual Studio Code Copilot has a 128-tool limit per request.

Use consolidated mode

Consolidated mode provides full functionality while staying well under client tool limits:

JSON

```
{
  "mcpServers": {
    "Azure MCP": {
      "command": "npx",
      "args": ["-y", "@azure/mcp@latest", "server", "start", "--mode",
"consolidated"]
    }
  }
}
```

This configuration exposes curated tools that group related operations, optimizing for both functionality and AI agent effectiveness.

Use selective tool loading

Load only the tools you need for your specific workflow:

JSON

```
{
  "mcpServers": {
    "Azure Essentials": {
      "command": "npx",
      "args": [
        "-y",
        "@azure/mcp@latest",
        "server",
        "start",
        "--tool",
        "azmcp_subscription_list",
        "--tool",
        "azmcp_group_list",
        "--tool",
        "azmcp_storage_account_get"
      ]
    }
  }
}
```

```
}  
}  
}  
]
```

Use custom chat modes

Visual Studio Code supports [custom chat modes](#) that let you configure different tool sets for different scenarios. This approach allows you to switch between tool configurations based on your current task while staying within client limits.

Token management

Effective token management improves response times and reduces costs when using language models.

Use specific prompts

Specific prompts reduce the number of tool calls and the amount of context needed:

Less effective:

- "Tell me about my Azure resources"

More effective:

- "List storage accounts in subscription 'my-subscription' in resource group 'my-rg'"
- "Show me the connection string for storage account 'mystorageaccount'"

Scope operations appropriately

When possible, scope operations to specific resources or resource groups rather than querying entire subscriptions:

Less efficient:

- "List all resources in my subscription and filter for storage accounts"

More efficient:

- "List storage accounts in resource group 'production-rg'"

Batch operations

The Azure MCP Server supports batch operations for certain scenarios. When working with multiple resources:

Sequential operations:

```
List storage accounts, then for each one show the containers
```

Parallel operations when appropriate:

```
Get details for storage accounts 'account1', 'account2', and 'account3'
```

The Azure MCP Server and underlying Azure SDKs handle batching automatically when appropriate, optimizing network calls and reducing latency.

Caching and reuse

The Azure MCP Server maintains connection state during a session. To optimize performance:

- Keep MCP server instances running rather than starting and stopping frequently
- Reuse subscription and resource group information in subsequent queries
- Cache resource details in your workflow when you need to reference them multiple times

Common use cases

The Azure MCP Server supports a wide range of real-world scenarios. This section provides examples of common patterns and workflows.

Deploying a new service

When deploying a new Azure service, you can use the Azure MCP Server to automate infrastructure setup and configuration:

1. Create resource group and resources:

```
Create a resource group called 'webapp-prod' in East US,  
then create a storage account called 'webappdata' in that resource group
```


2. Configure resources:

```
Get the storage account connection string and store it as a secret  
in key vault 'webapp-kv' with name 'StorageConnectionString'
```

3. Verify deployment:

```
List all resources in resource group 'webapp-prod'  
and verify that the storage account and key vault exist
```

Debugging a live site issue

The Azure MCP Server can help you diagnose and troubleshoot production issues:

1. Check resource health:

```
Check the health status of App Service 'my-webapp'  
and list any recent service health incidents
```

2. Query logs:

```
Query Application Insights for exceptions in the last hour  
from application 'my-webapp'
```

3. Inspect configuration:

```
Get the application settings for App Service 'my-webapp'  
and check if the database connection string is configured correctly
```

4. Check dependencies:

```
List the databases in SQL server 'my-sql-server'
```

```
and verify connectivity to database 'my-database'
```

Managing secrets and configuration

A common pattern involves managing secrets and configuration across environments:

1. Audit secrets:

```
List all secrets in key vault 'production-kv'  
and show me which ones are expiring in the next 30 days
```

2. Rotate secrets:

```
Generate a new connection string for storage account 'mystore',  
update the secret 'StorageConnection' in key vault 'production-kv',  
and restart App Service 'my-webapp'
```

3. Sync configuration:

```
Get all key-value pairs from App Configuration 'my-appconfig'  
with label 'production' and compare them to the staging environment
```

Data operations across services

The Azure MCP Server excels at coordinating data operations across multiple services:

1. Extract, Transform, Load (ETL):

```
Get data from Cosmos DB container 'raw-data',  
transform it using the provided logic,  
and upload the results to blob container 'processed-data'  
in storage account 'analytics'
```

2. Backup and archival:

```
Export all documents from Cosmos DB container 'transactions'
and create a snapshot in blob storage with timestamp
```

3. Cross-service queries:

```
Query Azure Data Explorer for events where error count is greater than 100,
then lookup the associated user data from Cosmos DB
```

Infrastructure audit and compliance

Use the Azure MCP Server to audit infrastructure and verify compliance:

1. Security audit:

```
List all storage accounts in my subscription
and check which ones don't have firewall rules configured
```

2. Cost analysis:

```
List all SQL databases in my subscription,
show their pricing tiers,
and identify candidates for downgrading
```

3. Resource tagging:

```
List all resources in subscription 'production-sub'
without an 'Environment' tag and show their resource groups
```

Development and testing workflows

Developers can use the Azure MCP Server to streamline development workflows:

1. Environment setup:

Create a development environment with a storage account, key vault, and App Service in resource group 'dev-environment'

2. Test data management:

Copy data from production storage container 'data' to development storage container 'test-data', anonymizing sensitive fields

3. Configuration management:

Compare App Service settings between 'staging-webapp' and 'production-webapp' and highlight differences

Related content

- [Get started using the Azure MCP Server](#)
- [Azure MCP Server tools](#)
- [Authentication guidance](#) [↗](#)
- [Troubleshooting guide](#) [↗](#)
- [Azure MCP Server repository](#) [↗](#)

Note: This article contains content created with AI. [Learn more](#)

Last updated on 01/28/2026

Connect GitHub Copilot coding agent to the Azure MCP Server

This article shows you how to set up and connect the GitHub Copilot coding agent to the Azure MCP Server so that the coding agent can understand your Azure-specific files and Azure resources to make edits to your code files.

In your GitHub repository, assigning GitHub issues to the GitHub Copilot coding agent creates a pull request with the changes to your code. If the requested changes require access to your Azure resources, the GitHub Copilot coding agent needs to use the Azure MCP Server. Before the GitHub Copilot coding agent can use the Azure MCP Server to make changes to your Azure-based project agentically, you need to configure the GitHub Copilot coding agent and give it the proper permissions in Azure. You could manually configure everything, however many steps are automated using `azd`, the Azure Developer CLI, and the `coding-agent` extension.

Prerequisites

- An Azure account and access to an Azure subscription. For details on how to set them up, see the [pricing page for Azure accounts](#) [↗].
- A GitHub account and a GitHub Copilot subscription. For details on how to set them up, see [Creating an account on GitHub](#) [↗] and [Quickstart for GitHub Copilot](#) [↗], respectively.
- An existing local clone of a GitHub repository. Since this article describes how to set up the connection between GitHub Copilot coding agent to the Azure MCP Server, the GitHub repository should include deployment scripts to Azure, like Bicep or Terraform templates.

Configure the GitHub repository to use the Azure MCP Server

The `azd` `coding-agent` extension simplifies the steps required to securely set up the connection between the GitHub Copilot coding agent and the Azure MCP Server for your Azure subscription. First, it creates an account in your Azure subscription and assigns it a role with

necessary permissions. Second, it provides a JSON snippet required to introduce the Azure MCP Server to the GitHub Copilot coding agent.

1. If you don't already have `azd` installed, [follow the instructions](#) to install it.
2. In the terminal, navigate into the local clone of the repository you want to work with.
3. Invoke the azd coding agent extension with the command: `azd coding-agent config`.
4. During installation, you are asked to select your:
 - Azure subscription
 - Which GitHub repository will use the Copilot Coding agent
 - Whether you want to create a new or existing User Managed Identity
 - An Azure location
 - An Azure resource group
 - The GitHub repository where a new branch will be created containing the generated GitHub Actions workflow setup file

When selecting the location and resource group, you may want to use the same target location and resource group as the Azure resources in the application.

5. After a few moments, the `azd` coding agent extension creates (or uses the existing) User Managed Identity and assigns it a role, stores identity values in the GitHub repository environment, and creates and pushes a branch containing the generated GitHub Actions workflow setup file.
6. You will see a message in the console:

Console

```
(!)\n(!) NOTE: Some tasks must still be completed, manually:\n(!)
```

Usually there are three tasks:

- Merge the branch containing the generated GitHub Actions workflow setup file.
- Configure Copilot coding agent's managed identity roles in the Azure portal. By default, the "Reader" role is assigned. However you may want to give it other permissions based on what you want the coding agent to do autonomously.

- Visit the link to set up the MCP Configuration. To navigate there manually, in GitHub go to Settings > Copilot > coding agent > MCP Configuration and paste in the JSON snippet provided. Here's an example:

JSON

```
{
  "mcpServers": {
    "Azure": {
      "type": "local",
      "command": "npx",
      "args": [
        "-y",
        "@azure/mcp@latest",
        "server",
        "start"
      ],
      "tools": [
        "*"
      ]
    }
  }
}
```

7. Finally, you can allow the `azd` coding agent extension to open the browser so you can create the pull request to merge the branch containing the generated GitHub Actions workflow setup file.

Create an Issue in GitHub to initiate GitHub Copilot coding agent

At this point, you successfully set up GitHub Copilot coding agent to use the Azure MCP Server for any GitHub Issues you assign to GitHub Copilot coding agent that require an understanding of Azure deployments and resources.

For example, suppose you want to increase the memory allocated to PostgreSQL when deployed to Azure Database for PostgreSQL. You would create an issue to modify your Bicep template to use the next tier of storage available and assign it to GitHub Copilot.

ⓘ Note

The User Managed Identity role is set to "Reader" by default, so the changes you request should be to modify deployment scripts like Bicep or Terraform templates. Asking to

directly modify existing resources in your Azure subscription isn't authorized due to the permissions of the "Reader" role.

1. In GitHub, in the repository containing your Azure-based project where you enabled the Azure MCP Server, go to Issues.
2. Select the "New Issue" button. Describe the change you want GitHub Copilot coding agent to make in the title and description fields. Select the "Create" button.

Borrowing from the example earlier, you might use the following text to describe your issue.

text

Title: Increase database storage

Currently, when deploying to Azure via Bicep, we're creating a PostgreSQL database with 32gb of storage. I need the next tier higher -- whatever that is.

This example issue makes a simple, clear request even if the user doesn't know exactly what they're asking for. It allows the Azure MCP Server to do research about available storage tiers for Azure Database for PostgreSQL Flexible Server and the setting in the Bicep template required to make that change.

Important

Make sure to use the word "Azure" in your prompt to ensure that GitHub Copilot requests tools from the Azure MCP Server.

3. Select the "Assign to Copilot" button under **Assignees**. The "Assign Copilot to issue" dialog appears allowing you to modify the target repository, the base branch, and add an optional prompt. Select the "Assign" button.
4. Once the issue is assigned to GitHub Copilot coding agent, you see a link to the pull request prefixed with "[WIP]" letting you know that the work is starting.
5. Select the "[WIP]" link to view the pull request.
6. In the body of the pull request, select the link to view the coding session. This shows the progress the Copilot Coding agent is making on the request, similar to the experience in

Visual Studio.

7. When finished, GitHub Copilot coding agent requests a code review. Use your normal workflow to iterate with GitHub, treating GitHub Copilot coding agent as a coworker.
8. When you approve the changes and merge the pull request, GitHub Copilot resolves the original issue you created.

Related content

- [GitHub Copilot coding agent](#) 

Last updated on 02/27/2026

Quickstart: Integrate Azure MCP Server with GitHub Copilot CLI

In this quickstart, you learn how to connect the Azure Model Context Protocol (MCP) Server to the GitHub Copilot CLI. This integration allows GitHub Copilot to interact with your Azure resources directly from your terminal.

Prerequisites

- [GitHub Copilot CLI](#) [↗] installed.
- [Azure CLI](#) installed and authenticated (`az login`).
- [Node.js](#) [↗] installed (for running the server via `npx`).

Sign in to Azure MCP Server for local development

Azure MCP Server authenticates to [Microsoft Entra ID](#) using the [Azure Identity library for .NET](#). The server supports [two authentication modes](#) [↗]:

- **Broker mode:** Uses your operating system's native authentication (like Windows Web Account Manager) with `InteractiveBrowserCredential`.
- **Credential chain mode:** Tries multiple authentication methods in sequence: environment variables, Visual Studio Code, Visual Studio, Azure CLI, Azure PowerShell, Azure Developer CLI, and interactive browser authentication.

Sign in using any of these methods:

Visual Studio Code

1. Open the Command Palette (`Ctrl+Shift+P` or `Cmd+Shift+P` on Mac).
2. Run **Azure: Sign In** and follow the prompts.

After signing in, Azure MCP Server can authenticate and run operations on Azure services based on your permissions.

Add Azure MCP Server

The GitHub Copilot CLI supports MCP servers through the `/mcp` command family.

- 1. Open your terminal.
- 2. Start the GitHub Copilot CLI in interactive mode:

Bash

copilot

- 3. In the interactive session, run the following command to open the MCP server configuration form:

Bash

/mcp add

- 4. Fill in the configuration fields with the following values:

[Expand table](#)

Field	Value
Server Name	azure-mcp
Server Type	1 (Local)
Command	npx -y @azure/mcp@latest server start
Environment Variables	(leave blank - utilizes Azure CLI authentication)
Tools	*

ⓘ Note

If you prefer using .NET, set the **Command** to `dotnet dnx -p Azure.Mcp server start`.

- 5. Press `Ctrl+S` (or `Cmd+S` on macOS) to save the server configuration.
- 6. When you've finished, press `esc` to close the server configuration.

Verify the connection

Confirm that you configured the Azure MCP Server correctly and that the GitHub Copilot CLI recognizes it:

1. In your Copilot CLI session, run:

Bash

```
/mcp show
```

2. Review the output. You should see `azure-mcp` listed in the configuration:

Output

```
• MCP Server Configuration:
  • azure-mcp (local): Command: npx

Total servers: 1
Config file: ~/.copilot/mcp-config.json
```

Use Azure MCP Server

Once connected, you can use natural language to interact with your Azure resources.

1. In the GitHub Copilot CLI session, type a prompt that requires Azure context. For example:

text

```
> List my Azure resource groups.
```

2. GitHub Copilot identifies the intent and uses the `azure-mcp` tools to fetch the information. It prints a response listing your Azure resource groups, similar to:

Output

```
I found the following resource groups in your subscription:

- **my-resource-group-1** (East US)
- **dev-environment** (West Europe)
- **production-app** (Central US)
```

Manage MCP servers

Manage your configured MCP servers using the following commands:

- List servers: `/mcp show`
- Remove a server: `/mcp remove azure-mcp`
- Get help: `/mcp help`

Next steps

- Learn more about [GitHub Copilot CLI](#) [↗].
- Explore [Azure MCP Server capabilities](#).

 **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 02/25/2026

Quickstart: Integrate Azure MCP Server with GitHub Copilot SDK

This guide explains how to configure the [GitHub Copilot SDK](#) to use Azure Model Context Protocol (MCP) tools for interacting with Azure resources.

Azure MCP provides a set of tools that enable AI assistants to interact with Azure resources directly. When integrated with the Copilot SDK, you can build applications that leverage natural language to manage Azure subscriptions, resource groups, storage accounts, and more.

Prerequisites

- [GitHub Copilot CLI](#) installed.
- [Node.js](#) installed (for running the server via `npx`).
- [Azure CLI](#) installed and authenticated (`az login`).

Sign in to Azure MCP Server for local development

Azure MCP Server authenticates to [Microsoft Entra ID](#) using the [Azure Identity library for .NET](#). The server supports [two authentication modes](#):

- **Broker mode:** Uses your operating system's native authentication (like Windows Web Account Manager) with `InteractiveBrowserCredential`.
- **Credential chain mode:** Tries multiple authentication methods in sequence: environment variables, Visual Studio Code, Visual Studio, Azure CLI, Azure PowerShell, Azure Developer CLI, and interactive browser authentication.

Sign in using any of these methods:

Visual Studio Code

1. Open the Command Palette (`Ctrl+Shift+P` or `Cmd+Shift+P` on Mac).
2. Run **Azure: Sign In** and follow the prompts.

After signing in, Azure MCP Server can authenticate and run operations on Azure services based on your permissions.

Azure MCP Server configuration example

Regardless of the programming SDK you use, Azure MCP server must be configured in the app context for tools to be available. The essential configuration resembles the following:

JSON

```
{
  "mcp_servers": {
    "azure-mcp": {
      "type": "local",
      "command": "npx",
      "args": ["-y", "@azure/mcp@latest", "server", "start"],
      "tools": ["*"]
    }
  }
}
```

The `tools: ["*"]` parameter is essential - it enables all tools from the MCP server for the session.

Integration examples

The following examples show how to integrate the SDK in different languages.

⚠ Note

For faster startup, you can install Azure MCP Server globally using `npm install -g @azure/mcp@latest`.

Python

Installation

Install the Python SDK package using pip.

Bash

```
pip install github-copilot-sdk
```

Sample code

The following code demonstrates a complete flow:

Python

```
import asyncio
from copilot import CopilotClient
from copilot.generated.session_events import SessionEventType

async def main():
    # Initialize the Copilot client
    client = CopilotClient({
        "cli_args": [
            "--allow-all-tools",
            "--allow-all-paths",
        ]
    })

    await client.start()

    # Configure Azure MCP server in session config
    azure_mcp_config = {
        "azure-mcp": {
            "type": "local",
            "command": "npx",
            "args": ["-y", "@azure/mcp@latest", "server", "start"],
            "tools": ["*"], # Enable all Azure MCP tools
        }
    }

    # Create session with MCP servers
    session = await client.create_session({
        "model": "gpt-4.1", # Default model; BYOK can override
        "streaming": True,
        "mcp_servers": azure_mcp_config,
    })

    # Handle events
    def handle_event(event):
        if event.type == SessionEventType.ASSISTANT_MESSAGE_DELTA:
            if hasattr(event.data, 'delta_content') and
event.data.delta_content:
                print(event.data.delta_content, end="", flush=True)
            elif event.type == SessionEventType.TOOL_EXECUTION_START:
                tool_name = getattr(event.data, 'tool_name', 'unknown')
                print(f"\n[TOOL: {tool_name}]")

    session.on(handle_event)

    # Send prompt
    await session.send_and_wait({
        "prompt": "List all resource groups in my Azure subscription"
    })

    await client.stop()
```



```
if __name__ == "__main__":  
    asyncio.run(main())
```

The preceding code:

- Initializes the Copilot client.
- Configures the Azure MCP server using `npx`.
- Creates a session with the GPT-4.1 model and streaming enabled.
- Handles events to print assistant responses and tool execution logs.
- Sends a prompt to list Azure resource groups.

 **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 02/25/2026

Connect Azure MCP Server to sovereign clouds

This article shows you how to configure Azure MCP Server to authenticate against a sovereign cloud instead of the Azure public cloud. For example, use these steps when your subscription is in Azure US Government or Azure operated by 21Vianet Cloud (Azure in China), or when you need to provide a custom authority host. For more background about Azure sovereign cloud offerings, see [What are sovereign clouds?](#).

Prerequisites

- Access to an Azure subscription in Azure US Government or Azure in China.
- A client or local setup that can start Azure MCP Server. For setup options, see [Get started with the Azure MCP Server](#).
- At least one supported local authentication tool, such as [Azure CLI](#), [Azure PowerShell](#), or [Azure Developer CLI](#).

Supported sovereign clouds

Azure MCP Server recognizes the following cloud names and maps them to the correct Microsoft Entra authority host.

 Expand table

Cloud	Authority host	Supported aliases
Azure Public Cloud	<code>https://login.microsoftonline.com</code>	<code>AzureCloud</code> , <code>AzurePublicCloud</code> , <code>Public</code> , <code>AzurePublic</code>
Azure US Government	<code>https://login.microsoftonline.us</code>	<code>AzureUSGovernment</code> , <code>USGov</code> , <code>AzureUSGovernmentCloud</code> , <code>USGovernment</code>
Azure in China	<code>https://login.chinacloudapi.cn</code>	<code>AzureChinaCloud</code> , <code>China</code> , <code>AzureChina</code>

Aliases are case-insensitive.

Configure the cloud for Azure MCP Server

You can set the cloud either in the server arguments or through configuration. If you specify the cloud in more than one place, Azure MCP Server resolves the value in this order.

 Expand table

Priority	Source	Setting
1	Command line	<code>--cloud</code>
2	.NET configuration providers	<code>AZURE_CLOUD</code> , <code>azure_cloud</code> , <code>cloud</code> , <code>Cloud</code>
3	Environment variable fallback	<code>AZURE_CLOUD</code>
Default	Fallback	<code>AzurePublicCloud</code>

Configure using a server argument

If your MCP client starts Azure MCP Server for you, add `--cloud` to the server arguments.

JSON

```
{
  "servers": {
    "Azure MCP Server": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y",
        "@azure/mcp@latest",
        "server",
        "start",
        "--cloud",
        "AzureUSGovernment"
      ]
    }
  }
}
```

Replace `AzureUSGovernment` with `AzureChinaCloud` when you connect to the Azure in China.

Configure using environment variables

If you start Azure MCP Server from a shell, or if your MCP client supports environment variables, set `AZURE_CLOUD` with the appropriate value before starting the server.

PowerShell

PowerShell

```
$env:AZURE_CLOUD = "AzureUSGovernment"  
azmcp server start
```

Authenticate your local tools to the same cloud

Before you start Azure MCP Server, sign in to the same sovereign cloud in the local tools that Azure MCP Server can use for authentication.

Azure CLI

Use Azure CLI when your workflow relies on `az login` or the Azure CLI credential.

Bash

```
az cloud set --name AzureUSGovernment  
az login
```

For Azure in China, replace `AzureUSGovernment` with `AzureChinaCloud`.

Configure a self-hosted remote server

If you deploy Azure MCP Server as a remote MCP server, make sure the host environment is configured for the target sovereign cloud before you publish the endpoint.

Set these environment variables on the host or container:

- `AZURE_CLOUD`.
- `AzureAd__ClientCredentials__0__TokenExchangeUrl`.

If you use one of the Microsoft-provided Azure Container Apps templates, this value is set for you. The template derives the correct value from the target cloud before it publishes the remote endpoint. For examples, see the [Foundry managed identity template](#), the [Copilot Studio managed identity template](#), and the [on-behalf-of template](#).

If you configure the remote server manually, set

`AzureAd__ClientCredentials__0__TokenExchangeUrl` to the token exchange audience for your cloud:

Cloud	Value
Azure Public Cloud	<code>api://AzureADTokenExchange</code>
Azure US Government	<code>api://AzureADTokenExchangeUSGov</code>
Azure in China	<code>api://AzureADTokenExchangeChina</code>

For template-based deployments, review the upstream [Azure MCP Server azd templates](#) if you want to confirm the generated host or container settings before you publish the remote endpoint.

Verify the connection

1. Restart your MCP client after you change the cloud configuration.
2. Run a simple prompt that requires Azure context, such as `List my resource groups`.
3. If the request fails, verify that your local tools are authenticated to the same sovereign cloud and tenant as the Azure subscription you want to use.

Troubleshoot sovereign cloud configuration

If authentication or discovery fails, start with these checks.

1. Verify the cloud configuration. Confirm that the cloud name or authority host is correct.
2. Check local authentication. Make sure you authenticated the local toolchain to the correct cloud.
3. Verify the tenant. Confirm that the tenant belongs to the sovereign cloud subscription you want to use. For example, run `az account show --query tenantId -o tsv`.
4. Check network connectivity. Confirm that you can reach the correct authority host for your cloud.
 - Azure US Government: `https://login.microsoftonline.us`
 - Azure in China: `https://login.chinacloudapi.cn`
5. For remote deployments, confirm that both `AZURE_CLOUD` and `AzureAd__ClientCredentials__0__TokenExchangeUrl` are set correctly.

Common error messages

Use the following table to map common failures to likely causes.

 Expand table

Error	Likely cause	Resolution
Authentication failed	Your local tool is still signed in to the wrong cloud, or not signed in at all.	Reauthenticate with the correct cloud by using <code>az login</code> , <code>Connect-AzAccount</code> , or <code>azd auth login</code> .
Cannot connect to authority host	The cloud value or custom authority host URL is invalid, or the endpoint is unreachable.	Verify the cloud name, custom authority host, and network connectivity.
Invalid tenant	The tenant doesn't match the sovereign cloud subscription.	Confirm the tenant ID and sign in again with the correct tenant and cloud.
The primary access token is from the wrong issuer	The token was issued for a different tenant than the subscription expects.	Check the active tenant, then restart the client and Azure MCP Server after switching to the correct tenant.

Verify the effective configuration

If the problem persists, start the server with debug logging and confirm that Azure MCP Server is using the expected authority host.

Bash

```
azmcp server start --cloud AzureUSGovernment --log-level Debug
```

The debug output should show the authority host being used for authentication.

Additional checks for enterprise environments

If you work behind enterprise network controls, also verify the following items.

- Firewall or proxy rules are not blocking outbound traffic to the sovereign cloud authority host and Azure Resource Manager endpoint.
- The correct account is being selected when multiple credentials are available.
- Private endpoint resources are reachable through VPN, ExpressRoute, or another approved network path.

If you still have problems, see the Azure MCP Server [troubleshooting guide](#).

Related content

- [Get started with the Azure MCP Server](#)
- [Azure MCP Server overview](#)
- [Deploy a self-hosted remote Azure MCP Server and connect to it using Copilot Studio](#)
- [Deploy a self-hosted remote Azure MCP Server and connect to it using Microsoft Foundry](#)

 **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 03/31/2026

Deploy a self-hosted remote Azure MCP Server and connect to it using Microsoft Foundry

Deploy the [Azure MCP Server](#) as a self-hosted remote server over HTTPS. Agents in [Microsoft Foundry](#) and [Microsoft Copilot Studio](#) can securely connect to and call MCP tools using the deployed Azure MCP Server to run Azure operations. This article focuses on the Microsoft Foundry connection scenario.

Prerequisites

- Azure subscription with **Owner** or **User Access Administrator** access
- [Azure Developer CLI \(azd\)](#)
- The list of Azure MCP Server tool areas (namespaces) you wish to enable (see [azmcp-commands.md](#)). The reference template in this article uses the `storage` namespace.
- [Azure Storage account](#)
- [Microsoft Foundry project](#)

Azure MCP Server template

Use the [Azure MCP Server - ACA with Managed Identity](#) `azd` template to deploy Azure MCP Server to Azure Container Apps with storage tools and a managed identity for secure access to Azure Storage. The Azure Developer CLI (`azd`) is an open source tool that simplifies provisioning and deploying Azure resources and offers concise commands (`azd deploy`, `azd provision`) that map to key stages in your development workflow.

Deploy the Azure MCP server

Deploy the Azure MCP server to Azure Container Apps:

1. Clone and initialize the `azmcp-foundry-aca-mi` template with the `azd init` command.

Bash

```
azd init -t azmcp-foundry-aca-mi
```


When prompted, enter an environment name.

2. Run the template with the `azd up` command.

```
Bash
```

```
azd up
```

`azd` prompts you for the following:

- **Subscription:** Select the subscription for the provisioned resources (listed below).
- **Project resource ID:** The Azure resource ID of the Microsoft Foundry project used for agent integration.
- **Storage Account resource ID:** The Azure resource ID of the storage account the MCP server accesses.
- **Resource group:** Create or select a resource group to store the provision resources.

`azd` uses the template files to provision the following resources and configurations:

- **Azure Container App:** Runs the Azure MCP server and provides the storage namespace.
- **Microsoft Entra ID role assignments:** Assign roles to the Azure Container Apps managed identity for outbound authentication to the storage account you specify with the storage resource ID:
 - Reader: Read-only access to storage account properties.
 - Storage Blob Data Reader: Read-only access to blob data.
- **Entra app registration:** Provides OAuth 2.0 authentication for clients like agents that have the `Mcp.Tools.ReadWrite.All` role. The template assigns this role to the managed identity of the Microsoft Foundry project you specify.
- **Application Insights:** Provides telemetry and monitoring.

Deployment output

1. After deployment finishes, retrieve the environment variables for the `azd` project using the `azd env get-values` command.

```
Bash
```

```
azd env get-values
```

Example output:

text

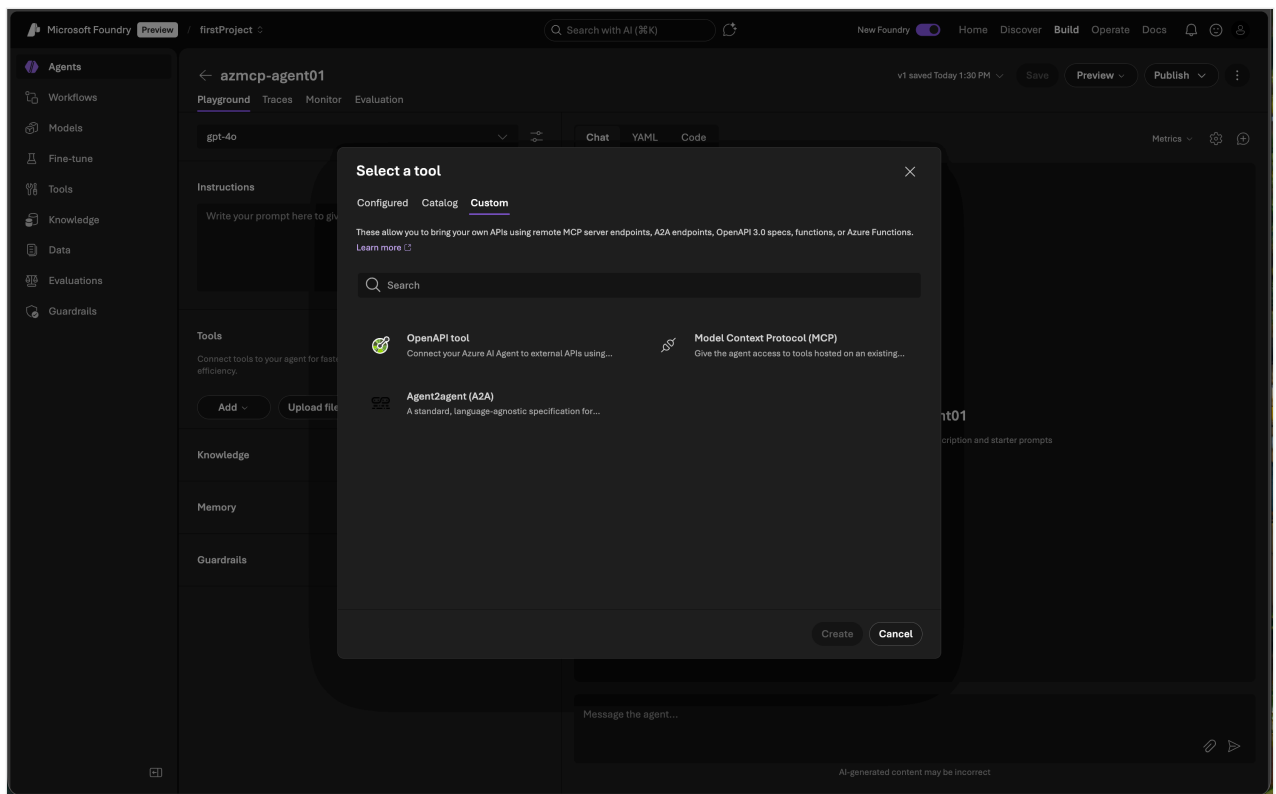
```
CONTAINER_APP_URL="https://azure-mcp-storage-server.<your-container-app-name>.eastus2.azurecontainerapps.io"  
ENTRA_APP_CLIENT_ID=<your-app-client-id>  
ENTRA_APP_IDENTIFIER_URI="api://<your-app-client-id>"  
ENTRA_APP_OBJECT_ID=<your-app-object-id>  
ENTRA_APP_ROLE_ID=<your-app-role-id>  
ENTRA_APP_SERVICE_PRINCIPAL_ID=<your-app-service-principal-id>
```

2. Copy the `CONTAINER_APP_URL` and `ENTRA_APP_CLIENT_ID` values to use in the next section, or leave the terminal open for reference.

Use the Azure MCP server from Foundry agent

After deployment, connect your Foundry agent to the Azure MCP Server running on Azure Container Apps. The agent authenticates using its managed identity to gain access to the configured Azure Storage tools.

1. Go to your Foundry project at <https://ai.azure.com/nextgen>.
2. Select **Build** → **Create agent**.
3. Select **+ Add** in the tools section.
4. Select the **Custom** tab.
5. Select **Model Context Protocol**, then select **Create**.



Add Model Context Protocol tool

Third-party services you connect to via Model Context Protocol are Non-Microsoft Products under the Product Terms. Use third-party services at your own risk and subject to third-party license terms and privacy policies.

Name *

Remote MCP Server endpoint * ⓘ

Authentication * ⓘ

Microsoft Entra

Type * ⓘ

Project Managed Identity

Audience * ⓘ

Connect
Cancel

6. Configure the MCP connection values:

- **Name:** Provide a name for the tool.
- **Remote MCP Server:** Enter the `CONTAINER_APP_URL` value from the `azd` output for the tool endpoint.
- **Authentication:** Select **Microsoft Entra** → **Project Managed Identity**.
- **Type:** Select **Project Managed Identity**.
- **Audience:** Enter the `ENTRA_APP_IDENTIFIER_URI` value from the `azd` output.

Select **Connect** to associate the connection with the agent.

The agent is ready to assist you. It answers questions and uses tools from the Azure MCP Server to perform Azure operations for you.

Clean up resources

Run `azd down` to delete Azure resources.

Bash

```
azd down
```

Explore the Bicep modules

The `azd` template includes the following Bicep modules:

- *main.bicep* orchestrates deployment of all resources.
- *aca-infrastructure.bicep* deploys the container app hosting Azure MCP Server.
- *aca-role-assignment-resource-storage.bicep* assigns Azure Storage RBAC roles to the container app's managed identity on the storage account specified by the input storage account resource ID.
- *entra-app.bicep* creates an Entra app registration and a custom app role for OAuth 2.0 authentication.
- *aif-role-assignment-entraapp.bicep* assigns the Entra app role to the Foundry project's managed identity (specified by the input Foundry resource ID) for Azure MCP Server access.
- *application-insights.bicep* deploys Application Insights for telemetry and monitoring when enabled.

Related content

- [Deploy a remote Azure MCP Server and connect to it using Copilot Studio](#)

 **Note:** This article contains content created with AI. [Learn more](#)

Deploy a self-hosted remote Azure MCP Server and connect to it using Copilot Studio

Deploy the [Azure MCP Server](#) over HTTPS as a self-hosted remote server. This setup lets AI agents in [Microsoft Foundry](#) and [Microsoft Copilot Studio](#) can securely connect to and call MCP tools using the deployed Azure MCP Server to run Azure operations. This article focuses on the Copilot Studio connection scenario.

Prerequisites

- Power Platform license that includes:
 - Copilot Studio
 - Power Apps
- Azure subscription with **Owner** or **User Access Administrator** permissions
- [Azure Developer CLI \(azd\)](#)
- The list of Azure MCP Server tool areas (namespaces) you wish to enable (see [azmcp-commands.md](#)). The reference template in this article uses the `storage` namespace.

Azure MCP Server template

This article uses the [Azure MCP Server - ACA with Copilot Studio agent](#) `azd` template to deploy the server to Azure Container Apps. The template enables storage tools and a managed identity for secure access to Azure Storage. The Azure Developer CLI (`azd`) is an open source tool that simplifies provisioning and deploying Azure resources and offers concise commands (`azd deploy`, `azd provision`) that map to key stages in your development workflow.

Deploy the Azure MCP server

Deploy the Azure MCP server to Azure Container Apps:

1. Clone and initialize the `azmcp-copilot-studio-aca-mi` template using `azd`.

Bash

```
azd init -t azmcp-copilot-studio-aca-mi
```

When prompted, enter an environment name.

2. Run the template with the `azd up` command.

Bash

```
azd up
```

`azd` prompts you for the following:

- **Subscription:** Select the subscription for the provisioned resources (listed below).
- **Resource Group:** The resource group in which to create the resources. You can create a new resource group on demand during this step.

`azd` uses the template files to provision the following resources and configurations:

- **Azure Container App:** Runs the Azure MCP Server and provides the storage namespace.
- **User-assigned managed identity:** A managed identity with the **Subscription Reader** role assigned to the container app and used by the Azure MCP server to make tool calls.
- **Entra App Registration (Client):** For the Power Apps custom connector to connect to the remote Azure MCP Server.
- **Application Insights:** Provides telemetry and monitoring.

Deployment output and configuration

1. After deployment completes, retrieve `azd` environment variables with the `azd env get-values` command.

Bash

```
azd env get-values
```

Example output:

text

```
AZURE_RESOURCE_GROUP="<your-resource-group-name>"
AZURE_SUBSCRIPTION_ID="<your-subscription-id>"
AZURE_TENANT_ID="<your-tenant-id>"
CONTAINER_APP_NAME="<your-container-app-name>"
CONTAINER_APP_URL="https://azure-mcp-storage-server.<your-container-app-name>.westus3.azurecontainerapps.io"
ENTRA_APP_CLIENT_ID="<your-client-app-registration-client-id>"
ENTRA_APP_SERVER_CLIENT_ID="<your-server-app-registration-client-id>"
```

2. You also need to add the created API scope as one of the permissions of the client app registration. Go to the Azure portal and search for the client app registration using the `ENTRA_APP_CLIENT_CLIENT_ID` output value.
3. Go to the API permissions blade and select **Add a permission**.
4. In the My APIs tab, select the **Server app registration** and add the `Mcp.Tools.ReadWrite` scope.

Call tools from Copilot Studio agent

The Copilot Studio agent connects to MCP servers by using a custom connector.

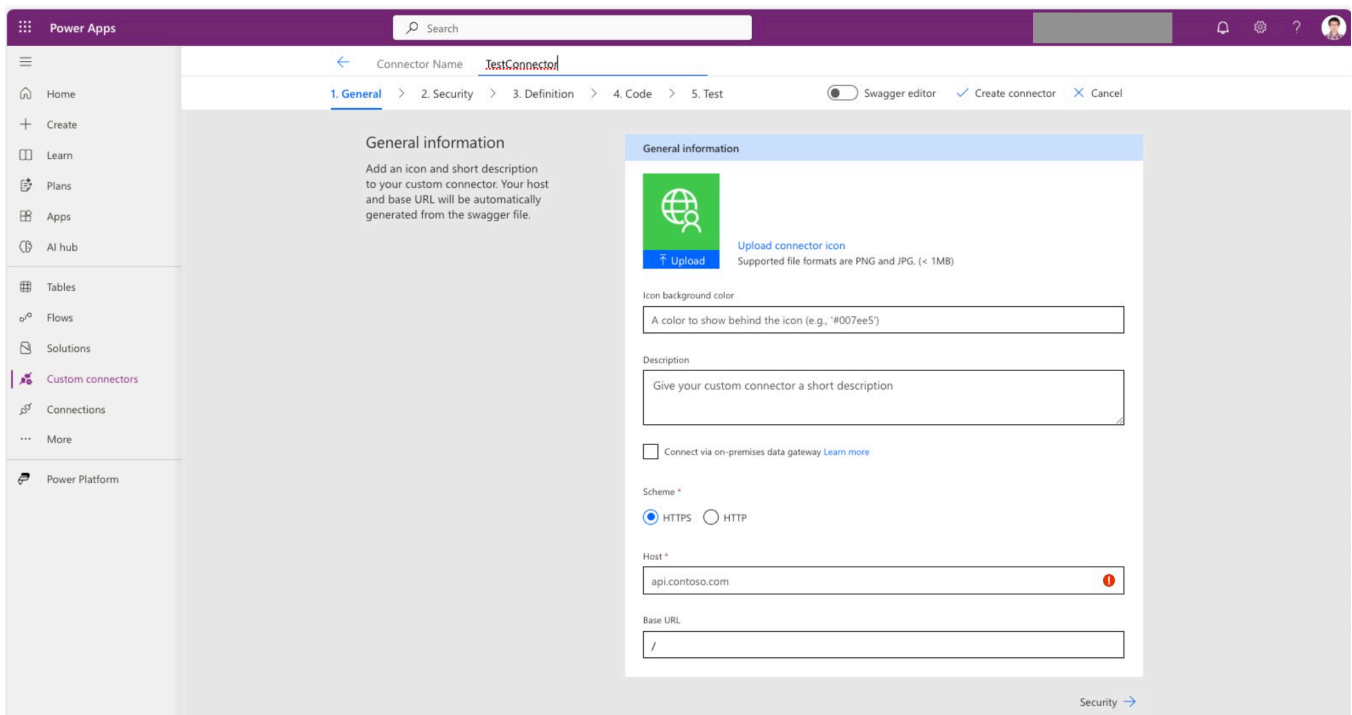
Configure a custom connector

1. Sign in to [Power Apps](#) and select the environment to host the custom connector.
2. Create a new custom connector using the **Create from blank** option. To learn more about custom connector configuration, see [create custom connector from scratch](#).
3. Complete the following sections for each step of the connector creation workflow.

General

On the **General** step:

- Provide a descriptive **Name** and **Description** for the custom connector.
- Set **Scheme** to `HTTPS`.
- Set **Host** to the `CONTAINER_APP_URL` value from the `azd` output.



Security

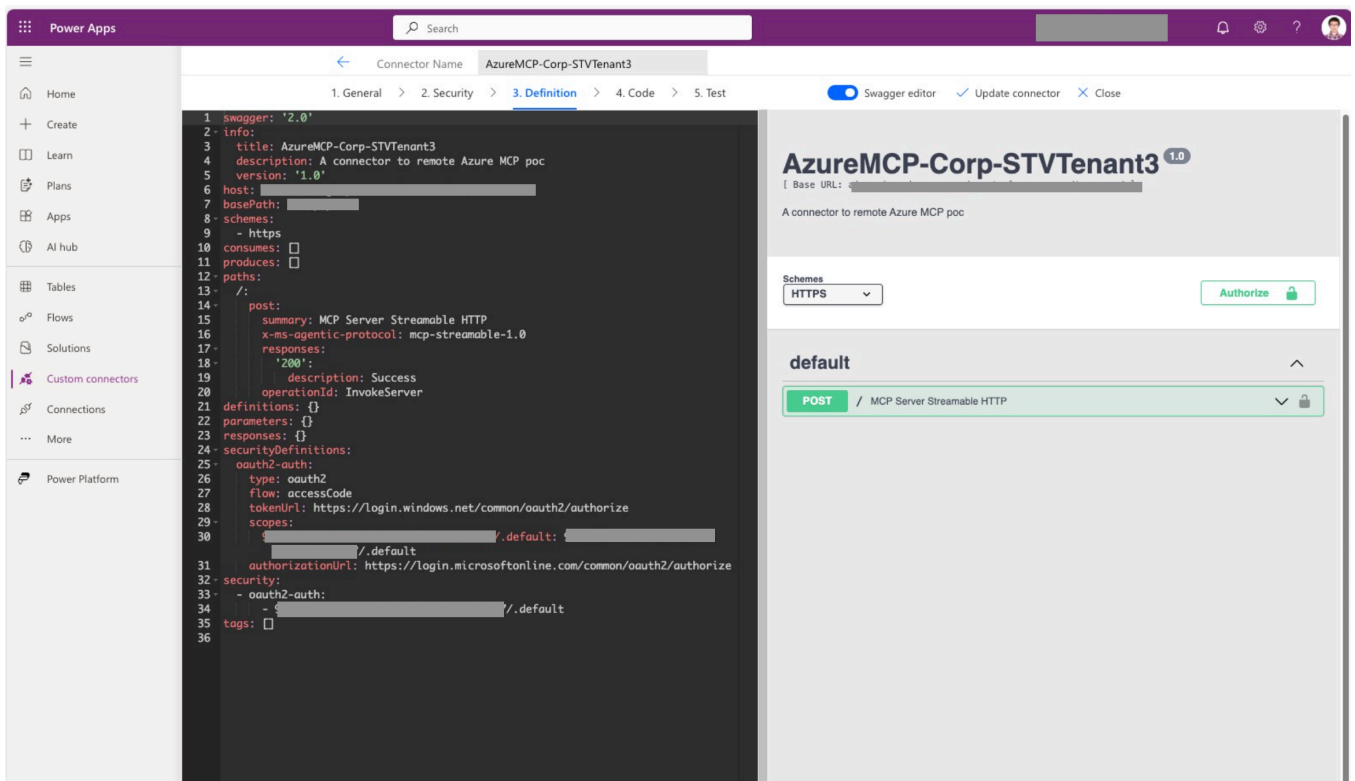
Skip the Security step for now and proceed to the **Definition** step.

Definition

1. Toggle **Swagger editor** to enter the editor view.
2. In the editor view:
 - Expose a POST method at the root path with a custom `x-ms-agentic-protocol: mcp-streamable-1.0` property. This property is required for the custom connector to interact with the API by using the MCP protocol.

ⓘ Note

See the [custom connector swagger example](#) [↗] for a reference template.



Custom connector: Security

Return to the **Security** step to configure authentication:

 Expand table

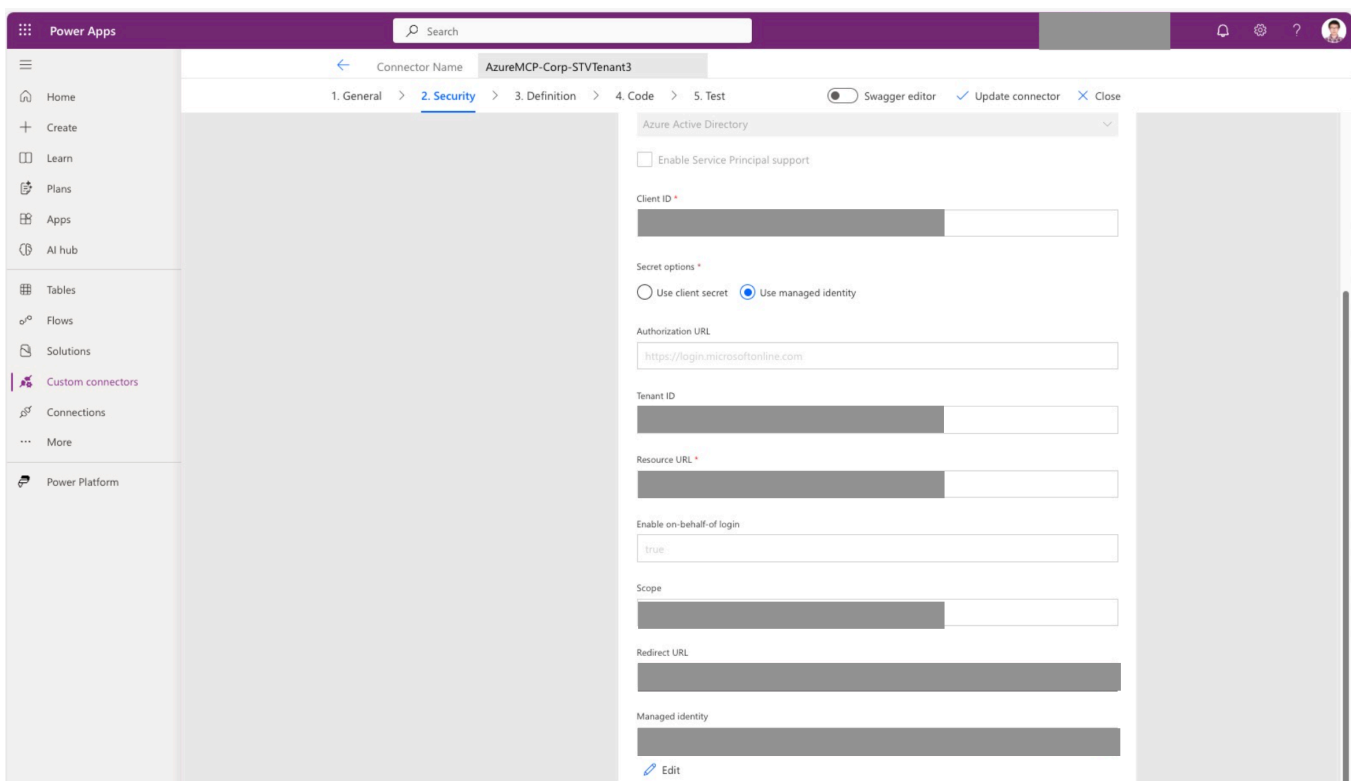
Parameter	Value	Notes
Authentication type	OAuth 2.0	Required
Identity provider	Azure Active Directory	Required
Client ID	ENTRA_APP_CLIENT_CLIENT_ID from azd output	Client app registration ID
Secret option	Use client secret OR Use managed identity	See below for setup
Authorization URL	https://login.microsoftonline.com	Default value
Tenant ID	AZURE_TENANT_ID from azd output	Your Azure tenant ID
Resource URL	ENTRA_APP_SERVER_CLIENT_ID from azd output	Server app registration client ID (not a URL)
On-behalf-of login	Enabled	Required
Scope	<ENTRA_APP_SERVER_CLIENT_ID>/.default	Format: {server_client_id}/.default

Secret option setup:

- **If using client secret:** Create a client secret in the client app registration (Azure portal). Copy the secret value and paste it into the client secret field.
- **If using managed identity:** Proceed with remaining steps until the custom connector is created.

Same-tenant requirement: The client and server app registrations must be in the same tenant for simplified authentication. For cross-tenant scenarios, see [Known issues](#).

Select **Create connector** and wait for completion. After creation, the UI shows a redirect URL and, if selected, a managed identity.



The screenshot shows the 'Security' configuration page for a custom connector named 'AzureMCP-Corp-STV/Tenant3' in the Power Apps environment. The page is divided into several sections for configuring authentication and authorization. The 'Secret options' section has 'Use managed identity' selected. The 'Authorization URL' is set to 'https://login.microsoftonline.com'. Other fields like 'Client ID', 'Tenant ID', 'Resource URL', 'Scope', 'Redirect URL', and 'Managed identity' are present but redacted with grey bars. The 'Enable on-behalf-of login' checkbox is checked. The left sidebar shows the 'Custom connectors' section under 'Power Apps'.

Power Apps

Search

Connector Name: AzureMCP-Corp-STV/Tenant3

1. General > 2. Security > 3. Definition > 4. Code > 5. Test

Swagger editor Update connector Close

Azure Active Directory

☐ Enable Service Principal support

Client ID *

Secret options *

☐ Use client secret ☒ Use managed identity

Authorization URL

https://login.microsoftonline.com

Tenant ID

Resource URL *

Enable on-behalf-of login

true

Scope

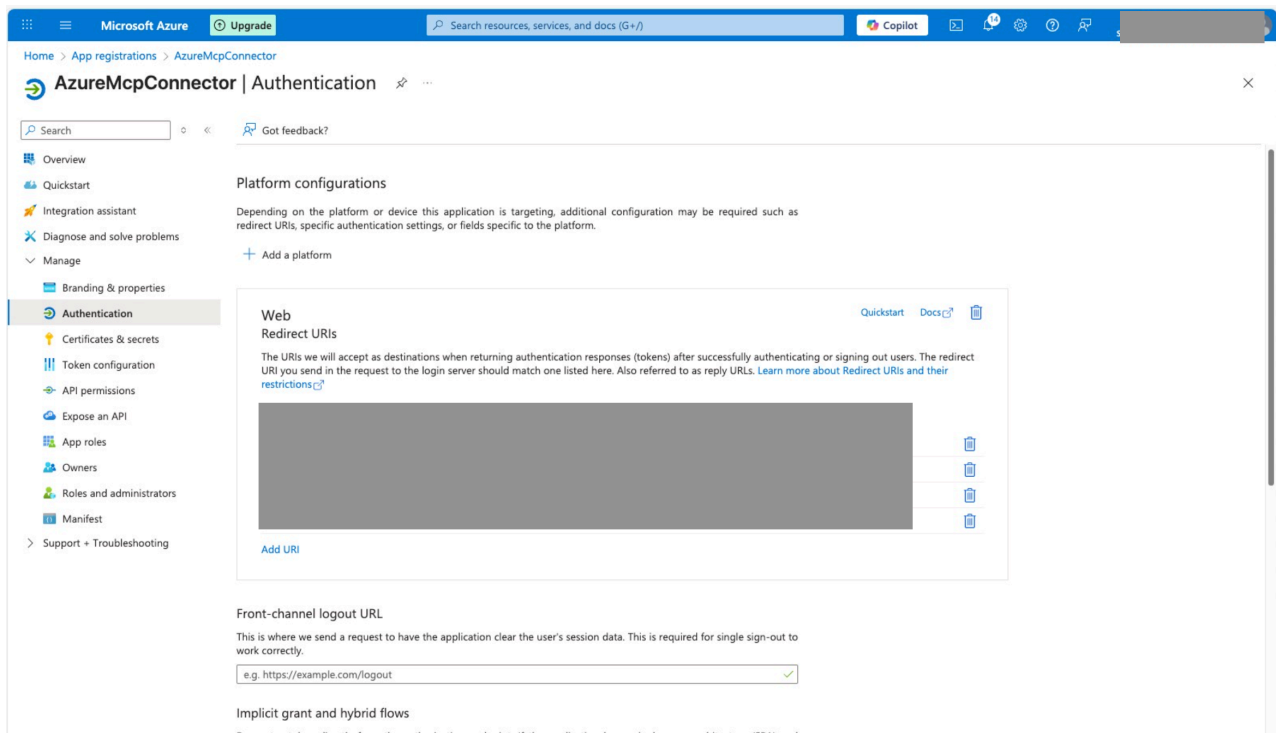
Redirect URL

Managed identity

Edit

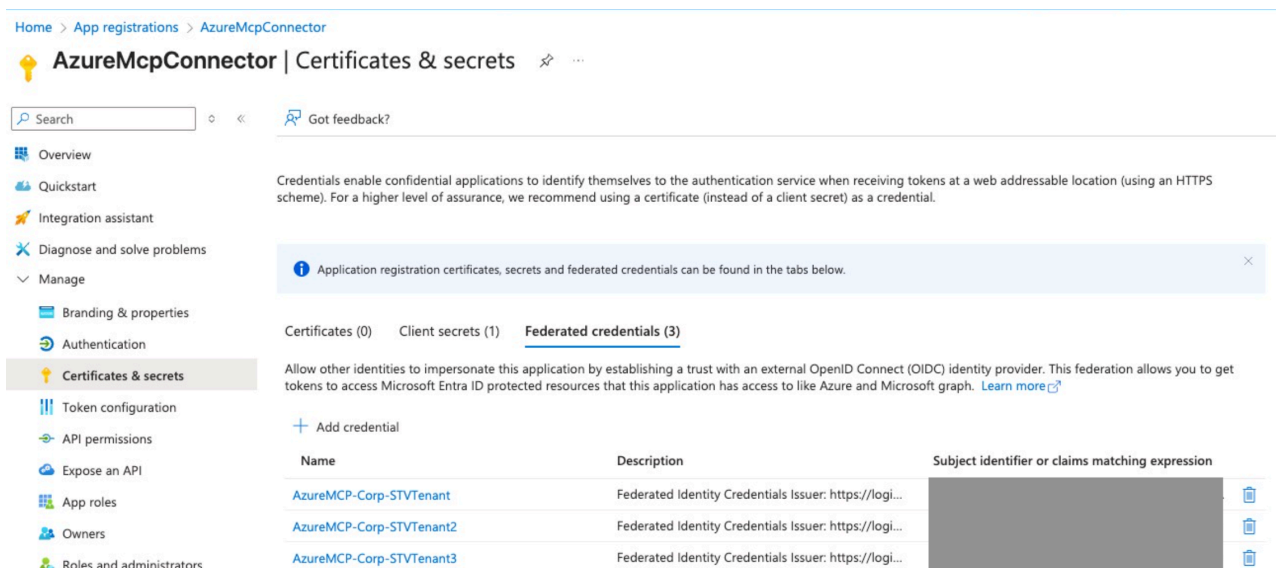
App registration: Configure redirect URI and credentials

1. In the Azure portal, add a redirect URI under the Web platform in the client app registration.



2. If you chose **Use managed identity** on the **Security** step, create a federated credential in the client app registration.

- Select **Other issuer** as the scenario.
- Copy the **issuer** and **subject** values from the custom connector into the credential fields.
- Provide a descriptive **Name** and **Description**, then select **Add**.



Test connection

1. Open the custom connector, select **Edit**, and go to the **Test** step.
2. Select any operation and choose **New connection**.

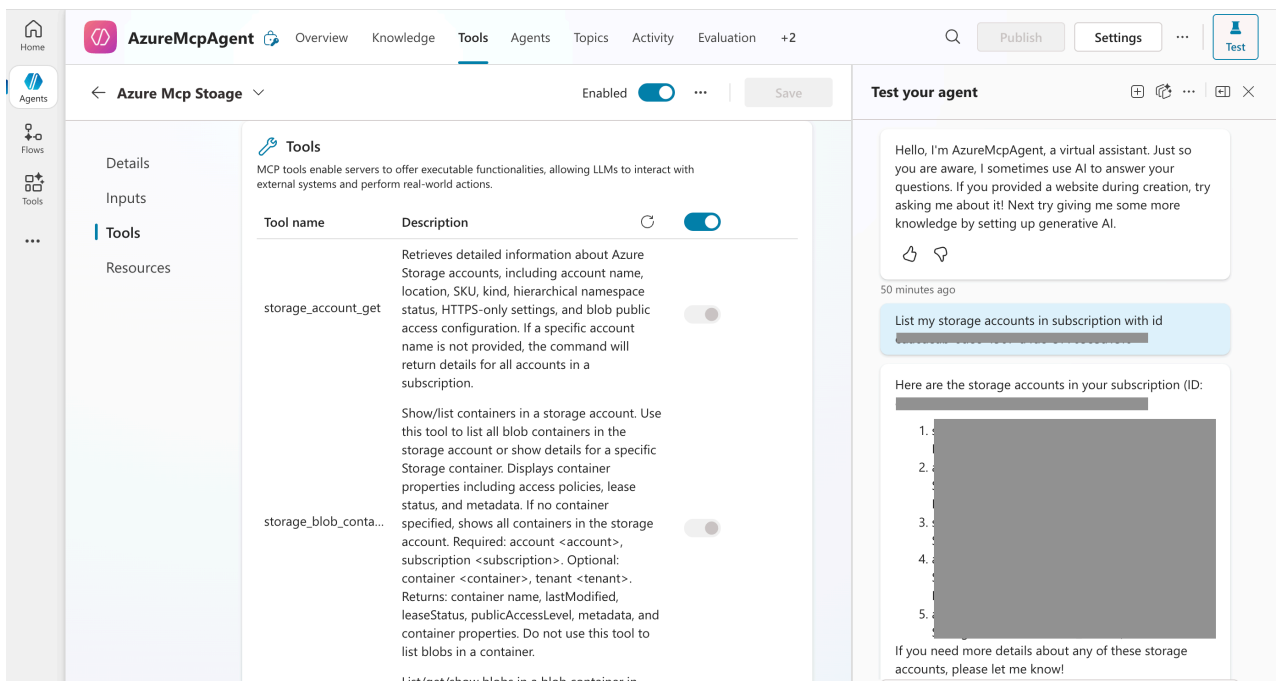
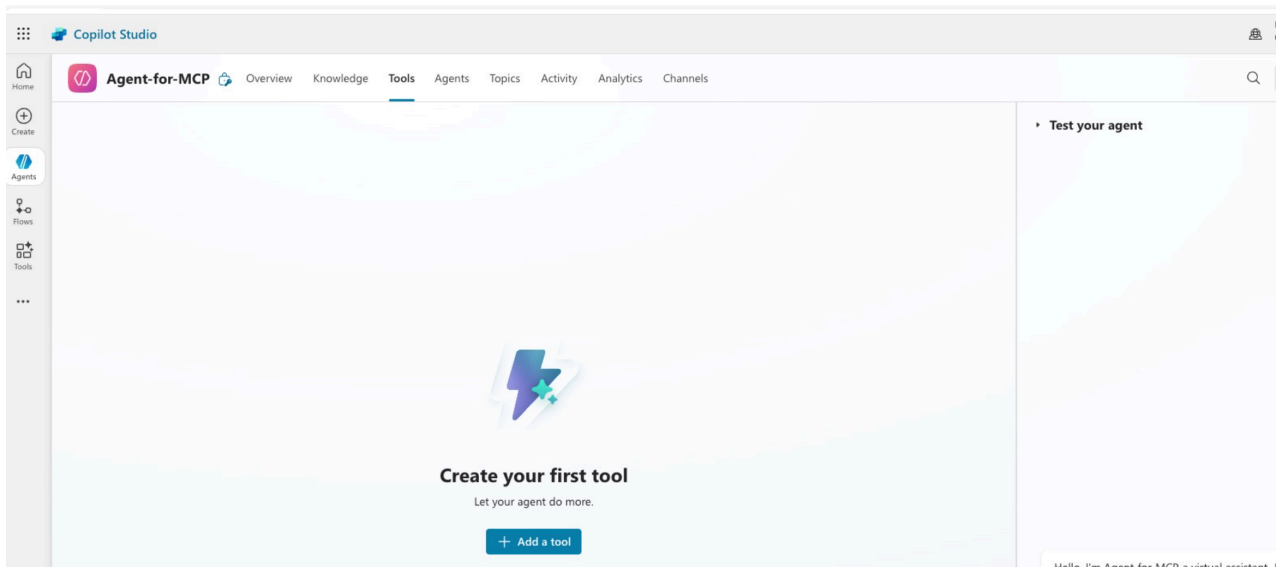
3. Sign in with the user account you plan to use to access the MCP tools. You might see a dialog requesting consent or an admin approval prompt. If you are unsure, see [Known issues](#).

If sign-in succeeds, the UI shows the connection is created successfully. If you encounter an error during sign-in, see [Known issues](#) and troubleshoot with your tenant admin.



Call an Azure MCP tool in Copilot Studio test playground

1. Sign in to [Copilot Studio](#) and select the environment to host the Copilot Studio agent. Create a new agent or use an existing one.
2. Open the agent details and select the **Tools** tab.
3. Select **Add a tool**.
4. Search for your custom connector name and add it.
5. After you add the custom connector, the Copilot Studio Agent attempts to list the tools from the MCP server. If successful, you see the available tool list under the connector.
6. Select **Test** to start a test playground session.
7. Prompt the agent to call an MCP tool, for example to list storage accounts in the subscription.



Clean up resources

Run the following command to delete the Azure resources this template created when you don't need them.

Bash

```
azd down
```

! Note

`azd` cannot delete the Entra app registrations created by this template. Delete the Entra app registrations by searching for the `ENTRA_APP_CLIENT_CLIENT_ID` and the `ENTRA_APP_SERVER_CLIENT_ID` values in the Azure portal and then delete the corresponding app registrations.

Delete the Copilot Studio agent, Power Apps custom connector, and connection to clean up Power Platform resources.

Template structure

The `azd` template includes these Bicep modules:

- `main.bicep` - Orchestrates deployment of all resources.
- `aca-storage-managed-identity.bicep` - Creates a user-assigned managed identity.
- `aca-storage-subscription-role.bicep` - Assigns an Azure RBAC role to the user-assigned managed identity. It defaults to the Subscription Reader role.
- `aca-infrastructure.bicep` - Deploys the Container App hosting the Azure MCP Server.
- `entra-app.bicep` - Creates Entra app registrations.
- `application-insights.bicep` - Deploys Application Insights for telemetry and monitoring when enabled.

Known issues

- **Single-tenant requirement:** The Power Apps custom connector doesn't support authenticating users from multiple tenants, so set the client app registration to accept only users from its tenant.
- **Consent options:** During authentication, the user or a tenant admin grants the client app access to their data. Learn more in [application consent experience](#). You can give consent in several ways:
 - A user can give consent during sign-in just for that user. Tenant security policy might block this.
 - A tenant admin can give consent for all users in the tenant in the client app registration under the **API permissions** blade in Azure portal.
 - Add the client app registration as a preauthorized client app in the server app registration under the **Expose an API** blade in Azure portal.
- **Cross-tenant scenario:** If the client app registration and server app registration are in different tenants, you might see the following error when you try to create the connection:

- "The app is trying to access a service 'server_app_registration_client_id'(server_app_registration_display_name) that your organization 'client_app_registration_tenant' lacks a service principal for."
- **Resolution:** A tenant admin of the client app registration must provision a service principal for the server app registration in that tenant:

Bash

```
az ad sp create --id <server_app_registration_client_id>
```

- After provisioning, create the connection again. The consent flow triggers.
- If the Power Apps environment has a tenant isolation policy, it blocks data flow when the client or server app registrations are in different tenants. Learn how to add exception rules to allow this data flow in [cross tenant restrictions](#).

Related content

- [Deploy a remote Azure MCP Server and connect to it using Microsoft Foundry](#)

 **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 12/13/2025

Deploy Azure MCP Server with on-behalf-of authentication

Deploy [Azure MCP Server](#) as a self-hosted remote server over HTTPS on Azure Container Apps. This article uses the on-behalf-of (OBO) authentication model, which lets the server call Azure services by using the identity of the signed-in user rather than the server's own managed identity. Agents in [Microsoft Foundry](#) and [Microsoft Copilot Studio](#) can connect to the deployed server and invoke Azure MCP tools that operate with the user's own permissions and access.

How the OBO flow works

The on-behalf-of flow is distinct from the managed identity approach used in other Azure MCP Server templates:

- **Managed identity approach:** The server authenticates to downstream Azure services by using its own managed identity. All users share the permissions granted to that identity. For a Microsoft Foundry example that uses this model, see [Deploy a remote Azure MCP Server and connect using Microsoft Foundry](#).
- **OBO approach:** When a user authenticates with the server, the server exchanges the user's token for a new token scoped to a downstream Azure service. The server calls Azure services *on behalf of* the user, so each user's own Azure permissions determine what they can do.

The template provisions two [Microsoft Entra](#) app registrations to enable this flow:

- **Server app registration:** Exposed to clients as the OAuth 2.0 resource. When a user's token arrives, the server uses a federated identity credential (backed by a managed identity) to perform the OBO token exchange to access downstream APIs like Azure Resource Manager and Azure Storage.
- **Client app registration:** Used by external clients (Foundry agents, Copilot Studio custom connectors) to authenticate against the server. The client app is pre-authorized on the server app so users don't need to consent separately.

Prerequisites

- Azure subscription with **Owner** or **User Access Administrator** permissions

- [Azure Developer CLI \(azd\)](#) installed
- [Azure CLI](#) installed
- The list of Azure MCP Server tool namespaces you want to enable. See [azmcp-commands.md](#). The template in this article enables the `storage` namespace by default.

Deploy the Azure MCP Server

This article shows how to use the [azmcp-obo-aca](#) `azd` template to deploy the Azure MCP Server to Azure Container Apps with OBO authentication. Deploy the server:

1. Initialize the `azmcp-obo-template` template by using the `azd init` command.

Bash

```
azd init -t azmcp-obo-template
```

When prompted, enter an environment name.

2. Run the template by using the `azd up` command.

Bash

```
azd up
```

`azd` prompts you for the following values:

- **Subscription:** Select the subscription for the provisioned resources.
- **Resource group:** Create or select a resource group to hold the resources.

`azd` uses the template files to provision the following resources and configurations:

- **Azure Container App:** Runs the Azure MCP Server with the `storage` namespace enabled.
- **User-assigned managed identity:** Provides a client credential for the server app registration through a [federated identity credential](#). The server uses this identity to perform the OBO token exchange.
- **Entra app registration (server):** The OAuth 2.0 resource exposed to clients. Has the `Mcp.Tools.ReadWrite` scope, and carries Azure Resource Manager and Azure Storage API permissions for the OBO exchange.
- **Entra app registration (client):** Used by clients such as Foundry agents and Power Apps custom connectors to authenticate with the server. Pre-authorized on the server app to eliminate the need for user consent.

- **Application Insights:** Provides telemetry and monitoring.

Retrieve deployment outputs

After the deployment finishes, use the `azd env get-values` command to get the `azd` environment variables.

Bash

```
azd env get-values
```

Example output:

text

```
AZURE_RESOURCE_GROUP="<your-resource-group-name>"
AZURE_SUBSCRIPTION_ID="<your-subscription-id>"
AZURE_TENANT_ID="<your-tenant-id>"
CONTAINER_APP_NAME="<your-container-app-name>"
CONTAINER_APP_URL="https://<your-container-app-name>.<region>.azurecontainerapps.io"
ENTRA_APP_CLIENT_CLIENT_ID="<client-app-registration-id>"
ENTRA_APP_SERVER_CLIENT_ID="<server-app-registration-id>"
```

Keep this output available. You need these values in the sections that follow.

Grant admin consent and add the API scope

After deployment, complete two required configuration steps before clients can connect.

Add the API scope to the client app registration

The client app registration needs permission to call the server app's `Mcp.Tools.ReadWrite` scope.

1. In the Azure portal, search for the client app registration by using the `ENTRA_APP_CLIENT_CLIENT_ID` value.
2. Go to **API permissions** → **Add a permission** → **My APIs** tab.
3. Select the server app registration and add the `Mcp.Tools.ReadWrite` scope.
4. Select **Grant admin consent** to apply the permission to all users.

📌 **Note**

If the server app registration doesn't appear under **My APIs**, the app might still be propagating. Wait a few minutes and refresh, or see the [Troubleshooting](#) section.

Grant admin consent for downstream API permissions

The server app registration has Azure Resource Manager and Azure Storage API permissions configured, but these permissions require admin consent before the OBO token exchange can succeed.

1. In the [Azure portal](#), search for the server app registration by using the `ENTRA_APP_SERVER_CLIENT_ID` value.
2. Go to **API permissions**.
3. Select **Grant admin consent for <your tenant>** and confirm.

ⓘ Note

If the **Grant admin consent** button is unavailable, your account lacks sufficient permissions. This template requires an Azure subscription with **Owner** or **User Access Administrator** access.

Alternatively, use the Azure CLI:

Bash

```
az ad app permission admin-consent --id <ENTRA_APP_SERVER_CLIENT_ID>
```

Connect to the server

After deploying and completing the post-deployment configuration, you can connect clients to the server. Select the option that fits your scenario.

C# client app

The template includes a .NET console app in the `client/` folder that you can use to verify the deployment locally. The app authenticates interactively through the browser by using the client app registration, connects to the MCP server, lists the available tools, and optionally calls the `storage_account_get` tool.

Prerequisites: [.NET 10 SDK](#)

1. In the `client/` folder, open `appsettings.json` and set the following values by using the `azd env get-values` output:

JSON

```
{
  "McpServer": {
    "Url": "<CONTAINER_APP_URL>"
  },
  "EntraClientClientId": "<ENTRA_APP_CLIENT_CLIENT_ID>",
  "SubscriptionId": "<AZURE_SUBSCRIPTION_ID>"
}
```

💡 Tip

You can also create an `appsettings.Development.json` file with your local values and set the `DOTNET_ENVIRONMENT` environment variable to `Development` to load it without modifying the committed file.

2. From the `client/` folder, run the app:

Bash

```
dotnet run
```

The app opens a browser window for you to sign in. After sign-in, it connects to the MCP server and prints the list of available tools.

3. To also call the `storage_account_get` tool and list storage accounts in your subscription, pass the `--list-accounts` flag:

Bash

```
dotnet run -- --list-accounts true
```

If you encounter authentication errors such as `MsalUiRequiredException`, see the [Troubleshooting](#) section.

Add more Azure tools

The template enables the `storage` namespace by default. To enable additional tool namespaces:

1. Identify the API permissions required for the tools you want. See the [API permissions reference](#).
2. Add the permissions to the server app registration by using the Azure CLI:

Bash

```
az ad app permission add \  
  --id <ENTRA_APP_SERVER_CLIENT_ID> \  
  --api <downstream-api-id> \  
  --api-permissions <permission-id>=Scope
```

3. Grant admin consent for the new permissions:

Bash

```
az ad app permission admin-consent --id <ENTRA_APP_SERVER_CLIENT_ID>
```

4. Update the Container App environment variables to pass the additional namespace flags to the server startup command.

Clean up resources

Run `azd down` to delete the Azure resources created by this template.

Bash

```
azd down
```

ⓘ Note

`azd down` doesn't delete the Entra app registrations. After running `azd down`, manually delete them in the Azure portal by searching for the `ENTRA_APP_CLIENT_CLIENT_ID` and `ENTRA_APP_SERVER_CLIENT_ID` values.

Troubleshooting

The following sections provide details on common errors you might encounter and how to resolve them.

IDW10502: MsalUiRequiredException

text

```
{"status":500,"message":"IDW10502: An MsalUiRequiredException was thrown due to a challenge for the user..."}
```

The server's OBO token exchange failed because admin consent isn't granted for the downstream API permissions on the server app registration. In the Azure portal, find the server app registration (using `ENTRA_APP_SERVER_CLIENT_ID`) → **API permissions** → **Grant admin consent**.

OBO token exchange failures

Check the Entra sign-in logs for details. In the Azure portal, go to **Microsoft Entra ID** → **Monitoring** → **Sign-in logs** → **User sign-ins (non-interactive)**. Look for entries where the application matches your server app registration and the resource matches the downstream Azure API.

Container App errors

Open the Azure portal, go to your Container App → **Monitoring** → **Log stream** to view real-time application logs. Application Insights telemetry is available under **Investigate** → **Search** or via Log Analytics queries on the `requests` and `traces` tables.

ServiceManagementReference error on redeploy

text

```
{"error":{"code":"BadRequest","message":"ServiceManagementReference field is required for Update..."}}
```

This error occurs when running `azd up` on an existing deployment that was originally created without a `serviceManagementReference` value. Add the parameter to `infra/main.parameters.json`:

JSON

```
{  
  "parameters": {
```

```
"serviceManagementReference": {  
  "value": "<your-guid>"  
}  
}  
}
```

ⓘ Note

For a list of other known issues, see [KnownIssues.md](#) [↗] in the template repository.

Related content

- [Deploy a remote Azure MCP Server and connect to it using Microsoft Foundry](#)
- [Deploy a remote Azure MCP Server and connect to it using Copilot Studio](#)

ⓘ **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 03/31/2026

Manage Azure App Service with Azure MCP Server

Manage web applications and configure database connections using natural language conversations with AI assistants through the Azure MCP Server.

[Azure App Service](#) is a fully managed platform for building, deploying, and scaling web applications. It supports multiple programming languages and frameworks with integrated developer tools. While the Azure portal and Azure CLI are powerful, the Azure MCP Server provides a more intuitive way to interact with your App Service resources through conversational AI.

What is the Azure MCP Server?

The Azure MCP Server enables AI agents and assistants to interact with Azure resources through natural language commands using the [Model Context Protocol \(MCP\)](#) [↗](#). Instead of manually navigating portals or writing scripts, you can describe what you want to accomplish, and the AI assistant uses the Azure MCP Server tools to perform the actions.

For Azure App Service developers, this means you can:

- Add a database connection for an App Service
- Diagnose web app issues using built-in detectors
- List available diagnostic detectors for a web app
- Retrieve detailed information about web app deployments
- Retrieve detailed information about web apps
- Retrieve application settings for an App Service web app
- Update application settings for an App Service web app

Prerequisites

To use the Azure MCP Server with Azure App Service, you need:

Azure requirements

- **Azure subscription:** An active Azure subscription. [Create one for free](#) [↗](#).

- **Azure App Service resources:** At least one App Service in your subscription, or permissions to create them.
- **Database resource:** An existing database resource to connect to your App Service.
- **Azure permissions:** Appropriate roles to perform the operations you want. See [Azure Built-in Roles](#).

MCP client requirements

You need an AI assistant or development environment that supports the Model Context Protocol. Choose one:

AI-powered code editors:

- [Cursor](#)
- [Cline](#)
- [IntelliJ IDEA with MCP support](#)
- [Visual Studio](#)
- [Visual Studio Code](#)
- [Windsurf](#)

Programmatic integration:

- [.NET with MCP SDK](#)
- [Python with MCP SDK](#)

For complete setup instructions, see [Get started with Azure MCP Server](#).

Where can you use Azure MCP Server?

The Azure MCP Server works in three primary contexts:

In AI-powered chat and code editors

Use the Azure MCP Server directly within AI assistants and code editors. As you chat about your Azure resources, the AI assistant automatically invokes Azure MCP Server tools to retrieve information, make changes, or answer questions. This is the most common usage pattern.

Get started with:

- [Cline](#)
- [Cursor](#)

- [Eclipse](#)
- [IntelliJ](#)
- [Visual Studio](#)
- [Visual Studio Code](#)
- [Windsurf](#)
- [GitHub Copilot coding agent](#)

In programmatic applications

Integrate the Azure MCP Server into your applications using the MCP SDK. Your app acts as an MCP client and invokes Azure MCP Server tools programmatically. This approach is useful for building custom automation, chatbots, or intelligent applications that need Azure integration.

Get started with:

- [Python](#)
- [.NET](#)

In self-hosted scenarios

Deploy the Azure MCP Server in your own environment for advanced control, security requirements, or custom modifications. You can run it locally, in containers, or integrate it into existing infrastructure. This pattern suits enterprise scenarios requiring air-gapped environments or custom authentication flows.

Learn how to:

- [Deploy a remote MCP server with Microsoft Foundry](#)
- [Deploy a remote MCP server with Copilot Studio](#)

Available tools for Azure App Service

Azure MCP Server provides the following tools for Azure App Service operations:

 Expand table

Tool	Description
<code>appservice database add</code>	Add a database connection for an app using a connection string.

Tool	Description
<code>appservice webapp diagnostic diagnose</code>	Run a diagnostic detector on a web app and return the results.
<code>appservice webapp diagnostic list</code>	List available diagnostic detectors for a web app.
<code>appservice webapp deployment get</code>	Retrieve detailed information about web app deployments.
<code>appservice webapp get</code>	Retrieve detailed information about Azure App Service web apps.
<code>appservice webapp settings get-appsettings</code>	Retrieve application settings for an App Service web app.
<code>appservice webapp settings update-appsettings</code>	Update application settings for an App Service web app.

For detailed information about each tool, including parameters and examples, see [Azure App Service tools for Azure MCP Server](#).

Get started

Ready to use Azure MCP Server with your Azure App Service resources?

- 1. Set up your environment:** Choose an AI assistant or development tool that supports MCP. For setup and authentication instructions, see the links in the [Where can you use Azure MCP Server?](#) section above.
- 2. Start exploring:** Ask your AI assistant questions about your App Service resources or request operations. Try prompts like:
 - "Add a database connection for 'myapp' to a SQL database named 'mydatabase' with Azure SQL server."
 - "Diagnose web app 'myapp' in resource group 'rg-prod' with detector 'Availability'."
 - "List the diagnostic detectors for web app 'myapp' in resource group 'rg-prod'."
 - "Retrieve details for deployment ID '12345' for web app 'myapp'."
 - "Retrieve details for web app 'myapp'."
 - "Retrieve application settings for web app 'myapp'."
 - "Update app setting 'API_URL' for web app 'myapp' to 'https://api.example.com'."
- 3. Learn more:** Review the [Azure App Service tools reference](#) for all available capabilities and detailed parameter information.

Best practices

When using Azure MCP Server with Azure App Service:

- **Use diagnostics to investigate issues:** Run detectors like `Availability` and `CpuAnalysis` to troubleshoot app health before making configuration changes.
- **Use version control for settings:** Track changes to application settings to prevent configuration drift.
- **Verify settings before applying updates:** Use `get-appsettings` to confirm current settings before making changes.

Related content

- [Azure MCP Server overview](#)
- [Get started with Azure MCP Server](#)
- [Azure App Service tools reference](#)
- [Azure App Service documentation](#)
- [App Service best practices](#)

ⓘ **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 03/24/2026

Manage Azure Functions with Azure MCP Server

Manage function apps, review configurations, and troubleshoot deployments using natural language conversations with AI assistants through the Azure MCP Server.

[Azure Functions](#) is a serverless compute service that enables you to run event-driven code without managing infrastructure. While the Azure portal, Azure CLI, and Azure Functions Core Tools are powerful, the Azure MCP Server provides a more intuitive way to interact with your function apps through conversational AI.

What is the Azure MCP Server?

The Azure MCP Server enables AI agents and assistants to interact with Azure resources through natural language commands using the [Model Context Protocol \(MCP\)](#). Instead of manually navigating portals or writing scripts, you can describe what you want to accomplish, and the AI assistant uses the Azure MCP Server tools to perform the actions.

For Azure Functions developers, this means you can:

- Retrieve function app details and settings without navigating the portal
- Check function app status and configuration through simple questions
- Review hostnames and deployment information conversationally
- Troubleshoot function app issues by asking about current state
- Compare function app configurations across environments
- Verify function app settings before and after deployments

Prerequisites

To use the Azure MCP Server with Azure Functions, you need:

Azure requirements

- **Azure subscription:** An active Azure subscription. [Create one for free](#).
- **Azure Functions resources:** At least one function app in your subscription, or permissions to create them.

- **Azure permissions:** Appropriate roles like Contributor or Website Contributor to perform the operations you want. See [Azure Functions security documentation](#).

MCP client requirements

You need an AI assistant or development environment that supports the Model Context Protocol. Choose one:

AI-powered code editors:

- [Cursor](#)
- [Cline](#)
- [IntelliJ IDEA with MCP support](#)
- [Visual Studio](#)
- [Visual Studio Code](#)
- [Windsurf](#)

Programmatic integration:

- [.NET with MCP SDK](#)
- [Python with MCP SDK](#)

For complete setup instructions, see [Get started with Azure MCP Server](#).

Where can you use Azure MCP Server?

The Azure MCP Server works in three primary contexts:

In AI-powered chat and code editors

Use the Azure MCP Server directly within AI assistants and code editors. As you chat about your Azure resources, the AI assistant automatically invokes Azure MCP Server tools to retrieve information, make changes, or answer questions. This is the most common usage pattern.

Get started with:

- [Cline](#)
- [Cursor](#)
- [Eclipse](#)
- [IntelliJ](#)
- [Visual Studio](#)

- [Visual Studio Code](#)
- [Windsurf](#)
- [GitHub Copilot coding agent](#)

In programmatic applications

Integrate the Azure MCP Server into your applications using the MCP SDK. Your app acts as an MCP client and invokes Azure MCP Server tools programmatically. This approach is useful for building custom automation, chatbots, or intelligent applications that need Azure integration.

Get started with:

- [Python](#)
- [.NET](#)

In self-hosted scenarios

Deploy the Azure MCP Server in your own environment for advanced control, security requirements, or custom modifications. You can run it locally, in containers, or integrate it into existing infrastructure. This pattern suits enterprise scenarios requiring air-gapped environments or custom authentication flows.

Learn how to:

- [Deploy a remote MCP server with Microsoft Foundry](#)
- [Deploy a remote MCP server with Copilot Studio](#)

Available tools for Azure Functions

The Azure MCP Server provides one tool specifically designed for Azure Functions operations, enabling you to retrieve function app information through natural language conversations.

Get function app details

Get detailed information about your function apps, including configuration, status, hostnames, and deployment details.

Common scenarios:

- Get function app details to verify configuration before deployments
- Get function app status to check runtime information across environments

- List hostnames for a function app to review custom domains

For detailed information about each tool, including parameters and examples, see [Azure Functions tools for Azure MCP Server](#).

Get started

Ready to use Azure MCP Server with your Azure Functions resources?

1. **Set up your environment:** Choose an AI assistant or development tool that supports MCP. For setup and authentication instructions, see the links in the [Where can you use Azure MCP Server?](#) section above.
2. **Start exploring:** Ask your AI assistant questions about your function apps or request operations. Try prompts like:
 - "Show me details for my function app 'my-functions' in resource group 'my-rg'"
 - "What's the current status of function app 'api-backend'?"
 - "List the hostnames for function app 'public-api'"
3. **Learn more:** Review the [Azure Functions tools reference](#) for all available capabilities and detailed parameter information.

Best practices

When using Azure MCP Server with Azure Functions:

- **Specify resource group:** Always include the resource group name when querying function apps to avoid ambiguity in subscriptions with many resources.
- **Verify before changes:** Use read queries to understand current function app state before making configuration changes or deployments.
- **Compare environments:** Leverage conversational queries to compare function app settings across development, staging, and production environments for configuration consistency.
- **Check status regularly:** Ask about function app status and health as part of your deployment verification workflow to catch issues early.
- **Document configurations:** Use the conversation history to document current function app configurations for team knowledge sharing and troubleshooting.
- **Combine with other tools:** Use Azure MCP Server for quick queries and Azure Functions Core Tools for deployments to optimize your development workflow.

Related content

- [Azure MCP Server overview](#)
- [Get started with Azure MCP Server](#)
- [Azure Functions tools reference](#)
- [Azure Functions documentation](#)
- [Azure Functions best practices](#)

ⓘ **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 01/21/2026

Manage Azure Key Vault with Azure MCP Server

Manage keys, secrets, and certificates using natural language conversations with AI assistants through the Azure MCP Server.

[Azure Key Vault](#) is a cloud service for securely storing and accessing secrets, keys, and certificates. It helps solve problems related to [secrets management](#), [key management](#), and [certificate management](#). While the Azure portal, Azure CLI, and Azure PowerShell are powerful, the Azure MCP Server provides a more intuitive way to interact with your key vaults through conversational AI.

What is the Azure MCP Server?

The Azure MCP Server enables AI agents and assistants to interact with Azure resources through natural language commands using the [Model Context Protocol \(MCP\)](#) [↗](#). Instead of manually navigating portals or writing scripts, you can describe what you want to accomplish, and the AI assistant uses the Azure MCP Server tools to perform the actions.

For Azure Key Vault administrators and developers, this means you can:

- Create, retrieve, and list keys, secrets, and certificates without navigating the portal
- Review cryptographic key properties and certificate expiration dates
- Import certificates into your vaults
- Query Managed HSM settings for high-security deployments

Prerequisites

To use the Azure MCP Server with Azure Key Vault, you need:

Azure requirements

- **Azure subscription:** An active Azure subscription. [Create one for free](#) [↗](#).
- **Azure Key Vault resources:** At least one key vault in your subscription. You can create a key vault using the [Azure CLI](#), [Azure PowerShell](#), or the [Azure portal](#).
- **Azure permissions:** Appropriate [Azure RBAC roles](#) like Key Vault Administrator, Key Vault Secrets Officer, Key Vault Certificates Officer, or Key Vault Crypto Officer to perform the

operations you want. See [Provide access to Key Vault keys, certificates, and secrets with Azure role-based access control](#).

MCP client requirements

You need an AI assistant or development environment that supports the Model Context Protocol. Choose one:

AI-powered code editors:

- [Cursor](#)
- [Cline](#)
- [IntelliJ IDEA with MCP support](#)
- [Visual Studio](#)
- [Visual Studio Code](#)
- [Windsurf](#)

Programmatic integration:

- [.NET with MCP SDK](#)
- [Python with MCP SDK](#)

For complete setup instructions, see [Get started with Azure MCP Server](#).

Where can you use Azure MCP Server?

The Azure MCP Server works in three primary contexts:

In AI-powered chat and code editors

Use the Azure MCP Server directly within AI assistants and code editors. As you chat about your Azure resources, the AI assistant automatically invokes Azure MCP Server tools to retrieve information, make changes, or answer questions. This is the most common usage pattern.

Get started with:

- [Cline](#)
- [Cursor](#)
- [Eclipse](#)
- [IntelliJ](#)
- [Visual Studio](#)

- [Visual Studio Code](#)
- [Windsurf](#)
- [GitHub Copilot coding agent](#)

In programmatic applications

Integrate the Azure MCP Server into your applications using the MCP SDK. Your app acts as an MCP client and invokes Azure MCP Server tools programmatically. This approach is useful for building custom automation, chatbots, or intelligent applications that need Azure integration.

Get started with:

- [Python](#)
- [.NET](#)

In self-hosted scenarios

Deploy the Azure MCP Server in your own environment for advanced control, security requirements, or custom modifications. You can run it locally, in containers, or integrate it into existing infrastructure. This pattern suits enterprise scenarios requiring air-gapped environments or custom authentication flows.

Learn how to:

- [Deploy a remote MCP server with Microsoft Foundry](#)
- [Deploy a remote MCP server with Copilot Studio](#)

Available tools for Azure Key Vault

The Azure MCP Server provides multiple tools for Azure Key Vault operations, enabling you to manage keys, secrets, and certificates through natural language conversations.

Manage keys

Create and retrieve [cryptographic keys](#) stored in your vault. Supported key types include RSA, RSA-HSM, EC, EC-HSM, oct, and oct-HSM.

Common scenarios:

- Create new RSA or EC keys for encryption or signing operations
- Retrieve key properties and metadata

- List all keys in a vault to audit key inventory

Manage secrets

Create, retrieve, and list [sensitive information](#) like API keys, passwords, and connection strings.

Common scenarios:

- Securely store API keys and database passwords
- Retrieve connection strings for application configuration
- Audit secret inventory to identify unused credentials

Manage certificates

Create, import, retrieve, and list [SSL/TLS certificates](#) and other certificate-based credentials.

Common scenarios:

- Generate or import SSL/TLS certificates for web applications
- Track certificate expiration dates to plan for [renewal](#)
- Retrieve certificate properties for compliance verification

Manage Managed HSM settings

Retrieve [Azure Key Vault Managed HSM](#) account settings for high-security deployments that require FIPS 140-3 Level 3 validated HSMs. This tool only applies to Managed HSM vaults, not standard Key Vault vaults.

Common scenarios:

- Review purge protection and soft-delete retention settings for Managed HSM
- Query HSM-specific configurations

For detailed information about each tool, including parameters and examples, see [Azure Key Vault tools for Azure MCP Server](#).

Get started

Ready to use Azure MCP Server with your Azure Key Vault resources?

1. **Set up your environment:** Choose an AI assistant or development tool that supports MCP. For setup and authentication instructions, see the links in the [Where can you use](#)

[Azure MCP Server?](#) section above.

2. **Start exploring:** Ask your AI assistant questions about your key vaults or request operations. Try prompts like:

- "List all secrets in my key vault 'my-vault'"
- "Get the certificate 'web-ssl-cert' from key vault 'prod-vault'"
- "Create a new RSA key named 'app-key' in key vault 'crypto-vault'"

3. **Learn more:** Review the [Azure Key Vault tools reference](#) for all available capabilities and detailed parameter information.

Best practices

When using Azure MCP Server with Azure Key Vault:

- **Specify vault name clearly:** Always include the exact key vault name when querying to avoid ambiguity, especially in subscriptions with many vaults.
- **Check certificate expiration:** Ask about certificate properties regularly to identify expiring certificates before they cause issues.
- **Audit your inventory:** Use list operations to review your keys, secrets, and certificates inventory for compliance and security audits.
- **Combine with other tools:** Use Azure MCP Server for quick queries and inventory checks. Use Azure CLI or PowerShell for vault configuration changes, access control management, and sensitive operations like [secret rotation](#).

For general Azure Key Vault security guidance beyond the Azure MCP Server, see [Secure your Azure Key Vault](#).

Related content

- [Azure MCP Server overview](#)
- [Get started with Azure MCP Server](#)
- [Azure Key Vault tools reference](#)
- [Azure Key Vault documentation](#)
- [Azure RBAC for Key Vault](#)

📘 **Note:** This article contains content created with AI. [Learn more](#)

Manage Azure Redis with Azure MCP Server

If you work with Azure Managed Redis, the Azure MCP Server can help you create new Redis resources, inventory existing caches, and manage Redis deployments using natural language conversations with AI assistants.

[Azure Managed Redis](#) provides in-memory data storage based on the Redis software. While the Azure portal and Azure CLI are powerful tools, the Azure MCP Server provides a more intuitive way to interact with your Redis resources through conversational AI.

What is the Azure MCP Server?

The Azure MCP Server enables AI agents and assistants to interact with Azure resources through natural language commands using the [Model Context Protocol \(MCP\)](#). Instead of manually navigating portals or writing scripts, you can describe what you want to accomplish, and the AI assistant uses the Azure MCP Server tools to perform the actions.

For Azure Redis users, this means you can:

- Create new Redis resources with specific SKUs and configurations using plain language
- List all Redis resources across subscriptions without complex queries
- Configure Redis modules like RedisJSON or RedisBloom through conversation
- Review Redis cache inventory and deployment details quickly
- Set up development and production Redis instances conversationally
- Verify Redis resource configurations across environments

Prerequisites

To use the Azure MCP Server with Azure Redis, you need:

Azure requirements

- **Azure subscription:** An active Azure subscription. [Create one for free](#).
- **Azure Redis resources:** At least one Azure Managed Redis cache in your subscription, or permissions to create them.

- **Azure permissions:** Appropriate roles like Contributor or Redis Cache Contributor to perform the operations you want. See [Use Microsoft Entra ID for cache authentication with Azure Managed Redis](#).

MCP client requirements

You need an AI assistant or development environment that supports the Model Context Protocol. Choose one:

AI-powered code editors:

- [Cursor](#)
- [Cline](#)
- [IntelliJ IDEA with MCP support](#)
- [Visual Studio](#)
- [Visual Studio Code](#)
- [Windsurf](#)

Programmatic integration:

- [.NET with MCP SDK](#)
- [Python with MCP SDK](#)

For complete setup instructions, see [Get started with Azure MCP Server](#).

Where can you use Azure MCP Server?

The Azure MCP Server works in three primary contexts:

In AI-powered chat and code editors

Use the Azure MCP Server directly within AI assistants and code editors. As you chat about your Azure resources, the AI assistant automatically invokes Azure MCP Server tools to retrieve information, make changes, or answer questions. This is the most common usage pattern.

Get started with:

- [Cline](#)
- [Cursor](#)
- [Eclipse](#)
- [IntelliJ](#)

- [Visual Studio](#)
- [Visual Studio Code](#)
- [Windsurf](#)
- [GitHub Copilot coding agent](#)

In programmatic applications

Integrate the Azure MCP Server into your applications using the MCP SDK. Your app acts as an MCP client and invokes Azure MCP Server tools programmatically. This approach is useful for building custom automation, chatbots, or intelligent applications that need Azure integration.

Get started with:

- [Python](#)
- [.NET](#)

In self-hosted scenarios

Deploy the Azure MCP Server in your own environment for advanced control, security requirements, or custom modifications. You can run it locally, in containers, or integrate it into existing infrastructure. This pattern suits enterprise scenarios requiring air-gapped environments or custom authentication flows.

Learn how to:

- [Deploy a remote MCP server with Microsoft Foundry](#)
- [Deploy a remote MCP server with Copilot Studio](#)

Available tools for Azure Redis

The Azure MCP Server provides two tools specifically designed for Azure Redis operations. These tools enable you to create and manage Redis resources through natural language conversations.

Create a Redis resource

Provision new Azure Managed Redis resources with specific configurations, SKUs, and modules.

Common scenarios:

- Create a Redis cache for a new application or environment

- Provision Redis with a specific SKU matching performance requirements
- Enable Redis modules like RedisJSON or RedisBloom during creation

List Redis resources

List and review all Redis resources in your subscription. Returns details of all Azure Managed Redis, Azure Cache for Redis, and Azure Redis Enterprise resources.

Common scenarios:

- List all Redis resources in your subscription to audit deployments
- View Redis caches to identify instances for cost optimization
- Get an inventory of Redis clusters across your environment

For detailed information about each tool, including parameters and examples, see [Azure Redis tools for Azure MCP Server](#).

Get started

Ready to use Azure MCP Server with your Azure Redis resources?

1. **Set up your environment:** Choose an AI assistant or development tool that supports MCP. For setup and authentication instructions, see the links in the [Where can you use Azure MCP Server?](#) section above.
2. **Start exploring:** Ask your AI assistant questions about your Redis resources or request operations. Try prompts like:
 - "List all Redis resources in my subscription"
 - "Create a Redis cache named 'test-cache' in eastus with Balanced_B0 SKU"
 - "Show me my Redis instances in the 'production' resource group"
3. **Learn more:** Review the [Azure Redis tools reference](#) for all available capabilities and detailed parameter information.

Best practices

When using Azure MCP Server with Azure Redis:

- **Choose appropriate SKUs:** Specify Redis SKU based on your performance and cost requirements. Use Balanced_B0 for development and higher SKUs for production workloads.

- **Enable required modules early:** Specify Redis modules like RedisJSON or RedisBloom during creation to avoid reconfiguration later.
- **Disable access keys:** For production environments, explicitly disable access key authentication and use managed identities or Microsoft Entra ID for better security.
- **Use consistent naming:** Follow naming conventions for Redis resources across environments (for example, `app-cache-dev`, `app-cache-prod`) to simplify management.
- **Review regularly:** Periodically list all Redis resources to identify unused instances and optimize costs.
- **Tag resources:** While creating Redis resources, consider adding Azure tags through the portal for better resource organization and cost tracking.

Related content

- [Azure MCP Server overview](#)
- [Get started with Azure MCP Server](#)
- [Azure Redis tools reference](#)
- [Azure Redis documentation](#)
- [Azure Managed Redis overview](#)

 **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 01/21/2026

Use Azure MCP Server with Azure AI Search

Azure Model Context Protocol (MCP) Server enables AI assistants like GitHub Copilot, Claude Desktop, and others to interact with Azure AI Search through natural language commands. This integration allows you to manage search resources and query both indexed and remote data without writing code or remembering complex CLI syntax.

Overview

Azure AI Search is a cloud-based search service that supports full-text, vector, and hybrid search over scattered enterprise content. You can load and query an index for classic search scenarios, or you can use a knowledge base for agentic retrieval over indexed and remote data.

With Azure MCP Server integration, you can use natural language to:

- List Azure AI Search services in your subscription.
- View index schemas, fields, and configurations.
- Query indexes to find relevant content.
- View knowledge base and knowledge source details.
- Retrieve from knowledge bases using queries or conversational messages.

Prerequisites

To use the Azure MCP Server with Azure Functions, you need:

Azure requirements

Before using Azure MCP Server with Azure AI Search, ensure you have:

- **Azure subscription:** An active Azure subscription. [Create one for free](#).
- **Azure AI Search service:** You need to have at least one Azure AI Search service provisioned to use its features.
- **Azure permissions:** Appropriate roles to perform the operations you want. See [Azure Built-in Roles](#).

MCP client requirements

You need an AI assistant or development environment that supports the Model Context Protocol. Choose one:

AI-powered code editors:

- [Cursor](#)
- [Cline](#)
- [IntelliJ IDEA with MCP support](#)
- [Visual Studio](#)
- [Visual Studio Code](#)
- [Windsurf](#)

Programmatic integration:

- [.NET with MCP SDK](#)
- [Python with MCP SDK](#)

For complete setup instructions, see [Get started with Azure MCP Server](#).

Where can you use Azure MCP Server?

The Azure MCP Server works in three primary contexts:

In AI-powered chat and code editors

Use the Azure MCP Server directly within AI assistants and code editors. As you chat about your Azure resources, the AI assistant automatically invokes Azure MCP Server tools to retrieve information, make changes, or answer questions. This is the most common usage pattern.

Get started with:

- [Cline](#)
- [Cursor](#)
- [Eclipse](#)
- [IntelliJ](#)
- [Visual Studio](#)
- [Visual Studio Code](#)
- [Windsurf](#)
- [GitHub Copilot coding agent](#)

In programmatic applications

Integrate the Azure MCP Server into your applications using the MCP SDK. Your app acts as an MCP client and invokes Azure MCP Server tools programmatically. This approach is useful for building custom automation, chatbots, or intelligent applications that need Azure integration.

Get started with:

- [Python](#)
- [.NET](#)

In self-hosted scenarios

Deploy the Azure MCP Server in your own environment for advanced control, security requirements, or custom modifications. You can run it locally, in containers, or integrate it into existing infrastructure. This pattern suits enterprise scenarios requiring air-gapped environments or custom authentication flows.

Learn how to:

- [Deploy a remote MCP server with Microsoft Foundry](#)
- [Deploy a remote MCP server with Copilot Studio](#)

Available tools for Azure AI Search

Azure MCP Server provides the following tools for Azure AI Search operations:

 [Expand table](#)

Tool	Description
<code>search index get</code>	Retrieve details of Azure AI Search indexes and their properties.
<code>search index query</code>	Query an Azure AI Search index for relevant results.
<code>search knowledge base get</code>	Get details of Azure AI Search knowledge bases and their sources.
<code>search knowledge base retrieve</code>	Retrieve information using a specific Azure AI Search knowledge base.
<code>search knowledge source get</code>	Get details of knowledge sources utilized in Azure AI Search.
<code>search service list</code>	List all Azure AI Search services available in the subscription.

For detailed information about each tool, including parameters and examples, see [Azure AI Search tools for Azure MCP Server](#).

Get started

Ready to use Azure MCP Server with your Azure AI Search resources?

1. **Set up your environment:** Choose an AI assistant or development tool that supports MCP. For setup and authentication instructions, see the links in the [Where can you use Azure MCP Server?](#) section above.
2. **Start exploring:** Ask your AI assistant questions about your search resources. Try prompts like:
 - "Get details of the search index named 'products'."
 - "Query the search index 'products' for items with 'laptop' in the title."
 - "Retrieve information from knowledge base 'faq-kb' with the question 'What are the return policies?'."
3. **Learn more:** Review the [Azure AI Search tools reference](#) for all available capabilities and detailed parameter information.

Best practices

- **Use detailed queries during retrieval:** When using the search index query tool, specify precise index and parameters to narrow results effectively.
- **Check index structures before querying:** Use the search index get command to understand index fields and optimize query performance.
- **Regularly review knowledge base settings:** Periodically retrieve knowledge base details to ensure configurations meet current needs.

Related content

- [Azure MCP Server documentation](#)
- [Azure AI Search MCP tools reference](#)
- [Azure AI Search documentation](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Use Azure MCP Server with Azure Speech in Foundry Tools

Azure Model Context Protocol (MCP) Server enables AI assistants like GitHub Copilot, Claude Desktop, and others to interact with Azure Speech in Foundry Tools through natural language commands. This integration allows you to manage text-to-speech and speech-to-text capabilities without writing code or remembering complex CLI syntax.

[Azure Speech in Foundry Tools](#) provides speech to text, text to speech, and other capabilities through a Microsoft Foundry resource. You can transcribe speech to text with high accuracy, produce natural-sounding text-to-speech voices, translate spoken audio, and conduct live AI voice conversations.

What is the Azure MCP Server?

The Azure MCP Server enables AI agents and assistants to interact with Azure resources through natural language commands using the [Model Context Protocol \(MCP\)](#). Instead of manually navigating portals or writing scripts, you can describe what you want to accomplish, and the AI assistant uses the Azure MCP Server tools to perform the actions.

For Azure Speech developers, this means you can:

- Transcribe speech from audio files.
- Synthesize speech from text input.

Prerequisites

To use the Azure MCP Server with Azure Speech in Foundry, you need:

- **Azure subscription:** An active Azure subscription. [Create one for free](#).
- **Foundry resource:** You must have an existing Azure AI Services endpoint to process speech and text.
- **Azure permissions:** Appropriate roles to perform the operations you want. See [Azure Built-in Roles](#).

MCP client requirements

You need an AI assistant or development environment that supports the Model Context Protocol. Choose one:

AI-powered code editors:

- [Cursor](#)
- [Cline](#)
- [IntelliJ IDEA with MCP support](#)
- [Visual Studio](#)
- [Visual Studio Code](#)
- [Windsurf](#)

Programmatic integration:

- [.NET with MCP SDK](#)
- [Python with MCP SDK](#)

For complete setup instructions, see [Get started with Azure MCP Server](#).

Where can you use Azure MCP Server?

The Azure MCP Server works in three primary contexts:

In AI-powered chat and code editors

Use the Azure MCP Server directly within AI assistants and code editors. As you chat about your Azure resources, the AI assistant automatically invokes Azure MCP Server tools to retrieve information, make changes, or answer questions. This is the most common usage pattern.

Get started with:

- [Cline](#)
- [Cursor](#)
- [Eclipse](#)
- [IntelliJ](#)
- [Visual Studio](#)
- [Visual Studio Code](#)
- [Windsurf](#)
- [GitHub Copilot coding agent](#)

In programmatic applications

Integrate the Azure MCP Server into your applications using the MCP SDK. Your app acts as an MCP client and invokes Azure MCP Server tools programmatically. This approach is useful for building custom automation, chatbots, or intelligent applications that need Azure integration.

Get started with:

- [Python](#)
- [.NET](#)

In self-hosted scenarios

Deploy the Azure MCP Server in your own environment for advanced control, security requirements, or custom modifications. You can run it locally, in containers, or integrate it into existing infrastructure. This pattern suits enterprise scenarios requiring air-gapped environments or custom authentication flows.

Learn how to:

- [Deploy a remote MCP server with Microsoft Foundry](#)
- [Deploy a remote MCP server with Copilot Studio](#)

Available tools for Azure Speech in Foundry

Azure MCP Server provides the following tools for Azure Speech in Foundry operations:

 Expand table

Tool	Description
<code>speech stt recognize</code>	Recognize speech from audio files and convert it to text.
<code>speech tts synthesize</code>	Generate audio files from text using neural text-to-speech.

For detailed information about each tool, including parameters and examples, see [Azure Speech in Foundry tools for Azure MCP Server](#).

Get started

Ready to use Azure MCP Server with your Azure Speech resources?

1. **Set up your environment:** Choose an AI assistant or development tool that supports MCP. For setup and authentication instructions, see the links in the [Where can you use](#)

[Azure MCP Server?](#) section above.

2. **Start exploring:** Ask your AI assistant questions about your speech apps or request operations. Try prompts like:

- "Recognize speech from the audio file 'meeting-recording.mp3'."
- "Convert 'meeting-recording.wav' audio to text and save it as 'transcript.txt'."
- "Get transcription results from the audio recording in the project folder."

3. **Learn more:** Review the [Azure Speech tools reference](#) for all available capabilities and detailed parameter information.

Best practices

- **Use supported audio formats:** Ensure you use WAV, MP3, or other supported formats for speech recognition.
- **Specify language for accuracy:** Always specify the language parameter to improve recognition accuracy.
- **Choose voice selection wisely:** Select appropriate voice options to match the desired tone for synthesized speech.

Related content

- [Azure MCP Server documentation](#)
- [Azure MCP Speech tools reference](#)
- [Azure Speech documentation](#)
- [Speech-to-text overview](#)
- [Text-to-speech overview](#)

ⓘ **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 03/07/2026

What are the Azure MCP Server tools?

The Azure Model Context Protocol (MCP) Server exposes many tools you can use from an existing `client` to interact with Azure services through natural language prompts. For example, you can use the Azure MCP Server to interact with Azure resources conversationally from GitHub Copilot agent mode in Visual Studio Code or other AI agents with commands like these:

- "Show me all my resource groups"
- "List blobs in my storage container named 'documents'"
- "What's the value of the 'ConnectionString' key in my app configuration?"
- "Query my log analytics workspace for errors in the last hour"
- "Show me all my Cosmos DB databases"

Developer credentials and security

The MCP server uses your Azure user credentials or managed identity to ensure authorized access. Access is secured through Azure Role-Based Access Control (RBAC), providing fine-grained permissions for approved users. The local MCP server is intended strictly for developer use within your organization. Don't use these tools for external applications or scenarios outside the approved development environment.

Azure MCP Server start parameters

The `azmcp` server supports the following options for server start parameters:

 Expand table

Option	Required or optional	Description
Debug	Optional	Enable debug mode with verbose logging to <code>stderr</code> . Default: <code>false</code> .
Enable insecure transports	Optional	Enable insecure transport. Default: <code>false</code> .

Option	Required or optional	Description
Disable user confirmation (Not recommended)	Optional	Disable user confirmation (elicitation) before allowing high risk commands to run, such as returning secrets (passwords) from KeyVault. When enabled, tools that handle secrets, credentials, or sensitive data will execute without user confirmation. This removes an important security layer designed to prevent unauthorized access to sensitive information. Only use this option in trusted, automated environments where user interaction isn't possible. Never use this option in production environments or when handling untrusted input. Default: <code>false</code> .
Mode	Optional	Server mode: <code>namespace</code> (default), <code>consolidated</code> , <code>all</code> , or <code>single</code> . Default: <code>namespace</code> .
Namespace	Optional	The Azure service namespaces to expose on the MCP server (for example, <code>storage</code> , <code>keyvault</code> , <code>cosmos</code>). Default: all namespaces.
Read only	Optional	Whether the MCP server should be read-only. If true, no write operations are allowed. Default: <code>false</code> .
Tool	Optional	Expose specific tools by name (for example, <code>azmcp_storage_account_get</code>). It automatically switches to <code>all</code> mode. Default: all tools.
Transport	Optional	Transport mechanism to use for Azure MCP Server. Default: <code>stdio</code> .

Tool parameters

The Azure MCP Server tools define parameters for the data they need to complete tasks. For example, these parameters might include the subscription ID, an account name, or a resource group.

You might include the data for these parameters in the prompt you use to call a tool, or the previous conversation context might establish the data. If the conversation context provides the data, the Azure MCP Server can use that information without requiring you to repeat it in every prompt. This context creates a more natural conversational experience while still ensuring all necessary data is available for the tools.

The tools reference articles document the parameters specific to each tool. All of the tools also share the following global parameters.

Parameter	Description
Subscription	Azure subscription ID or name for target resources. This parameter identifies the Azure subscription that contains the resources you want to manage. You can use either the subscription GUID or the display name. If not specified, the Azure MCP Server resolves the default subscription from your Azure CLI profile (set via az account set) or the <code>AZURE_SUBSCRIPTION_ID</code> environment variable. Required for most operations unless a default is configured.
Resource group	The name of the Azure resource group. This is a logical container for Azure resources that helps organize and manage related resources together. Required for most resource-specific operations.
Tenant Id	Azure tenant ID for authentication. This parameter specifies the Microsoft Entra ID tenant to authenticate against. Can be either the GUID identifier or the display name of your Entra ID tenant. Optional - uses default tenant if not specified.
Authentication method	Authentication method to use for Azure operations. Options include <code>credential</code> (Azure CLI/managed identity), <code>key</code> (access key), or <code>connectionString</code> . Default is <code>credential</code> , which uses Azure CLI authentication or managed identity.
Maximum retries	Maximum number of retry attempts for failed operations before giving up. Controls how many times the system attempts to retry a failed request. Default is 3 retries.
Retry delay	Initial delay in seconds between retry attempts. For exponential backoff, this value is used as the base delay that gets multiplied on each retry. Default is 2 seconds.
Retry delay maximum	Maximum delay in seconds between retries, regardless of the retry strategy. This parameter caps the delay time to prevent excessively long waits. Default is 10 seconds.
Retry mode	Retry strategy to use when operations fail. <code>fixed</code> uses consistent delays between retries, while <code>exponential</code> increases the delay between each attempt. Default is <code>exponential</code> for better handling of temporary issues.
Retry network timeout	Network operation timeout in seconds. When operations take longer than this timeout, they are canceled and might be retried if retries are enabled. Default is 100 seconds.

Example prompts include:

- **Set subscription:** "Use subscription 'my-subscription-id' for all operations"
- **Use tenant ID:** "Authenticate using tenant ID 'my-tenant-id'"
- **Set authentication method:** "Use 'credential' authentication for this session"
- **Configure retries:** "Set maximum retries to 5 with a 3-second delay"
- **Set retry mode:** "Use 'fixed' retry mode with a maximum delay of 5 seconds"
- **Set network timeout:** "Set network timeout to 120 seconds for all operations"
- **Configure retry parameters:** "Use exponential retry mode with a maximum of 4 retries and a delay of 2 seconds"

Tool annotations for Azure MCP Server

Tool annotations [↗](#) are hints that provide additional information about the characteristics of each tool. The following table describes the possible hints that can be associated with a tool.

[↗](#) Expand table

Name	Description
Destructive	Indicates whether the tool can make destructive updates to its environment. If true, the tool might delete or modify existing resources. If false, the tool only adds new resources without removing or altering existing ones.
Idempotent	Specifies if repeated calls with the same arguments produce the same result without side effects. If true, multiple executions with the same arguments produce the same result. If false, repeated executions might have additional effects or yield different results.
Open world	Defines whether the tool might interact with an "open world" of external entities. If true, the tool might interact with an unpredictable or dynamic set of entities (for example, web search). If false, the tool's domain of interaction is closed and well-defined (for example, memory access).
Read only	Indicates if the tool performs only read operations. If true, there's no change of state of the environment. If false, the tool might make modifications to its environment.
Secret	Microsoft proprietary annotation that indicates if the tool's response might contain sensitive data requiring sanitization. If true, the response might include secrets, credentials, or keys that should be sanitized before forwarding to the LLM or logging. If false, the response doesn't contain sensitive information.
Local required	Microsoft proprietary annotation to indicate if the tool requires local execution or resources. If true, the tool is only available when the Azure MCP server runs in Local (STDIO) mode. If false, the tool is available in both local and remote server modes.

User confirmation for sensitive data

Tools that handle sensitive data, such as secrets, require user consent before execution through a security mechanism called **elicitation**. When you use tools that access sensitive information, the MCP client prompts you to confirm the operation before proceeding.

Elicitation (user confirmation) Security Feature:

Elicitation prompts appear when tools might expose sensitive information like:

- Key Vault secrets
- Connection strings and passwords

- Certificate private keys
- Other confidential data

These prompts protect against unauthorized access to sensitive information. You can bypass elicitation only in automated scenarios.

Available tools

Azure MCP Server provides the following tools for Azure services and Azure-related functionality.

Best practices

 Expand table

Product, tool, or service	Namespace	Description
Azure best practices	<code>get azure bestpractices get</code>	Get guidance on Azure Functions development, deployment, and Azure SDK usage.
Azure AI best practices for app development	<code>get azure bestpractices ai app</code>	Get best practices and code generation guidance for building AI applications in Azure. Use it when you need recommendations on writing code for AI agents, chatbots, workflows, or any AI/LLM features. Additionally, this command provides guidance for code generation on Microsoft Foundry for application development.
Terraform best practices for Azure	<code>azureterraformbestpractices</code>	Get guidance on implementing Terraform for Azure resources.

AI and Machine Learning

 Expand table

Product, tool, or service	Namespace	Description
Microsoft Foundry	<code>foundry</code>	Work with Foundry models, deployments, and endpoints.
Azure AI Search	<code>search</code>	Manage Azure AI Search resources, including search services, indexes, and queries.

Product, tool, or service	Namespace	Description
Azure Speech in Foundry Tools	speech	Manage Speech resources such as speech-to-text and text-to-speech services.

Analytics

[Expand table](#)

Product, tool, or service	Namespace	Description
Azure App Lens	applens	Diagnose and analyze application performance issues using Azure App Lens.
Azure Data Explorer	kusto	Work with Azure Data Explorer clusters, databases, tables, and queries.
Azure Event Hubs	eventhubs	Manage Azure Event Hubs namespaces and event hubs.

Compute

[Expand table](#)

Product, tool, or service	Namespace	Description
Azure App Service	appservice	Manage web apps, database connections, diagnostics, deployments, and application settings for Azure App Service.
Azure Functions	functionapp	List Azure Functions.
Azure Kubernetes Service	aks	List Azure Kubernetes Service clusters.
Azure Service Fabric	servicefabric	Manage Service Fabric managed clusters, including node details and restarts.
Azure Compute	compute	Get details for Azure virtual machines, virtual machine scale sets, and managed disks.

Containers

[Expand table](#)

Product, tool, or service	Namespace	Description
Azure Container Registry	acr	List Azure Container Registry instances.
Azure Functions	functionapp	List Azure Functions.
Azure Kubernetes Service	aks	List Azure Kubernetes Service clusters.
Azure Service Fabric	servicefabric	Manage Service Fabric managed clusters, including node details and restarts.

Databases

 Expand table

Product, tool, or service	Namespace	Description
Azure Cosmos DB	cosmos	Work with Azure Cosmos DB accounts, databases, containers, and documents.
Azure Database for MySQL	mysql	Manage Azure Database for MySQL servers, databases, and tables.
Azure Database for PostgreSQL	postgres	Manage Azure Database for PostgreSQL servers, databases, and tables.
Azure Redis	redis	Create and list Azure Redis resources, including Managed Redis and Cache for Redis.
Azure SQL	sql	Work with Azure SQL Database servers, databases, firewall rules, and elastic pools.

Developer tools

 Expand table

Product, tool, or service	Namespace	Description
Azure App Configuration	appconfig	Manage centralized application settings and feature flags.
Azure Application Insights	applicationinsights	List Application Insights resources.

Product, tool, or service	Namespace	Description
Azure CLI	extension	Find Azure CLI commands and get installation instructions for CLI tools.
Azure Developer CLI	extension	Get installation instructions for Azure Developer CLI (azd), learn usage for deployment scenarios, and access azd MCP tools when azd is installed locally
Azure Load Testing	loadtesting	Create, run, and see load testing.

DevOps

 Expand table

Product, tool, or service	Namespace	Description
Azure Bicep schema	bicepschema	Retrieve Bicep schemas for Azure resources to use in Infrastructure as Code templates.
Azure Deploy	deploy	Deploy and manage Azure resources using templates and scripts.
Azure Developer CLI	extension	Get installation instructions for Azure Developer CLI (azd), learn usage for deployment scenarios, and access azd MCP tools when azd is installed locally
Azure Managed Grafana	grafana	List Azure Managed Grafana workspaces.
Azure Monitor	monitor	Query Azure Monitor logs and metrics.
Azure Workbooks	workbooks	Create, manage, and update Azure Workbooks for data visualization and reporting.

Hybrid and multicloud

 Expand table

Product, tool, or service	Namespace	Description
Azure Database for PostgreSQL	postgres	Manage Azure Database for PostgreSQL servers, databases, and tables.
Azure SQL	sql	Work with Azure SQL Database servers, databases, firewall rules, and elastic pools.

Identity

 Expand table

Product, tool, or service	Namespace	Description
Azure RBAC	role	View and manage Azure role-based access control assignments.

Integration

 Expand table

Product, tool, or service	Namespace	Description
Azure Event Grid	eventgrid	Manage Azure Event Grid resources, including topics and subscriptions.
Azure Native ISV	datadog	Work with Azure Native ISV services, including Datadog integration for monitoring and observability.
Azure Service Bus	servicebus	Manage Azure Service Bus resources, including queues, topics, and peek at messages.

Internet of Things (IoT)

 Expand table

Product, tool, or service	Namespace	Description
Azure Cosmos DB	cosmos	Work with Azure Cosmos DB accounts, databases, containers, and documents.
Azure Event Grid	eventgrid	Manage Azure Event Grid resources, including topics and subscriptions.
Azure Functions	functionapp	List Azure Functions.

Management and governance

 Expand table

Product, tool, or service	Namespace	Description
Azure Advisor	<code>advisor</code>	Get recommendations to optimize Azure resources and improve performance.
Azure Cloud Architect	<code>cloudarchitect</code>	Design cloud systems by gathering requirements through guided questions and recommending optimal solutions.
Azure Managed Grafana	<code>grafana</code>	List Azure Managed Grafana workspaces.
Azure Migrate	<code>azuremigrate</code>	Get step-by-step guidance for changing a Platform Landing Zone and make or download platform landing zone setups for Azure Migrate projects.
Azure Monitor	<code>monitor</code>	Query Azure Monitor logs and metrics.
Azure Policy	<code>policy</code>	Manage Azure Policy assignments, definitions, and initiatives.
Azure Pricing	<code>pricing</code>	Get Azure retail pricing, cost estimates, and billing details for services, SKUs, and regions.
Azure Quick Review CLI	<code>extension</code>	Generate compliance and security reports for Azure resources.
Azure Quotas	<code>quota</code>	Manage Azure resource quotas and limits.
Azure Resource Health	<code>resourcehealth</code>	Check availability status, view health events, and track service-impacting issues for Azure resources.
Azure Well-Architected Framework	<code>wellarchitectedframework</code>	Get architectural best practices, design patterns, and recommendations for Azure services based on the five Well-Architected Framework pillars.
Azure Workbooks	<code>workbooks</code>	Create, manage, and update Azure Workbooks for data visualization and reporting.
Resource Groups	<code>group</code>	List Azure resource groups and list resources within a group.
Subscription	<code>subscription</code>	List Azure subscriptions.

Messaging

 Expand table

Product, tool, or service	Namespace	Description
Azure Event Grid	eventgrid	Manage Azure Event Grid resources, including topics and subscriptions.
Azure Event Hubs	eventhubs	Manage Azure Event Hubs namespaces and event hubs.
Azure Service Bus	servicebus	Manage Azure Service Bus resources, including queues, topics, and peek at messages.

Mobile

 Expand table

Product, tool, or service	Namespace	Description
Foundry	foundry	Work with Foundry models, deployments, and endpoints.
Azure AI Search	search	Manage Azure AI Search resources, including search services, indexes, and queries.
Azure App Configuration	appconfig	Manage centralized application settings and feature flags.
Azure Communication Services	communication	Send SMS and email messages using Azure Communication Services.

Security

 Expand table

Product, tool, or service	Namespace	Description
Azure App Configuration	appconfig	Manage centralized application settings and feature flags.
Azure Confidential Ledger	confidentialledger	Manage Azure Confidential Ledger resources, including ledgers and transactions.
Azure Key Vault	keyvault	List and create keys, secrets, certificates in Azure Key Vault.

Storage

 Expand table

Product, tool, or service	Namespace	Description
Azure Confidential Ledger	confidentialledger	Manage Azure Confidential Ledger resources, including ledgers and transactions.
Azure File Sync	storagesync	Manage Azure file sync services, register servers, create sync groups, configure cloud endpoints, and set up server endpoints for Azure File Sync.
Azure Files	fileshares	Manage Azure file shares, including creating, updating, deleting, and snapshotting file shares.
Azure Managed Lustre	managedlustre	Manage Azure Managed Lustre file systems, including auto-import and auto-export jobs.
Azure Storage	storage	Manage Azure Storage accounts, containers, blobs, and tables.
Azure Storage	storage	List Azure Storage accounts, containers, blobs, and tables.

Virtual desktop infrastructure (VDI)

 Expand table

Product, tool, or service	Namespace	Description
Azure Virtual Desktop	virtualdesktop	Manage Azure Virtual Desktop host pools, session hosts, and user sessions.

Web

 Expand table

Product, tool, or service	Namespace	Description
Azure AI Search	search	Manage Azure AI Search resources, including search services, indexes, and queries.
Azure App Configuration	appconfig	Manage centralized application settings and feature flags.
Azure App Service	appservice	Manage web apps, database connections, diagnostics, deployments, and application settings for Azure App Service.

Product, tool, or service	Namespace	Description
Azure Communication Services	communication	Send SMS and email messages using Azure Communication Services.
Azure Functions	functionapp	List Azure Functions.
Azure SignalR	signalr	Manage Azure SignalR resources and runtimes.

Other

 Expand table

Product, tool, or service	Namespace	Description
Azure Marketplace	marketplace	Discover Azure Marketplace products and offers.
Azure MCP tool	No namespace	Discover and manage available Azure MCP Server tools.

Related content

- [What is the Azure MCP Server?](#)
- [Get started using Azure MCP Server](#)
- [Azure MCP Server repository](#) 

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 03/11/2026

Azure best practices tools for the Azure MCP Server overview

The Azure MCP Server lets you manage data retrieval and analysis, including working with AI applications and executing get commands, with natural language prompts.

Get Azure best practices for AI app

This command returns best practices and code generation guidance for building AI applications in Azure. Use it when you need recommendations on writing code for AI agents, chatbots, workflows, or any AI/LLM features. Additionally, this command provides guidance for code generation on Microsoft Foundry for application development.

Example prompts include:

- "Get best practices for code generation in AI applications?"
- "Show me code guidance for chatbots in Azure?"
- "Get recommendations for building workflows using AI components?"
- "Create an AI app that helps me improve customer interactions?"
- "Create an AI app that supports data analysis in Microsoft Foundry?"

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Get Azure best practices

This tool returns a list of best practices for code generation, operations, and deployment when working with Azure services. Call this tool for any code generation, deployment, or operations involving Azure, Azure Functions, Azure Kubernetes Service (AKS), Azure Container Apps (ACA), Bicep, Terraform, Azure Cache, Redis, CosmosDB, Entra, Azure Active Directory, Azure App Services.

Example prompts include:

- "Get the latest Azure coding agent best practices?"
- "Get the latest Azure operations best practices?"

- "Get the latest general Azure best practices?"
- "Get the latest Azure Static Web Apps code generation best practices?"
- "Get the latest Azure Static Web Apps deployment best practices?"
- "Get the latest Azure Static Web Apps best practices?"
- "Get the latest Azure Functions all best practices?"
- "What are coding agent best practices?"
- "Configure Azure MCP for my coding agent project?"

 Expand table

Parameter	Required or optional	Description
Action	Required	The action type for the best practices. Options: <code>all</code> , <code>code-generation</code> , <code>deployment</code> . Note: <code>static-web-app</code> and <code>coding-agent</code> resources only support <code>all</code> .
Resource	Required	The Azure resource type for which to get best practices. Options: <code>general</code> (general Azure), <code>azurefunctions</code> (Azure Functions), <code>static-web-app</code> (Azure Static Web Apps), <code>coding-agent</code> (Coding Agent).

Tool annotation hints: Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related resources

- [Azure Functions documentation](#)
- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 03/07/2026

Azure MCP Server tools for Azure Well-Architected Framework

The Azure Model Context Protocol (MCP) Server lets you get Azure Well-Architected Framework guidance, best practices, and recommendations for Azure services with natural language prompts.

Azure Well-Architected Framework is a set of guiding tenets that help you design, build, and optimize workloads across five pillars: reliability, security, cost optimization, operational excellence, and performance efficiency; for more information, see [Azure Well-Architected Framework documentation](#).

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Get Well-Architected Framework service guide

This tool retrieves Azure Well-Architected Framework guidance for a specific Azure service, or lists all supported services when no service is specified. When you provide a service, the tool returns architectural best practices, design patterns, and recommendations across the five pillars: reliability, security, cost optimization, operational excellence, and performance efficiency.

Example prompts include:

- "Show all services with Well-Architected Framework guidance."
- "Which services have architectural guidance under the Well-Architected Framework?"
- "Retrieve Well-Architected Framework guidance for service 'App Service'."
- "What's the WAF guidance for service 'Cosmos DB'?"
- "Show the architectural guidance for 'Azure Functions'."

Parameter	Required or optional	Description
Service	Optional	A single Azure service name. The value is case-insensitive and accepts hyphens, underscores, and spaces. If the name contains spaces, enclose it in double quotes. Examples: <code>cosmos-db</code> , <code>Cosmos_DB</code> , <code>Cosmos DB</code> , <code>cosmosdb</code> , <code>cosmos-database</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Well-Architected Framework documentation](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 03/27/2026

Terraform best practices for Azure tools for the Azure MCP Server overview

This article describes the Terraform best practices for Azure tool available in Azure MCP Server. The tool provides guidance for implementing Infrastructure as Code (IaC) with Terraform in Azure environments.

Terraform best practices for Azure help you implement secure, scalable, and maintainable infrastructure configurations by using recommended patterns and practices from Microsoft.

Get best practices

Returns best practices for implementing Terraform with Azure. Use this tool to get guidance on creating secure, scalable, and maintainable infrastructure as code for Azure resources.

Example prompts include:

- **General best practices:** "What are the best practices for using Terraform with Azure?"
- **Implementation guidance:** "Show me guidance for implementing Azure resources with Terraform"
- **Module structure:** "How should I structure my Terraform modules for Azure?"
- **State management:** "What are the recommended patterns for managing state files in Azure Terraform projects?"
- **Secure infrastructure:** "I need help implementing secure Azure infrastructure using Terraform"

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related resources

- [Terraform on Azure documentation](#)
- [Azure Architecture Center - Best practices in cloud applications](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Azure Advisor tools for the Azure MCP Server overview

The Azure MCP Server lets you manage resource optimization, cost recommendations, and performance improvements with natural language prompts.

Azure Advisor is a service that gives you actionable insights to help you optimize your Azure resources and improve performance. For more about Advisor, see [Azure Advisor documentation](#).

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Get recommendation list

Show Azure Advisor recommendations for a subscription.

Example prompts:

- "Show Azure Advisor recommendations for subscription 'MySubscription'."
- "What are the current Advisor recommendations for my subscription 'MySubscription'?"
- "Show me all Azure Advisor recommendations under subscription 'MySubscription'."

[Tool annotation hints](#):

Destructive: ✗ | Idempotent: ✓ | Open World: ✗ | Read Only: ✓ | Secret: ✗ | Local Required: ✗

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Advisor documentation](#)

ⓘ **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 02/20/2026

Application Insights tools for the Azure MCP Server overview

The Azure MCP Server allows you to list Application Insights resources using natural language prompts.

[Application Insights](#) is an extensible Application Performance Management (APM) service for developers and DevOps professionals. It provides insights into the performance and usage of your applications, helping you to detect and diagnose issues, understand user behavior, and improve application performance.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

List recommendations

Lists [Application Insights](#) code optimization recommendations in a subscription.

Example prompts include:

- **List code optimization recommendations:** "List code optimization recommendations across my Application Insights components."
- **Show recommendations for all resources:** "Show me code optimization recommendations for all Application Insights resources in my subscription."
- **List profiler recommendations by group:** "List profiler recommendations for Application Insights in resource group 'devops-group'."

[Tool annotation hints:](#)

Destructive: ✗ | Idempotent: ✔ | Open World: ✗ | Read Only: ✔ | Secret: ✗ | Local Required: ✗

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Application Insights](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/27/2026

Microsoft Foundry tools for the Azure MCP Server overview

The Azure MCP Server enables you to manage Azure resources, including Microsoft Foundry models and deployments, with natural language prompts. This capability helps you quickly manage your AI models without needing to remember complex syntax.

[Foundry](#) is a platform for deploying and managing custom AI models in Azure. It provides tools and services for training, fine-tuning, deploying, and monitoring AI models in production environments. With Foundry, you can more easily incorporate AI capabilities into your applications.

When connecting to your Foundry resource, the Azure MCP Server requires either the **endpoint** or the **resource group** of your Foundry resource. For operations that don't require a specific resource, such as listing available models, neither the endpoint or resource group is required.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Agents: Connect and run

Connect to a specific Azure AI Agent and run a query. This command returns the agent's response along with thread and run IDs for potential evaluation.

Example prompts include:

- **Connect to agent:** "Connect to agent 'support-bot' and ask 'What is the status of ticket #12345?' at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/support-project'"
- **Query specific agent:** "Ask agent 'sales-bot' for the latest sales report with query 'Show me this month's revenue' at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/sales'"

- **Use context:** "Connect to agent 'hr-bot' with query 'What are the benefits options?' at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/hr-dept'"

 Expand table

Parameter	Required or optional	Description
Agent	Required	The ID of the agent to interact with.
Query	Required	The query sent to the agent.
Endpoint	Required	The endpoint URL for the Foundry project or service in the format <code>https://<resource>.services.ai.azure.com/api/projects/<project-name>.</code>

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Agents: Create a new agent

Creates a Foundry Agent that processes messages according to a given system instruction using an existing Foundry model deployment.

Example prompts include:

- **Create customer support agent:** "Create an agent named 'customer-support-bot' using deployment 'gpt-4' at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/support-project' with system instruction 'You are a helpful customer support agent who assists with product inquiries and troubleshooting'"
- **Create sales assistant:** "Set up an agent called 'sales-assistant' with model deployment 'gpt-35-turbo' at 'https://my-example-resource.services.ai.azure.com/api/projects/sales' that follows the instruction 'You are a sales assistant who helps customers find products and provides pricing information'"
- **Create HR chatbot:** "Build an agent named 'hr-onboarding-bot' using deployment 'gpt-4-turbo' at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/hr-dept' with instruction 'You are an HR specialist who guides new employees through the onboarding process and answers policy questions'"

Parameter	Required or optional	Description
Endpoint	Required	The endpoint URL for the Foundry project or service in the format <code>https://<resource>.services.ai.azure.com/api/projects/<project-name>.</code>
Model deployment	Required	Name of the model deployment.
Agent name	Required	A human-readable name of the Agent.
System instruction	Required	System instruction for the agent to follow when process messages.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✖ | Open World: ✖ | Read Only: ✖ | Secret: ✖ | Local Required: ✖

Agents: Evaluate an agent

Run agent evaluation on agent data. Requires JSON strings for query, response, and tool definitions.

Example prompts include:

- **Evaluate task adherence:** "Evaluate the query 'What are the refund policies?' for task_adherence at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/support-project' using Azure OpenAI deployment 'gpt-4'"
- **Check intent resolution:** "Evaluate the query 'What pricing plans do you offer?' for intent_resolution at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/sales' using Azure OpenAI deployment 'gpt-4'"
- **Verify tool accuracy:** "Analyze tool_call_accuracy for query 'Check inventory for item 5678' at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/sales-bot' using Azure OpenAI deployment 'gpt-35-turbo'"
- **Assess agent performance:** "Evaluate query 'My app won't start' for task_adherence at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/support' using Azure OpenAI deployment 'gpt-4'"

- **Comprehensive evaluation:** "Run evaluation on query 'What documents do I need for onboarding?' for task_adherence at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/hr-dept' using Azure OpenAI deployment 'gpt-4'"

[Expand table](#)

Parameter	Required or optional	Description
Endpoint	Required	The endpoint URL for the Foundry project or service in the format <code>https://<resource>.services.ai.azure.com/api/projects/<project-name>.</code>
Azure OpenAI Deployment	Required	The deployment name for the Azure OpenAI model to be used in evaluation.
Query	Required	The query sent to the agent.
Evaluator	Required	The name of the evaluator to use (<code>intent_resolution</code> , <code>tool_call_accuracy</code> , <code>task_adherence</code>).
Response	Optional	The response from the agent.
Tool Definitions	Optional	Optional tool definitions made by the agent in JSON format.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Agents: Get agent sample for language SDK

Get code samples to interact with a Foundry Agent using Foundry SDK and programming language of your choice.

Example prompts include:

- **Get Python sample:** "Show me a Python code sample for interacting with a Foundry Agent"
- **Get C# sample:** "Generate a csharp code example to work with my Foundry agent"
- **Get TypeScript sample:** "Provide a typescript sample code for connecting to and using a Foundry Agent"

[Expand table](#)

Parameter	Required or optional	Description
Programming language	Required	The programming language of the sdk for interacting with a Foundry Agent. Supported values are <code>csharp</code> , <code>python</code> and <code>typescript</code> .

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Agents: List agents

List all Azure AI Agents in a Foundry project. Shows agents that can be used for AI workflows, evaluations, and interactive tasks.

Example prompts include:

- **View all agents:** "Show me all agents at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/my-project'"
- **List by project:** "List all AI agents at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/customer-service'"
- **Check available agents:** "What agents do I have at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/my-project?'"
- **Agent inventory:** "I need a complete list of agents at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/my-project'"
- **Find specific agents:** "Show me all chatbot agents at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/my-project'"

[Expand table](#)

Parameter	Required or optional	Description
Endpoint	Required	The endpoint URL for the Foundry project or service in the format <code>https://<resource>.services.ai.azure.com/api/projects/<project-name></code> .

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Agents: Query and execute an agent

Query an agent and evaluate its response in a single operation. This command returns both the agent response and evaluation results.

Example prompts include:

- **Query and evaluate:** "Query agent 'support-bot' with 'What is ticket 123 status?' and evaluate task adherence at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/support-project' using Azure OpenAI endpoint 'https://my-openai.openai.azure.com' and deployment 'gpt-4'"
- **Single operation:** "Ask agent 'sales-bot' query 'Show monthly revenue' and check intent resolution at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/sales' using Azure OpenAI endpoint 'https://my-openai.openai.azure.com' and deployment 'gpt-4'"
- **Combined action:** "Connect to agent 'hr-bot' with query 'What is the onboarding process?' and evaluate tool call accuracy at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/hr-dept' using Azure OpenAI endpoint 'https://my-openai.openai.azure.com' and deployment 'gpt-35-turbo'"
- **Full cycle:** "Query agent 'marketing-bot' with 'Suggest campaign ideas' and evaluate task adherence at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/marketing' using Azure OpenAI endpoint 'https://my-openai.openai.azure.com' and deployment 'gpt-4'"
- **End-to-end check:** "Ask agent 'devops-bot' query 'What is deployment status?' and evaluate intent resolution at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/devops' using Azure OpenAI endpoint 'https://my-openai.openai.azure.com' and deployment 'gpt-4'"

 Expand table

Parameter	Required or optional	Description
Endpoint	Required	The endpoint URL for the Foundry project or service in the format <code>https://<resource>.services.ai.azure.com/api/projects/<project-name></code>

Parameter	Required or optional	Description
Azure OpenAI Endpoint	Required	The endpoint URL for the Azure OpenAI service to be used in evaluation.
Azure OpenAI Deployment	Required	The deployment name for the Azure OpenAI model.
Agent ID	Required	The ID of the agent to interact with.
Query	Required	The query sent to the agent.
Evaluators	Optional	The list of evaluators to use for evaluation, separated by commas. If not specified, all evaluators are used.

Tool annotation hints:

Destructive: ✗ | Idempotent: ✗ | Open World: ✔ | Read Only: ✗ | Secret: ✗ | Local Required: ✗

Knowledge: List knowledge indexes

Get a list of knowledge indexes from Foundry:

- Find knowledge indexes created within Foundry projects.
- Use these indexes with AI agents for knowledge retrieval and RAG applications.
- The list updates as you create new indexes or update existing ones.

Example prompts include:

- **View all indexes:** "Show me all knowledge indexes at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/my-project'"
- **Filter by project:** "List knowledge indexes at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/support-bot'"
- **Search by name:** "Find the knowledge index named 'product-faqs' at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/my-project'"
- **Filter by tag:** "List knowledge indexes tagged with 'security' at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/my-project'"
- **Show index details:** "Show details for the 'customer-service' knowledge index at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/my-project'"

[Expand table](#)

Parameter	Required or optional	Description
Endpoint	Required	The endpoint URL for the Foundry project or service in the format <code>https://<resource>.services.ai.azure.com/api/projects/<project-name></code>

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Knowledge: Get index schema

Get the detailed schema configuration of a specific knowledge index from Foundry.

This operation shows you comprehensive information about the structure and configuration of a knowledge index, including field definitions, data types, searchable attributes, and other schema properties. Use this schema information to understand how the index structures and indexes your data for searching.

Example prompts include:

- **View index schema:** "Show me the schema for knowledge index 'product-facts' at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/my-project'"

[Expand table](#)

Parameter	Required or optional	Description
Endpoint	Required	The endpoint URL for the Foundry project or service in the format <code>https://<resource>.services.ai.azure.com/api/projects/<project-name></code>
Index	Required	The name of the knowledge index.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Models: List available models

List all available Azure OpenAI models and deployments in a Foundry resource. This tool retrieves information about Azure OpenAI models deployed in your Foundry resource including model names, versions, capabilities, and deployment status.

Example prompts include:

- **View all models:** "Show me all available AI models in Foundry"
- **Filter by free usage:** "List all free models available for prototyping in Foundry that I can use in the playground"
- **Filter by free usage:** "List all free models available for prototyping in Foundry"
- **Filter by publisher:** "Show me models published by Microsoft in Foundry"
- **Filter by license:** "What models with Apache license are available in Foundry?"
- **Search by name:** "Find the llama model in Foundry"

 Expand table

Parameter	Required or optional	Description
Search for free playground	Optional	If set to true, returns a list of models from Foundry that you can also use with GitHub inference endpoint and GitHub PAT token. If false, returns a list of models from Foundry, regardless of GitHub support. To learn more, see GitHub Models .
Publisher	Optional	A filter to specify the publisher of the models to retrieve.
License	Optional	A filter to specify the license type of the models to retrieve.
Model	Optional	The name of the model to search for.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Models: Deploy a model

Deploy an AI model to your Azure environment. Use this command to deploy selected models from Foundry and make them available for use in your applications.

Example prompts include:

- **Deploy with required parameters:** "Deploy GPT-4 model in OpenAI format to my ai-services account in resource group 'my-resource-group'"
- **Specify deployment name:** "Set up a deployment named text-embedding for the Ada embedding model in my Foundry Tools account in resource group 'my-resource-group' with Standard SKU"
- **Include model version:** "Deploy version 2 of Llama model from Meta to my Foundry Tools account in resource group 'my-resource-group' with scale capacity of 3"
- **Deploy to specific resource group:** "Create a deployment named content-generation with GPT-4 model in my ai-central service in resource group 'my-resource-group'"
- **Configure scaling:** "Deploy Claude model to my Foundry Tool in resource group 'my-resource-group' with autoscaling enabled and maximum capacity of 5"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Deployment	Required	The name of the Foundry model deployment.
Model	Required	The name of the model to deploy.
Model format	Required	The format of the model (for example, <code>OpenAI</code> , <code>Meta</code> , <code>Microsoft</code>).
Foundry Tools	Required	The name of the Foundry Tools account to deploy to.
Model version	Optional	The version of the model to deploy.
Model source	Optional	The source of the model.
SKU	Optional	The SKU name for the deployment.
SKU capacity	Optional	The SKU capacity for the deployment.
Scale type	Optional	The scale type for the deployment.
Scale capacity	Optional	The scale capacity for the deployment.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Models: List model deployments

List model deployments in a Foundry (Cognitive Services) project. Shows currently deployed AI models at the project level.

Example prompts include:

- **List deployments on production:** "Show me all model deployments at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/production'"
- **Check specific endpoint:** "What models are currently deployed to endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/my-project'?"
- **View regional deployments:** "List all deployments at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/west-region'"
- **Check deployment status:** "Show me the status of all models deployed to endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/my-project'"
- **See active models:** "What AI models are running at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/my-project' right now?"

 Expand table

Parameter	Required or optional	Description
Endpoint	Required	The endpoint URL for the Foundry project or service in the format <code>https://<resource>.services.ai.azure.com/api/projects/<project-name></code>

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

OpenAI: Create chat completions

Create chat completions using Azure OpenAI in Foundry. Send messages to Azure OpenAI chat models deployed in your Foundry resource and receive AI-generated conversational responses. Supports multi-turn conversations with message history, system instructions, and response customization.

Example prompts include:

- **Simple greeting:** "Create a chat completion with message array `[{"role": "user", "content": "Hello, how are you today?"}]` using deployment 'gpt-35-turbo' on resource 'openai-prod'"

- **With system message:** "Create a chat completion with system message 'You are a helpful assistant' and user message 'Explain quantum computing' using deployment 'gpt-35-turbo' on resource 'openai-west'"
- **Control creativity:** "Generate a chat completion for 'Write a creative story' using deployment 'gpt-4' with temperature 0.8 and max 150 tokens on resource 'ai-central'"
- **Deterministic response:** "Create chat completion with message 'List 5 facts about Mars' using deployment 'gpt-35-turbo' with temperature 0.1 and seed 12345 on resource 'ai-services-prod'"
- **Conversation with history:** "Continue chat completion with messages: system 'You are a coding assistant', user 'How do I create a function in Python?', assistant 'Here's how...', user 'Can you show an example?' using deployment 'gpt-4' on resource 'dev-openai'"
- **With penalties for repetition:** "Create completion for 'Describe the benefits of cloud computing' using deployment 'gpt-35-turbo' with frequency penalty 0.5 and presence penalty 0.3 on resource 'ai-services-main'"
- **Streaming response:** "Generate streaming chat completion for 'Explain machine learning step by step' using deployment 'gpt-4' with stream true on resource 'openai-research'"
- **With stop sequences:** "Create completion for 'Count from 1 to 10' using deployment 'gpt-35-turbo' with stop sequences ['5', 'STOP'] on resource 'ai-test'"
- **User tracking:** "Generate completion for 'What is Azure AI?' using deployment 'gpt-4' with user identifier 'user-123' on resource 'prod-openai'"
- **Fine-tuned control:** "Create chat completion for 'Summarize this article' using deployment 'gpt-35-turbo' with temperature 0.2, top_p 0.9, max tokens 200, and AAD authentication on resource 'secure-ai'"

[Expand table](#)

Parameter	Required or optional	Description
Resource name	Required	The name of the Azure OpenAI resource.
Deployment	Required	The name of the Foundry model deployment.
Message array	Required	Array of messages in the conversation (JSON format). Each message should have <code>role</code> and <code>content</code> properties.
Max tokens	Optional	The maximum number of tokens to generate in the completion.
Temperature	Optional	Controls randomness in the output. Lower values make it more deterministic.
Top p	Optional	Controls diversity via nucleus sampling (0.0 to 1.0). Default is <code>1.0</code> .

Parameter	Required or optional	Description
Frequency penalty	Optional	Penalizes new tokens based on their frequency (-2.0 to 2.0). Default is 0.
Presence penalty	Optional	Penalizes new tokens based on presence (-2.0 to 2.0). Default is 0.
Stop	Optional	Up to 4 sequences where the API will stop generating further tokens.
Stream	Optional	Whether to stream back partial progress. Default is false.
Seed	Optional	If specified, the system will make a best effort to sample deterministically.
User	Optional	Optional user identifier for tracking and abuse monitoring.
Authentication type	Optional	The type of authentication to use. Options are key (default) or aad.

Tool annotation hints:

Destructive: ✗ | Idempotent: ✗ | Open World: ✗ | Read Only: ✔ | Secret: ✗ | Local Required: ✗

OpenAI: Create embeddings

Create embeddings using Azure OpenAI in Foundry. Generate vector embeddings from text using Azure OpenAI deployments in your Foundry resource for semantic search, similarity comparisons, clustering, or machine learning.

Example prompts include:

- **Basic text embedding:** "Generate embeddings for the text 'Azure OpenAI Service' using my 'text-embedding-ada-002' deployment in resource group 'my-resource-group'"
- **Create vector embeddings:** "Create vector embeddings for my text using Azure OpenAI with deployment 'text-embedding-3-large' on resource 'ai-services-prod' in resource group 'my-resource-group'"
- **Document embedding:** "Generate embeddings for 'Machine learning revolutionizes data analysis' using deployment 'ada-002' on resource 'embedding-service' in resource group 'my-resource-group'"
- **Multiple sentences:** "Create embeddings for the text 'Cloud computing provides scalable infrastructure. It enables global accessibility.' using my embedding deployment in

resource group 'my-resource-group'"

- **With user tracking:** "Generate embeddings for 'Natural language processing applications' using deployment 'text-embedding-3-small' with user identifier 'analytics-team' in resource group 'my-resource-group'"
- **Specific dimensions:** "Create embeddings for 'Artificial intelligence transforms business operations' using deployment 'text-embedding-3-large' with 1536 dimensions on resource 'ai-central' in resource group 'my-resource-group'"
- **Base64 format:** "Generate embeddings for 'Deep learning neural networks' using deployment 'ada-002' with base64 encoding format on resource 'ml-services' in resource group 'my-resource-group'"
- **Research text:** "Create vector embeddings for 'Quantum computing demonstrates computational advantages in specific algorithms' using my text-embedding deployment in resource group 'my-resource-group'"
- **Product description:** "Generate embeddings for 'High-performance laptop with advanced graphics processing unit' using deployment 'text-embedding-3-small' on resource 'product-ai' in resource group 'my-resource-group'"
- **Technical documentation:** "Create embeddings for 'API authentication requires valid credentials and proper authorization headers' using deployment 'ada-002' with float encoding on resource 'docs-embedding' in resource group 'my-resource-group'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Resource name	Required	The name of the Azure OpenAI resource.
Deployment	Required	The name of the Foundry model deployment.
Input text	Required	The input text to generate embeddings for.
User	Optional	Optional user identifier for tracking and abuse monitoring.
Encoding format	Optional	The format to return embeddings in (<code>float</code> or <code>base64</code>).
Dimensions	Optional	The number of dimensions for the embedding output. Only supported in some models.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✖ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local
Required: ✖

OpenAI: Create completions

Create text completions using Azure OpenAI in Foundry. Send a prompt or question to Azure OpenAI models deployed in your Foundry resource and receive generated text answers. Use this when you need to create completions, get AI-generated content, generate answers to questions, or produce text completions from Azure OpenAI based on any input prompt. Supports customization with temperature and max tokens.

Example prompts include:

- **Basic completion:** "Create a completion with the prompt 'What is Azure?' using my 'gpt-35-turbo' deployment in resource group 'my-resource-group'"
- **With temperature control:** "Generate text completion for 'Explain machine learning' using deployment 'text-davinci-003' with temperature 0.3 in resource group 'my-resource-group'"
- **Limited tokens:** "Create a completion with prompt 'Write a summary' using my 'gpt-4' deployment with max 100 tokens in resource group 'my-resource-group'"
- **Creative writing:** "Generate completion for 'Tell me a story about AI' using deployment 'gpt-35-turbo' with temperature 0.8 and 200 max tokens in resource group 'my-resource-group'"
- **Technical explanation:** "Create completion with prompt 'How does cloud computing work?' using my OpenAI resource 'ai-services-east' and deployment 'gpt-4' in resource group 'my-resource-group'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group where the AI resource is hosted.
Resource name	Required	The name of the Azure OpenAI resource.
Deployment	Required	The name of the deployment.
Prompt text	Required	The prompt text to send to the completion model.

Parameter	Required or optional	Description
Max tokens	Optional	The maximum number of tokens to generate in the completion.
Temperature	Optional	Controls randomness in the output. Lower values make it more deterministic.

Tool annotation hints:

Destructive: ❌ | Idempotent: ❌ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

OpenAI: List models and deployments

List all available OpenAI models and deployments in an Azure resource. This tool retrieves information about deployed models including model names, versions, capabilities, and deployment status.

Example prompts include:

- **View all models:** "List all OpenAI models in my 'ai-services-prod' resource in resource group 'my-resource-group'"
- **Check deployments:** "Show me all deployed models and their status in resource 'openai-east' in resource group 'my-resource-group'"
- **Production inventory:** "What models are available in my 'production-openai' resource in resource group 'my-resource-group'?"
- **Development check:** "List all models and deployments in my 'dev-ai-services' resource in resource group 'my-resource-group'"
- **Model capabilities:** "Show me all available OpenAI models with their capabilities in resource 'ai-central' in resource group 'my-resource-group'"
- **Deployment status:** "What's the current status of all deployments in my 'openai-west' resource in resource group 'my-resource-group'?"
- **Regional models:** "List all models available in my 'europe-openai' resource in resource group 'my-resource-group'"
- **Service overview:** "Give me a complete overview of models and deployments in resource 'customer-ai' in resource group 'my-resource-group'"
- **Model versions:** "Show me all model versions available in my 'ai-services-main' resource in resource group 'my-resource-group'"

- **Resource audit:** "I need to audit all OpenAI models and deployments in resource 'enterprise-ai' in resource group 'my-resource-group'"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Resource name	Required	The name of the Azure OpenAI resource.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Resources: Get Foundry resource

Get detailed information about Foundry resources, including endpoint URL, location, SKU, and all deployed models with their configuration. If a specific resource name is provided, returns details for that resource only. If no resource name is provided, lists all Foundry resources in the subscription or resource group.

Example prompts include:

- **Get specific resource:** "Show me details for the 'ai-foundry-prod' Foundry resource including all deployed models"
- **List all resources:** "What Foundry resources do I have in my subscription?"
- **Resource with configuration:** "Get the endpoint URL, location, and SKU information for my 'customer-ai-foundry' foundry resource"

[Expand table](#)

Parameter	Required or optional	Description
Resource name	Optional	The name of the Azure OpenAI resource.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local Required: ✖

Threads: Create a new thread

Creates a Foundry agent thread that holds the messages between the Agent and the user.

Example prompts include:

- **Create support thread:** "Create a new thread at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/support-project' with user message 'I need help with my account login'"
- **Start conversation thread:** "Create a thread at 'https://my-example-resource.services.ai.azure.com/api/projects/customer-service' with message 'What are your business hours?'"
- **Initialize chat thread:** "Start a new thread at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/sales-bot' with the user message 'I'm interested in purchasing your premium plan'"

 Expand table

Parameter	Required or optional	Description
Endpoint	Required	The endpoint URL for the Foundry project or service in the format <code>https://<resource>.services.ai.azure.com/api/projects/<project-name></code> .
User message	Required	The user message to add to the thread.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✖ | Open World: ✖ | Read Only: ✖ | Secret: ✖ | Local Required: ✖

Threads: Get thread messages

Get messages in a Foundry agent thread.

Example prompts include:

- **Retrieve conversation history:** "Get all messages from thread 'thread_abc123xyz' at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/support-project'"
- **View thread messages:** "Show me the messages in thread 'thread_456def789' from endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/customer-service'"
- **Check thread content:** "Retrieve messages for thread ID 'thread_xyz789abc' at 'https://my-example-resource.services.ai.azure.com/api/projects/sales-bot'"

 Expand table

Parameter	Required or optional	Description
Endpoint	Required	The endpoint URL for the Foundry project or service in the format <code>https://<resource>.services.ai.azure.com/api/projects/<project-name>.</code>
Thread ID	Required	The Foundry Agent Thread ID.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Threads: List all threads

List Foundry agent threads.

Example prompts include:

- **List all threads:** "Show me all agent threads at endpoint 'https://my-example-resource.services.ai.azure.com/api/projects/support-project'"
- **View project threads:** "List all threads from 'https://my-example-resource.services.ai.azure.com/api/projects/customer-service'"
- **Get thread inventory:** "Retrieve all agent threads at 'https://my-example-resource.services.ai.azure.com/api/projects/sales-bot'"

 Expand table

Parameter	Required or optional	Description
Endpoint	Required	The endpoint URL for the Foundry project or service in the format <code>https://<resource>.services.ai.azure.com/api/projects/<project-name>.</code>

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Foundry documentation](#)
- [Foundry Tools overview](#)
- [Deploy and consume models](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 03/05/2026

Azure MCP Server tools for Azure AI Search

The Azure Model Context Protocol (MCP) Server lets you manage Azure AI Search resources, including search services, indexes, knowledge sources, and knowledge bases with natural language prompts.

[Azure AI Search](#) (formerly Azure Cognitive Search) is a cloud search service that provides APIs and tools for building applications and agents that follow the Retrieval Augmented Generation (RAG) pattern. It supports vector and keyword retrieval, reranking, and agentic retrieval; for more information, see [Azure AI Search documentation](#).

ⓘ Note

Each Azure AI Search knowledge base exposes a native MCP endpoint for direct retrieval. For more information, see [Call the MCP endpoint](#).

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Get search index

This tool retrieves detailed information about Azure AI Search indexes, including index schema, fields, analyzers, scoring profiles, and other index properties. When you provide an index name, it returns properties for that index; without an index name, it returns all indexes in the specified service.

Example prompts include:

- "Show me the details of index 'products-index' in Azure AI Search service 'my-search-service'."
- "List all indexes in the Azure AI Search service 'enterprise-search'."
- "What fields are in the 'users' index in service 'contoso-search'?"

[Expand table](#)

Parameter	Required or optional	Description
Service	Required	The name of the Azure AI Search service (for example, <code>my-search-service</code>).
Index	Optional	The name of the search index within the Azure AI Search service.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local Required: ✖

Query search index

This tool runs a search query against an Azure AI Search index and returns the matching documents and relevance metadata. Results typically include document fields, a relevance score, and any text highlights that match the query.

Example prompts include:

- "Search for 'machine learning' in the 'documents' index of my 'contoso-search' service."
- "Query index 'products' for 'noise-canceling headphones' in Azure AI Search service 'fabrikam-search'."
- "Search my 'content' index in 'my-search-service' for anything mentioning 'climate change'."

[Expand table](#)

Parameter	Required or optional	Description
Service	Required	The name of the Azure AI Search service (for example, <code>my-search-service</code>).
Index	Required	The name of the search index within the Azure AI Search service.
Query	Required	The search query to execute against the Azure AI Search index.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local Required: ✖

Get knowledge base

This tool gets details for Azure AI Search knowledge bases. Knowledge bases encapsulate retrieval and reasoning capabilities over one or more knowledge sources or indexes. If you don't provide a knowledge base name, it returns details for all knowledge bases within the specified service.

Example prompts include:

- "List all knowledge bases in the Azure AI Search service 'my-search-service'."
- "Show me the knowledge bases in service 'contoso-search'."
- "Get the details of knowledge base 'support-agent' in Azure AI Search service 'enterprise-search'."

 Expand table

Parameter	Required or optional	Description
Service	Required	The name of the Azure AI Search service (for example, <code>my-search-service</code>).
Knowledge base	Optional	The name of the knowledge base within the Azure AI Search service.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Retrieve from knowledge base

This tool executes a retrieval operation against an Azure AI Search knowledge base to find relevant information from its data sources. Provide either a single-turn query for retrieval or one or more conversational messages in `role:content` format. Specifying both query and messages isn't allowed.

Example prompts include:

- "Run a retrieval with knowledge base 'support-agent' in Azure AI Search service 'my-search-service' for the query 'password reset steps'."

- "Ask knowledge base 'product-docs' in search service 'contoso-search' to retrieve information about 'API rate limits'."
- "Query knowledge base 'hr-policies' in search service 'hr-search' about 'vacation accrual policy'."

[Expand table](#)

Parameter	Required or optional	Description
Service	Required	The name of the Azure AI Search service (for example, <code>my-search-service</code>).
Knowledge base	Required	The name of the knowledge base within the Azure AI Search service.
Query	Optional	Natural language query for retrieval when a conversational message history isn't provided.
Messages	Optional	Conversation history messages passed to the knowledge base. Each entry formatted as <code>role:content</code> , where role is <code>user</code> or <code>assistant</code> (for example, <code>user:What policies apply to archived invoices?</code>).

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Get knowledge source

This tool gets details of Azure AI Search knowledge sources. A knowledge source can point at an existing Azure AI Search index or represent external data (for example, a blob storage container) that Azure AI Search has indexed. Knowledge sources are used by knowledge bases during retrieval. If you don't provide a knowledge source name, it returns details for all knowledge sources in the specified service.

Example prompts include:

- "List all knowledge sources in the Azure AI Search service 'my-search-service'."
- "Show me the knowledge sources in the Azure AI Search service 'contoso-search'."
- "Get the details of knowledge source 'product-index' in search service 'enterprise-search'."

Parameter	Required or optional	Description
Service	Required	The name of the Azure AI Search service (for example, <code>my-search-service</code>).
Knowledge source	Optional	The name of the knowledge source within the Azure AI Search service.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

List search services

This tool lists Azure AI Search services in a subscription and returns details for each service, including name, location, SKU, provisioning state, and endpoint.

Example prompts include:

- "List Azure AI Search services in my subscription."
- "What AI Search services do I have?"
- "Show me my Azure AI Search resources."

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure AI Search documentation](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Azure Speech in Foundry Tools for the Azure MCP Server overview

Use the Azure MCP Server to manage Azure Speech in Foundry Tools functionalities such as speech-to-text (STT) and text-to-speech (TTS) with natural language prompts.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Speech-to-Text: Recognize

Recognize speech from an audio file using [Speech](#). This command takes an audio file and converts it to text using advanced speech recognition capabilities. Supported audio formats include WAV, MP3, OPUS/OGG, FLAC, ALAW, MULAW, MP4, M4A, and AAC. Compressed formats require GStreamer to be installed on the system.

Example prompts include:

- **Basic conversion:** "Convert the audio file `./meeting-recording.wav` to text using endpoint `https://myservice.cognitiveservices.azure.com/` with Azure Speech Services"
- **With language detection:** "Recognize speech from file `./recording.mp3` using endpoint `https://myservice.cognitiveservices.azure.com/` with language detection"
- **With profanity filtering:** "Transcribe speech from file `./interview.wav` using endpoint `https://myservice.cognitiveservices.azure.com/` with profanity option removed"
- **Specify endpoint:** "Convert speech to text from file `./audio.wav` using endpoint `https://myservice.cognitiveservices.azure.com/`"
- **Spanish language:** "Transcribe the audio file `./session.wav` using endpoint `https://myservice.cognitiveservices.azure.com/` in es-ES language"
- **Detailed output:** "Convert speech to text from file `./audio.wav` using endpoint `https://myservice.cognitiveservices.azure.com/` with detailed output format"
- **With phrase hints:** "Recognize speech from file `./notes.wav` using endpoint `https://myservice.cognitiveservices.azure.com/` with phrase hints 'Azure' for better

accuracy"

- **Multiple phrase hints:** "Transcribe file ./meeting.wav using endpoint `https://myservice.cognitiveservices.azure.com/` with phrase hints: 'Azure', 'cognitive services', 'machine learning'"
- **Comma-separated hints:** "Convert speech to text from file ./podcast.mp3 using endpoint `https://myservice.cognitiveservices.azure.com/` with phrase hints: 'Azure, cognitive services, API'"
- **Raw profanity output:** "Transcribe audio from file ./audio.wav using endpoint `https://myservice.cognitiveservices.azure.com/` with profanity option raw"

[Expand table](#)

Parameter	Required or optional	Description
Endpoint	Required	The Azure AI Services endpoint URL (for example, <code>https://your-service.cognitiveservices.azure.com/</code>).
File	Required	Path to the local audio file to recognize.
Language	Optional	The language for speech recognition (for example, <code>en-US</code> , <code>es-ES</code>). Default is <code>en-US</code> .
Phrases	Optional	Phrase hints to improve recognition accuracy. Can be specified multiple times or as comma-separated values.
Format	Optional	Output format: <code>simple</code> or <code>detailed</code> .
Profanity	Optional	Profanity filter: <code>masked</code> , <code>removed</code> , or <code>raw</code> . Default is <code>masked</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Text-to-Speech: Synthesize

Convert text to speech using Speech. This command takes text input and generates an audio file using advanced neural text-to-speech capabilities.

Example prompts include:

- **Basic synthesis:** "Convert the text 'Hello, welcome to Foundry Tools' to speech using endpoint `https://myservice.cognitiveservices.azure.com/` and save to output.wav"

- **With custom voice:** "Synthesize 'Thank you for using our service' to audio file greeting.mp3 using my custom voice my-custom-voice under service `https://myservice.cognitiveservices.azure.com/` and endpoint "guid-endpoint"
- **Different language:** "Generate Spanish speech for 'Bienvenido a Azure' and save to welcome-es.wav using my speech endpoint <https://myresource.cognitiveservices.azure.com/> in es-ES language"

 Expand table

Parameter	Required or optional	Description
Endpoint	Required	The Azure AI Services endpoint URL (for example, <code>https://your-service.cognitiveservices.azure.com/</code>).
Text	Required	The text to convert to speech.
Output file path	Required	Path where the synthesized audio file will be saved.
Language	Optional	The language for speech recognition (for example, <code>en-US</code> , <code>es-ES</code>). Default is <code>en-US</code> .
Voice	Optional	The voice to use for speech synthesis (for example, <code>en-US-JennyNeural</code>). If not specified, the default voice for the language will be used.
Format	Optional	Output format: <code>Riff24Khz16BitMonoPcm</code> , <code>Audio16Khz32KBitRateMonoMp3</code> , <code>Audio24Khz96KBitRateMonoMp3</code> , <code>Ogg16Khz16BitMonoOpus</code> , <code>Raw16Khz16BitMonoPcm</code> . Default is <code>Riff24Khz16BitMonoPcm</code> .
Endpoint ID	Optional	The endpoint ID of a custom voice model for speech synthesis.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Foundry Tools](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Azure App Configuration tools for the Azure MCP Server overview

The Azure MCP Server allows you to manage Azure resources, including App Configuration stores using natural language prompts. This allows you to quickly manage configuration settings and feature flags without remembering complex syntax.

[Azure App Configuration](#) provides a service to centrally manage application settings and feature flags. Modern programs, especially programs running in a cloud, generally have many components that are distributed. Spreading configuration settings across these components can lead to hard-to-troubleshoot errors during an application deployment. Use App Configuration to store all the settings for your application and secure their access in one place.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Account: List stores

The Azure MCP Server can list App Configuration stores in a subscription. This is useful for quickly checking the status of your App Configuration resources.

Example prompts include:

- **List stores:** "List all App Configuration stores in my subscription."
- **Show stores:** "What App Configuration stores do I have?"
- **Find stores:** "I need to see my App Configuration resources"
- **Query stores:** "Can you show me all my App Config stores?"
- **Check stores:** "App Configuration stores in subscription abc123"

[Tool annotation hints:](#)

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Key-value: Delete setting

The Azure MCP Server can delete a [key-value setting](#) from an App Configuration store.

Example prompts include:

- **Delete a setting:** "Remove the 'AppName:TemporaryConfig' key from my 'myappconfigstore' App Configuration store."
- **Delete a labeled setting:** "Delete the 'AppName:FeatureFlag' setting with label 'test' from App Configuration store 'myappconfigstore'"
- **Remove configuration:** "Delete the key 'ProductionEndpointUrl' from App Configuration store 'contoso-appconfig'"
- **Clean up settings:** "Delete the key 'TestSettings' with label 'deprecated' from App Configuration store 'myappconfigstore'"
- **Purge config:** "Delete the temporary API key 'TempAuth' from app-config-dev"

 Expand table

Parameter	Required or optional	Description
Account	Required	The name of the App Configuration store (for example, my-appconfig).
Key	Required	The name of the key to access within the App Configuration store.
Label	Optional	The label to apply to the configuration key. Labels are used to group and organize settings.
Content type	Optional	The content type of the configuration value. This is used to indicate how the value should be interpreted or parsed.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Key-value: Get key-values

Gets key-values in an App Configuration store. This command can provide one of the following actions:

- Retrieve a specific key-value by its key and optional label
- List key-values if no key is provided.

Listing key-values can optionally be filtered by a key filter and label filter. Each key-value includes its key, value, label, content type, ETag, last modified time, and lock status.

Example prompts include:

- **List all key-value settings:** "List all key-value settings in App Configuration store 'myappconfigstore'"
- **Show key-value settings:** "Show me the key-value settings in App Configuration store 'contoso-appconfig'"
- **Filter by key prefix:** "List all key-value settings with key name starting with 'prod-' in App Configuration store 'production-config'"
- **Get specific key content:** "Show the content for the key 'AppName:ConnectionString' in App Configuration store 'eastus-config'"
- **Environment-specific settings:** "List all key-value settings with key name starting with 'dev-' in App Configuration store 'development-config'"
- **Get labeled configuration:** "Show me the key-value settings with label 'staging' in App Configuration store 'app-config-staging'"
- **API configuration:** "Show the content for the key 'ApiSettings:Endpoint' in App Configuration store 'api-config'"
- **Database settings:** "List all key-value settings with key name starting with 'Database' in App Configuration store 'backend-config'"
- **Feature flags:** "Show me the key-value settings with label 'features' in App Configuration store 'feature-config'"
- **Application secrets:** "Show the content for the key 'Secrets:ApiKey' in App Configuration store 'secure-config'"

 Expand table

Parameter	Required or optional	Description
Account	Required	The name of the App Configuration store (for example, <code>my-appconfig</code>).
Key	Optional	The name of the key to access within the App Configuration store.
Label	Optional	The label to apply to the configuration key. Labels are used to group and organize settings.
Key filter	Optional	Specifies the key filter, if any, to be used when retrieving key-values. The filter can be an exact match, for example a filter of <code>foo</code> would get all key-values with a key of <code>foo</code> , or the filter can include a <code>*</code> character at the end of the

Parameter	Required or optional	Description
		string for wildcard searches (for example, <code>App*</code>). If omitted all keys is retrieved.
Label filter	Optional	Specifies the label filter, if any, to be used when retrieving key-values. The filter can be an exact match, for example a filter of <code>foo</code> would get all key-values with a label of <code>foo</code> , or the filter can include a <code>*</code> character at the end of the string for wildcard searches (for example, <code>Prod*</code>). This filter is case-sensitive. If omitted, all labels is retrieved.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Key-value: Set lock on key-value

Sets the lock state of a key-value in an App Configuration store. This command can lock and unlock key-values.

Example prompts include:

- **Lock a setting:** "Lock the key 'AppName:ConnectionString' in App Configuration store 'myappconfigstore'."
- **Lock a labeled setting:** "Lock the key 'AppName:ApiKey' with label 'production' in App Configuration store 'contoso-appconfig'."
- **Unlock a setting:** "Unlock the key 'AppName:ConnectionString' in App Configuration store 'myappconfigstore'."
- **Unlock a labeled setting:** "Unlock the key 'AppName:ApiKey' with label 'production' in App Configuration store 'contoso-appconfig'."

 Expand table

Parameter	Required or optional	Description
Account	Required	The name of the App Configuration store (for example, <code>my-appconfig</code>).
Key	Required	The name of the key to access within the App Configuration store.
Label	Optional	The label to apply to the configuration key. Labels are used to group and organize settings.

Parameter	Required or optional	Description
Content type	Optional	The content type of the configuration value. This is used to indicate how the value should be interpreted or parsed.
Lock	Optional	Whether a key-value is locked (set to <code>read-only</code>) or unlocked (<code>read-only</code> removed).

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Key-value: Set key-value setting

Set or update a [key-value setting](#) in an App Configuration store.

Example prompts include:

- **Create a setting:** "Create a new key 'AppName:ApiUrl' with value 'https://api.example.com' in my 'myappconfigstore' App Configuration store."
- **Update a setting:** "Update the key 'AppName:MaxRetries' to value '5' in App Configuration store 'myappconfigstore'"
- **Create a labeled setting:** "Set 'AppName:LogLevel' with value 'Debug' and label 'dev' in my 'contoso-appconfig' App Configuration store."
- **Add new config:** "Add a new setting called 'ApiEndpoint' with URL value 'https://api.contoso.com' to my 'eastus-config'"
- **Change existing value:** "Set key 'MaxThreads' to value '10' in App Configuration store 'appconfig-prod'"

 Expand table

Parameter	Required or optional	Description
Account name	Required	The name of the App Configuration store.
Key	Required	The key name of the setting to set.
Value	Required	The value to set for the key.
Label	Optional	The label of the setting to set.

Parameter	Required or optional	Description
Tags	Optional	The tags to associate with the configuration key. Tags should be in the format <code>key=value</code> . You can specify multiple tags.
Content type	Optional	The content type of the configuration value. This value indicates how the value should be interpreted or parsed.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure App Configuration](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 12/05/2025

Azure MCP Server tools for Azure AppLens

Use Azure MCP Server to diagnose Azure App Service resources by using natural language prompts.

Azure AppLens identifies and troubleshoots application and platform issues for Azure App Service. For more information, see [Azure AppLens documentation](#).

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Diagnose applens resource

Get diagnostic help from App Lens for Azure application and service issues to identify what's wrong with a service. Ask questions about performance, slowness, failures, errors, application state, or availability to receive expert analysis and solutions. Returns analysis, insights, and recommended solutions. Only the resource name and question are required. Subscription, resource group, and resource type are optional and used to narrow down results when multiple resources share the same name.

Example prompts include:

- "Diagnose resource 'webapp-prod' with AppLens, question: 'Why is the app responding slowly?'. "
- "Use AppLens to check resource 'api-backend', question: 'Why are requests timing out under load?'. "
- "What does AppLens report for resource 'orders-service', question: 'What is causing the recent 500 errors and failures?'. "

 Expand table

Parameter	Required or optional	Description
Question	Required	The diagnostic question to ask about the resource.
Resource	Required	The name of the resource to investigate or diagnose.
Resource group	Optional	Azure resource group name. Use to narrow results when multiple resources share the same name.
Resource type	Optional	The Azure resource type. Use to narrow results when multiple resources share the same name.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure App Service diagnostics documentation](#)

 **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 04/09/2026

Azure MCP Server tools for Azure App Service

The Azure Model Context Protocol (MCP) Server lets you manage Azure App Service resources with natural language prompts. You can deploy and update web apps and APIs, configure app settings, diagnose runtime issues, and list or retrieve app details.

Azure App Service is a fully managed platform for building, deploying, and scaling web apps and APIs; for more information, see [Azure App Service documentation](#).

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Add database connection

This tool adds a database connection to an Azure App Service by using the connection string for an existing database. It configures the App Service's connection settings so the app can access the specified database and database server. If you don't provide a connection string, the tool generates a default connection string.

Example prompts include:

- "Add database 'ordersdb' of database type 'SqlServer' with database server 'contoso-sql.database.windows.net' to app 'webapp-prod' in resource group 'rg-prod'."
- "Configure database 'userdb' database type 'MySQL' on database server 'contoso-mysql.mysql.database.azure.com' for app 'api-staging' in resource group 'rg-staging'."
- "Connect database 'appdata' database type 'PostgreSQL' with database server 'pgserver.postgres.database.azure.com' to app 'backend-app' in resource group 'rg-backend'."
- "Add CosmosDB database 'catalogdb' database type 'CosmosDB' with database server 'contoso-cosmos.documents.azure.com' to app 'ecommerce-app' in resource group 'rg-ecommerce'."

- "Add database 'inventory' of database type 'SqlServer' on database server 'adventure-works-sql.database.windows.net' to app 'inventory-service' in resource group 'rg-inventory'."
- "Configure database 'analytics' database type 'PostgreSQL' on database server 'contoso-pg.postgres.database.azure.com' for app 'analytics-api' in resource group 'rg-analytics' using connection string 'Server=tcp:contoso-pg.postgres.database.azure.com;Database=analytics;User Id= <your-username>;Password= <your-password>'."
- "Add database 'customers' database type 'MySQL' with database server 'adventure-works-mysql.mysql.database.azure.com' to app 'crm-web' in resource group 'rg-crm' using connection string 'Server=adventure-works-mysql.mysql.database.azure.com;Database=customers;Uid= <your-username>;Pwd= <your-password>'."
- "Connect database 'sessiondb' database type 'CosmosDB' on database server 'fabrikam-cosmos.documents.azure.com' to app 'session-service' in resource group 'rg-session'."
- "Set up database 'orders' database type 'SqlServer' on database server 'fabrikam-sql.database.windows.net' for app 'order-processor' in resource group 'rg-orders'."
- "Configure database 'logs' database type 'PostgreSQL' on database server 'fabrikam-pg.postgres.database.azure.com' for app 'logger-app' in resource group 'rg-logging' with connection string 'Host=fabrikam-pg.postgres.database.azure.com;Database=logs;Username= <your-username>;Password= <your-password>'."

 Expand table

Parameter	Required or optional	Description
App	Required	The name of the Azure App Service (for example, <code>my-webapp</code>).
Database	Required	The name of the database to connect to (for example, <code>mydb</code>).
Database server	Required	The server name or endpoint for the database (for example, <code>contoso-server.database.windows.net</code>).
Database type	Required	The type of database: <code>SqlServer</code> , <code>MySQL</code> , <code>PostgreSQL</code> , and <code>Cosmos DB</code> .
Resource group	Required	The name of the Azure resource group. This resource group is a logical container for Azure resources.
Connection string	Optional	The connection string for the database. If not provided, a default is generated.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✖ | Open World: ✔ | Read Only: ✖ | Secret: ✖ | Local
Required: ✖

Diagnose web app

This tool runs a specified detector on an Azure App Service web app and returns the detector's diagnostic results. The output includes the detector results and related diagnostic data to help you investigate app health and behavior.

Example prompts include:

- "Diagnose web app 'webapp-prod' in resource group 'rg-prod' with detector 'Availability'."
- "Diagnose web app 'my-webapp' in resource group 'webapp-dev' with detector 'CpuAnalysis' between '2025-03-01T00:00:00Z' and '2025-03-01T01:00:00Z' with interval 'PT5M'."

 Expand table

Parameter	Required or optional	Description
App	Required	The name of the Azure App Service (for example, <code>my-webapp</code>).
Detector name	Required	The name of the diagnostic detector to run (for example, <code>Availability</code> , <code>CpuAnalysis</code> , <code>MemoryAnalysis</code>).
Resource group	Required	The name of the Azure resource group. This resource group is a logical container for Azure resources.
End time	Optional	The end time in ISO format (for example, <code>2023-01-01T00:00:00Z</code>).
Interval	Optional	The time interval (for example, <code>PT1H</code> for 1 hour, <code>PT5M</code> for 5 minutes).
Start time	Optional	The start time in ISO format (for example, <code>2023-01-01T00:00:00Z</code>).

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local
Required: ✖

Get deployment details

This tool retrieves detailed information about deployments in an Azure App Service web app. It returns metadata such as deployment name, whether the deployment is active, start and end times, who authored and performed the deployment, and the deployment type.

Example prompts include:

- "List the deployments for web app 'webapp-prod' in resource group 'rg-prod'."
- "Get the deployment 'd4f8c9a2-1b3e-4c5d-9f7a-123456abcdef' for web app 'webapp-prod' in resource group 'rg-prod'."

 Expand table

Parameter	Required or optional	Description
App	Required	The name of the Azure App Service (for example, <code>my-webapp</code>).
Resource group	Required	The name of the Azure resource group. This resource group is a logical container for Azure resources.
Deployment ID	Optional	The ID of the deployment.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Get web app details

Retrieves detailed information about Azure App Service web apps, including app name, resource group, location, runtime stack, state, and hostnames.

Example prompts include:

- "List the web apps in my subscription."
- "Show me the web apps in resource group 'rg-prod'."
- "Get the details for web app 'api-staging' in resource group 'rg-staging'."

 Expand table

Parameter	Required or optional	Description
App	Optional	The name of the Azure App Service web app (for example, <code>contoso-webapp</code>). This tool returns details for a specific web app when you provide the <code>App</code> name; if you don't provide an <code>App</code> , it returns information for all web apps in the subscription or for the specified <code>resource group</code> and <code>subscription</code> .
Resource group	Optional	The name of the Azure resource group. This resource group is a logical container for Azure resources.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Get app settings

This tool retrieves the application settings for an Azure App Service web app and returns key-value pairs for each setting. App settings can include connection strings and other sensitive values, so treat returned values as secrets and limit their exposure.

Example prompts include:

- "List the application settings for web app 'my-webapp' in resource group 'rg-prod'."
- "Get the application settings for web app 'orders-api' in resource group 'rg-staging'."

 Expand table

Parameter	Required or optional	Description
App	Required	The name of the Azure App Service web app (for example, <code>my-webapp</code>).
Resource group	Required	The name of the Azure resource group that contains the web app (for example, <code>prod-rg</code>).

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ✅ | Local Required: ❌

List diagnostic detectors

This tool retrieves detailed information about detectors for a specified Azure App Service web app. For each detector, it returns the detector name, detector type, description, category, and analysis types. The results help you investigate issues and understand the diagnostics available for the web app.

Example prompts include:

- "List the diagnostic detectors for web app 'my-webapp' in resource group 'rg-prod'."

 Expand table

Parameter	Required or optional	Description
App	Required	The name of the Azure App Service (for example, <code>my-webapp</code>).
Resource group	Required	The name of the Azure resource group. This resource group is a logical container for Azure resources.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Update app settings

This tool updates an application setting for an App Service web app. You can choose one of three update types: `add`, `set`, or `delete`.

- `add`: Creates a new application setting with the specified name and value. If the application setting already exists, the operation fails and returns an error.
- `set`: Creates or updates the value of an application setting. If the application setting doesn't exist, `set` behaves like `add`. If it exists, the value is overwritten.
- `delete`: Removes the specified application setting. If the application setting doesn't exist, no action is taken.

For the `add` and `set` update types, both the application setting name and value are required. For the `delete` update type, only the application setting name is required.

Example prompts include:

- "Add application setting name 'feature-flag' with value 'true' to app 'my-webapp' in resource group 'rg-prod' with setting update type 'add'."

- "Set application setting name 'db-connection-string' with value 'Server=tcp:contoso-sql.database.windows.net;Initial Catalog=orders' on app 'orders-webapp' in resource group 'rg-orders' with setting update type 'set'."
- "Delete application setting name 'old-api-key' from app 'legacy-service' in resource group 'rg-archive' with setting update type 'delete'."

 Expand table

Parameter	Required or optional	Description
App	Required	The name of the Azure App Service (for example, <code>my-webapp</code>).
Resource group	Required	The name of the Azure resource group that contains the web app.
Setting name	Required	The name of the application setting.
Setting update type	Required	The type of update to perform on the application setting. Valid values: <code>add</code> , <code>set</code> , <code>delete</code> .
Setting value	Optional	The value of the application setting. Required for <code>add</code> and <code>set</code> update types.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure App Service documentation](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Azure Bicep Schema tools for the Azure MCP Server overview

The Azure MCP Server enables you to manage Azure resources, including Azure Bicep schemas, with natural language prompts. With this capability, you can quickly retrieve the latest API versions and property definitions for your Infrastructure as Code templates without needing to remember complex syntax.

[Azure Bicep](#) is a domain-specific language (DSL) that simplifies the authoring experience for Azure Resource Manager templates. Bicep offers concise syntax, reliable type safety, and support for all resource types and API versions.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Get Bicep Resource Schema

Gets the Bicep schema for the most recent apiVersion of an Azure resource. This operation helps you ensure your Bicep templates use the correct properties and values when defining Azure resources.

Example prompts include:

- **Get storage account schema:** "Get me the Bicep schema for Microsoft.Storage/storageAccounts"
- **Find service properties:** "How can I use Bicep to create an Azure OpenAI service with resource type 'Microsoft.CognitiveServices/accounts'?"
- **Check API version:** "What's the latest apiVersion for Microsoft.KeyVault/vaults?"
- **Need schema help:** "I'm creating a Bicep template for Microsoft.Cognitive/accounts"
- **Request schema guidance:** "Show me the properties for Microsoft.Web/sites"

Parameter	Required or optional	Description
Resource type	Required	The Azure resource type in format <code>{ResourceProvider}/{ResourceType}</code> (such as <code>Microsoft.Storage/storageAccounts</code> , <code>Microsoft.Compute/virtualMachines</code>)

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Bicep documentation](#)
- [Azure Resource Manager template reference](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/03/2026

Azure CLI tools for the Azure MCP Server overview

The Azure MCP Server provides comprehensive support for Azure CLI operations, including finding commands, generating command syntax, and providing installation instructions. Perform virtually any Azure resource management operation without needing to remember specific command syntax, parameters, or formatting.

[Azure Command-Line Interface \(CLI\)](#) is a cross-platform command-line tool to connect to Azure and execute administrative commands on Azure resources. It allows the execution of commands through a terminal using interactive command-line prompts or a script. For a complete list of Azure CLI commands this tool executes, see the [Azure CLI reference documentation](#).

ⓘ **Note**

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Generate Azure CLI commands

The Azure MCP Server generates Azure CLI commands to accomplish specific goals.

Example prompts include:

- **Generate creation commands:** "Generate an `az` command to create a storage account"
- **Generate query commands:** "Create an `az` command to list all virtual machines in a resource group"

 Expand table

Parameter	Required or optional	Description
Intent	Required	The user intent of the task to be solved by using the CLI tool. This user intent is used to generate the appropriate CLI command to accomplish the

Parameter	Required or optional	Description
desirable goal.		
Cli type	Required	The type of CLI tool to use. Supported values are <code>az</code> for Azure CLI.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Get CLI installation instructions

The Azure MCP Server provides installation instructions for CLI tools including Azure CLI (`az`), Azure Developer CLI (`azd`), and Azure Functions Core Tools CLI (`func`). It incorporates knowledge of the CLI tool beyond what the LLM knows. Use this tool to get installation instructions if you attempt to use the CLI tool but it's not installed.

Example prompts include:

- Azure CLI installation: "How do I install the `az` CLI?"
- Azure Developer CLI installation: "Show me how to install `azd`"
- Azure Functions Core tools: "What is Azure Functions Core tools and how to install it"

 Expand table

Parameter	Required or optional	Description
CLI type	Required	The type of CLI tool to use. Supported values are <code>az</code> for Azure CLI, <code>azd</code> for Azure Developer CLI, and <code>func</code> for Azure Functions Core Tools CLI.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure CLI reference documentation](#)

- [Azure Developer CLI reference documentation](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/27/2026

Azure MCP Server tools for Azure Cloud Architect

The Azure Model Context Protocol (MCP) Server lets you design cloud architectures through guided requirements gathering and receive optimal Azure solution recommendations with natural language prompts.

Azure Cloud Architect helps you design scalable, resilient Azure solutions and apply guidance from the Azure Architecture Center; for more information, see [Azure Architecture Center documentation](#).

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Design cloud architecture

This tool recommends architecture designs for cloud services, applications, and solutions — including file storage, banking, video streaming, e-commerce, SaaS, and more. It gathers requirements iteratively by asking 1–2 focused questions at a time, tracks a confidence score (0.0–1.0), and returns architecture guidance aligned with the Azure Well-Architected Framework. When the confidence score reaches 0.7 or higher, the tool stops asking follow-up questions and presents the architecture recommendation.

The tool covers all tiers: infrastructure, platform, application, data, security, and operations. Recommendations are conservative, actionable, and provide a high-level overview.

Example prompts include:

- "Please help me design an architecture for a scalable file upload, storage, and retrieval service."
- "Help me design an Azure-based ATM service architecture for user transactions and account management."
- "I want to design a cloud app for ordering groceries with inventory and delivery tracking."

- "How can I design an Azure cloud service to store, transcode, and serve videos to users?"

 Expand table

Parameter	Required or optional	Description
Answer	Optional	The user's response to the current question.
Confidence score	Optional	A value between 0.0 and 1.0 representing confidence in understanding requirements. When this reaches 0.7 or higher, <code>nextQuestionNeeded</code> should be set to false.
Next question needed	Optional	Whether another question is needed.
Question	Optional	The current question being asked.
Question number	Optional	Current question number.
State	Optional	The complete architecture state from the previous request as JSON. Tracks architecture components, tiers (infrastructure, platform, application, data, security, operations), requirements (explicit, implicit, assumed), and confidence factors.
Total questions	Optional	Estimated total questions needed.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Architecture Center documentation](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Azure Communication Services tools for the Azure MCP Server overview

The Azure MCP Server lets you manage Azure Communication Services using natural language prompts. Learn how to send emails and SMS messages without needing to remember complex command syntax, making communication automation more accessible and efficient.

[Azure Communication Services](#) is a set of rich communication APIs that enable developers to build intelligent communication solutions. These solutions include voice and video calling, chat, SMS, and telephony capabilities in applications.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Email: Send email

Send an email message using Azure Communication Services.

Example prompts include:

- **Simple email:** "Send email to 'user@example.com' with subject 'Welcome' and message 'Hello there!' from 'noreply@mydomain.com' using endpoint 'https://myservice.communication.azure.com'"
- **Email with sender name:** "Send email from 'Support Team [support@mydomain.com](#)' to 'customer@example.com' with subject 'Thank you' and message 'Thanks for your purchase' using endpoint 'https://myservice.communication.azure.com'"
- **HTML email:** "Send HTML email to 'subscriber@example.com' with subject 'Newsletter' and HTML message '<h1>Latest News</h1><p>Check out our updates!</p>' from 'newsletter@mydomain.com' using endpoint 'https://myservice.communication.azure.com'"

Parameter	Required or optional	Description
Endpoint	Required	The Communication Services URI endpoint (for example, <code>https://myservice.communication.azure.com</code>). Used for credential authentication.
From	Required	The email address to send from (must be from a verified domain).
Sender name	Optional	The display name of the sender.
To	Required	The recipient email addresses to send the email to.
Cc	Optional	CC recipient email addresses.
Bcc	Optional	BCC recipient email addresses.
Subject	Required	The email subject.
Message	Required	The email message content to send to the recipients.
Is html	Optional	Flag indicating whether the message content is HTML.
Reply to	Optional	Reply-to email addresses.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

SMS: Send SMS message

Sends SMS messages to one or more recipients using Azure Communication Services.

Example prompts include:

- **Simple SMS:** "Send an SMS message to '+12345550123' saying 'Hello' from '+12345550456' using endpoint 'https://myservice.communication.azure.com'"
- **Specify sender and recipient:** "Send SMS to '+12345550789' from '+12345550456' with message 'Test message' using endpoint 'https://myservice.communication.azure.com'"
- **Multiple recipients:** "Send SMS to multiple recipients: '+12345550123', '+12345550789' with message 'Group announcement' from '+12345550456' using endpoint 'https://myservice.communication.azure.com'"

 Expand table

Parameter	Required or optional	Description
Endpoint	Required	The Communication Services URI endpoint (for example, <code>https://myservice.communication.azure.com</code>). Used for credential authentication.
From	Required	The SMS-enabled phone number associated with your Communication Services resource (in E.164 format, for example, <code>+14255550123</code>). Can also be a short code or alphanumeric sender ID.
To	Required	The recipient phone numbers in <code>E.164</code> international standard format (for example, <code>+14255550123</code>). Multiple numbers can be provided.
Message	Required	The SMS message content to send to the recipients.
Enable delivery report	Optional	Whether to enable delivery reporting for the SMS message. When enabled, events are emitted when delivery is successful.
Tag	Optional	Optional custom tag to apply to the SMS message for tracking purposes.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Communication Services](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Azure Confidential Ledger tools for the Azure MCP Server overview

The Azure MCP Server enables you to manage Azure resources, including Azure Confidential Ledger Services, by using natural language prompts. This capability lets you work with confidential ledger services without needing to remember complex command syntax.

[Azure Confidential Ledger](#) is a fully managed, secure, and highly available ledger service that provides a trusted environment for storing sensitive data. It applies trusted execution environments (TEEs) to ensure data integrity and confidentiality, making it suitable for scenarios that require tamper-proof records, such as financial transactions, supply chain management, and compliance auditing.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Append an entry to the ledger

Appends an immutable (tamper-proof) entry to a Confidential Ledger instance and returns the transaction identifier.

Example prompts include:

- **Simple data entry:** "Append an entry to 'audit-ledger' with data {"key": "value"}"
- **Tamper-proof transaction:** "Write a tamper-proof entry to ledger 'financial-ledger' containing {"transaction": "data"}"
- **Collection-specific entry:** "Append {"hello": "from tool"} to my confidential ledger 'test-ledger' in collection 'user-data'"
- **Immutable audit log:** "Create an immutable ledger entry in 'compliance-ledger' with content {"audit": "log"}"
- **Basic ledger write:** "Write an entry to confidential ledger 'business-ledger' with data {"timestamp": "2025-10-08", "event": "user_login"}"

- **Financial transaction:** "Append a financial transaction to ledger 'bank-ledger' with content {"amount": 1000, "account": "123456", "type": "deposit"}"
- **Supply chain record:** "Write tamper-proof entry to ledger 'supply-chain' with data {"product_id": "ABC123", "location": "warehouse", "status": "shipped"}"
- **Compliance entry:** "Create an audit entry in confidential ledger 'regulatory-ledger' in collection 'gdpr-logs' with content {"user_id": "user123", "action": "data_deletion", "timestamp": "2025-10-08T10:30:00Z"}"
- **Security log:** "Append security event to ledger 'security-ledger' with data {"event_type": "login_attempt", "ip_address": "192.168.1.1", "success": true}"
- **Document hash:** "Write document integrity record to ledger 'document-ledger' in collection 'contracts' with content {"document_id": "contract_001", "hash": "sha256:abc123", "verified": true}"

 Expand table

Parameter	Required or optional	Description
Ledger	Required	The name of the Confidential Ledger instance (for example, <code>myledger</code>).
Content	Required	The JSON or text payload to append as a tamper-proof ledger entry.
Collection ID	Optional	Optional ledger collection identifier. If omitted the default collection is used.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Get an entry from the ledger

Retrieves the Confidential Ledger entry and its recorded contents for the specified transaction ID, optionally scoped to a collection.

Example prompts include:

- **Retrieve specific transaction:** "Get entry from confidential ledger 'audit-ledger' with transaction ID '2.199'"

- **Collection-specific retrieval:** "Get entry with transaction ID '3.275' from ledger 'compliance-ledger' in collection 'gdpr-logs'"

 Expand table

Parameter	Required or optional	Description
Ledger	Required	The name of the Confidential Ledger instance (for example, <code>myledger</code>).
Transaction ID	Required	The Confidential Ledger transaction identifier (for example: <code>2.199</code>).
Collection ID	Optional	Optional ledger collection identifier. If omitted the default collection is used.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Confidential Ledger](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/27/2026

Azure MCP Server tools for Azure Compute overview

The Azure MCP Server tools help you manage virtual machines, virtual machine scale sets, and disks by using natural language prompts. By using key capabilities such as creating, retrieving, and updating resources, you can efficiently control your cloud environment.

Azure Compute provides scalable computing resources for applications and workloads. For more information, see [Azure Compute documentation](#).

① Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Managed disk: create

Creates a new Azure managed disk in the specified resource group. You can create empty disks (specify `size-gb`), disks from a source such as a snapshot, another managed disk, or a blob URI (specify `source`), disks from a Shared Image Gallery image version (specify `gallery-image-reference`), or disks ready for upload (specify `upload-type` and `upload-size-bytes`). If you don't specify the location, it defaults to the resource group's location. You can configure disk size, storage SKU (for example, `Premium_LRS`, `Standard_LRS`, `UltraSSD_LRS`), OS type, availability zone, hypervisor generation, tags, encryption settings, performance tier, shared disk, on-demand bursting, and IOPS/throughput limits for UltraSSD disks. Create a disk with network access policy `DenyAll`, `AllowAll`, or `AllowPrivate`, and associate a disk access resource during creation.

Example prompts include:

- "Create a 128 GB managed disk named `<disk-name>` in resource group `<resource-group>`"
- "Create a new `Premium_LRS` disk called `<disk-name>` in resource group `<resource-group>` with 256 GB"
- "Create a managed disk `<disk-name>` in resource group `<resource-group>` in `eastus`"
- "Create a disk from snapshot `<snapshot-resource-id>` in resource group `<resource-group>`"
- "Create a managed disk `<disk-name>` in resource group `<resource-group>` from blob `<blob-uri>`"
- "Create a 64 GB `Standard_LRS` Linux disk named `<disk-name>` in resource group `<resource-group>` in zone 1"
- "Create a managed disk `<disk-name>` in resource group `<resource-group>` with tags `env=prod team=infra`"
- "Create a 128 GB `Premium_LRS` disk named `<disk-name>` in resource group `<resource-group>` with performance tier `P30`"
- "Create a disk `<disk-name>` in resource group `<resource-group>` with customer-managed encryption using disk encryption set `<disk-encryption-set-id>`"

- "Create a managed disk from gallery image version `<image-version-resource-id>` in resource group `<resource-group>` "
- "Create a data disk from LUN 0 of gallery image version `<image-version-resource-id>` in resource group `<resource-group>` "
- "Create a disk ready for upload named `<disk-name>` in resource group `<resource-group>` with upload size 20972032 bytes"
- "Create a Trusted Launch upload disk named `<disk-name>` in resource group `<resource-group>` with `UploadWithSecurityData` type and security type `TrustedLaunch` "
- "Create an `UltraSSD_LRS` disk named `<disk-name>` in resource group `<resource-group>` with 256 GB, 10000 IOPS, and 500 MBps throughput"
- "Create a shared managed disk named `<disk-name>` in resource group `<resource-group>` with 512 GB and max shares set to 3"
- "Create a managed disk `<disk-name>` in resource group `<resource-group>` with network access policy `DenyAll` and disk access `<disk-access-resource-id>` "
- "Create a 128 GB managed disk named `<disk-name>` in resource group `<resource-group>` with on-demand bursting enabled"
- "Create a managed disk `<disk-name>` in resource group `<resource-group>` with encryption type `EncryptionAtRestWithPlatformAndCustomerKeys` "
- "Create a V2 hypervisor generation disk named `<disk-name>` in resource group `<resource-group>` with 128 GB"

[Expand table](#)

Parameter	Required or optional	Description
Disk name	Required	The name of the disk.
Resource group	Required	The name of the Azure resource group. This name is a logical container for Azure resources.
Disk access	Optional	Resource ID of the disk access resource for using private endpoints on disks.
Disk encryption set	Optional	Resource ID of the disk encryption set to use for enabling encryption at rest.
Disk iops read write	Optional	The number of IOPS allowed for this disk. Only settable for UltraSSD disks.
Disk mbps read write	Optional	The bandwidth allowed for this disk in MBps. Only settable for UltraSSD disks.
Enable bursting	Optional	Enable on-demand bursting beyond the provisioned performance target of the disk. Doesn't apply to Ultra values: <code>true</code> , <code>false</code> .
Encryption type	Optional	Encryption type of the disk. Accepted values: <code>EncryptionAtRestWithCustomerKey</code> , <code>EncryptionAtRestWithPlatformAndCustomerKeys</code> , <code>EncryptionAtRestWithPlatformKey</code> .
Gallery image reference	Optional	Resource ID of a Shared Image Gallery image version to use as the source for the disk. Format: <code>/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.Compute/galleries/{gallery}/images/{image}</code>
Gallery image reference lun	Optional	LUN (Logical Unit Number) of the data disk in the gallery image version. If specified, the disk is created from this LUN. If not specified, the disk is created from the OS disk of the image.
Hyper v generation	Optional	The hypervisor generation of the Virtual Machine. Applicable to OS disks only. Accepted values: <code>v1</code> , <code>v2</code> .

Parameter	Required or optional	Description
Location	Optional	The Azure region/location. Defaults to the resource group's location if not specified.
Max shares	Optional	The maximum number of VMs that can attach to the disk at the same time. A value greater than one indicates
Network access policy	Optional	Policy for accessing the disk via network. Accepted values: <code>AllowAll</code> , <code>AllowPrivate</code> , <code>DenyAll</code> .
Os type	Optional	The Operating System type of the disk. Accepted values: <code>Linux</code> , <code>Windows</code> .
Security type	Optional	Security type of the managed disk. Accepted values: <code>ConfidentialVM_DiskEncryptedWithCustomerKey</code> , <code>ConfidentialVM_DiskEncryptedWithPlatformKey</code> , <code>ConfidentialVM_VMGuestStateOnlyEncryptedWithPlatformKey</code> , <code>TrustedLaunch</code> . Required when <code>upload-type</code> is <code>UploadWithSecurityData</code> .
Size gb	Optional	Size of the disk in GB. Max size: 4095 GB.
SKU	Optional	Underlying storage SKU. Accepted values: <code>Premium_LRS</code> , <code>PremiumV2_LRS</code> , <code>Premium_ZRS</code> , <code>StandardSSD_LRS</code> , <code>Standard_LRS</code> , <code>UltraSSD_LRS</code> .
Source	Optional	Source to create the disk from, including a resource ID of a snapshot or disk, or a blob URI of a VHD. When provided, <code>size-gb</code> is optional and defaults to the source size.
Tags	Optional	Space-separated tags in 'key=value' format. Use "" to clear existing tags.
Tier	Optional	Performance tier of the disk (for example, <code>P10</code> , <code>P15</code> , <code>P20</code> , <code>P30</code> , <code>P40</code> , <code>P50</code> , <code>P60</code> , <code>P70</code> , <code>P80</code>). Applicable to Premium only.
Upload size bytes	Optional	The size in bytes (including the VHD footer of 512 bytes) of the content to be uploaded. Required when <code>upload-type</code> is <code>Upload</code> .
Upload type	Optional	Type of upload for the disk. Accepted values: <code>Upload</code> , <code>UploadWithSecurityData</code> . When specified, the disk is in <code>ReadyToUpload</code> state.
Zone	Optional	Availability zone into which to provision the resource.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Managed disk: delete

Delete an Azure managed disk from the specified resource group. This operation is idempotent - it returns success whether the disk was removed or didn't exist.

Example prompts include:

- "Delete the managed disk 'temp-data-disk' in resource group 'dev-rg'."
- "Remove managed disk 'old-backup-disk' from resource group 'prod-rg'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group.
Disk name	Required	The name of the disk to delete.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Managed disk: list or get

Lists available Azure managed disks or retrieves detailed information about a specific disk. You can view all disks in a subscription or in a specific resource group, including disk size, SKU, provisioning state, and OS type. The tool supports wildcard patterns in disk names (for example, `win_0sDisk*`). If you provide a disk name without specifying a resource group, it searches across the entire subscription. Specifying a resource group scopes the search to that resource group. Both parameters are optional.

Example prompts include:

- "List all managed disks in my subscription."
- "Show me all disks in resource group `<resource-group>`."
- "Get details of disk `<disk-name>`."
- "What are the available disk sizes?"
- "Show me the disks with name pattern `win_0sDisk*` in resource group `<resource-group>`."
- "Get information about disk `<disk-name>` in resource group `<resource-group>`."

 Expand table

Parameter	Required or optional	Description
Disk name	Optional	The name of the disk.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Managed disk: update

Update or modify properties of an existing Azure managed disk that you previously created. If you don't specify the resource group, the disk is located by name within the subscription. This operation supports changing disk size (only increases are allowed), storage SKU, IOPS and throughput limits (for UltraSSD only), maximum shares for shared disk attachments, on-demand bursting, tags, encryption settings, disk access, and performance tier. You can modify the network access policy to `DenyAll`, `AllowAll`, or `AllowPrivate` on an existing disk. Only specified properties are updated; unspecified properties remain unchanged.

Example prompts include:

- "Update disk `<disk-name>` in resource group `<resource-group>` to 1024 GB"
- "Change the SKU of disk `<disk-name>` to `UltraSSD_LRS`"
- "Resize disk `<disk-name>` in resource group `<resource-group>` to 2048 GB"
- "Update disk `<disk-name>` to disable bursting"
- "Set the max shares on disk `<disk-name>` to 3"
- "Change the network access policy of disk `<disk-name>` to `AllowPrivate`"
- "Update disk `<disk-name>` in resource group `<resource-group>` with tags `env=production`"
- "Set the IOPS limit on Ultra disk `<disk-name>` in resource group `<resource-group>` to 15000"

- "Update the throughput of disk `<disk-name>` in resource group `<resource-group>` to 1000 MBps"
- "Change the performance tier of disk `<disk-name>` in resource group `<resource-group>` to `P50`"
- "Update disk `<disk-name>` in resource group `<resource-group>` to use disk encryption set `<disk-encryption-set-id>`"
- "Change the encryption type of disk `<disk-name>` in resource group `<resource-group>` to `EncryptionAtRestWithCustomerKey`"
- "Set disk access on disk `<disk-name>` in resource group `<resource-group>` to `<disk-access-resource-id>` with network access policy `DenyAll`"
- "Update disk `<disk-name>` to `PremiumV2_LRS` SKU with 256 GB size and tags `env=test`"

[Expand table](#)

Parameter	Required or optional	Description
Disk name	Required	The name of the disk.
Disk access	Optional	Resource ID of the disk access resource for using private endpoints on disks.
Disk encryption set	Optional	Resource ID of the disk encryption set to use for enabling encryption at rest.
Disk iops read write	Optional	The number of IOPS allowed for this disk. Only settable for UltraSSD disks.
Disk mbps read write	Optional	The bandwidth allowed for this disk in MBps. Only settable for UltraSSD disks.
Enable bursting	Optional	Enable on-demand bursting beyond the provisioned performance target of the disk. Doesn't apply to Ultra disks. Accepted values: <code>true</code> , <code>false</code> .
Encryption type	Optional	Encryption type of the disk. Accepted values: <code>EncryptionAtRestWithCustomerKey</code> , <code>EncryptionAtRestWithPlatformAndCustomerKeys</code> , <code>EncryptionAtRestWithPlatformKey</code> .
Max shares	Optional	The maximum number of VMs that can attach to the disk at the same time. Value greater than one indicates a shared disk.
Network access policy	Optional	Policy for accessing the disk via network. Accepted values: <code>AllowAll</code> , <code>AllowPrivate</code> , <code>DenyAll</code> .
Size gb	Optional	Size of the disk in GB. Max size: 4095 GB.
SKU	Optional	Underlying storage SKU. Accepted values: <code>Premium_LRS</code> , <code>PremiumV2_LRS</code> , <code>Premium_ZRS</code> , <code>StandardSSD_LRS</code> , <code>StandardSSD_ZRS</code> , <code>Standard_LRS</code> , <code>UltraSSD_LRS</code> .
Tags	Optional	Space-separated tags in <code>key=value</code> format. Use <code>''</code> to clear existing tags.
Tier	Optional	Performance tier of the disk (for example, <code>P10</code> , <code>P15</code> , <code>P20</code> , <code>P30</code> , <code>P40</code> , <code>P50</code> , <code>P60</code> , <code>P70</code> , <code>P80</code>). Applicable to Premium SSD disks only.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Virtual machine: create

Create, deploy, or provision a single Azure Virtual Machine (VM). This command launches a new Linux or Windows VM with either SSH key or password authentication. It automatically creates networking resources (VNet, subnet, NSG, NIC, public IP) if you don't specify them. The default VM size is `Standard_DS1_v2`, and the default OS is Ubuntu 24.04 LTS if you don't specify otherwise.

You can create a Linux VM using an SSH public key by providing the key content or the path to the key file. For example, you can specify your public key file at `~/.ssh/id_rsa.pub`.

This command doesn't support creating Virtual Machine Scale Sets with multiple identical instances. Instead, use `vmss create`.

Example prompts include:

- "Create a new Linux VM named `<vm-name>` with SSH key in resource group `<resource-group>`"
- "Launch a virtual machine with the Ubuntu2404 image in `<resource-group>`"
- "Create a Windows VM named `<vm-name>` with an admin password in resource group `<resource-group>`"
- "Deploy VM `<vm-name>` in `<location>` with `Standard_DS1_v2` size"
- "Spin up a VM with `Standard_B2s` size and no public IP in resource group `<resource-group>`"
- "Create a Linux VM named `<vm-name>` in `<location>` with a custom network security group"

[Expand table](#)

Parameter	Required or optional	Description
Admin username	Required	The admin username for the VM. Required for VM creation.
Location	Required	The Azure region or location. Defaults to the resource group's location if you don't specify it.
Resource group	Required	The name of the Azure resource group. This name is a logical container for Azure resources.
VM name	Required	The name of the virtual machine.
Admin password	Optional	The admin password for Windows VMs or when the SSH key isn't provided for Linux VMs.
Image	Optional	The OS image to use. Can be URN (publisher:offer:SKU:version) or an alias like <code>Ubuntu2404</code> or <code>Win2022Datacenter</code> . Defaults to Ubuntu 24.04 LTS.
Network security group	Optional	Name of the network security group to use or create.
No public IP	Optional	Don't create or assign a public IP address.
OS disk size GB	Optional	OS disk size in GB. Defaults based on image requirements.
OS disk type	Optional	OS disk type: <code>Premium_LRS</code> , <code>StandardSSD_LRS</code> , <code>Standard_LRS</code> . Defaults based on VM size.
OS type	Optional	The Operating System type of the disk. Accepted values: <code>Linux</code> , <code>Windows</code> .
Public IP address	Optional	Name of the public IP address to use or create.

Parameter	Required or optional	Description
Source address prefix	Optional	Source IP address range for NSG inbound rules (for example, <code>203.0.113.0/24</code> or a specific IP). Defaults to <code>*</code> (any source).
SSH public key	Optional	SSH public key for Linux VMs. Can be the key content or path to a file.
Subnet	Optional	Name of the subnet within the virtual network.
Virtual network	Optional	Name of an existing virtual network to use. If you don't specify it, the command creates a new one.
VM size	Optional	The VM size (for example, <code>Standard_D2s_v3</code> or <code>Standard_B2s</code>). Defaults to <code>Standard_DS1_v2</code> if you don't specify it.
Zone	Optional	Availability zone into which to provision the resource.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Virtual machine: delete

Delete an Azure virtual machine permanently. This operation is irreversible and the VM data is lost. Use the `Force deletion` parameter to force-delete a VM that is in a running or failed state.

Example prompts include:

- "Delete VM 'test-vm-01' in resource group 'dev-rg'."
- "Remove virtual machine 'staging-web' from resource group 'staging-rg'."
- "Force delete VM 'stuck-vm' in resource group 'prod-rg'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group.
VM name	Required	The name of the virtual machine to delete.
Force deletion	Optional	Force delete the resource even if it's in a running or failed state.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Virtual machine: list or get

List or get Azure Virtual Machines (VMs) in a subscription or resource group. This command returns VM details, including the name, location, size, provisioning state, OS type, and instance view with runtime status and power state.

Example prompts include:

- "List all virtual machines in my subscription."

- "Show me all VMs in my subscription."
- "What virtual machines do I have?"
- "List virtual machines in resource group `resource-group-name`."
- "Show me VMs in resource group `resource-group-name`."
- "What VMs are in resource group `resource-group-name`?"
- "Get details for virtual machine `vm-name` in resource group `resource-group-name`."
- "Show me virtual machine `vm-name` in resource group `resource-group-name`."
- "What are the details of VM `vm-name` in resource group `resource-group-name`?"
- "Get virtual machine `vm-name` with instance view in resource group `resource-group-name`."
- "Show me VM `vm-name` with runtime status in resource group `resource-group-name`."
- "What is the power state of virtual machine `vm-name` in resource group `resource-group-name`?"
- "Get VM `vm-name` status and provisioning state in resource group `resource-group-name`."
- "Show me the current status of VM `vm-name`."

[Expand table](#)

Parameter	Required or optional	Description
Instance view	Optional	Include instance view details (only available when retrieving a specific VM).
VM name	Optional	The name of the virtual machine.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Virtual machine: update

Update, modify, or reconfigure an existing Azure virtual machine (VM). You can resize a VM, update tags, configure boot diagnostics, or change user data. You might need to deallocate the VM before resizing it to certain sizes.

Here are some example prompts for using this tool:

- "Add license type `Windows_Server` to VM `<vm-name>` in resource group `<resource-group-name>`"
- "Update user data for VM `<vm-name>` in resource group `<resource-group-name>`"
- "Resize VM `<vm-name>` in resource group `<resource-group-name>` to `Standard_B2s`"
- "Enable boot diagnostics for VM `<vm-name>` in resource group `<resource-group-name>`"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This name is a logical container for Azure resources.
VM name	Required	The name of the virtual machine.
Boot diagnostics	Optional	Enable or disable boot diagnostics: <code>true</code> or <code>false</code> .

Parameter	Required or optional	Description
License type	Optional	License type for Azure Hybrid Benefit: <code>Windows_Server</code> , <code>Windows_Client</code> , <code>RHEL_BYOS</code> , <code>SLES_BYOS</code> , or <code>None</code> to disable.
Tags	Optional	Space-separated tags in <code>key=value</code> format. Use <code>''</code> to clear existing tags.
User data	Optional	Base64-encoded user data for the VM. Use to update custom data scripts.
VM size	Optional	The VM size (for example, <code>Standard_D2s_v3</code> , <code>Standard_B2s</code>). Defaults to <code>Standard_DS1_v2</code> if not specified.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Virtual machine scale set: create

Create, deploy, or provision an Azure Virtual Machine Scale Set (VMSS) for running multiple identical VM instances. This tool helps you deploy workloads that require horizontal scaling, load balancing, or high availability across instances. The default configuration creates two instances of size `Standard_DS1_v2` running Ubuntu 24.04 LTS.

Create a scale set by specifying the `resource_group`, `VMSS name`, and `admin username`, along with other optional settings. Here are some example commands:

- "Create a virtual machine scale set named `my-vmss` in resource group `my-rg`."
- "Create a VMSS with four instances in `my-rg`."
- "Deploy a scale set with a Manual upgrade policy and two instances in `my-rg`."
- "Create a Linux VMSS with SSH public key from `'~/ .ssh/id_rsa.pub'` in `my-rg`."

 Expand table

Parameter	Required or optional	Description
Admin username	Required	The admin username for the VM. Required for VM creation.
Location	Required	The Azure region or location. Defaults to the resource group's location if you don't specify it.
Resource group	Required	The name of the Azure resource group. This name is a logical container for Azure resources.
Virtual machine scale set (VMSS) name	Required	The name of the virtual machine scale set.
Admin password	Optional	The admin password for Windows VMs or when an SSH key isn't provided for Linux VMs.
Image	Optional	The OS image to use. Can be a URN (publisher:offer:SKU:version) or alias like <code>Ubuntu2404</code> , <code>Win2022Datacenter</code> . Defaults to Ubuntu 24.04 LTS.
Instance count	Optional	Number of VM instances in the scale set. Default is 2.
Os disk size gb	Optional	OS disk size in GB. Defaults based on image requirements.

Parameter	Required or optional	Description
Os disk type	Optional	OS disk type: <code>Premium_LRS</code> , <code>StandardSSD_LRS</code> , <code>Standard_LRS</code> . Defaults based on VM size.
Os type	Optional	The Operating System type of the disk. Accepted values: Linux, Windows.
Ssh public key	Optional	SSH public key for Linux VMs. Can be the key content or path to a file.
Subnet	Optional	Name of the subnet within the virtual network.
Upgrade policy	Optional	Upgrade policy mode: <code>Automatic</code> , <code>Manual</code> , or <code>Rolling</code> . Default is <code>Manual</code> .
Virtual network	Optional	Name of an existing virtual network to use. If you don't specify it, the tool creates a new one.
VM size	Optional	The VM size (for example, <code>Standard_D2s_v3</code> , <code>Standard_B2s</code>). Defaults to <code>Standard_DS1_v2</code> if not specified.
Zone	Optional	Availability zone into which to provision the resource.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Virtual machine scale set: delete

Delete an Azure Virtual Machine Scale Set and all its VM instances permanently. This operation is irreversible. Use the `Force deletion` parameter to force-delete a scale set that is in a running or failed state.

Example prompts include:

- "Delete scale set 'web-frontend-vmss' in resource group 'prod-rg'."
- "Remove VMSS 'test-scaleset' from resource group 'dev-rg'."
- "Force delete virtual machine scale set 'stuck-vmss' in resource group 'staging-rg'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group.
VMSS name	Required	The name of the virtual machine scale set to delete.
Force deletion	Optional	Force delete the resource even if it's in a running or failed state.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Virtual machine scale set: list or get

List or get Azure Virtual Machine Scale Sets (VMSS) and their instances in a subscription or resource group. This tool returns scale set details, including name, location, SKU, capacity, upgrade policy, and individual VM instance information.

Example prompts include:

- "List all virtual machine scale sets in my subscription."
- "List virtual machine scale sets in resource group `<resource-group-name>`."
- "What scale sets are in resource group `<resource-group-name>`?"
- "Get details for virtual machine scale set `<vmss-name>` in resource group `<resource-group-name>`."
- "Show me VMSS `<vmss-name>` in resource group `<resource-group-name>`."
- "Show me instance `<instance-id>` of VMSS `<vmss-name>` in resource group `<resource-group-name>`."
- "What is the status of instance `<instance-id>` in scale set `<vmss-name>`?"

 Expand table

Parameter	Required or optional	Description
Instance ID	Optional	The instance ID of the virtual machine in the scale set.
Virtual machine scale set (VMSS) name	Optional	The name of the virtual machine scale set.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Virtual machine scale set: update

Update, modify, or reconfigure an existing Azure Virtual Machine Scale Set (VMSS). You can scale the instance count, resize VMs, change the upgrade policy, or update tags on a scale set. Some changes require `update-instances` to roll out to existing VMs. This tool doesn't create a new VMSS. Use `VMSS create` instead. To update a single VM, use `VM update`.

Example prompts include:

- "Update the capacity of VMSS `myScaleSet` to 15."
- "Enable overprovisioning on the scale set `myScaleSet`."
- "Change the VM size to `Standard_D4s_v3` for `myScaleSet`."
- "Clear existing tags on scale set `myScaleSet` in resource group `myResourceGroup`."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This name is a logical container for Azure resources.
Virtual machine scale set (VMSS) name	Required	The name of the virtual machine scale set.
Capacity	Optional	Number of VM instances (capacity) in the scale set.

Parameter	Required or optional	Description
Enable auto os upgrade	Optional	Enable automatic OS image upgrades. Requires health probes or the Application Health extension.
Overprovision	Optional	Enable or disable overprovisioning. When enabled, Azure provisions more VMs than requested and deletes extra VMs after deployment.
Scale in policy	Optional	Scale-in policy to determine which VMs to remove: <code>Default</code> , <code>NewestVM</code> , or <code>OldestVM</code> .
Tags	Optional	Space-separated tags in <code>key=value</code> format. Use <code>''</code> to clear existing tags.
Upgrade policy	Optional	Upgrade policy mode: <code>Automatic</code> , <code>Manual</code> , or <code>Rolling</code> . Default is <code>Manual</code> .
VM size	Optional	The VM size (for example, <code>Standard_D2s_v3</code> , <code>Standard_B2s</code>). Defaults to <code>Standard_DS1_v2</code> if not specified.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Virtual Machines documentation](#)

Last updated on 04/13/2026

Azure MCP Server tools for Azure Container Apps

The Azure MCP Server lets you manage containerized applications on Azure Container Apps, including: list, with natural language prompts.

Azure Container Apps is a fully managed serverless container platform for building and running microservices and containerized applications. For more information, see [Azure Container Apps documentation](#).

⚠ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

List container apps

This tool, part of the Model Context Protocol (MCP), lists Azure Container Apps in a subscription. You can optionally filter results by a resource group. Each returned container app includes the following properties: `name`, `location`, `resourceGroup`, `managedEnvironmentId`, and `provisioningState`. If no container apps are found, this tool returns an empty list of results, consistent with other list tools.

Example prompts include:

- "List all Azure Container Apps in my subscription."
- "Show me my Azure Container Apps."
- "List container apps in resource group 'rg-prod'."
- "Show me the container apps in resource group 'webapp-dev'."

[Tool annotation hints](#):

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Container Apps documentation](#)

ⓘ **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 04/13/2026

Azure Container Registry tools for the Azure MCP Server overview

The Azure MCP Server enables you to manage Azure resources, including Azure Container Registries, by using natural language prompts. This capability lets you work with container registries without needing to remember complex command syntax.

[Azure Container Registry](#) enables you to build, store, and manage container images and artifacts in a private registry for all types of container deployments. Use Azure container registries with your existing container development and deployment pipelines. Use Azure Container Registry Tasks to build container images in Azure on-demand, or automate builds triggered by source code updates, updates to a container's base image, or timers.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Registry: List registry accounts in subscription

List accounts in a subscription. Optionally filter by resource group.

Example prompts include:

- **List registries:** "List all Azure Container Registries in my subscription."
- **Show registries:** "What container registries do I have?"
- **Find registries in a group:** "Show me all container registries in resource group 'devops-resources'."
- **Filter by resource group:** "List container registries in the resource group 'production-resources'."
- **Query registries:** "Can you list all my Azure Container Registries?"
- **Check registries:** "Container registries in subscription abc123"

[Tool annotation hints:](#)

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Registry: List repositories in registry

List repositories in Azure Container Registries. By default, the command lists repositories for all registries in the subscription.

Example prompts include:

- **List all repositories:** "List all repositories in my Azure Container Registries."
- **Show repositories for a registry:** "What repositories are in my registry 'myregistry'?"
- **Find specific repository:** "Show me the repository 'myapp' in registry 'myregistry'."
- **Filter by image name:** "List all repositories with images named 'myimage' in my registries."
- **Query repositories:** "Can you list all my container images?"
- **Check repository details:** "Get details for repository 'myapp' in registry 'myregistry'."

 Expand table

Parameter	Required or optional	Description
Registry	Optional	The name of the Azure Container Registry. This name is unique for your container registry.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Container Registry](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Azure MCP Server tools for Azure Cosmos DB

The Azure Model Context Protocol (MCP) Server lets you manage Azure Cosmos DB resources with natural language prompts. You can list accounts, databases, and containers, run SQL queries against containers, and inspect resource metadata.

Azure Cosmos DB is a globally distributed, multi-model database service. For more information, see [Azure Cosmos DB documentation](#).

ⓘ **Note**

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

List accounts, databases, or containers

List Azure Cosmos DB accounts, databases, or containers. By default, this tool returns all accounts in your subscription. Specify the `Account` to list databases in that account, or specify both the `Account` and the `Database` to list containers in that database. Results are returned at the level you specify: account, database, or container.

Example prompts include:

- "List all Azure Cosmos DB accounts in my subscription."
- "Show me the databases in the Azure Cosmos DB account 'prod-cosmos'."
- "List all the containers in the database 'orders-db' for the Azure Cosmos DB account 'my-cosmosdb'."

 Expand table

Parameter	Required or optional	Description
Account	Optional	The name of the Azure Cosmos DB account. When not specified, lists all accounts in the subscription. Specify this parameter to list databases, or

Parameter	Required or optional	Description
		combine with <code>Database</code> to list containers.
Database	Optional	The name of the database. Requires <code>Account</code> to be specified. When provided, lists containers within this database.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Query container items

Query items from an Azure Cosmos DB container. Provide the account name, database name, and container name, and optionally supply a SQL query to filter results. The query uses Azure Cosmos DB SQL API syntax and the tool returns matching items as JSON documents.

Example prompts include:

- "List all items from container 'orders' in database 'ecommerce-db' for Azure Cosmos DB account 'contoso-cosmos'."
- "Query items from container 'orders' in database 'ecommerce-db' for account 'contoso-cosmos' using the SQL query 'SELECT * FROM c WHERE c.status = shipped'."
- "Show items containing 'outage' in container 'orders' in database 'sales' for Azure Cosmos DB account 'my-cosmos-account'."

 Expand table

Parameter	Required or optional	Description
Account	Required	The name of the Azure Cosmos DB account to query (for example, <code>contoso-cosmos</code>).
Container	Required	The name of the container to query (for example, <code>orders</code>).
Database	Required	The name of the database to query (for example, <code>ecommerce-db</code>).
Query	Optional	SQL query to execute against the container. Uses Azure Cosmos DB SQL API syntax.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Cosmos DB documentation](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 03/24/2026

Azure Data Explorer tools for the Azure MCP Server overview

The Azure MCP Server allows you to manage Azure Data Explorer resources using natural language prompts. You can list clusters, view databases, query data with natural language. You can also use specific KQL queries for targeted responses.

[Azure Data Explorer](#) is a fast, fully managed data analytics service for real-time analysis on large volumes of data streaming from applications, websites, IoT devices, and more. Azure Data Explorer helps you analyze large volumes of diverse data from any data source, such as websites, applications, IoT devices, and more.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Conditional parameters

Some of the Azure Data Explorer tools require **one** of the following parameter sets within the conversation context:

- **Option 1:** Cluster URI
- **Option 2:** Both cluster name and subscription

Don't provide all three parameters (cluster URI, cluster name, and subscription) together, because this creates conflicting inputs.

Cluster: List clusters

The Azure MCP Server lists all Azure Data Explorer clusters in a subscription.

Example prompts include:

- **List clusters:** "Show me all Azure Data Explorer clusters in my subscription."

- **View clusters:** "What Azure Data Explorer clusters do I have available?"
- **Check clusters:** "List all my Azure Data Explorer clusters."
- **Query clusters:** "Show my Azure Data Explorer cluster organization."
- **Find clusters:** "Get all ADX clusters in my Azure subscription."

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Cluster: Get cluster details

The Azure MCP Server gets details for a specific Azure Data Explorer cluster.

Example prompts include:

- **Get details:** "Show me details of my Azure Data Explorer cluster 'analytics-cluster'."
- **View cluster:** "Give me information about my ADX cluster 'logs-prod'."
- **Cluster info:** "What are the details of Azure Data Explorer cluster 'data-explorer-dev'?"
- **Check configuration:** "Get configuration details of my ADX cluster 'telemetry-cluster'."
- **Cluster properties:** "Show properties of my Azure Data Explorer cluster in subscription 'my-sub'."

[Expand table](#)

Parameter	Required or optional	Description
Cluster	Required	The name of the Azure Data Explorer cluster.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Database: List databases

The Azure MCP Server lists all databases in an Azure Data Explorer cluster.

Example prompts include:

- **List databases:** "Show me all databases in Azure Data Explorer cluster 'analytics-cluster'."
- **View databases:** "What databases do I have in my ADX cluster 'analytics-cluster'?"

- **Check databases:** "List all databases in Data Explorer cluster 'analytics-cluster'."
- **Query databases:** "Show databases in Azure Data Explorer cluster URI 'https://mycluster.westus.kusto.windows.net'."
- **Find databases:** "Get all databases from ADX cluster 'analytics-cluster'."

 Expand table

Parameter	Required or optional	Description
Cluster URI	Conditionally required	The URI of the Azure Data Explorer cluster.
Cluster	Conditionally required	The name of the Azure Data Explorer cluster.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Table: List tables

The Azure MCP Server lists all tables in a specific Azure Data Explorer database.

Example prompts include:

- **List tables:** "Show me all tables in the 'logs' database of Azure Data Explorer cluster 'analytics-cluster'."
- **View tables:** "What tables do I have in database 'telemetry' in ADX cluster 'analytics-cluster'?"
- **Check tables:** "List all tables in Azure Data Explorer database 'analytics' in cluster 'analytics-cluster'."
- **Query tables:** "Show tables in the 'metrics' database of Data Explorer cluster 'analytics-cluster'."
- **Find tables:** "Get all tables from 'events' database in Azure Data Explorer cluster 'analytics-cluster'."

 Expand table

Parameter	Required or optional	Description
Cluster URI	Conditionally required	The URI of the Azure Data Explorer cluster.
Cluster	Conditionally required	The name of the Azure Data Explorer cluster.

Parameter	Required or optional	Description
Database	Required	The name of the Azure Data Explorer database.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local Required: ✖

Table: Get table schema

The Azure MCP Server gets the schema of a specific table in an Azure Data Explorer database.

Example prompts include:

- **View schema:** "Show me the schema of the 'Events' table in database 'logs' in Azure Data Explorer cluster 'analytics-cluster'."
- **Get structure:** "What columns does the 'Metrics' table have in database 'telemetry' in ADX cluster 'analytics-cluster'?"
- **Check schema:** "Describe the 'Logs' table in database 'logs' in Data Explorer cluster 'analytics-cluster'."
- **View columns:** "Show columns and types for 'Telemetry' table in database 'telemetry' in Azure Data Explorer cluster 'analytics-cluster'."
- **Examine table:** "Get the structure of 'Traces' table in database 'logs' in ADX cluster 'analytics-cluster'."

 Expand table

Parameter	Required or optional	Description
Cluster URI	Conditionally required	The URI of the Azure Data Explorer cluster.
Cluster	Conditionally required	The name of the Azure Data Explorer cluster.
Database	Required	The name of the Azure Data Explorer database.
Table	Required	The name of the table.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local Required: ✖

Sample data

The Azure MCP Server retrieves a sample of data from a specified Azure Data Explorer table.

Example prompts include:

- **Get sample data:** "Show me a sample of data from the 'Events' table in database 'logs' in Azure Data Explorer cluster 'analytics-cluster'."
- **Preview table:** "Give me a preview of records from the 'Logs' table in database 'logs' in ADX cluster 'analytics-cluster'."
- **View data examples:** "Show sample rows from 'Metrics' table in database 'telemetry' in Data Explorer cluster 'analytics-cluster'."
- **Check data format:** "Get a few sample records from the 'Telemetry' table in database 'telemetry' in Azure Data Explorer cluster 'analytics-cluster' to see the data structure."
- **Data exploration:** "Return 10 sample rows from 'UserActivity' table in database 'logs' in ADX cluster 'analytics-cluster'."

 Expand table

Parameter	Required or optional	Description
Cluster URI	Conditionally required	The URI of the Azure Data Explorer cluster.
Cluster	Conditionally required	The name of the Azure Data Explorer cluster.
Database	Required	The name of the Azure Data Explorer database.
Table	Required	The name of the table to sample data from.
Limit	Optional	The maximum number of rows to return in the sample.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Query

The Azure MCP Server executes a KQL query against an Azure Data Explorer database.

Example prompts include:

- **Run query:** "Execute 'Logs | where Timestamp > ago(1h) | count' in database 'logs' in Azure Data Explorer cluster 'analytics-cluster'."
- **Query data:** "Run KQL query 'Logs | where Level == \"Error\" and Timestamp > ago(24h)' to find all errors in the last 24 hours in database 'logs' in ADX cluster 'analytics-cluster'."
- **Fetch data:** "Get recent events with query 'Events | take 100' from database 'logs' in Data Explorer cluster 'analytics-cluster'."
- **Extract insights:** "Query user activity patterns with 'UserActivity | summarize count() by UserId' from database 'logs' in Azure Data Explorer cluster 'analytics-cluster'."
- **Analyze logs:** "Execute KQL 'Metrics | summarize avg(Duration) by Service' to summarize performance metrics by service in database 'telemetry' in ADX cluster 'analytics-cluster'."

 Expand table

Parameter	Required or optional	Description
Cluster URI	Conditionally required	The URI of the Azure Data Explorer cluster.
Cluster	Conditionally required	The name of the Azure Data Explorer cluster.
Database	Required	The name of the Azure Data Explorer database.
Query	Required	The KQL query to execute.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Data Explorer](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Azure MCP Server tools for Azure Database for MySQL

The Azure MCP Server tools help you manage Azure Database for MySQL servers, databases, configuration settings, and schemas. You can use the tools to get and list servers and databases, query table schemas and data, and set server parameters by using natural language prompts.

Azure Database for MySQL is a managed relational database service based on the MySQL community edition. For more information, see [Azure Database for MySQL documentation](#).

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Get MySQL servers databases

List Azure Database for MySQL servers, databases, or tables in your subscription. By default, this tool returns all servers. Specify the `server` parameter to list databases on a server, or specify both the `server` and `database` parameters to list tables in a database.

Example prompts include:

- "List all MySQL servers in resource group 'rg-prod' with user name 'dbadmin'."
- "Show me my MySQL servers for resource group 'web-rg' using user name 'mysqluser'."
- "What MySQL servers are in resource group 'rg-staging' for user name 'adminuser'?"
- "List all MySQL databases in server 'mysql-server-01' within resource group 'rg-prod' using user name 'dbadmin'."
- "Show me the MySQL databases on server 'mysql-dbserver' for resource group 'rg-dev' with user name 'mysqluser'."
- "List all tables in MySQL database 'salesdb' on server 'mysql-server-01' in resource group 'rg-prod' using user name 'dbadmin'."

- "Show me the tables in database 'inventory' on server 'mysql-dbserver' for resource group 'rg-test' with user name 'mysqluser'."

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group that contains the resources.
User name	Required	The user name to access the Azure Database for MySQL server.
Database name	Optional	The name of the Azure Database for MySQL database to list tables from. Requires the Server name parameter.
Server name	Optional	The name of the Azure Database for MySQL server to list databases from.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Examples

- List all Azure Database for MySQL servers in subscription 'contoso-subscription' and resource group 'prod-rg'.
- List databases on server 'mysql-prod-01' in resource group 'prod-rg'.
- List tables in database 'salesdb' on server 'mysql-prod-01' in resource group 'prod-rg'.

Query MySQL database

The Model Context Protocol (MCP) tool runs a safe, read-only SQL `SELECT` query against an Azure Database for MySQL Flexible Server database. Use this tool to retrieve or inspect table data without modifying it. The tool rejects non-`SELECT` statements such as `INSERT`, `UPDATE`, `DELETE`, `REPLACE`, `MERGE`, `TRUNCATE`, `ALTER`, `CREATE`, and `DROP`. It also rejects multistatements, comments that hide writes, transaction control (`BEGIN/COMMIT/ROLLBACK`), `INTO OUTFILE`, and other destructive keywords. This tool executes only a single `SELECT` statement to ensure data integrity.

For best results, list the columns you need instead of using `SELECT *`. Add `WHERE` filters, use `LIMIT/OFFSET` for paging, and add `ORDER BY` for deterministic results. Avoid returning

unnecessary sensitive data.

Example prompts include:

- "Execute query 'SELECT id, name, email FROM customers WHERE id > 100 ORDER BY name LIMIT 50' on database 'ecommerce_db' in resource group 'rg-prod' on server 'mysql-prod-server' as user 'readonlyuser'."

 Expand table

Parameter	Required or optional	Description
Database name	Required	The MySQL database to access.
Query	Required	Query to execute against a MySQL database.
Resource group	Required	The name of the Azure resource group. This resource group is a logical container for Azure resources.
Server name	Required	The MySQL server to access.
User name	Required	The user name to access MySQL server.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Get server config

This tool is part of the Model Context Protocol (MCP) tools. It retrieves comprehensive configuration details for a specified Azure Database for MySQL Flexible Server instance. The tool returns server settings, performance parameters, security configurations, and operational characteristics that help you manage and optimize the database. Output is JSON and includes ServerName, Location, Version, SKU, StorageSizeGB, BackupRetentionDays, and GeoRedundantBackup.

Example prompts include:

- "Show me the configuration of MySQL server 'mysql-prod' in resource group 'rg-prod' with user 'dbadmin'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group that contains the server.
Server name	Required	The name of the Azure Database for MySQL Flexible Server instance.
User name	Required	The user name to authenticate to the server.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Get server parameter

This Model Context Protocol (MCP) tool retrieves the current value of a single server configuration parameter on Azure Database for MySQL Flexible Server. Use this tool to inspect a setting, such as `max_connections`, `wait_timeout`, or `slow_query_log`, before you change it. This tool requires a user account with sufficient privileges to read server parameters.

Example prompts include:

- "Show me the value of parameter 'connection_timeout' in resource group 'rg-prod' for MySQL server 'my-mysql-server' with user name 'dbadmin'."

 Expand table

Parameter	Required or optional	Description
Parameter	Required	The MySQL parameter to access.
Resource group	Required	The name of the Azure resource group. This resource group is a logical container for Azure resources.
Server name	Required	The MySQL server to access.
User name	Required	The user name to access MySQL server.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Update server parameter

This tool, part of the Model Context Protocol (MCP), updates a single configuration setting on an Azure Database for MySQL server. You specify the resource group, server name, user name, and the value to set.

Example prompts include:

- "Set parameter 'connection_timeout' to value '20' on server name 'mysql-prod' in resource group 'rg-prod' with user name 'dbadmin'."

[🔗](#) Expand table

Parameter	Required or optional	Description
Parameter	Required	The MySQL parameter to access.
Resource group	Required	The name of the Azure resource group. This resource group is a logical container for Azure resources.
Server name	Required	The MySQL server to access.
User name	Required	The user name to access MySQL server.
Value	Required	The value to set for the MySQL parameter.

Tool annotation hints:

Destructive: ✅ | Idempotent: ✅ | Open World: ❌ | Read Only: ❌ | Secret: ❌ | Local Required: ❌

Examples

- Set `autocommit` to 'ON' for server 'my-mysql-server' in resource group 'prod-rg' using user 'dbadmin'.
- Set `slow_query_log` to 'ON' for server 'analytics-db' in resource group 'analytics-rg' using user 'monitor'.
- Set `max_connections` to '200' for server 'web-db-server' in resource group 'web-rg' using user 'dbadmin'.

Get table schema

This Model Context Protocol (MCP) tool retrieves detailed schema information for a specific table in an Azure Database for MySQL Flexible Server instance. It returns comprehensive metadata, including column definitions, data types, constraints, indexes, and relationships. This metadata helps you understand table structure and supports application development.

Example prompts include:

- "Show the schema of table 'orders' in database 'salesdb' on server 'mysql-prod' within resource group 'rg-db-prod' as user 'dbadmin'."

 Expand table

Parameter	Required or optional	Description
Database name	Required	The MySQL database to access.
Resource group	Required	The name of the Azure resource group that contains the server.
Server name	Required	The MySQL server that hosts the database.
Table name	Required	The table to retrieve schema information for.
User name	Required	The user name to authenticate to the MySQL server.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Database for MySQL documentation](#)

 **Note:** This article contains content created with AI. [Learn more](#)

Azure Database for PostgreSQL tools for the Azure MCP Server overview

The Azure MCP Server allows you to manage Azure Database for PostgreSQL resources using natural language prompts. You can query databases, list tables, retrieve schemas, and more without remembering complex query syntax.

[Azure Database for PostgreSQL](#) is a fully managed, intelligent, and scalable PostgreSQL database service in the cloud. It lets you focus on application development, not database management.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Database: List databases

The Azure MCP Server can list all databases in a PostgreSQL server.

Example prompts include:

- **List databases:** "Show me all databases in my PostgreSQL server 'my-pg-server' in resource group 'my-resource-group'"
- **View databases:** "What databases do I have in my PostgreSQL server 'my-pg-server' in resource group 'my-resource-group'?"
- **Check databases:** "Check that I have a database named 'xyz' in server 'my-pg-server' in resource group 'my-resource-group'"
- **Query databases:** "Show databases in PostgreSQL server 'my-pg-server' in resource group 'my-resource-group'"
- **Find databases:** "Get all databases from my PostgreSQL instance 'my-pg-server' in resource group 'my-resource-group'"

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
User	Required	The user name to access PostgreSQL server.
Server	Required	The PostgreSQL server to be accessed.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Database: Execute database query

The Azure MCP Server can execute a query on a PostgreSQL database.

Example prompts include:

- **Run query:** "Execute 'SELECT * FROM users LIMIT 10' in my PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'"
- **Query data:** "Run a query to get recent orders from PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'"
- **Fetch data:** "Get user information from my PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group' with query"
- **Extract data:** "Query customer data from my PostgreSQL server 'my-pg-server' in resource group 'my-resource-group'"
- **Retrieve records:** "Select top sales records from PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
User	Required	The user name to access PostgreSQL server.
Server	Required	The PostgreSQL server to be accessed.

Parameter	Required or optional	Description
Database	Required	The PostgreSQL database to be accessed.
Query	Required	Query to be executed against a PostgreSQL database.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local Required: ✖

Table: List tables

The Azure MCP Server can list all tables in a PostgreSQL database.

Example prompts include:

- **List tables:** "Show me all tables in my PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'"
- **View tables:** "What tables do I have in my PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'?"
- **Check tables:** "Check that I have a table named 'xyz' in PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'"
- **Query tables:** "Show tables in PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'"
- **Find tables:** "Get all tables from my PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
User	Required	The user name to access PostgreSQL server.
Server	Required	The PostgreSQL server to be accessed.
Database	Required	The PostgreSQL database to be accessed.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Table: Get table schema

The Azure MCP Server can get the schema of a specific table in a PostgreSQL database.

Example prompts include:

- **View schema:** "Show me the schema of the 'users' table in my PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'"
- **Get structure:** "What columns does the 'products' table have in my PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'?"
- **Check schema:** "Check if my schema has a not null constraint on the id column in database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'"
- **View columns:** "Show columns and types for 'customers' table in PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'"
- **Examine table:** "Get the structure of 'transactions' table in my PostgreSQL database 'my-db' on server 'my-pg-server' in resource group 'my-resource-group'"

[🔗 Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
User	Required	The user name to access PostgreSQL server.
Server	Required	The PostgreSQL server to be accessed.
Database	Required	The PostgreSQL database to be accessed.
Table	Required	The PostgreSQL table to be accessed.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Server: List servers

The Azure MCP Server can list all PostgreSQL servers in a subscription and resource group.

Example prompts include:

- **List servers:** "Show me all PostgreSQL servers in resource group 'my-resource-group'"
- **View servers:** "What PostgreSQL servers do I have in resource group 'my-resource-group'?"
- **Check servers:** "Check if resource group 'my-resource-group' has a server named 'xyz'"
- **Query servers:** "Show PostgreSQL servers in resource group 'my-resource-group'"
- **Find servers:** "Get all PostgreSQL instances in resource group 'my-resource-group'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
User	Required	The user name to access PostgreSQL server.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Server: Get server configuration

The Azure MCP Server can retrieve the configuration of a PostgreSQL server.

Example prompts include:

- **View configuration:** "Show me the configuration of my PostgreSQL server 'my-pg-server' in resource group 'my-resource-group'"
- **Get settings:** "What are the settings of my PostgreSQL server 'pg-prod' in resource group 'my-resource-group'?"
- **Check config:** "Check if my server 'my-pg-server' in resource group 'my-resource-group' configuration 'x' is set to 'y'"
- **View server params:** "Show me all configuration parameters of my PostgreSQL server 'my-pg-server' in resource group 'my-resource-group'"
- **Get server setup:** "What is the configuration of my PostgreSQL instance 'my-pg-server' in resource group 'my-resource-group'?"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
User	Required	The user name to access PostgreSQL server.
Server	Required	The PostgreSQL server to be accessed.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Server: Get server parameter

The Azure MCP Server can retrieve a specific parameter of a PostgreSQL server.

Example prompts include:

- **View parameter:** "Show me the 'max_connections' parameter of my PostgreSQL server 'my-pg-server' in resource group 'my-resource-group'"
- **Get setting:** "What is the value of 'shared_buffers' in my PostgreSQL server 'my-pg-server' in resource group 'my-resource-group'?"
- **Check parameter:** "Check if my server 'my-pg-server' in resource group 'my-resource-group' parameter 'x' is set to 'y'"
- **View server param:** "Show me the 'work_mem' parameter value in my PostgreSQL server 'my-pg-server' in resource group 'my-resource-group'"
- **Get configuration value:** "What is the 'maintenance_work_mem' set to in my PostgreSQL instance 'my-pg-server' in resource group 'my-resource-group'?"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
User	Required	The user name to access PostgreSQL server.

Parameter	Required or optional	Description
Server	Required	The PostgreSQL server to be accessed.
Param	Required	The PostgreSQL parameter to be accessed.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Server: Set server parameter

The Azure MCP Server can set or update a specific parameter on a PostgreSQL server. This allows you to configure server settings, optimize performance, and adjust database behavior according to your application requirements.

Example prompts include:

- **Update connection setting:** "Set the 'max_connections' parameter to '200' on my 'prod-postgres-server' in resource group 'my-resource-group'"
- **Configure memory:** "Update the 'shared_buffers' parameter to '256MB' on server 'database-server-east' in resource group 'my-resource-group'"
- **Adjust timeout:** "Set 'statement_timeout' to '30000' on my PostgreSQL server 'my-pg-server' in resource group 'my-resource-group'"
- **Configure logging:** "Update the 'log_statement' parameter to 'all' on server 'dev-postgres' in resource group 'my-resource-group'"
- **Set maintenance parameter:** "Configure 'maintenance_work_mem' to '64MB' on my database server 'my-pg-server' in resource group 'my-resource-group'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
User	Required	The user name to access the PostgreSQL server.
Server	Required	The PostgreSQL server name to configure.
Param	Required	The PostgreSQL parameter to be set.

Parameter	Required or optional	Description
Value	Required	The value to set for the parameter.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Database for PostgreSQL](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/27/2026

Azure Deploy tools for the Azure MCP Server overview

Azure MCP Server simplifies Azure resource deployment by providing a unified experience for deploying applications and infrastructure. This article explains how to use Azure MCP Server to streamline your deployment process and improve efficiency.

App: Get logs

This tool fetches logs from the [Log Analytics](#) workspace for Container Apps, App Services, and Function Apps deployed by using [Azure Developer CLI](#). Use it after a successful `azd up` to check app status or troubleshoot errors in deployed applications.

Example prompts include:

- **Fetch app logs:** "Get logs for my app service in workspace folder '/home/user/myapp' with AZD environment 'production'."
- **View deployment logs:** "Show me the latest deployment logs for my container app in workspace folder '/home/user/myapp' with AZD environment 'dev'."
- **Check function logs:** "Retrieve logs for my function app in workspace folder '/home/user/myapp' with AZD environment 'staging'."
- **Troubleshoot app:** "Show error logs for my web app deployed with azd in workspace folder '/home/user/myapp' and environment 'production'."
- **Get logs with limit:** "Get the last 50 logs for my app service in workspace folder '/home/user/myapp' with AZD environment 'prod' and limit 50."

 Expand table

Parameter	Required or optional	Description
Workspace folder	Required	The full path of the workspace folder.
AZD environment	Required	The name of the environment created by AZD in the AZURE_ENV_NAME variable during <code>azd init</code> or <code>azd up</code> .
Limit	Optional	The maximum row number of logs to retrieve. Use this to get a specific number of logs or to avoid the retrieved logs from reaching token limit. Default is 200.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Architecture: Generate mermaid diagram

Generate a [Mermaid](#) architecture diagram for the application topology.

Example prompts include:

- **Generate architecture diagram:** "Create a Mermaid diagram for my Azure application with input 'web app connected to SQL database'."
- **Show app topology:** "Visualize the architecture of my deployed services with input 'container app, function app, and cosmos db'."
- **Diagram resources:** "Generate a diagram for all resources with input 'app service, storage account, and key vault'."
- **App structure diagram:** "Show the structure with input 'container app processing queue messages and function app handling events'."
- **Service relationship diagram:** "Create a diagram with input 'web app connects to database and redis cache'."

[Expand table](#)

Parameter	Required or optional	Description
Raw input	Required	The raw input to process.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Infrastructure as Code: Get guidance

This tool offers guidance for creating [Bicep](#) or Terraform files to deploy applications on Azure. The guidelines outline rules to improve the quality of Infrastructure as Code files, ensuring they are compatible with the AZD tool and adhere to best practices.

Example prompts include:

- **laC guidance for Bicep:** "Give me best practices for Bicep files for my web app using deployment tool 'AZD' with laC type 'bicep' and resource types 'appservice'."
- **Terraform rules:** "What are the guidelines for writing Terraform for Azure Container Apps using deployment tool 'AZD' with laC type 'terraform' and resource types 'containerapp'?"
- **Resource-specific rules:** "Show me laC rules for deploying appservice and aks using deployment tool 'AzCli'."
- **AZD compatibility:** "How do I make my Bicep files compatible with deployment tool 'AZD' for laC type 'bicep'?"
- **laC for multiple resources:** "Provide guidance using deployment tool 'AZD' with laC type 'bicep' for resource types 'appservice,containerapp,function'."

[Expand table](#)

Parameter	Required or optional	Description
Deployment tool	Required	The deployment tool to use. Valid values: <code>AZD</code> , <code>AzCli</code> .
Infrastructure as code file type	Optional	The Infrastructure as Code type. Valid values: <code>bicep</code> , <code>terraform</code> . Leave empty if deployment tool is the Azure CLI.
Resource types	Optional	Specifies the Azure resource types to retrieve laC rules for. Use a comma-separated list. Supported values are: <code>appservice</code> , <code>containerapp</code> , <code>function</code> , <code>aks</code> . Leave empty if you don't use these services.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Pipeline: Get guidance

Get guidance to create a CI/CD pipeline that provisions Azure resources and builds and deploys applications to Azure. Use this tool before creating a GitHub Actions workflow file for deployment on Azure. Ensure infrastructure files are ready and the application is ready to be containerized.

Example prompts include:

- **CI/CD pipeline setup:** "How do I set up a CI/CD pipeline for my Azure app?"

- **GitHub Actions guidance:** "Give me guidance for creating a GitHub Actions workflow for deployment."
- **Pipeline for containerized app:** "What are the steps to build and deploy a container app using AZD?"
- **Environment-specific pipeline:** "Set up a pipeline for deploying to the 'staging' environment."
- **Pipeline configuration:** "Show me how to use azure.yaml for pipeline setup."

 Expand table

Parameter	Required or optional	Description
Use AZD pipeline configuration	Optional	Whether to use the AZD tool to set up the deployment pipeline. Set to true only if you provide <code>azure.yaml</code> or the context suggests AZD tools.
Organization	Optional	The name of the organization or the user account name of the current GitHub repository.
Repository	Optional	The name of the current GitHub repository.
GitHub environment	Optional	The name of the environment to which the deployment pipeline will be deployed.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Plan: Create deployment plan

Generates a deployment plan to build the infrastructure and deploy the application on Azure. The agent reads the output and creates a deployment plan in `.azure/plan.copilotmd` for execution steps, with recommended Azure services based on the information detected from the project.

Example prompts include:

- **Generate deployment plan:** "Create a deployment plan for my Azure web app in workspace folder '/home/user/myapp' with project 'mywebapp' using target app service 'WebApp' and provisioning tool 'AZD' with IaC option 'bicep'."

- **Plan for multiple services:** "Generate a plan to deploy container app in workspace folder '/home/user/myapp' with project 'mycontainerapp' using target app service 'ContainerApp' and provisioning tool 'AZD' with IaC option 'bicep'."
- **Deployment steps:** "Show me the steps to deploy my project 'myproject' in workspace folder '/home/user/myapp' using target app service 'WebApp' and provisioning tool 'AZD' with IaC option 'bicep'."
- **Service recommendation:** "Recommend Azure services and generate a plan for project 'myapp' in workspace folder '/home/user/myapp' using target app service 'ContainerApp' and provisioning tool 'AZD'."
- **Provisioning plan:** "Create a plan for project 'myproject' in workspace folder '/home/user/myapp' using target app service 'AKS' and provisioning tool 'AzCli'."

 Expand table

Parameter	Required or optional	Description
Workspace folder	Required	The full path of the workspace folder.
Project	Required	The name of the project to generate the deployment plan for. If not provided, the name is inferred from the workspace.
Target app service	Required	The Azure service to deploy the application. Valid values: <code>ContainerApp</code> , <code>WebApp</code> , <code>FunctionApp</code> , <code>AKS</code> . Recommend one based on user application.
Provisioning tool	Required	The tool to use for provisioning Azure resources. Valid values: <code>AZD</code> , <code>AzCli</code> . Use AzCli if TargetAppService is <code>AKS</code> .
Azd IaC options	Optional	The Infrastructure as Code option for AZD. Valid values: <code>bicep</code> , <code>terraform</code> . Leave empty if Deployment tool is <code>AzCli</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Developer CLI](#)

- [Azure CLI](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/03/2026

Azure CLI tools for the Azure MCP Server overview

The Azure MCP Server provides comprehensive support for Azure CLI operations, including finding commands, generating command syntax, and providing installation instructions. Perform virtually any Azure resource management operation without needing to remember specific command syntax, parameters, or formatting.

[Azure Command-Line Interface \(CLI\)](#) is a cross-platform command-line tool to connect to Azure and execute administrative commands on Azure resources. It allows the execution of commands through a terminal using interactive command-line prompts or a script. For a complete list of Azure CLI commands this tool executes, see the [Azure CLI reference documentation](#).

ⓘ **Note**

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Generate Azure CLI commands

The Azure MCP Server generates Azure CLI commands to accomplish specific goals.

Example prompts include:

- **Generate creation commands:** "Generate an `az` command to create a storage account"
- **Generate query commands:** "Create an `az` command to list all virtual machines in a resource group"

 Expand table

Parameter	Required or optional	Description
Intent	Required	The user intent of the task to be solved by using the CLI tool. This user intent is used to generate the appropriate CLI command to accomplish the

Parameter	Required or optional	Description
desirable goal.		
Cli type	Required	The type of CLI tool to use. Supported values are <code>az</code> for Azure CLI.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Get CLI installation instructions

The Azure MCP Server provides installation instructions for CLI tools including Azure CLI (`az`), Azure Developer CLI (`azd`), and Azure Functions Core Tools CLI (`func`). It incorporates knowledge of the CLI tool beyond what the LLM knows. Use this tool to get installation instructions if you attempt to use the CLI tool but it's not installed.

Example prompts include:

- **Azure CLI installation:** "How do I install the `az` CLI?"
- **Azure Developer CLI installation:** "Show me how to install `azd`"
- **Azure Functions Core tools:** "What is Azure Functions Core tools and how to install it"

 Expand table

Parameter	Required or optional	Description
CLI type	Required	The type of CLI tool to use. Supported values are <code>az</code> for Azure CLI, <code>azd</code> for Azure Developer CLI, and <code>func</code> for Azure Functions Core Tools CLI.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure CLI reference documentation](#)

- [Azure Developer CLI reference documentation](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/27/2026

Azure Event Grid tools for the Azure MCP Server overview

The Azure MCP Server allows you to manage Azure Event Grid resources using natural language prompts. You can list topics, view subscriptions, and more without remembering complex syntax.

[Azure Event Grid](#) is a highly scalable, serverless event broker that you can use to integrate applications using events. Events are delivered by Event Grid to subscriber destinations such as applications, Azure services, or any endpoint to which Event Grid has network access. The source of those events can be other applications, SaaS services, and Azure services.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Events: Publish

Publish custom events to Event Grid topics for event-driven architectures. This tool sends structured event data to Event Grid topics with schema validation and delivery guarantees for downstream subscribers. Returns publish operation status.

Example prompts include:

- **Publish with schema:** "Publish an event to Event Grid topic 'payment-events' using CloudEvents schema with data '{"orderId": "12345", "amount": 99.99}'."
- **Simple publish:** "Publish event to my Event Grid topic 'user-signups' with data '{"userId": "user123", "email": "user@example.com}"'."
- **Resource group context:** "Send an event to Event Grid topic 'analytics-events' in resource group 'data-processing' with data '{"eventType": "click", "timestamp": "2025-12-05T10:00:00Z}"'."

Parameter	Required or optional	Description
Topic	Required	The name of the Event Grid topic.
Data	Required	The event data as JSON string to publish to the Event Grid topic.
Schema	Optional	The event schema type (<code>CloudEvents</code> , <code>EventGrid</code> , or <code>Custom</code>). Defaults to <code>EventGrid</code> .

Tool annotation hints:

Destructive: ✖ | Idempotent: ✖ | Open World: ✖ | Read Only: ✖ | Secret: ✖ | Local Required: ✖

Subscription: List

List Event Grid subscriptions with filtering and endpoint configuration. This tool shows all active subscriptions including webhook endpoints, event filters, and delivery retry policies.

Example prompts include:

- **Topic in subscription:** "List Event Grid subscriptions for topic 'payment-events' in subscription"
- **View all subscriptions:** "Show all Event Grid subscriptions in my subscription"
- **Complete inventory:** "List all Event Grid subscriptions in subscription"
- **Resource group filter:** "Show Event Grid subscriptions in resource group 'notification-services' in subscription"
- **Resource group context:** "List Event Grid subscriptions for topic 'analytics-events' in resource group 'data-processing'"
- **Filter by topic:** "Show me all Event Grid subscriptions for topic 'user-signups'"
- **Location-based:** "List Event Grid subscriptions for subscription in location 'eastus'"

 Expand table

Parameter	Required or optional	Description
Topic	Optional	The name of the Event Grid topic.
Region	Optional	The Azure region to filter resources by (for example, <code>eastus</code> , <code>westus2</code>).

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Topic: List

List all Event Grid topics in an Event Grid subscription with configuration and status information. This tool retrieves topic details including endpoints, access keys, and subscription information for event publishing and management.

Example prompts include:

- **List topics:** "Show me all the Event Grid topics in my subscription."
- **View topic details:** "List Event Grid topics in resource group 'event-processing'"
- **Check available topics:** "What Event Grid topics do I have in my 'westus2' region?"
- **Topic inventory:** "I need a list of all my Event Grid resources"
- **Find endpoints:** "Show me the endpoints for all my Event Grid topics"

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Event Grid documentation](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/03/2026

Azure Event Hubs tools for the Azure MCP Server overview

The Azure MCP Server lets you manage Azure Event Hubs resources with natural language prompts. You don't need to remember specific command syntax.

[Azure Event Hubs](#) is a native data-streaming service in the cloud that streams millions of events per second, with low latency, from any source to any destination. Event Hubs is compatible with Apache Kafka and lets you run existing Kafka workloads without any code changes.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Event Hub: Delete consumer group

Delete a consumer group from the specified Event Hub.

Example prompts include:

- **Delete specific consumer group:** "Delete consumer group 'analytics-group' from Event Hub 'orders-hub' in namespace 'eventhub-prod' in resource group 'my-resource-group'"
- **Remove consumer group:** "Remove the consumer group 'monitoring-consumers' from my Event Hub 'telemetry-events' in namespace 'prod-eventhubs' in resource group 'my-resource-group'"
- **Clean up consumer group:** "Delete the consumer group 'test-group' from Event Hub 'user-events' in the 'development-eventhubs' namespace in resource group 'my-resource-group'"

[⌵](#) Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Namespace	Required	The name of the Event Hubs namespace.
Event hub	Required	The name of the Event Hub within the namespace.
Consumer group	Required	The name of the consumer group within the Event Hub.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Event Hub: Get consumer group

Get consumer groups from Azure Event Hubs. This tool can:

- List all consumer groups in an Event Hub
- Get a single consumer group by name

The event hub and namespace parameters are required for both get and list. You only need the consumer group parameter when getting a specific consumer group.

Example prompts include:

- **List all consumer groups:** "List all consumer groups in Event Hub 'orders-hub' in namespace 'eventhub-prod' in resource group 'my-resource-group'"
- **Get specific consumer group:** "Get details of consumer group 'analytics-group' from Event Hub 'orders-hub' in namespace 'eventhub-prod' in resource group 'my-resource-group'"
- **Show consumer group info:** "Show me the consumer group 'monitoring-consumers' from Event Hub 'telemetry-events' in namespace 'prod-eventhubs' in resource group 'my-resource-group'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Namespace	Required	The name of the Event Hubs namespace.
Event hub	Required	The name of the Event Hub within the namespace.
Consumer group	Optional	The name of the consumer group within the Event Hub.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Event Hub: Create or update consumer group

Create or update a consumer group within the specified Event Hub. The tool creates a new consumer group or updates an existing one.

Example prompts include:

- **Create new consumer group:** "Create a new consumer group 'analytics-group' in Event Hub 'orders-hub' in namespace 'eventhub-prod' in resource group 'my-resource-group'"
- **Update existing consumer group:** "Update the consumer group 'analytics-group' in Event Hub 'orders-hub' in namespace 'eventhub-prod' in resource group 'my-resource-group' with user metadata 'Updated for Q4 analytics'"
- **Set up consumer group:** "Set up a consumer group 'monitoring-consumers' in Event Hub 'telemetry-events' in namespace 'prod-eventhubs' in resource group 'my-resource-group'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Namespace	Required	The name of the Event Hubs namespace.
Event hub	Required	The name of the Event Hub within the namespace.

Parameter	Required or optional	Description
Consumer group	Required	The name of the consumer group within the Event Hub.
User metadata	Optional	User metadata for the consumer group.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Event Hub: Delete Event Hub

Delete an event hub from an Azure Event Hubs namespace. This action permanently deletes all messages and consumer groups in the Event Hub.

Example prompts include:

- **Delete specific Event Hub:** "Delete Event Hub 'orders-hub' from namespace 'eventhub-prod' in resource group 'my-resource-group'"
- **Remove Event Hub:** "Remove the Event Hub 'telemetry-events' from my 'prod-eventhubs' namespace in resource group 'my-resource-group'"
- **Clean up Event Hub:** "Delete the Event Hub 'test-events' from namespace 'dev-eventhubs' in resource group 'my-resource-group'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Namespace	Required	The name of the Event Hubs namespace.
Event hub	Required	The name of the Event Hub within the namespace.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Event Hub: Get Event Hub

Get event hubs from an Azure namespace. This tool:

- List all event hubs in a namespace
- Get a single event hub by name

When you retrieve a single event hub or list multiple event hubs, the command returns detailed information for all event hubs, including partition count, settings, and metadata.

Example prompts include:

- **List all Event Hubs:** "List all Event Hubs in my 'prod-eventhubs' namespace in resource group 'my-resource-group'"
- **Get specific Event Hub:** "Get the details of my Event Hub 'orders-hub' in namespace 'eventhub-prod' in resource group 'my-resource-group'"
- **Show Event Hub info:** "Show me the Event Hub 'telemetry-events' from namespace 'monitoring-hubs' in resource group 'my-resource-group'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Namespace	Required	The name of the Event Hubs namespace.
Event hub	Optional	The name of the Event Hub within the namespace.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Event Hub: Create or update Event Hub

Create or update an Event Hub within an Azure Event Hubs namespace. This command:

- Creates a new Event Hub if it doesn't exist
- Updates an existing Event Hub's configuration

You can configure these properties:

- Partition count (number of partitions for parallel processing)
- Message retention time (how long messages are retained, in hours)

Some properties like partition count can't be changed after creation. This is a potentially long-running operation that waits for completion.

Example prompts include:

- **Create new Event Hub:** "Create a new event hub 'orders-hub' in my namespace 'production-eventhubs' in resource group 'my-resource-group'"
- **Update existing Event Hub:** "Update my event hub 'telemetry-events' in my namespace 'monitoring-hubs' in resource group 'my-resource-group'"
- **Create with configuration:** "Create event hub 'user-activity' in namespace 'analytics-hubs' in resource group 'my-resource-group' with 4 partitions and 24 hours message retention"


Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Namespace	Required	The name of the Event Hubs namespace. Must be used with the resource group parameter.
Event hub	Required	The name of the Event Hub within the namespace.
Partition count	Optional	The number of partitions for the Event Hub. Must be between 1 and 32 (or higher based on namespace tier).
Message retention in hours	Optional	The message retention time in hours. Minimum is 1 hour, maximum depends on the namespace tier.
Status	Optional	The status of the Event Hub (such as Active, Disabled). Status might be read-only in some operations.

Tool annotation hints:

Destructive:  |
Idempotent:  |
Open World:  |
Read Only:  |
Secret:  |
Local Required: 

Namespace: Delete namespace

Delete an Event Hubs namespace. This operation is irreversible and permanently deletes all event hubs, consumer groups, and configurations within the namespace.

Example prompts include:

- **Delete specific namespace:** "Delete event hub namespace 'eventhub-prod' in resource group 'my-resource-group'"
- **Remove namespace:** "Remove the Event Hubs namespace 'test-eventhubs' in resource group 'my-resource-group'"
- **Clean up namespace:** "Delete the namespace 'dev-eventhubs' in resource group 'my-resource-group' permanently"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Namespace	Required	The name of the Event Hubs namespace.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Namespace: Get namespace

Get Event Hubs namespaces from Azure. The behavior depends on which parameters you provide. When you retrieve a single namespace, the tool returns detailed information including SKU, settings, and metadata. When you list namespaces, the tool returns the same detailed information for all namespaces in the specified scope.

Example prompts include:

- **List all namespaces:** "List all event hub namespaces in my subscription"
- **Get specific namespace:** "Get the details of my namespace 'eventhub-prod' in my resource group 'my-resource-group'"
- **Show namespace info:** "Show me the namespace 'monitoring-hubs' in resource group 'my-resource-group' details"

Parameter	Required or optional	Description
Resource group	Optional	The name of the Azure resource group. When omitted, the system lists all namespaces in the subscription. When provided without namespace, the system lists all namespaces in the resource group. Required when getting a specific namespace.
Namespace	Optional	The name of the Event Hubs namespace to retrieve. When provided, returns detailed information for the specific namespace (requires resource group). When omitted, returns a list of namespaces.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Namespace: Create or update namespace

Create or update a namespace within the specified resource group. This tool creates a new namespace or updates an existing one. The tool might modify existing configurations and is considered destructive. This tool might take a long time.

When updating an existing namespace, provide only the properties you want to change. Unspecified properties keep their existing values. You must provide at least one update property.

Common update scenarios include:

- Scale up or down by changing the SKU tier or capacity
- Enable or disable auto-inflate and set the maximum throughput units
- Enable or disable Kafka support
- Modify tags for resource management
- Enable or disable zone redundancy (Premium SKU only)

Example prompts include:

- **Create new namespace:** "Create a new Event Hubs namespace 'production-events' in resource group 'my-resource-group' in East US"
- **Update namespace capacity:** "Update my namespace 'eventhub-prod' in resource group 'my-resource-group' to increase capacity to 10 throughput units"

- **Enable Kafka support:** "Enable Kafka on my Event Hubs namespace 'monitoring-hubs' in resource group 'my-resource-group'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Namespace	Required	The name of the Event Hubs namespace.
Location	Optional	The Azure region where the namespace is located (for example, <code>eastus</code> , <code>westus2</code>).
SKU name	Optional	The SKU name for the namespace. Valid values: <code>Basic</code> , <code>Standard</code> , <code>Premium</code> .
SKU tier	Optional	The SKU tier for the namespace. Valid values: <code>Basic</code> , <code>Standard</code> , <code>Premium</code> .
SKU capacity	Optional	The SKU capacity (throughput units) for the namespace. The valid range depends on the SKU.
Is auto inflate enabled	Optional	Enable or disable auto-inflate for the namespace.
Maximum throughput units	Optional	The maximum throughput units when auto-inflate is enabled.
Kafka enabled	Optional	Enable or disable Kafka for the namespace.
Zone redundant	Optional	Enable or disable zone redundancy for the namespace.
Tags	Optional	Tags for the namespace in JSON format (for example, <code>{"key1": "value1", "key2": "value2"}</code>).

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Event Hubs tools](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/27/2026

Azure MCP Server tools for Azure File Shares

The Azure Model Context Protocol (MCP) Server lets you manage Azure file shares (`Microsoft.FileShares`) by using natural language prompts. You don't need to remember specific command syntax.

✔ **Applies to:** File shares created with the `Microsoft.FileShares` resource provider (preview)

✗ **Doesn't apply to:** Classic file shares created with the `Microsoft.Storage` resource provider

[Azure Files](#) is a managed file sharing service in the cloud. Azure file shares provide high-performance, fully managed storage for your applications and workloads. This article applies only to file shares created with the `Microsoft.FileShares` resource provider (preview), which is currently only available for Network File System (NFS) file shares. It doesn't apply to classic file shares created with the `Microsoft.Storage` resource provider.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Get limits

Get file share limits for a subscription and location.

Example prompts include:

- "Show me the current file share limits in the 'eastus' location."
- "What are the file share limits for location 'westeurope?'"
- "Get the file share limits for location 'centralus.'"
- "Provide the file share limits for location 'eastus2.'"
- "Retrieve file share limits for the 'westus' region."

Parameter	Required or optional	Description
Location	Required	The Azure region or location name (for example, <code>eastus</code> , <code>westeurope</code>).

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Get usage

Get file share usage data for a subscription and location.

Example prompts include:

- "Show me the usage details for file shares in location 'eastus.'"
- "I want to see usage for file shares in region 'westeurope.'"
- "Get usage statistics for file shares in location 'centralus.'"
- "Can you provide usage information for file shares in 'eastus2?'"
- "Display usage for file shares in the 'westus' region."

 Expand table

Parameter	Required or optional	Description
Location	Required	The Azure region or location name (for example, <code>eastus</code> , <code>westeurope</code>).

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Get recommendations

Get provisioning parameter recommendations for a file share based on the desired storage size.

Example prompts include:

- "Get recommendations for a 1,000-GiB file share in location 'eastus.'"
- "Can you provide recommendations for a 500-GiB file share in 'westeurope?'"
- "Get details for a 2,000-GiB file share in location 'centralus.'"
- "I want to see recommendations for a 5,000-GiB file share in 'eastus2.'"
- "Retrieve recommendations for a 250-GiB file share in the 'westus' region."

 Expand table

Parameter	Required or optional	Description
Location	Required	The Azure region or location name (for example, <code>eastus</code> , <code>westeurope</code>).
Provisioned storage in GiB (gibibytes)	Required	The desired provisioned storage size of the share in GiB.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

File Share: Check name availability

Check if a file share name is available in a specific location.

Example prompts include:

- "Can you check if the file share name 'projectdata' is available in location 'eastus?'"
- "I want to see if 'salesbackup' is an available file share name in 'westeurope.'"
- "Check the availability of the file share name 'teamfiles' in location 'EastUS.'"
- "Is the name 'archive2024' free for a new file share in 'WestUS?'"
- "Verify whether 'clientdocs' can be used as a file share name in location 'centralus.'"

 Expand table

Parameter	Required or optional	Description
Name	Required	The name of the file share.
Location	Required	The Azure region or location name (for example, <code>EastUS</code> , <code>WestEurope</code>).

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local Required: ✖

File Share: Create file share

Create a new Azure file share resource in a resource group. This operation creates a high-performance, fully managed file share accessible through the NFS protocol.

Example prompts include:

- "Create a new file share named 'project-data' in resource group 'rg-prod' at location 'eastus.'"
- "I need to create a file share called 'backupshare' in resource group 'rg-backup' at location 'westeurope.'"
- "Set up a file share 'userdocs' in resource group 'rg-dev' at location 'centralus.'"
- "Create the file share 'archive2024' in resource group 'rg-archive' at location 'eastus2.'"
- "Generate a file share named 'mediafiles' in resource group 'rg-media' at location 'westus.'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. The resource group is a logical container for Azure resources.
Name	Required	The name of the file share.
Location	Required	The Azure region or location name (for example, <code>EastUS</code> , <code>WestEurope</code>).
Mount name	Optional	The mount name of the file share as seen by end users.
Media tier	Optional	The storage media tier (for example, <code>SSD</code>).
Redundancy	Optional	The redundancy level (for example, <code>Local</code> , <code>Zone</code>).
Protocol	Optional	The file sharing protocol (for example, <code>NFS</code>).
Provisioned storage in GiB (gibibytes)	Optional	The desired provisioned storage size of the share in GiB.
Provisioned io per sec	Optional	The provisioned IO operations per second.

Parameter	Required or optional	Description
Provisioned throughput in MiB per sec (mebibytes)	Optional	The provisioned throughput in MiB per second.
Public network access	Optional	Public network access setting (Enabled or Disabled).
NFS root squash	Optional	NFS root squash setting (NoRootSquash, RootSquash, or AllSquash).
Allowed subnets	Optional	Comma-separated list of subnet IDs allowed to access the file share.
Tags	Optional	Resource tags as JSON (for example, {"key1": "value1", "key2": "value2"}).

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

File Share: Get file share

Get details of a specific file share or list all file shares. If you provide a name, the command returns a specific file share. Otherwise, it lists all file shares in the subscription or resource group.

Example prompts include:

- "Show me all file shares in resource group 'rg-prod.'"
- "List every file share available under the resource group 'rg-backup.'"
- "Get details for the file share 'reports2024' in resource group 'rg-production.'"
- "Can you retrieve information on file share 'archive-logs' for resource group 'rg-data.'"
- "I need to see the file share 'projectfiles' from resource group 'rg-dev.'"

 Expand table

Parameter	Required or optional	Description
Resource group	Optional	The name of the Azure resource group. The resource group is a logical container for Azure resources.

Parameter	Required or optional	Description
Name	Optional	The name of the file share.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

File Share: Update file share

Update an existing Azure file share resource. You can update mutable properties such as provisioned storage, input/output operations per second (IOPS), throughput, and network access settings.

Example prompts include:

- "Update the file share named 'projectdocs' in resource group 'rg-prod' to modify its quota."
- "Make changes to the file share 'shareddata' within resource group 'rg-backup' by updating access settings."
- "Apply new settings to the file share 'datahub' in resource group 'rg-data' to increase throughput."
- "Change configuration settings for the file share 'prod-share' in resource group 'rg-production'."
- "I need to update the file share 'reports2024' in resource group 'rg-analytics'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. The resource group is a logical container for Azure resources.
Name	Required	The name of the file share.
Provisioned storage in GiB (gibibytes)	Optional	The desired provisioned storage size of the share in GiB.
Provisioned io per sec	Optional	The provisioned IO operations per second.

Parameter	Required or optional	Description
Provisioned throughput in MiB per sec (mebibytes)	Optional	The provisioned throughput in MiB per second.
Public network access	Optional	Public network access setting (Enabled or Disabled).
NFS root squash	Optional	NFS root squash setting (NoRootSquash, RootSquash, or AllSquash).
Allowed subnets	Optional	Comma-separated list of subnet IDs allowed to access the file share.
Tags	Optional	Resource tags as JSON (for example, {"key1": "value1", "key2": "value2"}).

Tool annotation hints:

Destructive: ☒ | Idempotent: ☒ | Open World: ☒ | Read Only: ☒ | Secret: ☒ | Local Required: ☒

File Share: Delete file share

Delete a file share permanently. You can't undo this operation.

Example prompts include:

- "Delete the file share named 'backup-share' in resource group 'rg-prod'."
- "Remove the file share 'project-files' in resource group 'rg-marketing'."
- "I want to delete the file share 'temp-data' in resource group 'rg-dev'."
- "Can you delete the file share 'archive-old' in resource group 'rg-archive'?"
- "Remove the file share 'cleanup-share' from resource group 'rg-cleanup'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. The resource group is a logical container for Azure resources.
Name	Required	The name of the file share.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Private Endpoint Connection: Get private endpoint connection

Get details of a specific private endpoint connection or list all private endpoint connections for a file share. If you provide `connection-name`, the command returns a specific connection. Otherwise, it lists all connections.

Example prompts include:

- "List all private endpoint connections for file share 'projectfiles' in resource group 'rg-prod.'"
- "Get the private endpoint connection named 'myconnection' for file share 'datashare' in resource group 'rg-data.'"
- "Show me private endpoint connections on file share 'backupshare' in resource group 'rg-backup.'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. The resource group is a logical container for Azure resources.
File share name	Required	The name of the file share.
Connection name	Optional	The name of the private endpoint connection.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Private Endpoint Connection: Update private endpoint connection

Update the state of a private endpoint connection for a file share. Use this operation to approve or reject private endpoint connection requests.

Example prompts include:

- "Approve the private endpoint connection 'myconnection' for file share 'projectfiles' in resource group 'rg-prod.'"
- "Reject the private endpoint connection named 'extconnection' on file share 'datashare' in resource group 'rg-data' with description 'Not authorized.'"
- "Update the private endpoint connection 'pendingconn' to 'Approved' for file share 'backupshare' in resource group 'rg-backup.'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. The resource group is a logical container for Azure resources.
File share name	Required	The name of the file share.
Connection name	Required	The name of the private endpoint connection.
Status	Required	The connection status (<code>Approved</code> , <code>Rejected</code> , or <code>Pending</code>).
Description	Optional	Description for the connection state change.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

File Share Snapshot: Create snapshot

Create a snapshot of an Azure file share. Snapshots are read-only point-in-time copies used for backup and recovery.

Example prompts include:

- "Create a snapshot named 'backup-snap-jan23' for file share 'backups' in resource group 'rg-data-prod'"

- "I want to create a snapshot called 'project-snapshot' for the file share 'projectfiles' in resource group 'rg-marketing'"
- "Generate a snapshot named 'weekly-backup' for file share 'companyshare' in resource group 'rg-finance'"
- "Create a snapshot called 'reports-snapshot' on the file share 'reports' in resource group 'rg-analytics'"
- "Initiate snapshot creation named 'dev-snapshot-01' for file share 'devfiles' in resource group 'rg-development'"


Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. The resource group is a logical container for Azure resources.
File share name	Required	The name of the parent file share.
Snapshot name	Required	The name of the snapshot.
Metadata	Optional	Custom metadata for the snapshot as a JSON object (for example, <code>{"key1": "value1", "key2": "value2"}</code>).

Tool annotation hints:

Destructive: 
| Idempotent: 
| Open World: 
| Read Only: 
| Secret: 
| Local Required: 

File Share Snapshot: Get snapshot

Get details of a specific file share snapshot or list all snapshots. If you provide the snapshot name, the command returns a specific snapshot. Otherwise, it lists all snapshots for the file share.

Example prompts include:

- "Show me all snapshots for file share 'projectfileshare' in resource group 'rg-prod.'"
- "List snapshots available on the file share 'teamdata' in resource group 'rg-finance.'"
- "Get the snapshot named 'snapshot20240601' from file share 'backupshare' in resource group 'rg-backup.'"

- "Retrieve details for the snapshot 'dailybackup' on file share 'reports' in resource group 'rg-analytics.'"
- "I need to see the snapshot 'weekendcopy' from file share 'mediafiles' in resource group 'rg-media.'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. The resource group is a logical container for Azure resources.
File share name	Required	The name of the parent file share.
Snapshot name	Optional	The name of the snapshot.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

File Share Snapshot: Update snapshot

Update the properties and metadata of an Azure file share snapshot, such as tags or retention policies.

Example prompts include:

- "Update the snapshot 'backup-snap-jan23' for file share 'backupshare' in resource group 'rg-backup.'"
- "Apply updates to snapshot 'data-snapshot' on file share 'datafiles' in resource group 'rg-data.'"
- "Can you update snapshot 'snapshot2024' for file share 'reports' in resource group 'rg-analytics.'"
- "Update the snapshot named 'endofmonth' on file share 'finance-data' in resource group 'rg-finance.'"
- "Modify the properties of snapshot 'weeklybackup' for file share 'devfileshare' in resource group 'rg-dev.'"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. The resource group is a logical container for Azure resources.
File share name	Required	The name of the parent file share.
Snapshot name	Required	The name of the snapshot.
Metadata	Optional	Custom metadata for the snapshot as a JSON object (for example, <code>{"key1": "value1", "key2": "value2"}</code>).

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

File share snapshot: Delete snapshot

Delete a file share snapshot permanently. This operation can't be undone.

Example prompts include:

- "Delete snapshot 'backup-snap-jan15' from file share 'datafiles' in resource group 'rg-prod'."
- "Remove snapshot 'weekly-backup' from file share 'backupshare' in resource group 'rg-backup'."
- "Delete snapshot 'snapshot2024-03-15' from file share 'reports' in resource group 'rg-analytics'."
- "Delete the snapshot 'dailybackup' from file share 'projectfiles' in resource group 'rg-dev'."
- "Remove snapshot 'weekllysnap' from file share 'archive' in resource group 'rg-archive'."

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. The resource group is a logical container for Azure resources.

Parameter	Required or optional	Description
File share name	Required	The name of the parent file share.
Snapshot name	Required	The name of the snapshot.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Files](#)

 **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 04/21/2026

Azure File Sync Tools

Azure File Sync tools in Azure MCP Server help you manage Azure File Sync services through natural language prompts. You can manage Storage Sync services, register servers, create sync groups, configure cloud endpoints, and set up server endpoints to synchronize files between on-premises servers and Azure File Shares. These tools simplify storage sync management and reduce configuration complexity.

[Azure File Sync](#) is a service that centralizes an organization's file shares in Azure Files while keeping the flexibility, performance, and compatibility of a Windows file server.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Storage Sync service: Create service

Create a new Azure Storage Sync service resource in a resource group. This service acts as the top-level service container that manages sync groups, registered servers, and synchronization workflows.

Example prompts include:

- "Create a storage sync service named 'FileSyncService01' in resource group 'rg-storage-sync' at location 'EastUS'"
- "Please set up a storage sync service called 'BackupSyncService' inside 'rg-backup' resource group located in 'WestEurope'"
- "I need a new storage sync service with name 'DataSyncProd' deployed to 'rg-prod-sync' resource group in 'CentralUS'"
- "Can you create a storage sync service named 'ArchiveSync' within resource group 'rg-archive' located at 'NorthEurope'?"
- "Set up storage sync service 'SyncServiceWest' in the resource group 'rg-west-sync' at Azure region 'WestUS2'"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Location	Required	The Azure region or location name (for example, <code>EastUS</code> , <code>WestEurope</code>).

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Storage Sync service: Delete service

Delete an Azure Storage Sync service and all its associated resources.

Example prompts include:

- "Delete the storage sync service named 'SyncServiceEastUS' in resource group 'rg-storage-prod'"
- "Can you remove the storage sync service 'FilesSync01' from resource group 'rg-file-sync'?"
- "Please delete storage sync service 'BackupSyncService' located in resource group 'rg-backups'"
- "I need to delete the storage sync service called 'CorporateSync' within resource group 'rg-corp-resources'"
- "Remove the storage sync service 'DataSyncPrimary' from resource group 'rg-data-sync'"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Storage Sync service: Get service

Retrieve Azure Storage Sync service details or list all Storage Sync services. The command shows service properties, location, provisioning state, and configuration. If you provide the resource name parameter, the command returns a specific Storage Sync service. Otherwise, it lists all Storage Sync services in the subscription or resource group.

Example prompts include:

- "Get details for the storage sync service named 'SyncServiceEastUS'"
- "Can you retrieve info on storage sync service 'CorporateFileSync'?"
- "I need to see the configuration of the service called 'DataSyncProd'"
- "Show me the storage sync service with the name 'BackupSync01'"
- "Fetch the service details for 'SyncServiceWestEurope' storage sync"

 Expand table

Parameter	Required or optional	Description
Name	Optional	The name of the storage sync service.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Storage Sync service: Update service

Update properties of an existing Azure Storage Sync service.

Example prompts include:

- "Update the storage sync service named 'filesyncservice01' in resource group 'rg-storage-prod'"
- "Can you update the service 'sync-service-eastus' within the resource group 'rg-sync-eastus'?"

- "Please update the storage sync service 'CorpFileSync' located in resource group 'rg-corp-filesync'"
- "Make changes to storage sync service 'ArchiveSyncService' in the resource group 'rg-archive-data'"
- "I need to update the service called 'BackupSync' in resource group 'rg-backup-resources'"


Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Incoming traffic policy	Optional	Incoming traffic policy for the service (<code>AllowAllTraffic</code> or <code>AllowVirtualNetworksOnly</code>).
Tags	Optional	Tags to assign to the service (space-separated key=value pairs).
Identity type	Optional	Managed service identity type (<code>None</code> , <code>SystemAssigned</code> , <code>UserAssigned</code> , <code>SystemAssigned,UserAssigned</code>).

Tool annotation hints:

Destructive: ✖ | Idempotent: ✖ | Open World: ✖ | Read Only: ✖ | Secret: ✖ | Local Required: ✖

Registered server: Get registered server

List all registered servers in a Storage Sync service or retrieve details about a specific registered server. The command returns server properties including server ID, registration status, agent version, OS version, and last heartbeat. If you provide the server ID parameter, the command returns a specific registered server. Otherwise, it lists all registered servers in the Storage Sync service.

Example prompts include:

- "Get details of the registered server for storage sync service 'SyncServiceWest' in resource group 'rg-storage-west'"

- "Show me the registered server for storage sync 'FileSyncService' within resource group 'rg-europe-central'"
- "Retrieve info on the registered server named 'RegisteredServer01' in resource group 'rg-prod-storage' for sync service 'ProdFileSync'"
- "Can you fetch the registered server for the storage sync service 'BackupSync' located in resource group 'rg-backup'"
- "I need the registered server data for storage sync service 'CorpSync' under resource group 'rg-corp-data'"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Server ID	Optional	The ID/name of the registered server.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Registered server: Unregister registered server

Unregister a server from a Storage Sync service.

Example prompts include:

- "Unregister the registered server with ID 'server123' from the storage sync service 'SyncServiceWest' in resource group 'rg-storage-sync'"
- "Can you unregister the server named 'server456' from storage sync service 'FileSyncService' within resource group 'prod-storage-rg'?"
- "Please remove the registered server 'backupServer01' from the storage sync service 'DataSyncService' in resource group 'dev-sync-resources'"
- "I need to unregister the server ID 'syncServer789' from the storage sync service called 'MainStorageSync' located in resource group 'rg-sync-apps'"
- "How do I unregister the registered server 'appServer42' from storage sync service 'SyncServiceEast' under resource group 'rg-eastern-storage'?"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Server ID	Required	The ID/name of the registered server.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Registered server: Update registered server

Update properties of a registered server.

Example prompts include:

- "Update the registered server with ID 'server123' in storage sync service 'SyncServiceOne' within resource group 'rg-sync-prod'"
- "Can you update the registered server named 'serverABC' for storage sync service 'DataSyncService' in resource group 'rg-data-sync'?"
- "Please update the registered server 'server789' in storage sync service 'MySyncService' under resource group 'rg-sync-dev'"
- "I need to update registered server 'server456' in 'SyncServicePrimary' storage sync service located in resource group 'rg-sync-main'"
- "Modify the registered server having ID 'server321' for the storage sync service 'BackupSync' inside resource group 'rg-backup-sync'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.

Parameter	Required or optional	Description
Server ID	Required	The ID/name of the registered server.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✖ | Open World: ✖ | Read Only: ✖ | Secret: ✖ | Local Required: ✖

Sync group: Create sync group

Create a sync group within an existing Storage Sync service. Sync groups define a sync topology and contain cloud endpoints (Azure File Shares) and server endpoints (local server paths) that sync together.

Example prompts include:

- "Create a sync group named 'FinanceSync' in storage sync service 'FinanceSyncService' within resource group 'rg-finance-prod'"
- "I need to create a sync group called 'HRSyncGroup' under storage sync service 'HRDataService' in 'rg-hr-resources'"
- "Set up a sync group with the name 'ProjectSync' for the storage sync service 'ProjectFilesService' inside resource group 'rg-projects-dev'"
- "Can you create a sync group named 'BackupSyncGroup' in 'BackupSyncService' located in resource group 'rg-backup-eastus'?"
- "Make a new sync group 'SalesDataSync' in the storage sync service 'SalesService' under resource group 'rg-sales-central'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Sync group name	Required	The name of the sync group.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Sync group: Delete sync group

Remove a sync group from a Storage Sync service. When you delete a sync group, you also remove all associated cloud endpoints and server endpoints within that group.

Example prompts include:

- "Delete the sync group named 'filesSyncGroup' in storage sync service 'DataSyncService' within resource group 'rg-storage-prod'"
- "Can you remove sync group 'backupSyncGroup' from storage sync service 'MySyncService' in resource group 'rg-westus'?"
- "Please delete sync group 'archiveSync' under the storage sync service 'ArchiveService' located in resource group 'rg-europe'"
- "Remove the sync group called 'syncGroup01' from 'FastSyncService' inside resource group 'rg-dev-storage'"
- "I want to delete the sync group 'photosSyncGroup' from storage sync service 'PhotoService' in resource group 'rg-photoapp'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Sync group name	Required	The name of the sync group.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Sync group: Get sync group

Get details about a specific sync group or list all sync groups. If you provide the sync group name parameter, the command returns a specific sync group. Otherwise, it lists all sync groups in the Storage Sync service.

Example prompts include:

- "Get the sync group from storage sync service 'SyncServiceWestUS' in resource group 'rg-storage-prod'"
- "Show me the details of storage sync service 'BackupSyncService' located in resource group 'rg-data-archive'"
- "I need to retrieve sync group information from service 'FileSyncServiceEast' within resource group 'rg-enterprise-files'"
- "Can you fetch the sync group data for storage sync service 'CorpSyncService' in 'rg-corp-resources'?"
- "Retrieve the sync group for 'FileShareSync' storage sync service under resource group 'rg-devops-staging'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Sync group name	Optional	The name of the sync group.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Cloud endpoint: Create cloud endpoint

Add a cloud endpoint to a sync group by connecting an Azure File Share. Cloud endpoints represent the Azure storage side of the sync relationship.

Example prompts include:

- "Create a cloud endpoint named 'backupEndpoint' in sync group 'filesSync' within storage sync service 'FileSyncService1' and resource group 'rg-data' using storage account resource ID '/subscriptions/12345/resourceGroups/rg-data/providers/Microsoft.Storage/storageAccounts/storageacct01' with Azure file share 'backupshare'"
- "Set up cloud endpoint 'userDocsEndpoint' in sync group 'docSyncGroup' under storage sync service 'UserFilesService' inside resource group 'rg-users', linking to storage account resource ID '/subscriptions/67890/resourceGroups/rg-users/providers/Microsoft.Storage/storageAccounts/usersstorage' and Azure file share 'userdocs'"
- "I need to create a cloud endpoint called 'mediaEndpoint' for sync group 'mediaSync' on service 'MediaSyncService' in resource group 'rg-media', using storage account resource ID '/subscriptions/abcde/resourceGroups/rg-media/providers/Microsoft.Storage/storageAccounts/mediastorage' and the Azure file share 'mediashare1'"
- "Please add cloud endpoint 'logsEndpoint' within sync group 'logSyncGroup' for storage sync service 'LogSyncService' located in resource group 'rg-logs' with storage account resource ID '/subscriptions/54321/resourceGroups/rg-logs/providers/Microsoft.Storage/storageAccounts/logstor' referencing Azure file share 'logfiles'"
- "Create cloud endpoint 'archiveEndpoint' in 'archiveSyncGroup' sync group for the storage sync service named 'ArchiveService' inside resource group 'rg-archive' using storage account resource ID '/subscriptions/98765/resourceGroups/rg-archive/providers/Microsoft.Storage/storageAccounts/archivestorage' and file share 'archivefiles'"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Sync group name	Required	The name of the sync group.
Cloud endpoint name	Required	The name of the cloud endpoint.
Storage account	Required	The resource ID of the Azure storage account.

Parameter	Required or optional	Description
resource ID		
Azure file share name	Required	The name of the Azure file share.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Cloud endpoint: Delete cloud endpoint

Delete a cloud endpoint from a sync group.

Example prompts include:

- "Delete the cloud endpoint named 'ArchiveEndpoint' in sync group 'SyncGroupAlpha' of storage sync service 'StorageSyncWest' within resource group 'rg-storage-prod'"
- "Can you remove cloud endpoint 'BackupEndpoint01' from sync group 'MainSyncGroup' under storage sync service 'FileSyncService' in resource group 'rg-data-center'?"
- "I need to delete the cloud endpoint 'UserFilesEndpoint' from the sync group 'UserSyncGroup' in the storage sync service 'UserSyncServiceEast' located in resource group 'rg-eastus1'"
- "Please delete cloud endpoint 'LogsStorage' for sync group 'LogSyncGroup' in storage sync service 'LogsSyncService' inside the resource group 'rg-logging-prod'"
- "Remove cloud endpoint 'PhotosEndpoint' in the sync group 'MediaSync' from storage sync service 'MediaSyncService' in resource group 'rg-media-services'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Sync group name	Required	The name of the sync group.
Cloud endpoint name	Required	The name of the cloud endpoint.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Cloud endpoint: Get cloud endpoint

List all cloud endpoints in a sync group or retrieve details about a specific cloud endpoint. The command returns cloud endpoint properties, including Azure File Share configuration, storage account details, and provisioning state. If you provide the cloud endpoint name parameter, the command returns a specific cloud endpoint. Otherwise, it lists all cloud endpoints in the sync group.

Example prompts include:

- "Get the cloud endpoint details for sync group 'SyncGroup1' in storage sync service 'StorageSyncEast' within resource group 'rg-storage-sync'"
- "Show me the cloud endpoint info in resource group 'rg-syncservices', storage sync service 'FileSyncService', and sync group 'PrimarySync'"
- "Retrieve cloud endpoint from storage sync service 'BackupSyncService' and sync group 'GroupA' in resource group 'rg-backups'"
- "I need the cloud endpoint for sync group 'DataSync' under storage sync service 'SyncServiceWest' in 'rg-westus-sync'"
- "Fetch cloud endpoint data for sync group 'MainGroup' of storage sync service 'CentralStorageSync' located in resource group 'rg-central-storage'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Sync group name	Required	The name of the sync group.
Cloud endpoint name	Optional	The name of the cloud endpoint.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Cloud endpoint: Trigger change detection

Trigger change detection on a cloud endpoint to sync file changes.

Example prompts include:

Example prompts include:

- "Trigger change detection for cloud endpoint 'endpointOne' in sync group 'SyncGroupA' of storage sync service 'SyncService123' within resource group 'rg-storage-sync'"
- "Can you start the change detection on cloud endpoint 'filesBackup' for sync group 'PrimaryGroup' under storage sync service 'FileSyncService' in resource group 'rg-prod-sync'?"
- "Initiate cloud endpoint change detection named 'BackupEndpoint' in storage sync service 'DataSync' and sync group 'MainSync' within resource group 'rg-data-sync'"
- "I need to trigger the change detection process on cloud endpoint 'archiveEndpoint' under sync group 'GroupOne' of storage sync service 'StorageSync01' located in 'rg-sync-resources'"
- "Please activate change detection for cloud endpoint 'cloudEndpointX' in sync group 'GroupX' for storage sync service 'SyncServiceX' inside resource group 'rg-sync-project'"

[🔗](#) Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Sync group name	Required	The name of the sync group.
Cloud endpoint name	Required	The name of the cloud endpoint.

Tool annotation hints:

Destructive: ❌ | Idempotent: ❌ | Open World: ❌ | Read Only: ❌ | Secret: ❌ | Local Required: ❌

Server endpoint: Create server endpoint

Add a server endpoint to a sync group by specifying a local server path to sync. Server endpoints represent the on-premises side of the sync relationship and include cloud tiering configuration.

Example prompts include:

- "Create a server endpoint named 'syncEndpoint01' in sync group 'filesSyncGroup' using storage sync service 'FileSyncService' within resource group 'rg-storage' with server resource ID '/subscriptions/12345/resourceGroups/rg-storage/providers/Microsoft.StorageSync/registeredServers/server01' and local path 'D:\Data'"
- "I need to add a server endpoint called 'backupEndpoint' to sync group 'BackupGroup' under storage sync service 'BackupService' in resource group 'rg-backup' using server resource ID '/subscriptions/67890/resourceGroups/rg-backup/providers/Microsoft.StorageSync/registeredServers/serverBackup' with local path 'E:\Backup'"
- "Set up server endpoint 'MediaSync' for sync group 'MediaGroup' on storage sync service 'MediaSyncService' inside the resource group 'rg-media' specifying server resource ID '/subscriptions/abcde/resourceGroups/rg-media/providers/Microsoft.StorageSync/registeredServers/mediaServer' and local sync path 'C:\MediaFiles'"
- "Please create a server endpoint named 'LogsEndpoint' in resource group 'rg-logs' using storage sync service 'LogSyncService' with sync group 'LogSync' providing server resource ID '/subscriptions/fghij/resourceGroups/rg-logs/providers/Microsoft.StorageSync/registeredServers/logServer' and local folder 'F:\Logs'"
- "Can you add server endpoint 'DocsEndpoint' to the sync group 'DocsGroup' in 'DocSyncService' within resource group 'rg-docs'? The server resource ID is '/subscriptions/klmno/resourceGroups/rg-docs/providers/Microsoft.StorageSync/registeredServers/docServer' and the local path is 'G:\Documents'"

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Sync group name	Required	The name of the sync group.
Server endpoint name	Required	The name of the server endpoint.
Server resource ID	Required	The resource ID of the registered server.
Server local path	Required	The local folder path on the server for syncing.
Cloud tiering	Optional	Enable cloud tiering on this endpoint.
Volume free space percent	Optional	Volume free space percentage to maintain (1-99, default 20).
Tier files older than days	Optional	Archive files not accessed for this many days.
Local cache mode	Optional	Local cache mode: <code>DownloadNewAndModifiedFiles</code> , <code>UpdateLocallyCachedFiles</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Server endpoint: Delete server endpoint

Delete a server endpoint from a sync group.

Example prompts include:

- "Delete the server endpoint 'BackupEndpoint01' from sync group 'MainSyncGroup' in storage sync service 'CorpSyncService' within resource group 'rg-data-sync'"
- "Please remove the server endpoint named 'FileServerEndpoint' in sync group 'DocumentsSync' belonging to storage sync service 'FileSyncService' located in resource group 'rg-file-services'"
- "I want to delete server endpoint 'ServerEndpoint3' under sync group 'DailySync' from the storage sync service 'DataSyncPrimary' in resource group 'rg-production-sync'"

- "Can you delete 'FinanceEndpoint' server endpoint from the 'FinanceSyncGroup' sync group at storage sync service 'FinanceDataSync' in resource group 'rg-finance'"
- "Remove the server endpoint 'ArchiveServer01' in sync group 'ArchiveSync' from storage sync service 'ArchiveSyncService' within resource group 'rg-archive-management'"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Sync group name	Required	The name of the sync group.
Server endpoint name	Required	The name of the server endpoint.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Server endpoint: Get server endpoint

List all server endpoints in a sync group or retrieve details about a specific server endpoint. The command returns server endpoint properties, including local path, cloud tiering status, sync health, and provisioning state. If you provide the server endpoint name parameter, the command returns a specific server endpoint. Otherwise, it lists all server endpoints in the sync group.

Example prompts include:

- "Get details of the server endpoint named 'endpoint1' in sync group 'SyncGroupA' from storage sync service 'StorageSync01' within resource group 'rg-storage-sync'"
- "Show me the server endpoint information for sync group 'SyncGroupB' under storage sync service 'FastSync' in resource group 'resource-group-prod'"
- "Retrieve server endpoint data for storage sync service 'CentralSync' and sync group 'SyncMain' located in resource group 'rg-data-sync'"
- "I need to get the server endpoint for sync group 'GroupSyncX' in storage sync service 'SyncServiceX' from resource group 'rg-sync-services'"

- "Please fetch the server endpoint info belonging to sync group 'PrimarySyncGroup' and storage sync service 'SyncServicePro' in resource group 'rg-sync-prod'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Sync group name	Required	The name of the sync group.
Server endpoint name	Optional	The name of the server endpoint.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Server endpoint: Update server endpoint

Update properties of a server endpoint.

Example prompts include:

- "Update the server endpoint 'endpoint01' in sync group 'SyncGroupA' for storage sync service 'SyncServiceX' within resource group 'rg-filesync'"
- "Can you modify server endpoint 'FileServer01' under sync group 'PrimarySync' in storage sync service 'CorpSync' in resource group 'rg-data'?"
- "Please update server endpoint 'BackupEndpoint' on sync group 'BackupGroup' for storage sync service 'StorageSyncProd' located in resource group 'rg-prod-storage'"
- "Set new settings for server endpoint 'EdgeNode' in storage sync service 'SyncServiceEast' sync group 'EastSync' within resource group 'rg-east'"
- "I need to update the server endpoint named 'ArchiveEndpoint' in sync group 'ArchiveSync' of storage sync service 'ArchiveService' inside resource group 'rg-archives'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The name of the storage sync service.
Sync group name	Required	The name of the sync group.
Server endpoint name	Required	The name of the server endpoint.
Cloud tiering	Optional	Enable cloud tiering on this endpoint.
Volume free space percent	Optional	Volume free space percentage to maintain (1-99, default 20).
Tier files older than days	Optional	Archive files not accessed for this many days.
Local cache mode	Optional	Local cache mode: DownloadNewAndModifiedFiles, UpdateLocallyCachedFiles.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✖ | Open World: ✖ | Read Only: ✖ | Secret: ✖ | Local Required: ✖

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure File Sync](#)

 **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 01/26/2026

Azure MCP Server tools for Azure Functions

The Azure MCP Server enables you to manage Azure Functions resources by using natural language prompts. You can manage existing function app resources and generate Azure Functions code, including function templates, project scaffolding, and language discovery.

Azure Functions is a serverless compute service for running event-driven code without managing infrastructure. For more information, see [Azure Functions documentation](#).

 Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Function app: list or get

Get details for a specific function app or list all function apps in your subscription. Returns information including name, location, status, and app service plan.

Example prompts include:

- "List all Function Apps in my subscription."
- "Show me all Function Apps in resource group 'rg-production'."
- "Retrieve details for the Function App named 'HealthMonitor' in resource group 'rg-production'."
- "Can you get the configuration of Function App 'DataProcessor' within resource group 'rg-analytics'?"

 Expand table

Parameter	Required or optional	Description
Function app	Optional	The name of the function app.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local
Required: ✖

Functions: get language list

This tool lists supported programming languages for Azure Functions development. It helps you discover available languages, compare language options, and choose a language to start a project. It returns language names, runtime versions, prerequisites, recommended development tools, and init, run, and build commands. Review this information before you use functions project get and functions template get.

Example prompts include:

- "Which programming languages does Azure Functions support?"
- "Show a side-by-side comparison of all Azure Functions languages."
- "Which runtime versions are available for Azure Functions?"

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local
Required: ✖

Functions: get project get

This tool returns project scaffolding information for a new Azure Functions app. It provides a project structure overview, setup instructions, and a file list that help you initialize a serverless project. The output helps you create the files and folders for the selected programming language.

Example prompts include:

- "Set up a new Azure Functions project in language 'python'."
- "Generate the project files for a TypeScript Azure Functions app, language 'typescript'."
- "Create the boilerplate for a Java Azure Functions app using JDK 21, language 'java'."

[🔗](#) Expand table

Parameter	Required or optional	Description
Language	Required	Programming language for the Azure Functions project. Valid values: <code>python</code> , <code>typescript</code> , <code>javascript</code> , <code>java</code> , <code>csharp</code> , <code>powershell</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Functions: list or get template

Generate Azure Functions code from templates that include triggers, bindings, AI agents, Durable Functions, and Model Context Protocol (MCP) servers, or list available templates. This tool generates serverless function code for a specified language. Without the template parameter, this tool lists available templates for the specified language. With the template parameter, this tool generates function code using the specified trigger and optional input and output bindings. You specify one trigger and zero or more bindings. Run this tool after you run `functions language list` and `functions project get`.

Example prompts include:

- "What triggers and bindings are available for Language 'csharp' Azure Functions?"
- "Show me all the Azure Function templates for Language 'python'."
- "Create a function from template 'TimerTrigger' in Language 'csharp' that runs every 5 minutes."
- "Show me template 'CosmosDBTrigger' with an output binding in Language 'java'."
- "I need template 'McpToolTrigger' in Language 'typescript' with runtime version '22'."

 Expand table

Parameter	Required or optional	Description
Language	Required	Programming language for the Azure Functions project. Valid values: <code>python</code> , <code>typescript</code> , <code>javascript</code> , <code>java</code> , <code>csharp</code> , <code>powershell</code> .
Runtime version	Optional	Optional runtime version for Java or TypeScript/JavaScript. When provided, template placeholders like <code>{{javaVersion}}</code> or <code>{{nodeVersion}}</code> are replaced automatically. See 'functions language list' for supported versions.
Template	Optional	Name of the function template to retrieve. Omit to list all available

Parameter	Required or optional	Description
name		templates for the specified language and valid values of template name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Functions documentation](#)

 **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 04/13/2026

Azure Key Vault tools for the Azure MCP Server overview

The Azure MCP Server lets you manage Azure Key Vault resources, including keys, secrets, and certificates with natural language prompts. You don't need to remember specialized command syntax to manage these resources.

[Azure Key Vault](#) is a cloud service for securely storing and accessing secrets, keys, and certificates. A secret is anything that you want to tightly control access to, such as API keys, passwords, or connection strings.

ⓘ **Note**

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Administration: Get all managed HSM settings

Retrieves all Key Vault Managed HSM account settings for a given vault. This includes settings such as purge protection and soft-delete retention days. This tool ONLY applies to Managed HSM vaults.

Example prompts include:

- **Get account settings:**
 - "Get the account settings for my managed HSM 'myhsm'"
 - "Show me the account settings for managed HSM 'contoso-hsm'"
- **Query a specific setting:** "What's the value of the 'purgeProtection' setting in my managed HSM with name 'myhsm'"

 Expand table

Parameter	Required or optional	Description
Vault	Required	The name of the Key Vault.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local
Required: ✖

Keys: Create key

The Azure MCP Server can create a new key in an Azure Key Vault. This operation lets you add cryptographic keys for your applications.

Example prompts include:

- **Create RSA key:** "Create a new RSA key named 'app-encryption-key' in my key vault 'mykeyvault'"
- **Generate EC key:** "Generate a new EC key called 'signing-key' in key vault 'security-kv'"
- **Add encryption key:** "Add a new 2048-bit RSA key named 'data-key' to key vault 'mykeyvault'"
- **Set up signing key:** "Create an EC key named 'signing-key' for JWT signing in key vault 'mykeyvault'"
- **Make new key:** "Create a P-256 EC key called 'jwt-signing' in my key vault 'api-vault'"

 Expand table

Parameter	Required or optional	Description
Vault	Required	The name of the Key Vault.
Key name	Required	The name of the key to create.
Key type	Required	The type of key to create. Supported types: <code>RSA</code> , <code>RSA-HSM</code> , <code>EC</code> , <code>EC-HSM</code> , <code>oct</code> , <code>oct-HSM</code> .

Tool annotation hints:

Destructive: ✔ | Idempotent: ✖ | Open World: ✖ | Read Only: ✖ | Secret: ✖ | Local
Required: ✖

Keys: Get or list keys

The Azure MCP Server can retrieve details of a specific key or list all keys in an Azure Key Vault. When you provide a key name, it returns details for that specific key. When you omit the key

name, it lists all keys in the vault.

Example prompts include:

- **Get key details:** "Show me details of the 'app-encryption-key' in my 'mykeyvault' Key Vault."
- **View key info:** "Get information about the 'signing-key' in Key Vault 'security-kv'"
- **List all keys:** "Show me all keys in my 'mykeyvault' Key Vault."
- **View keys:** "What keys do I have in Key Vault 'security-kv'?"
- **Query keys:** "Show all keys including managed keys in my Key Vault"

 Expand table

Parameter	Required or optional	Description
Vault	Required	The name of the Key Vault.
Key name	Optional	The name of the key to retrieve. Omit to list all keys.
Include managed	Optional	Whether or not to include managed keys when listing.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Secrets: Create secret

The Azure MCP Server can create a new secret in an Azure Key Vault. This operation lets you securely store sensitive information like passwords, API keys, and connection strings. This operation requires [user consent](#).

Example prompts include:

- **Create API secret:** "Create a secret named 'api-key' with value 'xyz123' in my key vault named 'production-vault'."
- **Store password:** "Add a secret called 'database-password' with value 'myP@ssw0rd' to key vault 'security-kv'"
- **Save connection string:** "Create a secret 'db-connection' with value 'Server=myserver;Database=mydb' in key vault 'mykeyvault'"
- **Add credentials:** "Store secret 'sp-secret' with value 'abc123def456' in key vault 'api-vault'"

[Expand table](#)

Parameter	Required or optional	Description
Vault	Required	The name of the Key Vault.
Secret name	Required	The name of the secret to create.
Value	Required	The value of the secret to store.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Secrets: Get or list secrets

The Azure MCP Server can retrieve a specific secret or list all secrets in a Key Vault. When you provide a secret name, it returns that specific secret value. When you omit the secret name, it lists all secrets in the vault. This operation requires [user consent](#) when retrieving a specific secret value.

Example prompts include:

- **Get specific secret:** "Retrieve the 'database-connection-string' secret from my 'production-vault' Key Vault."
- **Access API key:** "Get the 'third-party-api-key' secret from the 'api-secrets' vault"
- **List all secrets:** "Show me all secrets in my 'production-vault' Key Vault."
- **View secrets:** "What secrets do I have in Key Vault 'api-secrets'?"
- **Find secrets:** "List secrets in my Key Vault 'configuration-kv'"

[Expand table](#)

Parameter	Required or optional	Description
Vault	Required	The name of the Key Vault.
Secret name	Optional	The name of the secret to retrieve. Omit to list all secrets.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  (when retrieving specific secret) | Local Required: 

Certificates: Create certificate

The Azure MCP Server can create a new certificate in an Azure Key Vault by using the default policy. With this operation, you can generate SSL/TLS certificates for your applications.

Example prompts include:

- **Create SSL certificate:** "Create a certificate named 'web-ssl-cert' in my key vault 'production-vault'."
- **Generate certificate:** "Create a new certificate called 'api-tls-cert' in key vault 'security-kv'"
- **Add certificate:** "Generate a certificate 'webapp-cert' for my web application in key vault 'mykeyvault'"
- **Set up TLS cert:** "Create a certificate named 'app-certificate' in key vault 'mykeyvault'"
- **Make new cert:** "Create a certificate called 'service-cert' in key vault 'certificates-vault'"

 Expand table

Parameter	Required or optional	Description
Vault	Required	The name of the Key Vault.
Certificate name	Required	The name of the certificate to create.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Certificates: Get or list certificates

The Azure MCP Server can retrieve details of a specific certificate or list all certificates in an Azure Key Vault. When you provide a certificate name, it returns details for that specific certificate. When you omit the certificate name, it lists all certificates in the vault.

Example prompts include:

- **Get certificate details:** "Show me details of the 'web-ssl-cert' certificate in my 'production-vault' Key Vault."
- **View certificate info:** "Get information about the 'api-tls-cert' certificate in Key Vault 'security-kv'"
- **List all certificates:** "Show me all certificates in my 'production-vault' Key Vault."

- **View certificates:** "What certificates do I have in Key Vault 'security-kv'?"
- **Check certificates:** "What certificates are available in my 'ssl-vault'?"

 Expand table

Parameter	Required or optional	Description
Vault	Required	The name of the Key Vault.
Certificate name	Optional	The name of the certificate to retrieve. Omit to list all certificates.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Certificates: Import certificates

Imports an existing certificate (PFX or PEM with private key) into an Azure Key Vault without generating a new certificate or key material. If the certificate is a password-protected PFX, provide a password.

Example prompts include:

- **Import certificate from file:** "Import the certificate in file '/path/to/cert.pfx' into the key vault 'mykeyvault'."
- **Import certificate with name:** "Import a certificate into the key vault 'security-kv' using the name 'web-ssl-cert'."
- **Add PFX certificate:** "Import a PFX certificate from 'C:\certs\api.pfx' into key vault 'api-vault' as 'api-cert'."
- **Import PEM certificate:** "Import a PEM certificate into my key vault 'prod-vault' named 'prod-cert'."
- **Import password-protected certificate:** "Import the certificate 'secure.pfx' into key vault 'ssl-vault' with password 'mypassword'."

 Expand table

Parameter	Required or optional	Description
Vault	Required	The name of the Key Vault.
Certificate name	Required	The name of the certificate as it appears in Key Vault.
Certificate data or path	Required	Either the path to a PFX or PEM file, a base64 encoded PFX, or raw PEM text beginning with <code>-----BEGIN</code> .
Password	Optional	Password for a protected PFX being imported.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Manage Azure Key Vault with Azure MCP Server](#)
- [Azure Key Vault documentation](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/13/2026

Azure Kubernetes Service tools for the Azure MCP Server overview

Use the Azure MCP Server to manage Azure resources, including Azure Kubernetes Service (AKS) clusters, with natural language prompts. You can quickly manage your container workloads without remembering complex syntax.

[Azure Kubernetes Service \(AKS\)](#) is a managed container orchestration service that simplifies Kubernetes deployment and management. AKS provides serverless Kubernetes, integrated CI/CD, and enterprise-grade security and governance. With AKS, you focus on application development instead of infrastructure management.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Cluster: Get cluster details

Get or list Azure Kubernetes Service (AKS) clusters. If a specific cluster name is provided, that cluster will be retrieved. Otherwise, all clusters are listed in the specified subscription. Returns detailed cluster information including configuration, network settings, and status.

Example prompts include:

- **Get cluster configuration:** "Get the configuration of AKS cluster 'production-aks'"
- **Cluster details with resource group:** "Show me the details of AKS cluster 'web-app-cluster' in resource group 'containers-rg'"
- **Network configuration:** "Show me the network configuration for AKS cluster 'ml-workloads'"
- **Detailed cluster info:** "What are the details of my AKS cluster 'dev-kubernetes' in 'development-rg'?"
- **List all clusters:** "List all AKS clusters in my subscription for cluster 'production-aks'"
- **Show clusters:** "Show me Azure Kubernetes Service cluster 'web-app-cluster'"

- **Cluster inventory:** "What is the status of AKS cluster 'dev-kubernetes'?"
- **Production environment:** "Get configuration details for AKS cluster 'prod-aks-001' in resource group 'production'"
- **Development setup:** "Show me the setup of AKS cluster 'staging-aks' in 'staging-resources'"
- **Microservices cluster:** "What's the configuration of AKS cluster 'microservices-cluster' in 'apps-rg'?"

[Expand table](#)

Parameter	Required or optional	Description
Cluster name	Required	The name of the AKS cluster to get details for.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Node pool: Get details for a specific node pool

Get or list Azure Kubernetes Service (AKS) node pools (agent pools) in a cluster. If a specific node pool name is provided, that node pool is retrieved. Otherwise, all node pools are listed in the specified cluster. Returns key configuration and status including size, count, OS, mode, autoscaling, and provisioning state.

Example prompts include:

- **Get node pool details:** "Get details for node pool 'agentpool1' in AKS cluster 'production-aks' in 'containers-rg'"
- **Node pool configuration:** "Show me the configuration for node pool 'spotpool' in AKS cluster 'web-app-cluster' in resource group 'apps-rg'"
- **Setup information:** "What is the setup of node pool 'gpu-pool' for AKS cluster 'ml-workloads' in 'ai-resources'?"
- **List all node pools:** "List node pools for AKS cluster 'dev-kubernetes' in 'development-rg'"
- **Show node pool list:** "Show me the node pool list for AKS cluster 'microservices-cluster' in 'production'"
- **Node pool inventory:** "What node pools do I have for AKS cluster 'analytics-aks' in 'data-rg'?"

- **Production node pools:** "Get details for node pool 'systempool' in AKS cluster 'prod-aks-001' in 'production-rg'"
- **User node pools:** "Show configuration for node pool 'userpool' in AKS cluster 'staging-aks' in 'staging-resources'"
- **Specialized pools:** "What's the setup of node pool 'gpupool' for AKS cluster 'training-cluster' in 'ml-rg'?"
- **Scale information:** "List all node pools with scaling details for AKS cluster 'scalable-apps' in 'compute-rg'"

 Expand table

Parameter	Required or optional	Description
Cluster name	Required	AKS cluster name.
Node pool	Required	AKS node pool (agent pool) name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Kubernetes Service documentation](#)
- [Kubernetes best practices](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 12/09/2025

Azure Load Testing tools for the Azure MCP Server overview

The Azure MCP Server lets you manage Azure resources, including Azure Load Testing services, using natural language prompts. This feature helps you quickly create and manage load tests without needing to remember complex syntax.

[Azure Load Testing](#) is a fully managed load testing service that helps you generate high-scale load to identify application performance bottlenecks. With Azure Load Testing, you can stress test your applications and validate performance, scalability, and capacity.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Test: Create test

Creates a new load test in Azure Load Testing. Use this command to define and configure a load test for your application.

Example prompts include:

- **Create load test:** "Create a new load test with test resource 'loadtest-resource' test ID 'api-stress-001' display name 'API Stress Test' description 'Stress testing API endpoints' endpoint 'https://api.example.com' with 100 virtual users for 300 seconds duration and 60 seconds ramp-up time"
- **Set up test:** "Configure load test using test resource 'perf-test' test ID 'cart-load-001' display 'Shopping Cart Load Test' description 'Load test for cart API' endpoint 'https://cart.example.com/api' with 500 virtual users duration 600 seconds ramp-up 120 seconds"
- **New performance test:** "Create load test with test resource 'test-res' test ID 'peak-sim-001' display 'Peak Traffic Simulation' description 'Simulate peak traffic' endpoint 'https://app.example.com' 1000 virtual users 900 seconds duration 180 seconds ramp-up"

- **Initialize test:** "Set up load test with test resource 'ecommerce-test' test ID 'ecom-load-001' display 'E-commerce Load Test' description 'Load test for e-commerce site' endpoint 'https://shop.example.com' 200 virtual users 300 seconds duration 60 seconds ramp-up"
- **Test definition:** "Create test with test resource 'webapp-test' test ID 'web-load-001' display 'Web App Load Test' description 'Concurrent user simulation' endpoint 'https://webapp.example.com' 1000 virtual users 600 seconds duration 120 seconds ramp-up"

 Expand table

Parameter	Required or optional	Description
Test resource	Required	The name of the test resource to use.
Test ID	Required	A unique identifier for the test.
Display	Required	A user-friendly name for the new load test.
Description	Required	A description of the test and its purpose.
Endpoint	Required	The URL endpoint to test.
Virtual users	Required	The number of concurrent virtual users for the load test.
Duration	Required	The total duration of the test in seconds.
Ramp-up time	Required	The time period over which to gradually increase load to the specified number of virtual users.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Test: Get test details

Gets details about a specific load test in Azure Load Testing. Use this command to view the configuration and properties of an existing test.

Example prompts include:

- **View test details:** "Show me the configuration of load test with test resource 'loadtest-resource' and test ID 'api-stress-001'"

- **Check test setup:** "Get the details of test resource 'perf-test' with test ID 'peak-load-001' in resource group 'perf-testing'"
- **Test configuration:** "What are the settings for test resource 'prod-test' with test ID 'prod-readiness-001'?"
- **Examine test:** "Let me see the configuration of test resource 'test-res' with test ID 'recent-test-001'"
- **Test parameters:** "Show the parameters for test resource 'db-test' with test ID 'database-benchmark-001'"

 Expand table

Parameter	Required or optional	Description
Test resource	Required	The name of the test resource.
Test ID	Required	The unique identifier of the test.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Test resources: List test resources

Lists all test resources in the specified Azure subscription. Use this command to track and manage your load testing resources.

Example prompts include:

- **List test resources:** "Show me test resource 'loadtest-resource' in my subscription"
- **View available resources:** "What is test resource 'dev-loadtest' in my dev subscription?"
- **Resource inventory:** "List test resource 'perf-test-resource' in resource group 'performance-testing'"
- **Check environment:** "Show me test resource 'prod-test-resource' we provisioned in our subscription"
- **Find resources:** "Where is test resource 'webapp-test-resource' deployed?"

 Expand table

Parameter	Required or optional	Description
Test resource	Required	The name of a specific test resource to filter by.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local
Required: ✖

Test resources: Create test resource

Creates a test resource in Azure Load Testing. Use this command to set up resources needed for running load tests.

Example prompts include:

- **Create test resource:** "Create a new test resource 'loadtest-resource' for my load testing in resource group 'load-test-rg'"
- **Provision resources:** "Set up test resource 'perf-test-resource' for my performance testing in subscription 'test-sub'"
- **Initialize resource:** "Create test resource 'api-test-resource' for my 'api-load-test' in resource group 'perf-resources'"

 Expand table

Parameter	Required or optional	Description
Test resource	Required	A name for the new test resource.

Tool annotation hints:

Destructive: ✔ | Idempotent: ✖ | Open World: ✖ | Read Only: ✖ | Secret: ✖ | Local
Required: ✖

Test runs: Create or update test run

Create or update a load test run execution. This command creates a new test run for a specified test in the load testing resource or updates metadata and display properties of an existing test run. This command doesn't modify the test plan configuration or create a new test/resource; it solely manages test run executions.

When creating, it triggers a new test run execution based on the existing test configuration. Use the `testrun ID` to specify the new run identifier. The create operations are NOT idempotent—each call starts a new test run with unique timestamps and execution states.

When updating, this command modifies descriptive information (like display name and description) of a completed or in-progress test run for better organization and documentation. Update operations are idempotent, meaning repeated calls with the same values produce the same result.

Example prompts include:

- "Create a new test run for the load test with ID 'test-id-123'"
- "Update the display name of test run ID 'testrun-456' to 'Updated Test Run'"
- "I need to create a new load test run for the test ID 'test-id-789'"
- "How can I update the description for test run ID 'testrun-101' to 'New test execution with changes'?"
- "Show me how to create a test run for load test 'test-id-112' with a better display name"

 Expand table

Parameter	Required or optional	Description
Test ID	Required	The ID of the load test for which you want to fetch the details.
Description	Optional	The description for the load test run, providing more context.
Display name	Optional	A user-friendly display name to identify the load test run.
Old testrun ID	Optional	The ID of an existing test run to update. If provided, it triggers a rerun of the specified test run ID.
Test resource name	Optional	The name of the load test resource for which you want to fetch the details.
Testrun ID	Optional	The ID of the load test run for which you want to fetch the details.

Tool annotation hints: Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Test runs: Get or list test runs

Retrieve load test run details by `testrun` ID or list all test runs by `test` ID. This command returns execution details, including status, start and end times, progress, metrics, and artifacts. It doesn't return test configuration or resource details.

Example prompts include:

- "Show me all load test runs for test ID `test123`"
- "List all the test runs associated with load test ID `loadtest456`"
- "Get details for load test run ID `testrun789`"
- "What are the results for test run `testrun101` under load test ID `loadtest202`?"
- "Can you fetch the details for test run `testrun303`?"

 Expand table

Parameter	Required or optional	Description
Test ID	Optional	The ID of the load test for which you want to fetch the details.
Test resource name	Optional	The name of the load test resource for which you want to fetch the details.
Testrun ID	Optional	The ID of the load test run for which you want to fetch the details.

Tool annotation hints: Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Load Testing documentation](#)
- [Create and run a load test](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/19/2026

Azure Managed Grafana tools for the Azure MCP Server overview

The Azure MCP Server allows you to manage Azure resources, including Azure Managed Grafana workspaces using natural language prompts. This enables you to quickly manage your monitoring and visualization resources without remembering complex syntax.

[Azure Managed Grafana](#) is a fully managed service that offers Grafana dashboards as a service. It enables you to analyze metrics, logs, and traces without having to worry about setting up, maintaining, or scaling the Grafana infrastructure.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

List Grafana workspaces

Lists all Azure Managed Grafana workspaces in your subscription. This command helps you view and manage your Grafana workspace resources across your Azure environment.

Example prompts include:

- **Show all workspaces:** "Show me all Grafana workspaces in my subscription"
- **List monitoring resources:** "What Grafana workspaces do I have available?"
- **Find visualization dashboards:** "List all my Azure Managed Grafana resources"
- **Check workspace status:** "Are there any Grafana workspaces in my dev subscription?"
- **Dashboard inventory:** "I need to see all Grafana workspace resources in my account"

[Tool annotation hints:](#)

Destructive: ✗ | Idempotent: ✔ | Open World: ✗ | Read Only: ✔ | Secret: ✗ | Local Required: ✗

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Managed Grafana documentation](#)
- [Azure Monitor overview](#)
- [Data visualization in Azure](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/27/2026

Azure Managed Lustre tools for Azure MCP Server overview

Azure MCP Server lets you manage Azure resources, including Azure Managed Lustre, using natural language prompts. You can optimize AI and HPC workloads with scalable Lustre file systems.

[Azure Managed Lustre](#) is a high-performance, scalable file system built on the open-source Lustre technology and optimized for AI and HPC workloads on Azure. It provides the throughput, parallelism, and low-latency access required for large-scale simulation, model training, and fine-tuning. With [auto-import](#) and [auto-export](#) features, you can sync data between Azure Blob Storage and your Lustre file system. For one-time data movement, use manual import jobs.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

File system: Create a file system

Create an Azure Managed Lustre (AMLFS) file system using the specified network, capacity, maintenance window, and availability zone.

Example prompts include:

- **Basic filesystem creation:** "Create Azure Managed Lustre filesystem 'amlfs-prod-001' in resource group 'my-resource-group' in eastus with SKU 'AMLFS-Durable-Premium-125', size 128 TiB, in subnet '/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/my-rg/providers/Microsoft.Network/virtualNetworks/vnet-001/subnets/subnet-001', zone 1, maintenance on Sunday at 02:00"
- **Development environment:** "Create test filesystem 'dev-amlfs' in resource group 'my-resource-group' in westus2 using 'AMLFS-Durable-Premium-40' SKU with 32-TiB capacity in subnet '/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/dev-rg/providers/Microsoft.Network/virtualNetworks/dev-vnet/subnets/amlfs-subnet', availability zone 2, maintenance Wednesday at 14:00"
- **Secure filesystem with encryption:** "Create encrypted filesystem 'secure-amlfs' in resource group 'my-resource-group' in northeurope with 'AMLFS-Durable-Premium-125' SKU, 64-TiB capacity, subnet '/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/security-rg/providers/Microsoft.Network/virtualNetworks/secure-vnet/subnets/lustre-subnet', zone 1, maintenance Friday at 23:00, using custom encryption with key vault '/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/security-rg/providers/Microsoft.KeyVault/vaults/secure-kv' and key 'https://secure-kv.vault.azure.net/keys/lustre-key/a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p'"
- **Budget-optimized setup:** "Create cost-effective filesystem 'budget-fs' in resource group 'my-resource-group' in eastus2 with 'AMLFS-Durable-Premium-40', 48 TiB, subnet '/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/budget-rg/providers/Microsoft.Network/virtualNetworks/budget-vnet/subnets/storage-subnet', zone 1, maintenance Sunday at 05:00"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Name	Required	The AMLFS resource name. Must be DNS-friendly (letters, numbers, hyphens). Example: <code>amlfs-001</code> .
Location	Required	Azure region/region short name (use Azure location token, lowercase). Examples: <code>uaenorth</code> , <code>swedencentral</code> , <code>eastus</code> .
SKU	Required	The AMLFS SKU. Exact allowed values: <code>AMLFS-Durable-Premium-40</code> , <code>AMLFS-Durable-Premium-125</code> , <code>AMLFS-Durable-Premium-500</code> .
Size	Required	The AMLFS size in TiB as an integer (no unit). Examples: <code>4</code> , <code>12</code> , <code>128</code> .
Subnet ID	Required	Full subnet resource ID. Required format: <code>/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.Network/virtualNetworks/{vnet}/subnets/{subnet}</code> <code>/subscriptions/0000/resourceGroups/my-rg/providers/Microsoft.Network/virtualNetworks/vnet-001/subnets/subnet-001</code>
Zone	Required	Availability zone identifier. Use a single digit string matching the region's availability zone labels (for example <code>1</code>).
Maintenance day	Required	Preferred maintenance day. Allowed values: <code>Monday</code> , <code>Tuesday</code> , <code>Wednesday</code> , <code>Thursday</code> , <code>Friday</code> , <code>Saturday</code> , <code>Sunday</code> .
Maintenance time	Required	Preferred maintenance time in UTC. Format: <code>HH:MM</code> (24-hour). Examples: <code>00:00</code> , <code>23:00</code> .
HSM container	Optional	Full blob container resource ID for HSM integration. HPC Cache Resource Provider must have before deployment and Storage Account Contributor roles on parent Storage Account. Format: <code>/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.Storage/storageAccounts/{account}/blobServices/{blobService}</code> Example: <code>/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/rg/providers/Microsoft.Storage/storageAccounts/stacc/blobServices/default/container</code>
HSM log container	Optional	Full blob container resource ID for HSM logging. HPC Cache Resource Provider must have before deployment Storage Account Contributor roles on parent Storage Account. Same format as HSM container. Example: <code>/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/rg/providers/Microsoft.Storage/storageAccounts/stacc/blobServices/default/container</code>
Import prefix	Optional	Optional HSM import prefix (path prefix inside the container starting with <code>/</code>). Examples: <code> '/ingest/'</code> , <code> '/archive/2020/'</code> .
Root squash mode	Optional	Root squash mode. Allowed values: <code>All</code> , <code>RootOnly</code> , <code>None</code> .
No squash NID list	Optional	Comma-separated list of NIDs (network identifiers) not to squash. Example: <code>'10.0.2.4@tcp;10.0.2.[6-8]@tcp'</code> .
Squash UID	Optional	Numeric UID to squash root to. Required in case root squash mode isn't <code>None</code> . Example: <code>1000</code> .
Squash GID	Optional	Numeric GID to squash root to. Required in case root squash mode isn't <code>None</code> . Example: <code>1000</code> .
Custom encryption	Optional	Enable customer-managed encryption using a Key Vault key. When <code>true</code> , key URL and source vault required, with configured for Key Vault key access.
Key URL	Optional	Full Key Vault key URL. Format: <code>https://{vaultName}.vault.azure.net/keys/{keyName}/{keyVersion}</code> . Example: <code>https://amlfs-001.vault.azure.net/keys/key-amlfs-001/a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p</code> .
Source vault	Optional	Full Key Vault resource ID. Format: <code>/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.KeyVault/vaults/{vaultName}</code> <code>/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/rg/providers/Microsoft.KeyVault/vaults/vault-001</code>
User assigned identity ID	Optional	User-assigned managed identity resource ID (full resource ID) to use for Key Vault access when custom encryption have RBAC role to access the encryption key. Format: <code>/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.ManagedIdentity/userAssignedIdentities/{name}</code> <code>/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/rg/providers/Microsoft.ManagedIdentity/userAssignedIdentities/identity-001</code>

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

File system: List file systems

Get an inventory of Azure Managed Lustre file systems and check their properties.

Example prompts include:

- **List all file systems:** "List all Azure Managed Lustre file systems."
- **Show file system details:** "Get details for filesystem 'lustre-fs-01'."
- **Check file system status:** "What is the status of filesystem 'lustre-fs-01'?"
- **Filter by resource group:** "List Azure Managed Lustre file systems in resource group 'bigdata-rg'."
- **Filter by size:** "Show file systems larger than 100 TiB."

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local Required: ✖

File system: Calculate required subnet size

Calculates the required subnet size for an Azure Managed Lustre file system, given a SKU, and size. Use this calculation to plan network deployment for AMLFS.

Example prompts include:

- **Basic calculation:** "What is the required subnet size for filesystem with SKU 'AMLFS-Durable-Premium-125' and size 128 TiB?"
- **Small deployment:** "Calculate subnet size for Azure Managed Lustre filesystem with SKU 'AMLFS-Durable-Premium-250' and size 8 TiB"
- **Large scale planning:** "What subnet size do I need for a 512-TiB filesystem using 'AMLFS-Durable-Premium-500' SKU?"
- **Development environment:** "Calculate required subnet size for test filesystem with 'AMLFS-Durable-Premium-125' SKU and 32-TiB capacity"
- **Production planning:** "What is the subnet size requirement for production filesystem 'prod-amlfs-001' with 256 TiB using 'AMLFS-Durable-Premium-250'?"
- **High-performance setup:** "Calculate subnet requirements for AI training filesystem with 'AMLFS-Durable-Premium-500' and 1,024 TiB"
- **Research environment:** "What subnet size is needed for research filesystem 'ml-data-fs' with SKU 'AMLFS-Durable-Premium-125' and 64 TiB?"

 Expand table

Parameter	Required or optional	Description
SKU	Required	The AMLFS SKU. Allowed values: AMLFS-Durable-Premium-40, AMLFS-Durable-Premium-125, AMLFS-Durable-Premium-250, AMLFS-Durable-Premium-500.
Size	Required	The AMLFS size in tebibytes (TiB).

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local Required: ✖

File system: Get SKU

Retrieves the available Azure Managed Lustre SKU, including increments, bandwidth, scale targets, and zonal support.

Example prompts include:

- **List available SKUs:** "Show me the available Azure Managed Lustre SKUs."
- **Get SKUs by region:** "Display the available Azure Managed Lustre SKUs in West Europe."

[Expand table](#)

Parameter	Required or optional	Description
Location	Optional	Azure region. Examples: <code>uaenorth</code> , <code>swedencentral</code> , <code>eastus</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local

Required: 

File system: Update a file system

Update maintenance window and/or root squash settings of an existing Azure Managed Lustre (AMLFS) file system. Provide either maintenance day and time or root squash fields (`no-squash-nid-list`, `squash-uid`, `squash-gid`). Root squash fields must be provided if root squash isn't None. If updating the maintenance window, both maintenance day and maintenance time should be provided.

Example prompts include:

- **Basic maintenance window update:** "Update the maintenance window of the Azure Managed Lustre filesystem 'amlfs-prod-001' in resource group 'my-resource-group' to Sunday at 02:00"
- **Weekend maintenance schedule:** "Change maintenance window for filesystem 'hpc-lustre-fs' in resource group 'my-resource-group' to Saturday at 23:00"
- **Business hours maintenance:** "Update Azure Managed Lustre filesystem 'dev-amlfs' in resource group 'my-resource-group' maintenance to Wednesday at 14:30"
- **Off-peak scheduling:** "Set maintenance window for filesystem 'analytics-lustre' in resource group 'my-resource-group' to Monday at 01:00"
- **Root squash configuration:** "Update filesystem 'secure-amlfs' in resource group 'my-resource-group' with root squash mode 'All' and squash UID 1000 and GID 1000 with no squash NID list '10.0.2.4@tcp;10.0.2.[6-8]@tcp'"
- **Combined update:** "Update filesystem 'ml-amlfs' in resource group 'my-resource-group' maintenance to Friday at 03:00 and set root squash mode to 'None'"
- **Security hardening:** "Configure Azure Managed Lustre filesystem 'production-fs' in resource group 'my-resource-group' with no squash NID list '10.0.2.4@tcp;10.0.2.[6-8]@tcp', and squash GID 999"
- **Development environment:** "Update filesystem 'test-lustre' in resource group 'my-resource-group' maintenance window to Thursday at 12:00 for development testing"
- **Regional maintenance:** "Set maintenance schedule for filesystem 'europe-amlfs' in resource group 'my-resource-group' to Tuesday at 04:00 for minimal impact"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. A resource group is a logical container for Azure resources.
Name	Required	The AMLFS resource name. Must be DNS-friendly (letters, numbers, hyphens). Example: <code>amlfs-001</code> .
Maintenance day	Optional	Preferred maintenance day. Allowed values: <code>Monday</code> , <code>Tuesday</code> , <code>Wednesday</code> , <code>Thursday</code> , <code>Friday</code> , <code>Saturday</code> , <code>Sunday</code> .
Maintenance time	Optional	Preferred maintenance time in UTC. Format: <code>HH:MM</code> (24-hour). Examples: <code>00:00</code> , <code>23:00</code> .
No squash NID list	Optional	Comma-separated list of NIDs (network identifiers) not to squash. Example: <code>'10.0.2.4@tcp;10.0.2.[6-8]@tcp'</code> .

Parameter	Required or optional	Description
Squash UID	Optional	Numeric UID to squash root to. Required in case root squash mode isn't <code>None</code> . Example: <code>1000</code> .
Squash GID	Optional	Numeric GID to squash root to. Required in case root squash mode isn't <code>None</code> . Example: <code>1000</code> .
Root squash mode	Optional	Root squash mode. Allowed values: <code>All</code> , <code>RootOnly</code> , <code>None</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

File system: Validate subnet size

Validates that the provided subnet can host an Azure Managed Lustre filesystem for the given SKU and size.

Example prompts include:

- Basic validation:** "Validate if the network '/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/my-rg/providers/Microsoft.Network/virtualNetworks/vnet-001/subnets/subnet-001' can host Azure Managed Lustre filesystem of size 128 TiB using the SKU 'AMLFS-Durable-Premium-125'"
- Production environment:** "Check if subnet '/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/hpc-rg/providers/Microsoft.Network/virtualNetworks/hpc-vnet/subnets/lustre-subnet' can support AMLFS filesystem of 256 TiB with SKU 'AMLFS-Durable-Premium-250' in eastus"
- Development setup:** "Validate subnet capacity for Azure Managed Lustre filesystem size 48 TiB using SKU 'AMLFS-Durable-Premium-40' in subnet '/subscriptions/dev-sub/resourceGroups/dev-rg/providers/Microsoft.Network/virtualNetworks/dev-vnet/subnets/amlfs-subnet' in westus2"
- Large scale deployment:** "Can subnet '/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/ai-rg/providers/Microsoft.Network/virtualNetworks/ai-vnet/subnets/storage-subnet' host a 512-TiB Azure Managed Lustre filesystem using 'AMLFS-Durable-Premium-500' SKU in swedencentral?"

 Expand table

Parameter	Required or optional	Description
SKU	Required	The AMLFS SKU. Exact allowed values: <code>AMLFS-Durable-Premium-40</code> , <code>AMLFS-Durable-Premium-125</code> , <code>AMLFS-Durable-Premium-250</code> , <code>AMLFS-Durable-Premium-500</code> .
Size	Required	The AMLFS size in TiB as an integer (no unit). Examples: <code>4</code> , <code>12</code> , <code>128</code> .
Subnet ID	Required	Full subnet resource ID. Required format: <code>/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.Network/virtualNetworks/{vnet}/subnets/{subnet}</code> Example: <code>/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/my-rg/providers/Microsoft.Network/virtualNetworks/vnet-001/subnets/subnet-001</code> .
Location	Required	Azure region/region short name (use Azure location token, lowercase). Examples: <code>uaenorth</code> , <code>swedencentral</code> , <code>eastus</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Manual import: Create manual import job

Creates a manual import job for an Azure Managed Lustre filesystem to import files from the linked blob storage container. The import job performs a one-time sync of data from the configured HSM blob container to the Lustre filesystem. Use this job to import specific prefixes or all data from blob storage into the filesystem at a point in time.

Example prompts include:

- "Create a manual import job for filesystem 'LustreFs01' in resource group 'rg-lustre-prod'."
- "Start a manual import job on filesystem 'DataLakeFS' in resource group 'rg-data-core'."
- "Set up a manual import job for filesystem 'ProjectDataFS' within resource group 'rg-analytics'."
- "Initiate a manual import job on filesystem 'ArchiveFS' specifying the container 'importcontainer' in resource group 'rg-mlustredemo'."
- "Create a manual import job for filesystem 'TrainingFS' in resource group 'rg-lustre-eastus'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Filesystem name	Required	The name of the Azure Managed Lustre filesystem.
Job name	Optional	The name of the autoimport job. If not specified, the system generates a timestamped name.
Conflict resolution mode	Optional	Conflict resolution method for the autoimport job. Default: <code>Skip</code> . Allowed values: <code>Fail</code> , <code>Skip</code> , <code>OverwriteIfDirty</code> , <code>OverwriteAlways</code> . Please reference Conflict resolution mode for a thorough explanation of these resolution modes.
Import prefixes	Optional	Array of blob paths or prefixes to import from blob storage. Default: <code>/</code> . Maximum: 100 paths. Examples: <code>/data</code> , <code>/logs</code> .
Maximum errors	Optional	Total non-conflict-oriented errors (for example, OS errors) that the import job tolerates before exiting with failure. <code>-1</code> : infinite. <code>0</code> : exit immediately on any error.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Manual import: Get manual import job

Gets import job details or lists all import jobs for an Azure Managed Lustre filesystem. If the job name is provided, return details for that specific job. If job name is omitted, return a list of all import jobs for the filesystem.

Example prompts include:

- "Show me all import jobs for managed Lustre filesystem 'LustreFs01' in resource group 'rg-storageprod'"
- "List every import job in managed Lustre filesystem 'BackupFS' under resource group 'rg-lustre-dev'"
- "Get details for import job 'import123' from filesystem 'DataLakeFS' in resource group 'rg-storageprod'"

- "Retrieve information about import job 'dailybackup' on filesystem 'ArchiveFS' within resource group 'rg-lustre-dev'"
- "Can you show all import jobs linked to managed Lustre filesystem 'ProjectDataFS' in resource group 'rg-storageprod'?"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. A resource group is a logical container for Azure resources.
Filesystem name	Required	The name of the Azure Managed Lustre filesystem.
Job name	Optional	The name of the autoimport job. If not specified, the system generates a timestamped name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Manual import: Cancel manual import job

Cancels a running import job for an Azure Managed Lustre filesystem. This stops the import operation and prevents further processing. The job can't be resumed after cancellation.

Example prompts include:

- "Cancel the import job 'import1234' for filesystem 'LustreFs01' in resource group 'rg-storage-prod'"
- "I need to cancel the fs import job 'backup-sync' on filesystem 'DataLakeFS' in resource group 'rg-lustre-dev'"
- "Cancel the import job with ID 'dailybackup-import' from the managed Lustre filesystem 'ArchiveFS' in resource group 'rg-fileservices'"
- "Stop the fs import job 'import5678' on filesystem 'ProjectDataFS' in resource group 'rg-ml-prod'"
- "Cancel the managed Lustre fs import job named 'weeklysync' on filesystem 'TrainingFS' within resource group 'rg-analytics'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. A resource group is a logical container for Azure resources.
Filesystem name	Required	The name of the Azure Managed Lustre filesystem.
Job name	Required	The name of the autoexport or autoimport job.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Manual import: Delete manual import job

Deletes an import job for an Azure Managed Lustre filesystem. This operation removes the job record and history. The job must be completed or cancelled before it can be deleted.

Example prompts include:

- "Delete the import job 'import123' from filesystem 'LustreFs01' in resource group 'rg-lustre-prod'"
- "Remove the import job 'import456' on filesystem 'DataLakeFS' within resource group 'rg-data-central'"
- "Delete the fs import job with ID 'import789' from filesystem 'ArchiveFS' in resource group 'rg-backup'"
- "Remove fs import job 'importJob42' from filesystem 'ProjectDataFS' under resource group 'rg-analytics'"
- "Delete the import job 'syncJob01' on filesystem 'LustreFsX' within resource group 'rg-cluster-01'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. A resource group is a logical container for Azure resources.
Filesystem name	Required	The name of the Azure Managed Lustre filesystem.
Job name	Required	The name of the autoexport or autoimport job.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Auto-import: Get details of auto-import jobs

Get status, configuration, and progress details of [autoimport](#) jobs for an Azure Managed Lustre filesystem. Autoimport jobs sync data from the linked blob storage container to the Lustre filesystem. If you provide a job name, the tool returns details of that specific job. Otherwise, it returns all jobs for the filesystem.

Example prompts include:

- "Get the autoimport jobs for filesystem 'LustreFs01' in resource group 'rg-storage-prod'"
- "Show me the blob autoimport jobs for filesystem 'archiveLustre' within resource group 'rg-data-lake'"
- "Retrieve autoimport job details of the Managed Lustre filesystem 'fastLustreCompute' in resource group 'rg-hpc-environment'"
- "Can you fetch the autoimport job info for filesystem 'Lustre2024' from resource group 'rg-lustre-main'"
- "I need to get the fs blob autoimport job details for filesystem 'analyticsLustreFS' under resource group 'rg-analytics'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. A resource group is a logical container for Azure resources.
Filesystem name	Required	The name of the Azure Managed Lustre filesystem.
Job name	Optional	The name of the auto export or auto import job.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Auto-import: Create an auto-import job

Create an [autoimport](#) job to continuously import new or modified files from the linked blob storage container to your Azure Managed Lustre filesystem. The job syncs changes from the configured HSM blob container to the Lustre filesystem, keeping your filesystem updated with changes in blob storage.

Example prompts include:

- "Create an autoimport job for filesystem 'ProjectDataFS' in resource group 'rg-managedlustre-prod'"
- "Set up a blob autoimport on filesystem 'LustreMainFS' within resource group 'rg-dev-cluster'"
- "I need to create a Managed Lustre autoimport for filesystem 'AnalyticsFS' in resource group 'rg-analytics-eastus'"
- "Start an autoimport for the filesystem named 'ResearchFS' in resource group 'rg-research-lustre'"
- "Establish an autoimport job on filesystem 'FSBackup' under resource group 'rg-backup-westus2'"
- "Create an autoimport job for filesystem 'DataFS' in resource group 'rg-prod' with prefix '/data/incoming' and conflict resolution mode 'OverwriteIfDirty'"
- "Create an autoimport job for filesystem 'trainingLustre01' in resource group 'rg-training-lustre' with prefix '/datasets' and conflict resolution mode 'OverwriteAlways'"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	Azure resource group name.
Filesystem name	Required	Azure Managed Lustre filesystem name.
Job name	Optional	Autoimport job name. If you don't specify a name, a timestamped name is generated.
Conflict resolution mode	Optional	Conflict resolution method for the autoimport job. Default: <code>Skip</code> . Allowed values: <code>Fail</code> , <code>Skip</code> , <code>OverwriteIfDirty</code> , <code>OverwriteAlways</code> . Please reference Conflict resolution mode for a thorough explanation of these resolution modes.
Autoimport prefixes	Optional	Array of blob paths or prefixes to autoimport to the cluster namespace. Default: <code>/</code> . Maximum: 100 paths. Example: <code>/data</code> , <code>/logs</code> .
Admin status	Optional	Administrative status of the autoimport job. <code>Enable</code> : job is active. <code>Disable</code> : disables the current active autoimport job. Default: <code>Enable</code> . Allowed values: <code>Enable</code> , <code>Disable</code> .
Enable deletions	Optional	Specifies whether to enable deletions during autoimport. This parameter only affects overwrite-dirty mode. Default: <code>false</code> .
Maximum errors	Optional	Total non-conflict-oriented errors (for example, OS errors) that the import can tolerate before exiting with failure. <code>-1</code> : infinite. <code>0</code> : exits immediately on any error.

Tool annotation hints:

Destructive: ✅ | Idempotent: ❌ | Open World: ❌ | Read Only: ❌ | Secret: ❌ | Local Required: ❌

Auto-import: Cancel an auto-import job

Cancel a running [autoimport](#) job for your Azure Managed Lustre filesystem. This action stops the ongoing sync operation from the linked blob storage container to the Lustre filesystem.

Example prompts include:

- "Cancel the autoimport job named 'dailySyncJob' on filesystem 'LustreFs01' in resource group 'rg-storage-prod'"
- "I need to stop the job 'autoimportJob42' for filesystem 'ProjectLustre' within 'rg-data-central'"
- "Cancel the autoimport job 'importJobA1' on the Lustre filesystem 'FsBackup2024' in the resource group 'rg-backup'"
- "How do I cancel the job 'nightlyAutoImport' running on filesystem 'fastLustreFs' in resource group 'rg-performance'?"
- "Stop the autoimport job 'urgentSync' on Managed Lustre filesystem 'MainLustreFS' inside resource group 'rg-enterprise'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	Azure resource group name.
Filesystem name	Required	Azure Managed Lustre filesystem name.
Job name	Required	Autoimport job name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Auto-import: Delete an auto-import job

Delete an [autoimport](#) job for your Azure Managed Lustre filesystem. This action permanently removes the job record from the filesystem. Use this tool to clean up completed, failed, or canceled autoimport jobs.

Example prompts include:

- "Delete the autoimport job named 'importJob123' from filesystem 'LustreFs1' in resource group 'rg-lustre-prod'"
- "Remove autoimport job 'dailySync' for filesystem 'FsData2024' within resource group 'rg-storage-eus'"
- "I want to delete the job 'autoImportApril' from Managed Lustre filesystem 'DataLakeFs' inside resource group 'rg-datalake-west'"
- "Delete the fs blob autoimport job 'syncJob01' on filesystem 'LustreFsX' located in resource group 'rg-cluster-01'"
- "Can you delete the autoimport job 'weekly-import' on filesystem 'LustreMain' under resource group 'rg-enterprise-services'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	Azure resource group name.
Filesystem name	Required	Azure Managed Lustre filesystem name.
Job name	Required	Autoimport job name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Auto-export: Get details of an auto-export job

Get status, configuration, and progress details of [autoexport](#) jobs for your Azure Managed Lustre filesystem. Autoexport jobs sync data from the Lustre filesystem to the linked blob storage container. If you provide a job name, the tool returns details of that specific job. Otherwise, it returns all jobs for the filesystem.

Example prompts include:

- "Get the blob autoexport jobs for filesystem 'LustreFs01' in resource group 'rg-lustre-prod'"
- "Show me the autoexport jobs of the Managed Lustre filesystem named 'AnalyticsFs' within 'rg-data-center'"
- "Retrieve blob autoexport job details for filesystem 'ProjectXFs' in resource group 'rg-projectx'"
- "Can you provide the autoexport job information for the Lustre filesystem 'SalesDataFs' under resource group 'rg-salesapp'?"
- "Can you provide the autoexport job information for the Lustre filesystem 'TrainingDataFs' under resource group 'rg-training'?"
- "I need to see the blob autoexport jobs for 'ArchiveFs' in resource group 'rg-archives'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	Azure resource group name.
Filesystem name	Required	Azure Managed Lustre filesystem name.
Job name	Optional	Auto export job name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Auto-export: Create an auto-export job

Create an [autoexport](#) job to continuously export modified files from your Azure Managed Lustre filesystem to the linked blob storage container. The job syncs changes from the Lustre filesystem to the configured HSM blob container, keeping your blob storage updated with changes in the filesystem.

Example prompts include:

- "Create an autoexport job for filesystem 'DataLakeFS' in resource group 'rg-lustre-prod'"
- "Set up autoexport on Managed Lustre filesystem 'LustreMain' within resource group 'rg-hpc-cluster'"
- "I need to create a blob autoexport for filesystem 'faststorage' in resource group 'rg-data-analytics'"
- "Deploy a new autoexport task for the Lustre filesystem named 'ArchiveFS' under resource group 'rg-archive'"
- "Initiate autoexport on filesystem 'ResearchFS' located in resource group 'rg-science-projects'"
- "Create an autoexport job for filesystem 'OutputFS' in resource group 'rg-prod' with prefix '/results/processed'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	Azure resource group name.
Filesystem name	Required	Azure Managed Lustre filesystem name.
Job name	Optional	Autoexport job name. If you don't specify a name, a timestamped name is generated.
Autoexport prefix	Optional	Blob path or prefix to autoexport from the cluster namespace. Default: <code>/</code> . Note: Only one prefix is supported for autoexport jobs. Example: <code>/data</code> .
Admin status	Optional	Administrative status of the autoexport job. <code>Enable</code> : job is active. <code>Disable</code> : disables the current active autoexport job. Default: <code>Enable</code> . Allowed values: <code>Enable</code> , <code>Disable</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Auto-export: Cancel an auto-export job

Cancel a running [autoexport](#) job for your Azure Managed Lustre filesystem. This action stops the ongoing sync operation from the Lustre filesystem to the linked blob storage container.

Example prompts include:

- "Cancel the autoexport job named 'dailyBackupJob' on filesystem 'lustreProdFs' in resource group 'rg-lustre-apps'"
- "Stop the job 'trainingLustre01-autoexport' for filesystem 'trainingLustre01' in resource group 'rg-training-lustre'"
- "I need to cancel the autoexport job 'weeklySync' from the 'dataLustreFs' filesystem in resource group 'rg-data-services'"
- "How do I cancel the autoexport job called 'exportJob123' on filesystem 'prodLustreFs' within resource group 'rg-production'"
- "Abort the autoexport job 'monthlyExport' on filesystem 'archiveLustre' under resource group 'rg-archive-management'"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	Azure resource group name.
Filesystem name	Required	Azure Managed Lustre filesystem name.
Job name	Required	Autoexport job name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Auto-export: Delete an auto-export job

Delete an [autoexport](#) job for your Azure Managed Lustre filesystem. This action permanently removes the job record from the filesystem. Use this tool to clean up completed, failed, or canceled autoexport jobs.

Example prompts include:

- "Delete the autoexport job 'archiveExportJob' from filesystem 'LustreProdFs' in resource group 'rg-cloud-storage'"
- "Remove autoexport job 'dailyBackup' on filesystem 'LustreFS1' within resource group 'rg-datahub'"
- "Can you delete the job named 'autoExportJob42' for filesystem 'AzureLustreFs' in resource group 'rg-az-lustre'?"
- "Delete autoexport job 'monthlyExport' from the Managed Lustre filesystem 'LustreFsEast' located in resource group 'rg-eastus-lustre'"
- "I need to delete the autoexport job 'exportJob2024' in filesystem 'LustreMain' under resource group 'rg-production-lustre'"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	Azure resource group name.
Filesystem name	Required	Azure Managed Lustre filesystem name.
Job name	Required	Auto export job name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Managed Lustre](#)
- [Learn more about autoimport](#)
- [Learn more about autoexport](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/12/2026

Marketplace tools for the Azure MCP

Server overview

The Azure MCP Server lets you manage Azure resources, including Azure Marketplace products, by using natural language prompts. With this capability, you can quickly discover and retrieve information about marketplace offerings without needing to remember complex syntax.

[Azure Marketplace](#) is an online store for solutions that are built on or built for Azure. It's designed for IT professionals and developers. The marketplace offers a catalog of applications, services, and solutions from Microsoft and partners that help you accelerate your cloud adoption and digital transformation.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Get Marketplace product information

Get detailed information about a specific product or offer from Azure Marketplace. This operation helps you get comprehensive details about marketplace solutions, including pricing, plans, and availability information for evaluation and procurement decisions.

Example prompts include:

- **Get product details:** "Show me information about product ID 'microsoft-ads.windows-data-science-vm'"
- **Check specific plan:** "Get marketplace product details for product ID 'canonical.ubuntuuserver' with plan ID 'standard-plan'"

[↗](#) Expand table

Parameter	Required or optional	Description
Product ID	Required	The unique identifier for the marketplace product you want to retrieve information about.
Include stop sold plans	Optional	Whether to include plans that are no longer available for purchase in the results.
Language	Optional	The language code for localized product information (for example, <code>en-us</code> , <code>fr-fr</code>).
Market	Optional	The market or region code to get region-specific pricing and availability (for example, <code>US</code> , <code>FR</code>).
Lookup offer in tenant level	Optional	Whether to look up the offer at the tenant level for organization-specific information.
Plan ID	Optional	The specific plan identifier within the product to get detailed plan information.
SKU ID	Optional	The specific pricing SKU identifier for a specific product variant or configuration.
Include service instruction templates	Optional	Whether to include service instruction templates in the response for deployment guidance.
Pricing audience	Optional	The target audience for pricing information (for example, <code>public</code> , <code>private</code> , <code>government</code>).

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

List Marketplace information

Gets and lists all marketplace products (offers) available to a subscription in the Azure Marketplace. Use this tool to search, select, browse, or filter marketplace offers by product name, publisher, pricing, or metadata. Returns information for each product, including display name, publisher details, category, pricing data, and available plans.

Example prompts include:

- **List all products:** "List all marketplace products available in my subscription"
- **Search products:** "Find marketplace products related to `database`"
- **Filter by category:** "Show marketplace products in the `Analytics` category"

Parameter	Required or optional	Description
Language	Optional	Product language code (for example, <code>en</code> for English, <code>fr</code> for French).
Search	Optional	Search for products using a short general term (up to 25 characters).
Filter	Optional	OData filter expression to filter results based on ProductSummary properties (for example, <code>displayName eq 'Azure'</code>).
Orderby	Optional	OData orderby expression to sort results by ProductSummary fields (for example, <code>displayName asc</code> or <code>popularity desc</code>).
Select	Optional	OData select expression to choose specific ProductSummary fields to return (for example, <code>displayName,publisherDisplayName,uniqueProductId</code>).
Next cursor	Optional	Pagination cursor to retrieve the next page of results. Use the <code>NextPageLink</code> value from a previous response.
Expand	Optional	OData expand expression to include related data in the response (for example, <code>plans</code> to include plan details).

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Marketplace documentation](#)
- [Find solutions in Azure Marketplace](#)
- [Azure Marketplace purchasing overview](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Azure MCP tools management for the Azure MCP Server overview

The Azure MCP Server provides tools to discover and manage the available Azure tools within the server. You can list all available tools using natural language prompts without remembering specific command syntax.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

List tools

The Azure MCP Server can list all available tools and their capabilities. This helps you discover what Azure services and operations you can manage through the MCP server.

Example prompts include:

- **List all tools:** "Show me all available Azure MCP tools"
- **Discover capabilities:** "What tools are available in the Azure MCP server?"
- **View tool inventory:** "List all Azure tools I can use"
- **Check available services:** "What Azure services can I manage with MCP?"
- **Find tools:** "Show me what Azure operations are available"

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Azure Migrate tools for the Azure MCP Server overview

The Azure MCP Server lets you manage migration planning, guidance retrieval, and request submissions with natural language prompts.

Azure Migrate provides a centralized hub for assessing and migrating on-premises workloads to Azure. For more information, see [Azure Migrate documentation](#).

ⓘ **Note**

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Get guidance for landing zone

Get how-to guidance for modifying, configuring, or customizing an existing Platform Landing Zone.

Example prompts include:

- "How do I turn off Bastion in my Platform Landing Zone?"
- "Get guidance on changing resource naming prefixes for my Landing Zone."
- "Show me how to enable DDoS protection for my Platform Landing Zone."
- "I need instructions to adjust CIDR ranges in my Platform Landing Zone."
- "What steps do I take to disable the Azure Monitoring Agent in my Platform Landing Zone?"

 Expand table

Parameter	Required or optional	Description
Scenario	Required	The modification scenario key.

Parameter	Required or optional	Description
Policy name	Optional	The policy assignment name to look up (for example, <code>Enable-DDoS-VNET</code>). Used with <code>policy-enforcement</code> or <code>policy-assignment</code> scenarios.
List policies	Optional	Set to true to list all available policies organized by archetype. Useful for finding the exact policy name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Request platform landing zone

Generate and download platform landing zone configurations for Azure Migrate projects, update parameters, check existing landing zones, review parameter statuses and create a new Azure Migrate project if one doesn't exist.

Example prompts include:

- "Check if a platform landing zone exists for Azure Migrate project 'migrate-project-1' in resource group 'rg-prod'."
- "Update the parameters for the platform landing zone related to 'migrate-project-1' in resource group 'rg-dev'."
- "Generate the platform landing zone for Azure Migrate project 'migrate-project-2' located in resource group 'rg-test'."
- "Download the landing zone files for Azure Migrate project 'migrate-project-3' in resource group 'rg-production'."
- "What is the current status of parameters for Azure Migrate project 'migrate-project-4' in resource group 'rg-staging'?"

 Expand table

Parameter	Required or Optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Migrate	Required	The Azure Migrate project name for Platform Landing Zone generation

Parameter	Required or Optional	Description
project name		context.
Action	Required	The action to perform: <code>update</code> (set parameters), <code>check</code> (check existing platform landing zone), <code>generate</code> (generate platform landing zone), <code>download</code> (get download instructions), <code>status</code> (view parameter status), <code>createmigrateproject</code> (create a new Azure Migrate project if one doesn't exist, requires location parameter).
Region type	Optional	The region type for the Platform Landing Zone. Allowed values: <code>single</code> , <code>multi</code> .
Firewall type	Optional	The firewall type for the Platform Landing Zone. Allowed values: <code>azurefirewall</code> , <code>nva</code> .
Network architecture	Optional	The network architecture for the Platform Landing Zone. Allowed values: <code>hubspoke</code> , <code>vwan</code> .
Identity subscription ID	Optional	The Azure subscription ID for the identity management group in the Platform Landing Zone (GUID format).
Management subscription ID	Optional	The Azure subscription ID for the management group in the Platform Landing Zone (GUID format).
Connectivity subscription ID	Optional	The Azure subscription ID for the connectivity group in the Platform Landing Zone (GUID format).
Regions	Optional	Comma-separated list of Azure regions for the Platform Landing Zone (for example, <code>eastus,westus2</code>).
Environment name	Optional	The environment name for the Platform Landing Zone.
Version control system	Optional	The version control system for the Platform Landing Zone. Allowed values: <code>local</code> , <code>github</code> , <code>azuredevops</code> .
Organization name	Optional	The organization name for the Platform Landing Zone.
Migrate project resource ID	Optional	The full resource ID of the Azure Migrate project for the Platform Landing Zone (alternative to <code>subscription/resourceGroup/migrateProjectName</code>).

Tool annotation hints: Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
 - [Get started using Azure MCP Server](#)
 - [Azure Migrate documentation](#)
-

Last updated on 02/26/2026

Azure MCP Server tools for Azure Monitor and Workbooks

The Azure Model Context Protocol (MCP) Server lets you manage Azure Monitor and Workbooks resources with natural language prompts. You can query Log Analytics workspaces, analyze operational data, monitor resource health, retrieve performance metrics, and manage Azure Monitor workbooks.

[Azure Monitor](#) helps you maximize the availability and performance of your applications and services. It provides a comprehensive solution for collecting, analyzing, and acting on telemetry from your cloud and on-premises environments.

Workbooks provide a flexible canvas for data analysis and the creation of rich visual reports within the Azure portal. They allow you to tap into multiple data sources from across Azure and combine them into unified interactive experiences. Workbooks let you combine multiple kinds of visualizations and analyses, making them great for freeform exploration. For more information, see [Azure Monitor workbooks documentation](#).

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Activity Log: Get activity logs

Lists Azure Monitor activity logs for a specified Azure resource for a given number of past hours. This tool helps you understand resource deployment history, configuration changes, and access patterns. It returns activity log events that include timestamp, operation name, status, and caller information. Use the results to investigate failed deployments, unexpected changes, or access issues.

Example prompts include:

- "List the activity logs for the last '720' hours for resource 'webapp-prod'."

 Expand table

Parameter	Required or optional	Description
Resource name	Required	The name of the Azure resource to retrieve activity logs for.

Parameter	Required or optional	Description
Event level	Optional	The level of activity logs to retrieve. Valid levels are: <code>Critical</code> , <code>Error</code> , <code>Informational</code> , <code>Verbose</code> , <code>Warning</code> . If not provided, returns all levels.
Hours	Optional	The number of hours before now to retrieve activity logs for.
Resource type	Optional	The type of the Azure resource (for example, <code>'Microsoft.Storage/storageAccounts'</code>). Only provide this if needed to disambiguate between multiple resources with the same name.
Top	Optional	The maximum number of activity logs to retrieve.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Web Tests: Create or update web test

Part of the Model Context Protocol (MCP) tooling, this tool creates or updates a standard web test in Azure Monitor to check endpoint availability. You specify monitoring settings such as the URL, frequency, locations, and expected responses. If the test doesn't exist, this tool creates it; otherwise it updates the existing test with the new settings.

Example prompts include:

- "Create a new Standard Web Test with webtest resource 'webtest-prod-availability' in resource group 'rg-prod-monitoring' and associate it with AppInsights component '/subscriptions/12345678-1234-1234-123456789abc/resourceGroups/rg-ai/providers/microsoft.insights/components/appinsights-prod'."
- "Update an existing Standard Web Test for webtest resource 'webtest-prod-availability' in resource group 'rg-prod-monitoring' to link it to AppInsights component '/subscriptions/12345678-1234-1234-123456789abc/resourceGroups/rg-ai/providers/microsoft.insights/components/appinsights-prod'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group.
Webtest resource	Required	The name of the Web Test resource to operate on.
Appinsights component	Optional	The resource ID of the Application Insights component to associate with the web test.
Description	Optional	A brief description of the web test.
Enabled	Optional	Whether the web test is enabled.

Parameter	Required or optional	Description
Expected status code	Optional	Expected HTTP status code.
Follow redirects	Optional	Whether to follow HTTP redirects.
Frequency	Optional	Test frequency in seconds. Supported values: 300, 600, 900.
Headers	Optional	HTTP headers to include in the request, as comma-separated KEY=VALUE pairs.
HTTP verb	Optional	HTTP method to use, for example get or post.
Ignore status code	Optional	Whether to ignore the status code validation.
Location	Optional	The location where the web test resource is created. This should match the Application Insights component location.
Parse requests	Optional	Whether to parse dependent requests.
Request body	Optional	The body to send with the request.
Request URL	Optional	The absolute URL to test.
Retry enabled	Optional	Whether retries are enabled.
SSL check	Optional	Whether to validate SSL certificates.
SSL lifetime check	Optional	Number of days to check SSL certificate lifetime.
Timeout	Optional	Request timeout in seconds. Supported values: 30, 60, 90, 120.
Web test name	Optional	The name of the test within the web test resource.
Webtest locations	Optional	Comma-separated list of locations to run the test from. Location refers to the geo-location population tag for Availability Tests.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Web Tests: Get web test

This tool gets details for a specific web test or lists all web tests. When you specify the Webtest resource, this tool returns detailed information for that web test. When you don't specify the Webtest resource, this tool returns a list of all web tests in the subscription, and you can filter the list by resource group.

Example prompts include:

- "Get Web Test details for webtest resource 'webtest-prod' in my subscription in resource group 'rg-monitoring'."

- "List all Web Test resources in my subscription."
- "List all Web Test resources in my subscription in resource group 'rg-prod'."

 Expand table

Parameter	Required or optional	Description
Webtest resource	Optional	The name of the Web Test resource to operate on.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Log Analytics: List workspaces

This tool lists Log Analytics workspaces in a subscription. It retrieves each workspace's name, ID, location, and other key properties. You can use it to identify workspaces before you query their logs or examine workspace settings.

Example prompts include:

- "List Log Analytics workspaces in my subscription."
- "Display my Log Analytics workspaces."
- "Get the Log Analytics workspaces in my subscription."

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Log Analytics: List tables

This tool lists all tables in a Log Analytics workspace. For example, list tables in workspace 'prod-law' in resource group 'rg-monitoring' to preview available columns and data types. It returns table names and schemas you use to build Kusto Query Language (KQL) queries. You can filter by table type, for example `CustomLog` or `AzureMetrics`.

Example prompts include:

- "List all tables in Log Analytics workspace 'prod-law' of table type 'CustomLog' in resource group 'rg-prod'."
- "Show me tables of table type 'AzureMetrics' for workspace 'f1b2c3d4-5678-90ab-cdef-1234567890ab' in resource group 'rg-monitoring'."

 Expand table

Parameter	Required or optional	Description
Workspace name	Optional	The Log Analytics workspace ID or name. This can be either the unique identifier (GUID) or the display name of your workspace.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Log Analytics: Get table types

This Model Context Protocol (MCP) tool lists available table types in an Azure Log Analytics workspace. It returns the names of the table types. You can use those names when you write queries against Azure Monitor Logs.

Example prompts include:

- "List all available table types in Log Analytics workspace name 'prod-law-01' in resource group 'rg-prod'."
- "What table types are available in Log Analytics workspace name 'analytics-workspace' in resource group 'rg-logs'?"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group that contains the workspace.
Workspace name	Required	The name or ID of the Log Analytics workspace. You can use the workspace GUID or the display name.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Log Analytics: Query workspace logs

Query logs across an entire Log Analytics workspace using Kusto Query Language (KQL). This tool runs workspace-wide queries that return logs across all resources and tables in the workspace. This tool is part of the Model Context Protocol (MCP) tools. For example, you can ask: 'show all errors in my workspace', 'what happened in my workspace in the last 24 hours', 'list failed requests across the workspace'.

Example prompts include:

- "Show logs with query 'errors' from table 'Syslog' in Log Analytics workspace 'my-workspace' in resource group 'rg-prod'."

 Expand table

Parameter	Required or optional	Description
Query	Required	The Kusto Query Language (KQL) query to run against the Log Analytics workspace. You can use predefined queries by name: <code>recent</code> shows the most recent logs, ordered by TimeGenerated; <code>errors</code> shows error-level logs, ordered by TimeGenerated. Or, provide a custom KQL query.
Resource group	Required	The name of the Azure resource group that contains the workspace.
Table name	Required	The name of the table to query within the workspace.
Workspace name	Required	The Log Analytics workspace ID or name. You can provide either the globally unique identifier (GUID) or the display name of the workspace.
Hours	Optional	The number of hours to query back from now.
Limit	Optional	The maximum number of results to return.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Log Analytics: Query resource logs

Query diagnostic and activity logs for a specific Azure resource in a Log Analytics workspace by using Kusto Query Language (KQL). This tool filters results to the specified resource and runs the provided KQL query against the chosen table. For example, ask "Show logs for resource 'app-monitor' for the last 24 hours."

Example prompts include:

- "Show logs with query 'recent' for resource ID '/subscriptions/12345678-1234-1234-1234-123456789012/resourceGroups/rg-prod/providers/Microsoft.Compute/virtualMachines/my-vm' in table 'AzureDiagnostics'."

 Expand table

Parameter	Required or optional	Description
Query	Required	The KQL query to execute against the Log Analytics workspace. You can use predefined queries like <code>recent</code> shows the most recent logs ordered by TimeGenerated; <code>errors</code> shows error-level logs ordered by TimeGenerated. Otherwise, provide a custom KQL query.

Parameter	Required or optional	Description
Resource ID	Required	The Azure Resource ID of the resource to query. Example: /subscriptions/<sub>/resourceGroups/<rg>/providers/Microsoft.OperationalInsights/workspace
Table name	Required	The name of the table to query within the workspace.
Hours	Optional	The number of hours to query back from now.
Limit	Optional	The maximum number of results to return.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Health: Get entity health

This tool retrieves the health status and recent health events for a specific entity in an Azure Monitor health model. The Model Context Protocol (MCP) tool reports application-level health based on custom health models, not basic resource availability. For basic resource availability, use Azure Resource Health or the `azmcp_resourcehealth_availability-status_get` tool. To query logs in a Log Analytics workspace, use `azmcp_monitor_workspace_log_query`. To query logs for a specific Azure resource, use `azmcp_monitor_resource_log_query`.

Example prompts include:

- "Show me the health status of entity 'order-service' using the health model 'app-health-v1' in resource group 'rg-prod'."

 Expand table

Parameter	Required or optional	Description
Entity name	Required	The entity to get health for.
Health model	Required	The name of the health model for which to get the health.
Resource group	Required	The name of the Azure resource group. This resource group is a logical container for Azure resources.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Metrics: Query metrics

Query Azure Monitor metrics for a resource. This tool returns time series data for the specified metrics, helping you analyze resource performance and availability. This tool is part of the Model Context Protocol (MCP) tools.

Example prompts include:

- "Analyze performance trends and response times for Application Insights resource 'appinsights-prod' with metrics 'requests/duration' and metric namespace 'microsoft.insights/components'."
- "Check the availability metric 'availabilityResults/availabilityPercentage' for Application Insights resource 'appinsights-staging' using metric namespace 'microsoft.insights/components'?"
- "Get the metric 'requests/duration' with aggregation 'Average' and interval 'PT1M' for resource 'appinsights-prod' using metric namespace 'microsoft.insights/components'."
- "Investigate error rates and failed requests for Application Insights resource 'appinsights-prod' using metrics 'requests/failed,exceptions/count' and metric namespace 'microsoft.insights/components'."
- "Query the metric 'requests/count' for resource type 'Microsoft.Insights/components' resource 'appinsights-qa' with metric namespace 'microsoft.insights/components' and interval 'PT5M'."
- "What's the requests per second rate using metric 'requests/count' with aggregation 'Count' for Application Insights resource 'appinsights-prod' and metric namespace 'microsoft.insights/components'?"

 Expand table

Parameter	Required or optional	Description
Metric names	Required	The names of metrics to query, comma-separated.
Metric namespace	Required	The metric namespace to query. Obtain this value from the azmcp-monitor-metrics-definitions tool.
Resource name	Required	The name of the Azure resource to query metrics for.
Aggregation	Optional	The aggregation type to use, such as Average, Maximum, Minimum, Total, or Count.
End time	Optional	The end time for the query in ISO format (for example, 2023-01-01T00:00:00Z). Defaults to now.
Filter	Optional	The OData filter to apply to the metrics query.
Interval	Optional	The time interval for data points (for example, PT1H for 1 hour, PT5M for 5 minutes).
Max buckets	Optional	The maximum number of time buckets to return. Defaults to 50.
Resource type	Optional	The Azure resource type (for example, Microsoft.Storage/storageAccounts,

Parameter	Required or optional	Description
		<code>Microsoft.Compute/virtualMachines</code>). If not specified, the tool attempts to infer the type from the resource name.
Start time	Optional	The start time for the query in ISO format (for example, <code>2023-01-01T00:00:00Z</code>). Defaults to 24 hours ago.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Metrics: List metric definitions

This tool lists metric definitions for an Azure resource. It returns metadata about each metric, including namespaces, descriptions, and aggregation types, so you can determine which metrics to query for a resource.

Example prompts include:

- "Get metric definitions for resource name 'app-insights-prod'."
- "List metric definitions for resource name 'mystorageacct' with resource type 'Microsoft.Storage/storageAccounts' and metric namespace 'Storage'."
- "Show metric definitions for resource name 'vm-prod-01' with search string 'cpu' and limit '20'."

 Expand table

Parameter	Required or optional	Description
Resource name	Required	The name of the Azure resource to query metrics for.
Limit	Optional	The maximum number of metric definitions to return. Defaults to 10.
Metric namespace	Optional	The metric namespace to query. Obtain this value from the <code>azmcp-monitor-metrics-definitions</code> tool.
Resource type	Optional	The Azure resource type (for example, <code>Microsoft.Storage/storageAccounts</code> , <code>Microsoft.Compute/virtualMachines</code>). If you don't specify it, the tool attempts to infer the resource type from the resource name.
Search string	Optional	A string to filter the metric definitions. The filter performs case-insensitive matching on metric name and description.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Instrumentation: Get learning resource

This tool lists all available learning resources for Azure Monitor instrumentation, or it retrieves the content of a specific resource by path. By default, the tool returns all resource paths. If you specify a path, the tool returns the full resource content. To instrument an application, use the orchestrator-start tool.

Example prompts include:

- "Get the onboarding learning resource at path 'onboarding/get-started.md'."
- "Show me the content of the Azure Monitor onboarding learning resource at path 'onboarding/quickstart.md'."
- "Retrieve the content of the Azure Monitor learning resource file at path 'samples/instrumentation-guide.html'."
- "List all Azure Monitor onboarding learning resources."
- "Show me all learning resource paths for Azure Monitor instrumentation."
- "Which learning resources are available for Azure Monitor instrumentation onboarding?"

 Expand table

Parameter	Required or optional	Description
Path	Optional	Learning resource path.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Instrumentation: Start orchestration

Start here for Model Context Protocol (MCP) tools that instrument Azure Monitor. This tool analyzes the workspace and returns the first action to execute. After you execute the action, call orchestrator-next to continue. Follow the action in the `instruction` field exactly.

Example prompts include:

- "Start Azure Monitor instrumentation orchestration for workspace path '/home/dev/workspace-monitoring'."
- "Analyze workspace path '/src/projects/my-app-workspace' and return the first Azure Monitor instrumentation step."
- "Begin guided Azure Monitor onboarding for project at workspace path '/workspace/my-app' and give me step one."

 Expand table

Parameter	Required or optional	Description
Workspace path	Required	Absolute path to the workspace folder.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✖ | Open World: ✖ | Read Only: ✖ | Secret: ✖ | Local
Required: ✔

Instrumentation: Continue orchestration

Get the next instrumentation action after you complete the current one.

This tool is part of the Model Context Protocol (MCP) suite.

After you execute the exact `instruction` from the previous response, run this tool to receive the next action.

Expected workflow:

1. You receive an action from orchestrator-start or orchestrator-next.
2. You execute the `instruction` field exactly.
3. You run this tool with a concise `Completion note` to get the next action.

Returns: The next action to execute, or `complete` status when all steps are done.

Example prompts include:

- "After completing the previous Azure Monitor instrumentation step, get the next action for session ID 'session-abc123' with completion note 'Added UseAzureMonitor() to Program.cs'."
- "Get the next onboarding action for session ID 'workspace/session-2026' with completion note 'Ran dotnet add package Microsoft.ApplicationInsights'."
- "After finishing the previous instrumentation step, return the next step for session ID 'session-789xyz' with completion note 'Updated appsettings.json to enable Application Insights'."

 Expand table

Parameter	Required or optional	Description
Completion note	Required	One sentence describing what you executed, for example, 'Ran dotnet add package command' or 'Added UseAzureMonitor() to Program.cs'.
Session ID	Required	The workspace path returned as sessionId from orchestrator-start.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✖ | Open World: ✖ | Read Only: ✖ | Secret: ✖ | Local
Required: ✔

Instrumentation: Send brownfield analysis

Sends brownfield code analysis findings after `orchestrator-start` returns status `analysis_needed`. This tool is part of the Model Context Protocol (MCP) workflow. You must scan the workspace source files and fill in the analysis template before you call this tool. After this tool succeeds, continue with `orchestrator-next`.

Example prompts include:

- "Send brownfield code analysis findings JSON
'{"serviceOptions":null,"initializers":null,"processors":null,"clientUsage":null,"sampling": {"found":false,"hasCustomSampling":false},"telemetryPipeline":null,"logging":null}' to Azure Monitor instrumentation session 'workspace-7a3b' after analysis was requested."
- "Continue migration orchestration by submitting findings JSON '{"serviceOptions": {"found":true,"details":"AddApplicationInsightsTelemetry used"},"initializers": [],"processors":[],"clientUsage":null,"sampling": {"found":false,"hasCustomSampling":false},"telemetryPipeline":null,"logging":null}' to session 'sess-01234'."
- "Send completed brownfield telemetry analysis as findings JSON '{"serviceOptions":null,"initializers":null,"processors":null,"clientUsage": {"found":true},"sampling": {"found":false,"hasCustomSampling":false},"telemetryPipeline":null,"logging": {"found":true}}' for onboarding session 'session-9f3b'."

[Expand table](#)

Parameter	Required or optional	Description
Findings JSON	Required	JSON object with brownfield analysis findings. Required properties: serviceOptions (service options findings from analyzing AddApplicationInsightsTelemetry() call, null if not found), initializers (telemetry initializer findings from analyzing ITelemetryInitializer or IConfigureOptions<TelemetryConfiguration> implementations, null if none found), processors (telemetry processor findings from analyzing ITelemetryProcessor implementations, null if none found), clientUsage (TelemetryClient usage findings from analyzing direct TelemetryClient usage, null if not found), sampling (custom sampling configuration findings, null if no custom sampling), telemetryPipeline (custom ITelemetryChannel or TelemetrySinks usage findings, null if not found), logging (explicit logger provider and filter findings, null if not found). For sections that don't exist in the codebase, pass an empty default object, for example found: <code>false</code> or hasCustomSampling: <code>false</code> , instead of null.
Session ID	Required	The workspace path returned as <code>sessionId</code> from <code>orchestrator-start</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Instrumentation: Send enhancement selection

Submit the user's enhancement selection after `orchestrator-start` returns status `enhancement_available`. Present the enhancement choices to the user, then call this tool with the chosen enhancement keys. You can select multiple enhancements by passing a comma-separated list, for example, `redis,processors`. After this tool succeeds, continue with `orchestrator-next`.

Example prompts include:

- "Submit enhancement keys 'redis,processors' for Azure Monitor instrumentation session ID 'workspaces/my-app/session-abc123'."
- "Continue instrumentation enhancement flow by sending enhancement keys 'redis' to session ID 'workspaces/prod-app/session-789'."
- "Send chosen enhancement keys 'entityframework,otlp' for onboarding session ID 'workspaces/onboard/session-456'."

 Expand table

Parameter	Required or optional	Description
Enhancement keys	Required	One or more enhancement keys, comma-separated (for example, <code>redis</code> , <code>redis,processors</code> , <code>entityframework,otlp</code>).
Session ID	Required	The workspace path returned as <code>sessionId</code> from <code>orchestrator-start</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Workbooks: List workbooks

Search Azure Workbooks using Resource Graph for fast metadata queries. This tool helps you discover, filter, and count workbooks across different scopes.

It returns workbook metadata, including `id`, `name`, `location`, `category`, and timestamps. By default, it doesn't return full workbook content (`serializedData`) — use the `show` tool for that, or set `Output format` to `full`.

By default, the search targets workbooks in your current Azure context (tenant/subscription). You can use `Resource group` to explicitly specify your search scope. The tool returns the server-side total count by default. The maximum results returned is 50, with a maximum limit of 1000; adjust this with `Max results`. Choose `Output format` as `summary` for minimal tokens or `full` for complete `serializedData` output.

Example prompts include:

- "Show me all workbooks in resource group 'monitoring-rg'."
- "List the shared workbooks in resource group 'prod-rg'."
- "What workbooks were modified after 2024-01-15 in resource group 'analytics-rg'?"

 Expand table

Parameter	Required or optional	Description
Resource group	Optional	The name of the Azure resource group to scope the search.
Category	Optional	Filter workbooks by category (for example, <code>workbook</code> , <code>sentinel</code> , <code>TSG</code>). If not specified, all categories are returned.
Include total count	Optional	Include the total count of all matching workbooks in the response (default: true).
Kind	Optional	Filter workbooks by kind (for example, <code>shared</code> , <code>user</code>). If not specified, all kinds are returned.
Max results	Optional	Maximum number of results to return (default: 50, max: 1000).
Modified after	Optional	Filter workbooks modified after this date (ISO 8601 format, for example, <code>2024-01-15</code>).
Name contains	Optional	Filter workbooks where the display name contains this text (case-insensitive).
Output format	Optional	Output format: <code>summary</code> (ID and name only, minimal tokens), <code>standard</code> (metadata without content, default), <code>full</code> (includes <code>serializedData</code>).
Source ID	Optional	Filter workbooks by source resource ID (for example, <code>/subscriptions/abc123/resourceGroups/prod/providers/Microsoft.Insights/components/myapp</code>). If not specified, all workbooks are returned.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Workbooks: Show workbook details

Retrieve full workbook details via the Azure Resource Manager (ARM) API, including the `serializedData` content. This command lets you get the complete workbook definition, including the visualization JSON.

It returns full workbook properties, `serializedData`, tags, and `ETag`. You can provide multiple `Workbook IDs` for batch operations. The command reports partial failures for individual workbooks. For better performance, use the list tool to discover workbooks first, then use show for specific workbooks.

Example prompts include:

- "Show me the details of the workbook with resource ID '/subscriptions/abc123/resourceGroups/monitoring/providers/Microsoft.Insights/workbooks/a0a0a0a0-bbbb-cccc-dddd-e1e1e1e1e1e1'."
- "Get the full definition of the workbook '/subscriptions/xyz789/resourceGroups/prod-rg/providers/Microsoft.Insights/workbooks/b1b1b1b1-cccc-dddd-eeee-f2f2f2f2f2f2'."

[Expand table](#)

Parameter	Required or optional	Description
Workbook IDs	Required	The Azure resource IDs of the workbooks to retrieve. Supports multiple values for batch operations.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Workbooks: Create workbook

Create a new workbook in the specified resource group and subscription. You can set the display name and the serialized JSON content for the workbook. This command returns the created workbook information upon successful completion.

Example prompts include:

- "Create a new workbook named 'Performance Dashboard' in resource group 'monitoring-rg' with the serialized content for a basic notebook."
- "Create a workbook called 'Infrastructure Overview' in resource group 'prod-rg' with content showing VM metrics."

[Expand table](#)

Parameter	Required or optional	Description
Display name	Required	The display name of the workbook.
Resource group	Required	The name of the Azure resource group containing the workbook.
Serialized content	Required	The serialized JSON content of the workbook.
Source ID	Optional	The linked resource ID for the workbook. By default, this is <code>azure monitor</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Workbooks: Update workbook

Update properties of an existing Azure workbook by adding new steps, modifying content, or changing the display name. This action returns the updated workbook details. You need the workbook resource ID and can specify either new serialized content or a new display name.

Example prompts include:

- "Update the workbook '/subscriptions/abc123/resourceGroups/monitoring-rg/providers/Microsoft.Insights/workbooks/a0a0a0a0-bbbb-cccc-dddd-e1e1e1e1e1' with display name 'Monthly Report'."
- "Change the serialized content of workbook '/subscriptions/xyz789/resourceGroups/prod-rg/providers/Microsoft.Insights/workbooks/b1b1b1b1-cccc-dddd-eeee-f2f2f2f2f2' to include a new metrics chart."

 Expand table

Parameter	Required or optional	Description
Workbook ID	Required	The Azure resource ID of the workbook to update.
Display name	Optional	The display name of the workbook.
Serialized content	Optional	The JSON serialized content of the workbook.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Workbooks: Delete workbooks

Delete one or more workbooks by their Azure resource IDs. This command performs a soft delete on workbooks, retaining them for 90 days. You can restore them from the Recycle Bin through the Azure portal if needed.

For batch operations, you can provide multiple `Workbook IDs` values. The command reports partial failures per workbook, ensuring that individual failures don't affect the entire batch operation.

To learn more, see [Manage Azure Monitor workbooks](#).

Example prompts include:

- "Delete the workbook with resource ID '/subscriptions/abc123/resourceGroups/monitoring/providers/Microsoft.Insights/workbooks/a0a0a0a0-bbbb-cccc-dddd-e1e1e1e1e1'."

- "Remove the workbooks with resource IDs '/subscriptions/xyz789/resourceGroups/prod-rg/providers/Microsoft.Insights/workbooks/b1b1b1b1-cccc-dddd-eeee-f2f2f2f2f2' and '/subscriptions/def456/resourceGroups/analytics-rg/providers/Microsoft.Insights/workbooks/c2c2c2c2-dddd-eeee-ffff-a3a3a3a3a3a3'."

[Expand table](#)

Parameter	Required or optional	Description
Workbook IDs	Required	The Azure resource IDs of the workbooks to delete. Supports multiple values for batch operations.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Monitor](#)
- [Application Insights](#)
- [Workbooks in Azure Monitor](#)
- [Metrics in Azure Monitor](#)

 **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 04/10/2026

Azure Native ISV tools for Azure MCP Server overview

The Azure MCP Server allows you to manage Azure resources, including Azure Native ISV partner solutions, using natural language prompts. This enables you to quickly manage third-party services that are natively integrated with Azure without remembering complex syntax, improving productivity and reducing operational overhead.

[Azure Native Integrations](#) enable you to easily provision, manage, and tightly integrate software and services from partner companies on Azure. Microsoft and partner organizations develop these services and manage them together, providing a seamless experience through the Azure portal.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

ISV partners

- [Datadog](#) - A monitoring and analytics platform for large-scale applications that encompasses infrastructure monitoring, application performance monitoring, log management, and user-experience monitoring.

Datadog monitored resources

The Azure MCP Server can list monitored resources in Datadog. [Datadog](#) is a monitoring and analytics platform for large-scale applications that encompasses infrastructure monitoring, application performance monitoring, log management, and user-experience monitoring.

Datadog's Azure Native Integration allows you to manage Datadog directly in the Azure console as an integrated service. This streamlined workflow covers everything from procurement to configuration, making it easy to start monitoring the health and performance of your applications across Azure, hybrid, or multicloud environments.

Example prompts include:

- **List monitored resources:** "Show me all resources being monitored by Datadog resource 'my-datadog' in resource group 'my-resource-group'"
- **Check monitoring status:** "What resources are being monitored by Datadog 'main-datadog' in resource group 'my-resource-group'?"
- **View monitoring coverage:** "List all monitored resources for Datadog resource 'company-datadog' in resource group 'my-resource-group'"
- **Audit monitoring:** "Show me what's being monitored by Datadog 'prod-datadog' in resource group 'my-resource-group'"
- **Inventory check:** "Get the list of resources monitored by Datadog 'monitor-datadog' in resource group 'my-resource-group'"

 Expand table

Parameter	Required	Description
Datadog resource	Required	The name of the Datadog resource in Azure.
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Partner Solutions documentation](#)
- [Datadog Azure integration documentation](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Azure Policy tools for Azure MCP Server overview

Azure MCP Server enables you to view Azure Policy assignments, definitions, and initiatives using natural language prompts. Simplify policy management without complex syntax.

[Azure Policy](#) is a service in Azure that allows you to create, assign, and manage policies to enforce rules and effects on your resources. It helps ensure compliance with organizational standards and regulatory requirements.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Policy: list assignments

List policy assignments in a subscription or scope. This command retrieves all Azure Policy assignments along with their complete policy definition details (rules, effects, parameters schema), enforcement modes, assignment parameters, and metadata. You can optionally filter by scope to list assignments at a specific resource group, resource, or management group level.

Example prompts include:

- "Show me all policy assignments in resource group 'rg-contoso'"
- "List every policy assignment available in resource group 'rg-production'"
- "Get details for policy assignment 'AuditVMUpdates' within resource group 'rg-contoso'"
- "Can you fetch the policy assignment called 'EnforceTagging' in resource group 'rg-marketing'?"
- "Retrieve the policy assignment named 'SecureStorage' from resource group 'rg-security'"

[🔗](#) Expand table

Parameter	Required or optional	Description
Scope	Optional	The scope of the policy assignment (for example, <code>/subscriptions/{subscriptionId},</code> <code>/subscriptions/{subscriptionId}/resourceGroups/{resourceGroupName}</code>).

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Policy](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 01/24/2026

Azure pricing tools for the Azure MCP Server

Azure [pricing tools](#) in MCP Server help you manage cost estimates, billing questions, and budget tracking using natural language prompts. This article explains how these tools let organizations learn about Azure spending and manage costs effectively.

ⓘ **Note**

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Get pricing information

Get Azure retail pricing information. The tool can estimate deployment costs from an ARM or Bicep template. You can provide the template within the prompt or as a file input.

Example prompts include:

- "This is my Bicep template: <bicep_template>. Estimate my deployment costs."
- "What is the pricing for SKU `Standard_D4s_v5` in region `eastus`?"
- "Can I get the pricing details for service `Virtual Machines` and SKU `Standard_E64-16ds_v4`?"
- "Show me the `Consumption` pricing for SKU `Standard_D4s_v5` in `westeurope` and include savings plan."
- "List pricing information for all SKUs in the `Storage` service in region `westus2`."

 Expand table

Parameter	Required or optional	Description
Currency	Optional	Currency code for pricing (for example, <code>USD</code> , <code>EUR</code>). The default is <code>USD</code> .
SKU	Optional*	ARM SKU name (for example, <code>Standard_D4s_v5</code> , <code>Standard_E64-16ds_v4</code>).

Parameter	Required or optional	Description
Service	Optional*	Azure service name (for example, <code>Virtual Machines</code> , <code>Storage</code> , <code>SQL Database</code>).
Region	Optional*	Azure region (for example, <code>eastus</code> , <code>westeurope</code> , <code>westus2</code>).
Service family	Optional*	Service family (for example, <code>Compute</code> , <code>Storage</code> , <code>Databases</code> , <code>Networking</code>).
Price type	Optional*	Price type filter (for options: <code>Consumption</code> , <code>Reservation</code> , <code>DevTestConsumption</code>).
Include savings plan	Optional	Include savings plan pricing information (uses preview API version).
Filter	Optional*	Raw OData filter expression for advanced queries (for example, <code>meterId eq 'abc-123'</code>). For more about OData, see OData documentation .

- At least one filter option is required.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Pricing Calculator documentation](#)

Last updated on 02/20/2026

Azure Quick Review CLI tools for the Azure MCP Server overview

The Azure MCP Server allows you to execute Azure Quick Review (azqr) commands using natural language prompts. This enables you to generate compliance and security reports for your Azure resources to identify non-compliant configurations and areas for improvement without needing to remember specific command syntax.

[Azure Quick Review CLI \(azqr\)](#)  is a powerful command-line interface (CLI) tool that specializes in analyzing Azure resources to ensure compliance with Azure's best practices and recommendations. Its main objective is to offer users a comprehensive overview of their Azure resources, allowing them to easily identify any non-compliant configurations or areas for improvement.

Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Generate compliance report

The Azure MCP Server can execute Azure Quick Review CLI commands to generate compliance and security reports for Azure resources. This helps identify non-compliant configurations and areas for improvement in your Azure environment.

Example prompts include:

- **Scan subscription:** "Generate compliance report for my subscription"
- **Scan resource group:** "Run security assessment for production resource group"
- **Quick review:** "Check my subscription for compliance issues"
- **Security scan:** "scan resources in dev-rg for security problems"
- **Generate report:** "Create compliance report for subscription abc123 and resource group web-apps"

[Tool annotation hints:](#)

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Quick Review CLI GitHub repository](#) ↗

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/27/2026

Azure Quotas for the Azure MCP Server overview

Azure MCP Server helps you manage Azure quotas efficiently by using natural language prompts. This article explains how to monitor quota usage, set alerts, and optimize resource allocation for your Azure resources.

[Azure Quotas](#) enables you to monitor and create alerts for specific quotas. You can receive notifications when the usage reaches predefined thresholds.

Region: Availability list

Given a list of Azure resource types, this tool returns a list of regions where the resource types are available. Always get the user's subscription ID before calling this tool.

Example prompts include:

- **Check region availability:** "Which regions support Microsoft.App/containerApps and Microsoft.Web/sites?"
- **Resource region list:** "Show me all regions where I can deploy Microsoft.CognitiveServices/accounts."
- **Find available regions for resources:** "List available regions for Microsoft.App/containerApps, Microsoft.Web/sites, and Microsoft.CognitiveServices/accounts."
- **Region support for cognitive services:** "Where can I deploy the 'gpt-4' model for Microsoft.CognitiveServices/accounts?"
- **Deployment options:** "What regions allow deployment of Microsoft.App/containerApps and Microsoft.Web/sites?"

 Expand table

Parameters	Required or optional	Description
Resource types to check	Required	Comma-separated list of Azure resource types to check available regions for. For example: <code>Microsoft.App/containerApps</code> , <code>Microsoft.Web/sites</code> , <code>Microsoft.CognitiveServices/accounts</code> .

Parameters	Required or optional	Description
Cognitive service model name	Optional	Model name for cognitive services. Only needed when <code>Microsoft.CognitiveServices</code> is included in resource types.
Cognitive service model version	Optional	Model version for cognitive services. Only needed when <code>Microsoft.CognitiveServices</code> is included in resource types.
Cognitive service deployment SKU name	Optional	Deployment SKU name for cognitive services. Only needed when <code>Microsoft.CognitiveServices</code> is included in resource types.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Usage: Check usage and quotas

This tool checks the usage and quota information for Azure resources in a region.

Example prompts include:

- **Check quota usage:** "Check my quota usage for Microsoft.App/containerApps in eastus."
- **Resource quota status:** "Show me the current quota and usage for Microsoft.Web/sites and Microsoft.CognitiveServices/accounts in westus."
- **Quota limits:** "What are the quota limits for Microsoft.App/containerApps in centralus?"
- **Usage report:** "Get a usage report for Microsoft.Compute/virtualMachines in region 'eastus2'."
- **Quota and usage details:** "Can you provide quota and usage details for Microsoft.App/containerApps, Microsoft.Web/sites in westeurope?"

[🔗 Expand table](#)

Parameters	Required or optional	Description
Region for deployment	Required	The Azure region where you want to check the usage and quota. For example: <code>eastus</code> , <code>westus</code> .
Resource types to deploy	Required	The Azure resource types that you want to check the usage and quota for (comma-separated). For example: <code>Microsoft.App/containerApps</code> , <code>Microsoft.Web/sites</code> , <code>Microsoft.CognitiveServices/accounts</code> .

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local
Required: ✖

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Quotas](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/03/2026

Azure RBAC tools for the Azure MCP Server overview

Azure RBAC tools in the Azure MCP Server allow you to manage Azure role-based access control using natural language prompts. This allows you to quickly view and manage role assignments without remembering complex syntax, streamlining your Azure access management workflow.

[Azure role-based access control \(Azure RBAC\)](#) is the authorization system used to manage access to Azure resources. The way you control access to resources using Azure RBAC is to assign Azure roles. This is a key concept to understand – it's how permissions are enforced. A role assignment consists of three elements: security principal, role definition, and scope.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

List role assignments

The Azure MCP Server can list Azure RBAC [role assignments](#) at a specific scope. This allows you to view who has access to what resources and what permissions they have.

Example prompts include:

- **List assignments:** "Show me all role assignments in my subscription."
- **View scope assignments:** "List role assignments for resource group 'myresourcegroup'"
- **Check access:** "What role assignments exist at the subscription scope?"
- **Query assignments:** "Show me all RBAC assignments for my Azure subscription"
- **Find assignments:** "List all role assignments in scope '/subscriptions/12345678-1234-1234-1234-123456789012'"
- **Resource group scope:** "Show role assignments for resource group 'production-rg' in my subscription"

- **Specific resource scope:** "List role assignments for storage account 'mystorageaccount' in resource group 'storage-rg'"
- **Virtual machine access:** "What role assignments exist for VM 'prod-vm01' in the production resource group?"
- **Database permissions:** "Show me who has access to SQL database 'proddb' in resource group 'database-rg'"

[Expand table](#)

Parameter	Required	Description
Scope	Required	The scope to list role assignments for. Can be a subscription, resource group, or resource.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure role-based access control \(Azure RBAC\)](#)

Last updated on 02/27/2026

Azure Redis tools for Azure MCP Server overview

The Azure MCP Server lets you manage Azure Redis instances using natural language prompts. You can create new Redis resources and list existing Redis resources without remembering complex syntax or commands.

[Azure Redis](#) provides an in-memory data store based on the Redis software. Redis improves the performance and scalability of applications that use backend data stores heavily. Redis processes large volumes of application requests by keeping frequently accessed data in server memory, which you can write to and read from quickly.

The Azure Redis tools support both [Azure Managed Redis](#) and [Azure Cache for Redis](#).

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Create a Redis resource

Create a new Azure Managed Redis resource in Azure. Use this command to provision a new Redis resource in your subscription.

Example prompts include:

- "Create a new Redis instance named 'my-redis' in resource group 'rg-backend' located in 'eastus'"
- "Set up a Redis cache called 'cache-prod' within resource group 'rg-production' at location 'westus2'"
- "I need to create Redis resource 'fastcache' in 'rg-apps' resource group with location 'centralus'"
- "Provision Redis named 'session-store' in resource group 'rg-sessions' located in 'northcentralus'"

- "Please create Redis resource 'redis-main' in resource group 'rg-main' at 'eastus2' location"

 Expand table

Parameter	Required or optional	Description
Resource	Required	The name of the Redis resource (for example, <code>my-redis</code>).
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
SKU	Optional	The SKU for the Redis resource. (Default: <code>Balanced_B0</code>).
Location	Required	The location for the Redis resource (for example <code>eastus</code>).
Access keys authentication	Optional	Whether to enable access keys for authentication for the Redis resource. (Default: <code>false</code>).
Modules	Optional	A list of modules to enable on the Azure Managed Redis resource (for example, <code>RedisBloom</code> , <code>RedisJSON</code>).

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

List Redis resources

Lists all Redis resources in a subscription. Returns details of all Azure Managed Redis, Azure Cache for Redis, and Azure Redis Enterprise resources. Use this command to explore and view which Redis resources are available in your subscription.

Example prompts include:

- "Show me all Redis caches available in my Azure subscription"
- "List every Redis instance I have under tenant 'contoso.com'"
- "Get details for Redis cache 'redisCacheWestUS' in subscription 'ProductionSub'"
- "Can you retrieve information about Redis instance 'sales-redis-cache'?"
- "I want to see the Redis cache named 'inventory-redis' in my tenant 'contoso.com'"

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Redis](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/27/2026

Azure MCP Server tools for Azure Resource Health

The Azure MCP Server lets you manage resource health, including checking availability status, viewing health events, and tracking service-impacting issues across your Azure resources, with natural language prompts.

Azure Resource Health provides information about the health of your individual Azure resources and helps you diagnose and mitigate issues; for more information, see [Azure Resource Health documentation](#).

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Get resource health availability status

This tool retrieves the Azure Resource Health availability status for a specific resource or for all resources in a subscription or resource group. It reports whether a resource is `Available`, `Unavailable`, `Degraded`, or `Unknown`, and includes the reason and details to help you investigate and troubleshoot. You can check the health of Azure resources such as virtual machines and storage accounts.

Example prompts include:

- "Get the availability status for resource 'vm-web-01'."
- "What is the Azure Resource Health availability status of the storage account 'mystorageacct'?"
- "What is the availability status of virtual machine 'app-server-01' in resource group 'rg-prod'?"
- "Get Azure Resource Health availability status for all resources in my subscription."
- "Show me the health status of all my Azure resources."
- "What resources in resource group 'rg-monitoring' have health issues?"

 Expand table

Parameter	Required or optional	Description
ResourceId	Optional	The Azure resource ID to get health status for such as <code>/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.Compute/virtualMachines/{</code>

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Get resource health events

List Azure Service Health events for your subscription to track incidents, planned maintenance, advisories, and security events over a specified time range (for example, `the last 30 days`). Query planned maintenance, past or ongoing incidents, advisories, and security events to retrieve details about resource availability, potential issues, and timestamps. The tool returns `trackingId`, `title`, `summary`, `eventType`, `status`, `startTime`, `endTime`, and `impactedServices`. Filter results by `Event type`, `Status`, `Tracking ID`, time range (`Query start time` and `Query end time`), or apply an OData `Filter` to narrow the results.

Example prompts include:

- "Show all service health events in my subscription."
- "Show Azure service health events for subscription `<subscription_id>`."
- "Which service issues occurred in the last 30 days?"
- "List active service health events in my subscription."
- "Show planned maintenance events for my Azure services."

 Expand table

Parameter	Required or optional	Description
Event type	Optional	Filter by event type (ServiceIssue, PlannedMaintenance, HealthAdvisory, Security). If not specified, all event types are included.
Filter	Optional	Additional OData filter expression to apply to the service health events query.
Query end time	Optional	End time for the query in ISO 8601 format (for example, <code>2024-01-31T23:59:59Z</code>). Events up to this time will be included.
Query start time	Optional	Start time for the query in ISO 8601 format (for example, <code>2024-01-01T00:00:00Z</code>). Events from this time onwards will be included.
Status	Optional	Filter by status (Active, Resolved). If not specified, all statuses are included.

Parameter	Required or optional	Description
Tracking ID	Optional	Filter by tracking ID to get a specific service health event.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Resource Health documentation](#)

Last updated on 04/15/2026

Azure Service Bus tools for the Azure MCP Server overview

The Azure MCP Server lets you to manage Azure Service Bus resources, including queues and topics with natural language prompts. You can peek at messages and view message details without specialized knowledge of messaging protocols.

[Azure Service Bus](#) is a fully managed enterprise message broker with message queues and publish-subscribe topics. Service Bus decouples applications and services from each other.

ⓘ **Note**

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Get queue runtime details

The Azure MCP Server can retrieve runtime details about a Service Bus queue, including its message count and status.

Example prompts include:

- **Details queue:** "Show me details about the 'orders' queue in my 'app-messaging' namespace."
- **Queue info:** "What's the status of queue 'notifications' in namespace 'messaging-hub'?"
- **Check queue:** "Get details for queue 'user-events' in namespace 'app-messaging'"
- **Queue status:** "Show me message count for queue 'orders' in namespace 'retail-messaging'"

 Expand table

Parameter	Required or optional	Description
Namespace	Required	The fully qualified Service Bus namespace host name.
Queue name	Required	The queue name.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local Required: ✖

Get topic runtime details

The Azure MCP Server can retrieve runtime details about a Service Bus topic, including its subscription count and status.

Example prompts include:

- **Details topic:** "Show me runtime details about the 'product-events' topic in my 'retail-messaging' namespace."
- **Topic info:** "What's the runtime status of topic 'system-updates' in namespace 'app-messaging'?"
- **Check topic:** "Get details for topic 'notifications' in namespace 'messaging-hub'"
- **Topic status:** "Show me subscription count for topic 'events' in namespace 'app-messaging'"
- **View topic:** "Tell me about topic 'broadcast-topic' in namespace 'retail-messaging'"

 Expand table

Parameter	Required or optional	Description
Namespace	Required	The fully qualified Service Bus namespace host name.
Topic name	Required	The name of the topic.

Tool annotation hints:

Destructive: ✖ | Idempotent: ✔ | Open World: ✖ | Read Only: ✔ | Secret: ✖ | Local Required: ✖

Get topic subscription runtime details

The Azure MCP Server can retrieve runtime details about a subscription within a Service Bus topic, including message counts.

Example prompts include:

- **Details subscription:** "Show me details about subscription 'mobile-app' in topic 'notifications' in namespace 'app-messaging'."
- **Subscription info:** "What's the status of subscription 'admin' in topic 'system-updates' in namespace 'messaging-hub'?"
- **Check subscription:** "Get message count for subscription 'premium-users' in topic 'offers' in namespace 'retail-messaging'"
- **Subscription status:** "Show me details for subscription 'email-service' in topic 'notifications' in namespace 'app-messaging'"
- **View subscription:** "Tell me about subscription 'analytics' in topic 'events' in namespace 'data-messaging'"

 Expand table

Parameter	Required or optional	Description
Namespace	Required	The fully qualified Service Bus namespace host name.
Topic name	Required	The name of the topic containing the subscription.
Topic subscription name	Required	The name of the topic subscription.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Service Bus](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 12/09/2025

Azure Service Fabric tools for the Azure MCP Server overview

The Azure MCP Server lets you manage microservices and containerized applications, including deployment, scaling, and monitoring, with natural language prompts.

Service Fabric is a distributed systems platform that makes it easy to package, deploy, and manage scalable and reliable microservices and containers. For more information, see [Azure Service Fabric documentation](#).

⚠ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Get managed cluster node details

Get nodes for a Service Fabric managed cluster. By default, this command returns all nodes. You can also specify a node name to retrieve details for a single node. The output includes the name, node type, status, IP address, fault domain, upgrade domain, health state, and seed node status.

Example prompts include:

- "Get all nodes for the managed cluster `myManagedCluster` in resource group `rg-prod`."
- "Show me the nodes for the Service Fabric managed cluster `testCluster` within resource group `rg-dev`."
- "What is the status of node `node1` in managed cluster `myCluster` located in resource group `rg-test`?"
- "I need details for the node named `node2` from the managed cluster `productionCluster` in resource group `rg-prod`."
- "List the nodes for the Service Fabric managed cluster `stagingCluster` within resource group `rg-staging`."

[Expand table](#)

Parameter	Required or optional	Description
Cluster	Required	Service Fabric managed cluster name.
Resource group	Required	The name of the Azure resource group, which is a logical container for Azure resources.
Node	Optional	The node name. When specified, this parameter returns a single node instead of all nodes.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Restart managed cluster nodes based on type

Restart nodes of a specific node type in a Service Fabric managed cluster.

Example prompts include:

- "Restart nodes `node1` and `node2` of node type `frontend` in managed cluster `myservicefabric` within resource group `rg-prod`."
- "I need to restart the nodes `node3` and `node4` of the `backend` node type in the `mycluster` managed cluster located in resource group `rg-dev`."
- "Can you restart all nodes of type `worker` in managed cluster `mycluster` that are located in resource group `rg-staging`?"
- "Restart nodes `node5` and `node6` of node type `database` in managed cluster `clustername` under resource group `rg-production`?"
- "Restart the nodes `node7` and `node8` of `app` node type in managed cluster `servicefabriccluster` under resource group `rg-test` and use update type `ByUpgradeDomain`."

[Expand table](#)

Parameter	Required or optional	Description
Cluster	Required	Service Fabric managed cluster name.
Node type	Required	The node type name within the managed cluster.

Parameter	Required or optional	Description
Nodes	Required	The list of node names to restart. You can provide multiple node names.
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Update type	Optional	The update type for the restart operation. Update types include: <code>Default</code> or <code>ByUpgradeDomain</code> .

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Service Fabric documentation](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/26/2026

Azure SignalR tools for the Azure MCP Server overview

The Azure MCP Server lets you manage Azure resources, including Azure SignalR resources, using natural language prompts. This feature lets you quickly manage your SignalR resources without remembering complex syntax.

[Azure SignalR](#) is a fully managed real-time messaging service that enables you to build and integrate real-time communication into your applications.

⚠ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Runtime: Get or list runtime information

Gets or lists details of an Azure SignalR runtimes. If a specific SignalR name is used, the details of that SignalR runtime will be retrieved. Otherwise, all SignalR runtimes in the specified subscription or resource group will be retrieved. Returns runtime information including identity, network ACLs, upstream templates.

Example prompts include:

- **Get specific SignalR details:** "Show me the details of SignalR 'my-signalr-service'"
- **Network information:** "Show me the network information of SignalR runtime 'chat-signalr'"
- **Resource group specific:** "Describe the SignalR runtime 'realtime-hub' in resource group 'messaging-rg'"
- **Detailed runtime info:** "Get information about my SignalR runtime 'notification-service' in 'production-rg'"
- **List by resource group:** "Show all the SignalRs information in 'communication-rg'"
- **Subscription-wide listing:** "List all SignalRs in my subscription"

- **Production environment:** "Show details of SignalR runtime 'prod-signalr-001' in resource group 'prod-messaging'"
- **Development setup:** "Get runtime information for SignalR 'dev-chat-service' in 'development-rg'"
- **Gaming application:** "Show me the details of SignalR 'game-hub' used for real-time gaming"
- **IoT messaging:** "Display network ACLs and upstream templates for SignalR 'iot-signalr' in 'iot-resources'"

 Expand table

Parameter	Required or optional	Description
Signalr	Optional	The name of the SignalR runtime.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure SignalR](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/27/2026

Azure MCP Server tools for Azure SQL Database

The Azure MCP Server lets you manage Azure SQL Database resources, including creating, deleting, updating, and listing databases, with natural language prompts.

Azure SQL Database is a relational database service in the Microsoft Azure cloud that provides high availability, scalability, and security. For more information, see [Azure SQL Database documentation](#).

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Create SQL Database

Create a new Azure SQL Database on an existing SQL Server. Create a database with configurable performance tiers, size limits, and other settings. It returns the newly created database information, including configuration details.

Example prompts include:

- "Create a SQL database named 'my-database' with SKU tier Premium in server 'my-sql-server'."
- "Create a new SQL database called 'products-db' in resource group 'my-resource-group' on server 'my-sql-server'."
- "Create a SQL database 'reports-db' with a maximum size of 2GB in server 'my-sql-server'."

[⌵](#) Expand table

Parameter	Required or optional	Description
Database name	Required	The Azure SQL Database name.
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Server name	Required	The Azure SQL Server name.
Collation	Optional	The collation for the database (for example, <code>SQL_Latin1_General_CP1_CI_AS</code>).
Elastic pool name	Optional	The name of the elastic pool to assign the database to.
Max size bytes	Optional	The maximum size of the database in bytes.
Read scale	Optional	Read scale option for the database (Enabled or Disabled).
SKU capacity	Optional	The SKU capacity (DTU or vCore count) for the database.
SKU name	Optional	The SKU name for the database (for example, <code>Basic</code> , <code>S0</code> , <code>P1</code> , <code>GP_Gen5_2</code>).
SKU tier	Optional	The SKU tier for the database (for example, <code>Basic</code> , <code>Standard</code> , <code>Premium</code> , <code>GeneralPurpose</code>).
Zone redundant	Optional	Indicates whether the database should be zone redundant.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Create SQL Server

Create a new Azure SQL server in the specified resource group and location. The server is configured with the provided administrator credentials and optional settings. The command returns the created server along with its properties, including the fully qualified domain name.

Example prompts include:

- "Create an Azure SQL server named 'my-sql-server' in location 'eastus' with admin login 'sqladmin'."

- "Set up a new SQL server called 'prod-sql-server' in resource group 'my-resource-group' with your administrator password."
- "Create a SQL server with name 'dev-sql-server' in resource group 'dev-resource-group' located in 'westus2'."

[Expand table](#)

Parameter	Required or optional	Description
Administrator login	Required	The administrator login name for the SQL server.
Administrator password	Required	The administrator password for the SQL server.
Location	Required	The Azure region where the SQL server will be created.
Resource group	Required	The name of the Azure resource group, which is a logical container for Azure resources.
Server name	Required	The Azure SQL Server name.
Public network access	Optional	Whether public network access is enabled for the SQL server (<code>Enabled</code> or <code>Disabled</code>).
Version	Optional	The version of SQL Server to create (currently only <code>12.0</code> is supported).

Tool annotation hints:

Destructive: ☒ | Idempotent: ☐ | Open World: ☐ | Read Only: ☐ | Secret: ☐ | Local Required: ☐

Create SQL Server firewall rule

Creates a firewall rule for an Azure SQL Server. Firewall rules control which IP addresses are allowed to connect to the SQL Server. You can specify either a single IP address (by setting the start and end IP to the same value) or a range of IP addresses. This command returns the created firewall rule with its properties.

Example prompts include:

- "Create a firewall rule named 'allow-office-ip' for SQL Server 'my-sql-server' in resource group 'my-resource-group'."

- "Add a firewall rule for SQL Server 'my-sql-server' allowing IP range '203.0.113.0' to '203.0.113.255'."
- "Create a new firewall rule for Azure SQL Server 'prod-sql-server' with IP limits from '198.51.100.0' to '198.51.100.255'."

[Expand table](#)

Parameter	Required or optional	Description
End IP address	Required	The end IP address of the firewall rule range.
Firewall rule name	Required	The name of the firewall rule.
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Server name	Required	The Azure SQL Server name.
Start IP address	Required	The start IP address of the firewall rule range.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Delete SQL Database

Deletes a database from an Azure SQL Server. This idempotent operation removes the specified database from the server, returning `Deleted = false` if the database doesn't exist or `Deleted = true` if it was successfully removed.

Example prompts include:

- "Delete the SQL database 'my-database' from server 'my-sql-server'."
- "Remove the database 'old-database' from resource group 'my-resource-group' on server 'my-sql-server'."
- "Delete the database 'test-database' from SQL server 'dev-sql-server'."

[Expand table](#)

Parameter	Required or optional	Description
Database name	Required	The Azure SQL Database name.
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Server name	Required	The Azure SQL Server name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Delete SQL Server

Remove the specified Azure SQL server from your Azure subscription, including all associated databases. This operation permanently deletes all server data and cannot be reversed. Use `force` to bypass confirmation.

Example prompts include:

- "Delete SQL server 'my-sql-server' in resource group 'my-resource-group'."
- "Remove the Azure SQL server 'old-sql-server' from my resource group."
- "Permanently delete SQL server 'test-sql-server' without confirmation."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Server name	Required	The Azure SQL Server name.
Force	Optional	Force delete the server without confirmation prompts.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Delete SQL Server firewall rule

Delete a firewall rule from an Azure SQL Server. This operation removes the specified firewall rule, which may restrict access for the IP addresses that were previously allowed by this rule. The operation is idempotent; if the rule does not exist, no error is returned.

Example prompts include:

- "Delete the firewall rule 'allow-office-ip' from resource group 'my-resource-group' in SQL server 'my-sql-server'."
- "Remove firewall rule 'temp-access-rule' for SQL server 'my-sql-server' in resource group 'my-resource-group'."
- "Delete firewall rule 'old-firewall-rule' from my SQL server 'dev-sql-server'."

 Expand table

Parameter	Required or optional	Description
Firewall rule name	Required	The name of the firewall rule.
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Server name	Required	The Azure SQL Server name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Get Azure SQL Database details

Retrieve information about Azure SQL databases in a SQL Server. You can show details for a specific Azure SQL database by name or list all Azure SQL databases within the specified SQL Server. This tool provides database information, including configuration details and current status.

Example prompts include:

- "List all databases in resource group 'my-rg' for server 'my-server'."

- "Get details for the Azure SQL database 'my-database' in resource group 'my-rg' and server 'my-server'."
- "Show all Azure SQL databases in resource group 'my-rg' within server 'my-server'."
- "Retrieve the Azure SQL database 'my-database' from resource group 'my-rg' in server 'my-server'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Server name	Required	The Azure SQL Server name.
Database name	Optional	The Azure SQL Database name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Get elastic pool list

Lists all SQL elastic pools in an Azure SQL Server, including their SKU, capacity, state, and database limits. You can view the elastic pool inventory, check pool utilization, compare pool configurations, or find available pools for database placement. The tool returns a JSON array of elastic pools with complete configuration details.

Example prompts include:

- "List all elastic pools in resource group 'my-resource-group' for SQL server 'my-sql-server'."
- "Show me the elastic pools in resource group 'prod-resource-group' for SQL server 'prod-sql-server'."
- "What elastic pools exist in my SQL server 'dev-sql-server' under resource group 'dev-resource-group'?"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Server name	Required	The Azure SQL Server name.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Get firewall rule list

Retrieve a list of firewall rules for an Azure SQL Server. This command retrieves all firewall rules configured for the specified SQL server, including their IP address ranges and rule names. It returns an array of firewall rule objects with their properties.

Example prompts include:

- "List all firewall rules in resource group 'my-resource-group' for SQL server 'my-sql-server'."
- "Show me the firewall rules in resource group 'prod-resource-group' for SQL server 'prod-sql-server'."
- "What firewall rules are set for SQL server 'dev-sql-server' in resource group 'dev-resource-group'?"

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Server name	Required	The name of the Azure SQL Server.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Get SQL server information

Retrieve details about Azure SQL servers in a resource group. Display information for a specific Azure SQL server by name or list all Azure SQL servers within the specified resource group. It returns comprehensive server information, including configuration details and the current state.

Example prompts include:

- "List all Azure SQL servers in resource group 'my-resource-group'."
- "Show me every Azure SQL server in resource group 'prod-resource-group'."
- "Show me the details of Azure SQL server 'my-sql-server'."
- "Get information for Azure SQL server 'prod-sql-server'."
- "Display the properties of Azure SQL server 'dev-sql-server'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Server name	Optional	The Azure SQL server name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

List Microsoft Entra ID administrators

List the Microsoft Entra ID administrators configured for a SQL server. This command retrieves all Entra ID administrators, including their display names, object IDs, and tenant information.

Example prompts include:

- "List Microsoft Entra ID administrators for SQL server 'prod-sql-server' in resource group 'prod-resource-group'."
- "Show me the Entra ID administrators configured for SQL server 'dev-sql-server' in resource group 'dev-resource-group'."
- "What Microsoft Entra ID administrators are set up for my SQL server 'analytics-sql-server' in resource group 'data-resource-group'?"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group.
Server name	Required	The Azure SQL Server name (for example, <code>prod-sql-server</code>).

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Rename SQL Database

Renames an existing Azure SQL Database to a new name within the same SQL Server. This command changes the database resource's identifier while preserving its configuration and data. It returns the updated database information with the new name.

Example prompts include:

- "Rename the database 'my-database' on server 'my-sql-server' to 'my-database-v2' in resource group 'my-resource-group'."
- "Rename my SQL database 'old-database' to 'new-database' on server 'prod-sql-server'."

[Expand table](#)

Parameter	Required or optional	Description
Database name	Required	The Azure SQL Database name.
New database name	Required	The new name for the Azure SQL Database.
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Server name	Required	The Azure SQL Server name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Update SQL Database

Scale and configure Azure SQL Database performance settings. Update an existing database's SKU, compute tier, storage capacity, or redundancy options to meet changing performance requirements. This command returns the updated database configuration, including applied scaling changes.

Example prompts include:

- "Change the collation of SQL database 'my-database' on server 'my-sql-server' in resource group 'my-resource-group'."
- "Update SQL database 'my-database' on server 'my-sql-server' to have a maximum size of 2GB."

 Expand table

Parameter	Required or optional	Description
Database name	Required	The Azure SQL Database name.
Resource group	Required	The name of the Azure resource group. This is a logical container for Azure resources.
Server name	Required	The Azure SQL Server name.
Collation	Optional	The collation for the database (for example, <code>SQL_Latin1_General_CP1_CI_AS</code>).
Elastic pool name	Optional	The name of the elastic pool to assign the database to.
Max size bytes	Optional	The maximum size of the database in bytes.
Read scale	Optional	Read scale option for the database (Enabled or Disabled).
SKU capacity	Optional	The SKU capacity (DTU or vCore count) for the database.
SKU name	Optional	The SKU name for the database (for example, <code>Basic</code> , <code>S0</code> , <code>P1</code> , <code>GP_Gen5_2</code>).
SKU tier	Optional	The SKU tier for the database (for example, <code>Basic</code> , <code>Standard</code> , <code>Premium</code> , <code>GeneralPurpose</code>).
Zone redundant	Optional	Whether the database should be zone redundant.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure SQL Database documentation](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 04/10/2026

Azure Storage tools for the Azure MCP Server overview

The Azure MCP Server lets you manage Azure Storage resources, including storage accounts, containers, blobs, and tables with natural language prompts.

[Azure Storage](#) is Microsoft's cloud storage solution for modern data storage scenarios.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Account: Create

Create a new Azure Storage account.

Prerequisites: The conversation context establishes [global parameters](#) (subscription, resource group). Caller must have Storage Account Contributor role or equivalent permissions on the target subscription.

Example prompts include:

- **Create storage account:** "Create a storage account named 'mystorageaccount' in resource group 'my-resource-group' in location 'eastus'."
- **With SKU:** "Create a storage account 'mydata' in resource group 'my-resource-group' in location 'eastus' with Standard_GRS."
- **Enable Data Lake:** "Create a storage account 'datalakeacct' in resource group 'my-resource-group' in location 'westeurope' with hierarchical namespace enabled."

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. The resource group is a logical container for Azure resources.
Storage account	Required	The globally unique name of the Azure Storage account (3-24 characters, lowercase letters, and numbers only).
Location	Required	The Azure region where Azure creates the storage account (for example, <code>eastus</code> , <code>westus2</code>).
SKU	Optional	The storage account SKU. Valid values: <code>Standard_LRS</code> , <code>Standard_GRS</code> , <code>Standard_RAGRS</code> , <code>Standard_ZRS</code> , <code>Premium_LRS</code> , <code>Premium_ZRS</code> , <code>Standard_GZRS</code> , <code>Standard_RAGZRS</code> .
Access tier	Optional	The default access tier for blob storage. Valid values: <code>Hot</code> , <code>Cool</code> .
Enable hierarchical namespace	Optional	Whether to enable hierarchical namespace (Data Lake Storage Gen2) for the storage account.

Success verification: The tool returns the created storage account details.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Account: Get details

Retrieves detailed information about Azure Storage accounts, including account name, location, SKU, kind, hierarchical namespace status, HTTPS-only settings, and blob public access configuration. If you don't provide a specific account name, the tool returns details for all accounts in the subscription.

Prerequisites: The conversation context establishes [global parameters](#) (subscription, authentication). Caller must have Storage Account Reader role or equivalent permissions.

Example prompts include:

- **Get storage account details:** "Show me details for the storage account 'mystorageaccount'."
- **List all accounts:** "What storage accounts are in my subscription?"
- **Check properties:** "What are the settings for storage account 'mydata'?"

 Expand table

Parameter	Required or optional	Description
Storage account	Optional	The globally unique name of the Azure Storage account (for example, 'mystorageaccount').

Success verification: Returns JSON with storage account properties or a list of all accounts if you don't specify a name.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Container: Create container

Create a blob container with optional blob public access.

Prerequisites: The conversation context establishes [global parameters](#) (subscription, authentication). Caller must have Storage Blob Data Contributor role or equivalent on the storage account.

Example prompts include:

- **Create container:** "Create a private container named 'mycontainer' in storage account 'mystorageaccount'."
- **With access level:** "Create a blob container named 'logs' in storage account 'mydata' with access level 'private'."

 Expand table

Parameter	Required or optional	Description
Storage account	Required	The globally unique name of the Azure Storage account (for example, 'mystorageaccount').
Container	Required	The name of the container to create within the storage account.

Success verification: The tool returns the created container properties.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Container: Get container details

List all blob containers in a storage account or show details for a specific container. Displays container properties including access policies, lease status, and metadata.

Prerequisites: The conversation context establishes [global parameters](#) (subscription, authentication). Caller must have Storage Blob Data Reader role or equivalent on the storage account.

Example prompts include:

- **Get container details:** "Show me details about the 'documents' container in storage account 'mystorageaccount'."
- **List containers:** "What containers are in storage account 'media_files'?"
- **Check access policy:** "Check access policy for container 'user_data' in storage account 'prodstore'."

 Expand table

Parameter	Required or optional	Description
Storage account	Required	The globally unique name of the Azure Storage account.
Container	Optional	The name of the container. If you don't specify a name, the tool lists all containers in the storage account.

Success verification: Returns JSON with container properties or a list of all containers if you don't specify a name.

[Tool annotation hints:](#)

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Blob: Get blob details

List blobs in a container or get details for a specific blob. Shows blob properties including metadata, size, last modification time, and content properties.

Prerequisites: The conversation context establishes [global parameters](#) (subscription, authentication). Caller must have Storage Blob Data Reader role or equivalent on the storage account.

Example prompts include:

- **Get blob details:** "Show me details for 'file.txt' in container 'documents' in storage account 'mystorageaccount'."
- **List blobs:** "What blobs are in container 'photos' in storage account 'mediafiles'?"
- **Blob metadata:** "What is the metadata for 'backup.zip' in container 'backups' in storage account 'mydata'?"

 [Expand table](#)

Parameter	Required or optional	Description
Storage account	Required	The globally unique name of the Azure Storage account (for example, 'mystorageaccount').
Container	Required	The name of the container within the storage account.
Blob	Optional	The name of the blob within the container, including the full path (for example, <code>file.txt</code> or <code>folder/file.txt</code>). If you don't specify a name, the tool lists all blobs in the container.

Success verification: Returns JSON with blob properties or a list of all blobs if you don't specify a name.

[Tool annotation hints:](#)

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Blob: Upload

Uploads a local file to a blob in Azure Storage if the blob doesn't exist.

Prerequisites: The conversation context establishes [global parameters](#) (subscription, authentication). Caller must have Storage Blob Data Contributor role or equivalent on the storage account. Local file must exist and be accessible.

Example prompts include:

- **Upload file:** "Upload local file 'report.pdf' to blob 'documents/report.pdf' in container 'documents' in storage account 'mystorageaccount'."
- **Upload and overwrite:** "Upload local file 'data.csv' to blob 'archive/data.csv' in container 'archive' in storage account 'mydata', overwriting if it exists'."

 Expand table

Parameter	Required or optional	Description
Storage account	Required	The globally unique name of the Azure Storage account (for example, <code>mystorageaccount</code>).
Container	Required	The name of the container within the storage account.
Blob	Required	The name of the blob within the container, including the full path (for example, <code>file.txt</code> or <code>folder/file.txt</code>).
Local file path	Required	The full path to the local file on your system.

Success verification: The tool returns the last modified time, ETag, and content hash of the uploaded blob.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Table: List

List all tables in an Azure Storage account.

Prerequisites: The conversation context establishes [global parameters](#) (subscription, authentication). Caller must have Storage Account Reader role or equivalent permissions.

Example prompts include:

- "Show me all tables in storage account 'dataarchives'"
- "List every table in storage account 'storagesample01'"
- "Get details for table 'UserLogs' in storage account 'appstorage01'"
- "I need to see the table named 'InventoryRecords' in storage account 'warehouseacct'"
- "Can you provide the tables inside storage account 'mystorageaccount'"

Parameter	Required or optional	Description
Storage account	Required	The name of the Azure Storage account. This is the unique name you chose for your storage account (for example, <code>mystorageaccount</code>).

Success verification: The tool returns the list of tables in the specified storage account.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Storage](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 01/27/2026

Azure Virtual Desktop tools for the Azure MCP Server overview

The Azure MCP Server enables you to manage Azure Virtual Desktop resources by using natural language prompts. You can list host pools, view session hosts, monitor user sessions, and manage your virtual desktop infrastructure without needing to remember complex command syntax.

[Azure Virtual Desktop](#) is a desktop and app virtualization service that runs on the cloud. It provides a full Windows experience with optimized Office 365 ProPlus, and supports Remote Desktop Services environments.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Conditional parameters

Some of the Azure Virtual Desktop tools require **one** of the following parameter options within the conversation context:

- **Option 1:** Host pool name
- **Option 2:** Host pool resource ID

Don't provide both parameters (host pool name and host pool resource ID) together, because this combination creates conflicting inputs. When you provide a host pool resource ID, the tool uses it instead of searching by name.

Host pools: List host pools

The Azure MCP Server can list all host pools in a subscription or resource group. This feature provides an overview of your virtual desktop infrastructure and helps you manage your desktop deployment.

Example prompts include:

- **List all host pools:** "Show me all host pools in my subscription."
- **View host pools:** "What host pools do I have available?"
- **Find host pools:** "List all virtual desktop host pools."
- **Query host pools:** "Show available host pools in my environment."
- **Check infrastructure:** "Get all Azure Virtual Desktop host pools."

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Host pools: List session hosts in a host pool

The Azure MCP Server can list all session hosts in a host pool. This functionality helps you monitor your virtual machines and understand the capacity and status of your virtual desktop environment.

Example prompts include:

- **List session hosts:** "Show me all session hosts in the 'production-hostpool' host pool."
- **View VMs:** "What session hosts are in my host pool?"
- **Find hosts:** "List all virtual machines in hostpool 'dev-environment'."
- **Query capacity:** "Show session hosts in my virtual desktop pool."
- **Check hosts:** "Get all session hosts for host pool ID
'/subscriptions/abc123/resourceGroups/rg/providers/Microsoft.DesktopVirtualization/hostPools/pool1'."

 Expand table

Parameter	Required or optional	Description
Host pool	Conditionally required	The name of the Azure Virtual Desktop host pool. This is the unique name you choose for your host pool.
Host pool resource ID	Conditionally required	The Azure resource ID of the host pool. When you provide this ID, the server uses it instead of searching by name.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Host pools: List user sessions

The Azure MCP Server can list all user sessions on a specific session host in a host pool. This capability helps you monitor active users, troubleshoot connection issues, and manage user workloads.

Example prompts include:

- **List user sessions:** "Show me all user sessions on session host 'vm-prod-001' in host pool 'production-pool'."
- **View active users:** "What users are connected to session host 'desktop-vm-02' in host pool 'dev-pool'?"
- **Find sessions:** "List all active sessions on session host 'avd-host-001' in host pool 'test-pool'."
- **Monitor users:** "Show user sessions for session host 'vm-host-05' in host pool 'production-pool'."
- **Check connections:** "Get all user sessions on session host 'session-host-03' in host pool 'main-pool'."

 Expand table

Parameter	Required or optional	Description
Host pool	Conditionally required	The name of the Azure Virtual Desktop host pool. This is the unique name you choose for your host pool.
Host pool resource ID	Conditionally required	The Azure resource ID of the host pool. When you provide this ID, the server uses it instead of searching by name.
Session host	Required	The name of the session host. This name is the computer name of the virtual machine in the host pool.

Tool annotation hints:

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local
Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Virtual Desktop documentation](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 12/06/2025

Azure MCP Server tools for Azure Compute overview

The Azure MCP Server tools help you manage virtual machines, virtual machine scale sets, and disks by using natural language prompts. By using key capabilities such as creating, retrieving, and updating resources, you can efficiently control your cloud environment.

Azure Compute provides scalable computing resources for applications and workloads. For more information, see [Azure Compute documentation](#).

① Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

Managed disk: create

Creates a new Azure managed disk in the specified resource group. You can create empty disks (specify `size-gb`), disks from a source such as a snapshot, another managed disk, or a blob URI (specify `source`), disks from a Shared Image Gallery image version (specify `gallery-image-reference`), or disks ready for upload (specify `upload-type` and `upload-size-bytes`). If you don't specify the location, it defaults to the resource group's location. You can configure disk size, storage SKU (for example, `Premium_LRS`, `Standard_LRS`, `UltraSSD_LRS`), OS type, availability zone, hypervisor generation, tags, encryption settings, performance tier, shared disk, on-demand bursting, and IOPS/throughput limits for UltraSSD disks. Create a disk with network access policy `DenyAll`, `AllowAll`, or `AllowPrivate`, and associate a disk access resource during creation.

Example prompts include:

- "Create a 128 GB managed disk named `<disk-name>` in resource group `<resource-group>`"
- "Create a new `Premium_LRS` disk called `<disk-name>` in resource group `<resource-group>` with 256 GB"
- "Create a managed disk `<disk-name>` in resource group `<resource-group>` in `eastus`"
- "Create a disk from snapshot `<snapshot-resource-id>` in resource group `<resource-group>`"
- "Create a managed disk `<disk-name>` in resource group `<resource-group>` from blob `<blob-uri>`"
- "Create a 64 GB `Standard_LRS` Linux disk named `<disk-name>` in resource group `<resource-group>` in zone 1"
- "Create a managed disk `<disk-name>` in resource group `<resource-group>` with tags `env=prod team=infra`"
- "Create a 128 GB `Premium_LRS` disk named `<disk-name>` in resource group `<resource-group>` with performance tier `P30`"
- "Create a disk `<disk-name>` in resource group `<resource-group>` with customer-managed encryption using disk encryption set `<disk-encryption-set-id>`"

- "Create a managed disk from gallery image version `<image-version-resource-id>` in resource group `<resource-group>` "
- "Create a data disk from LUN 0 of gallery image version `<image-version-resource-id>` in resource group `<resource-group>` "
- "Create a disk ready for upload named `<disk-name>` in resource group `<resource-group>` with upload size 20972032 bytes"
- "Create a Trusted Launch upload disk named `<disk-name>` in resource group `<resource-group>` with `UploadWithSecurityData` type and security type `TrustedLaunch` "
- "Create an `UltraSSD_LRS` disk named `<disk-name>` in resource group `<resource-group>` with 256 GB, 10000 IOPS, and 500 MBps throughput"
- "Create a shared managed disk named `<disk-name>` in resource group `<resource-group>` with 512 GB and max shares set to 3"
- "Create a managed disk `<disk-name>` in resource group `<resource-group>` with network access policy `DenyAll` and disk access `<disk-access-resource-id>` "
- "Create a 128 GB managed disk named `<disk-name>` in resource group `<resource-group>` with on-demand bursting enabled"
- "Create a managed disk `<disk-name>` in resource group `<resource-group>` with encryption type `EncryptionAtRestWithPlatformAndCustomerKeys` "
- "Create a V2 hypervisor generation disk named `<disk-name>` in resource group `<resource-group>` with 128 GB"

[Expand table](#)

Parameter	Required or optional	Description
Disk name	Required	The name of the disk.
Resource group	Required	The name of the Azure resource group. This name is a logical container for Azure resources.
Disk access	Optional	Resource ID of the disk access resource for using private endpoints on disks.
Disk encryption set	Optional	Resource ID of the disk encryption set to use for enabling encryption at rest.
Disk iops read write	Optional	The number of IOPS allowed for this disk. Only settable for UltraSSD disks.
Disk mbps read write	Optional	The bandwidth allowed for this disk in MBps. Only settable for UltraSSD disks.
Enable bursting	Optional	Enable on-demand bursting beyond the provisioned performance target of the disk. Doesn't apply to Ultra values: <code>true</code> , <code>false</code> .
Encryption type	Optional	Encryption type of the disk. Accepted values: <code>EncryptionAtRestWithCustomerKey</code> , <code>EncryptionAtRestWithPlatformAndCustomerKeys</code> , <code>EncryptionAtRestWithPlatformKey</code> .
Gallery image reference	Optional	Resource ID of a Shared Image Gallery image version to use as the source for the disk. Format: <code>/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.Compute/galleries/{gallery}/images/{image}</code>
Gallery image reference lun	Optional	LUN (Logical Unit Number) of the data disk in the gallery image version. If specified, the disk is created from this LUN. If not specified, the disk is created from the OS disk of the image.
Hyper v generation	Optional	The hypervisor generation of the Virtual Machine. Applicable to OS disks only. Accepted values: <code>v1</code> , <code>v2</code> .

Parameter	Required or optional	Description
Location	Optional	The Azure region/location. Defaults to the resource group's location if not specified.
Max shares	Optional	The maximum number of VMs that can attach to the disk at the same time. A value greater than one indicates
Network access policy	Optional	Policy for accessing the disk via network. Accepted values: <code>AllowAll</code> , <code>AllowPrivate</code> , <code>DenyAll</code> .
Os type	Optional	The Operating System type of the disk. Accepted values: <code>Linux</code> , <code>Windows</code> .
Security type	Optional	Security type of the managed disk. Accepted values: <code>ConfidentialVM_DiskEncryptedWithCustomerKey</code> , <code>ConfidentialVM_DiskEncryptedWithPlatformKey</code> , <code>ConfidentialVM_VMGuestStateOnlyEncryptedWithPlatformKey</code> , <code>TrustedLaunch</code> . Required when <code>upload-type</code> is <code>UploadWithSecurityData</code> .
Size gb	Optional	Size of the disk in GB. Max size: 4095 GB.
SKU	Optional	Underlying storage SKU. Accepted values: <code>Premium_LRS</code> , <code>PremiumV2_LRS</code> , <code>Premium_ZRS</code> , <code>StandardSSD_LRS</code> , <code>Standard_LRS</code> , <code>UltraSSD_LRS</code> .
Source	Optional	Source to create the disk from, including a resource ID of a snapshot or disk, or a blob URI of a VHD. When provided, <code>size-gb</code> is optional and defaults to the source size.
Tags	Optional	Space-separated tags in 'key=value' format. Use "" to clear existing tags.
Tier	Optional	Performance tier of the disk (for example, <code>P10</code> , <code>P15</code> , <code>P20</code> , <code>P30</code> , <code>P40</code> , <code>P50</code> , <code>P60</code> , <code>P70</code> , <code>P80</code>). Applicable to Premium only.
Upload size bytes	Optional	The size in bytes (including the VHD footer of 512 bytes) of the content to be uploaded. Required when <code>upload-type</code> is <code>Upload</code> .
Upload type	Optional	Type of upload for the disk. Accepted values: <code>Upload</code> , <code>UploadWithSecurityData</code> . When specified, the disk is in <code>ReadyToUpload</code> state.
Zone	Optional	Availability zone into which to provision the resource.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Managed disk: delete

Delete an Azure managed disk from the specified resource group. This operation is idempotent - it returns success whether the disk was removed or didn't exist.

Example prompts include:

- "Delete the managed disk 'temp-data-disk' in resource group 'dev-rg'."
- "Remove managed disk 'old-backup-disk' from resource group 'prod-rg'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group.
Disk name	Required	The name of the disk to delete.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Managed disk: list or get

Lists available Azure managed disks or retrieves detailed information about a specific disk. You can view all disks in a subscription or in a specific resource group, including disk size, SKU, provisioning state, and OS type. The tool supports wildcard patterns in disk names (for example, `win_0sDisk*`). If you provide a disk name without specifying a resource group, it searches across the entire subscription. Specifying a resource group scopes the search to that resource group. Both parameters are optional.

Example prompts include:

- "List all managed disks in my subscription."
- "Show me all disks in resource group `<resource-group>`."
- "Get details of disk `<disk-name>`."
- "What are the available disk sizes?"
- "Show me the disks with name pattern `win_0sDisk*` in resource group `<resource-group>`."
- "Get information about disk `<disk-name>` in resource group `<resource-group>`."

 Expand table

Parameter	Required or optional	Description
Disk name	Optional	The name of the disk.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Managed disk: update

Update or modify properties of an existing Azure managed disk that you previously created. If you don't specify the resource group, the disk is located by name within the subscription. This operation supports changing disk size (only increases are allowed), storage SKU, IOPS and throughput limits (for UltraSSD only), maximum shares for shared disk attachments, on-demand bursting, tags, encryption settings, disk access, and performance tier. You can modify the network access policy to `DenyAll`, `AllowAll`, or `AllowPrivate` on an existing disk. Only specified properties are updated; unspecified properties remain unchanged.

Example prompts include:

- "Update disk `<disk-name>` in resource group `<resource-group>` to 1024 GB"
- "Change the SKU of disk `<disk-name>` to `UltraSSD_LRS`"
- "Resize disk `<disk-name>` in resource group `<resource-group>` to 2048 GB"
- "Update disk `<disk-name>` to disable bursting"
- "Set the max shares on disk `<disk-name>` to 3"
- "Change the network access policy of disk `<disk-name>` to `AllowPrivate`"
- "Update disk `<disk-name>` in resource group `<resource-group>` with tags `env=production`"
- "Set the IOPS limit on Ultra disk `<disk-name>` in resource group `<resource-group>` to 15000"

- "Update the throughput of disk `<disk-name>` in resource group `<resource-group>` to 1000 MBps"
- "Change the performance tier of disk `<disk-name>` in resource group `<resource-group>` to `P50`"
- "Update disk `<disk-name>` in resource group `<resource-group>` to use disk encryption set `<disk-encryption-set-id>`"
- "Change the encryption type of disk `<disk-name>` in resource group `<resource-group>` to `EncryptionAtRestWithCustomerKey`"
- "Set disk access on disk `<disk-name>` in resource group `<resource-group>` to `<disk-access-resource-id>` with network access policy `DenyAll`"
- "Update disk `<disk-name>` to `PremiumV2_LRS` SKU with 256 GB size and tags `env=test`"

[Expand table](#)

Parameter	Required or optional	Description
Disk name	Required	The name of the disk.
Disk access	Optional	Resource ID of the disk access resource for using private endpoints on disks.
Disk encryption set	Optional	Resource ID of the disk encryption set to use for enabling encryption at rest.
Disk iops read write	Optional	The number of IOPS allowed for this disk. Only settable for UltraSSD disks.
Disk mbps read write	Optional	The bandwidth allowed for this disk in MBps. Only settable for UltraSSD disks.
Enable bursting	Optional	Enable on-demand bursting beyond the provisioned performance target of the disk. Doesn't apply to Ultra disks. Accepted values: <code>true</code> , <code>false</code> .
Encryption type	Optional	Encryption type of the disk. Accepted values: <code>EncryptionAtRestWithCustomerKey</code> , <code>EncryptionAtRestWithPlatformAndCustomerKeys</code> , <code>EncryptionAtRestWithPlatformKey</code> .
Max shares	Optional	The maximum number of VMs that can attach to the disk at the same time. Value greater than one indicates a shared disk.
Network access policy	Optional	Policy for accessing the disk via network. Accepted values: <code>AllowAll</code> , <code>AllowPrivate</code> , <code>DenyAll</code> .
Size gb	Optional	Size of the disk in GB. Max size: 4095 GB.
SKU	Optional	Underlying storage SKU. Accepted values: <code>Premium_LRS</code> , <code>PremiumV2_LRS</code> , <code>Premium_ZRS</code> , <code>StandardSSD_LRS</code> , <code>StandardSSD_ZRS</code> , <code>Standard_LRS</code> , <code>UltraSSD_LRS</code> .
Tags	Optional	Space-separated tags in <code>key=value</code> format. Use <code>''</code> to clear existing tags.
Tier	Optional	Performance tier of the disk (for example, <code>P10</code> , <code>P15</code> , <code>P20</code> , <code>P30</code> , <code>P40</code> , <code>P50</code> , <code>P60</code> , <code>P70</code> , <code>P80</code>). Applicable to Premium SSD disks only.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Virtual machine: create

Create, deploy, or provision a single Azure Virtual Machine (VM). This command launches a new Linux or Windows VM with either SSH key or password authentication. It automatically creates networking resources (VNet, subnet, NSG, NIC, public IP) if you don't specify them. The default VM size is `Standard_DS1_v2`, and the default OS is Ubuntu 24.04 LTS if you don't specify otherwise.

You can create a Linux VM using an SSH public key by providing the key content or the path to the key file. For example, you can specify your public key file at `~/.ssh/id_rsa.pub`.

This command doesn't support creating Virtual Machine Scale Sets with multiple identical instances. Instead, use `VMSS create`.

Example prompts include:

- "Create a new Linux VM named `<vm-name>` with SSH key in resource group `<resource-group>`"
- "Launch a virtual machine with the Ubuntu2404 image in `<resource-group>`"
- "Create a Windows VM named `<vm-name>` with an admin password in resource group `<resource-group>`"
- "Deploy VM `<vm-name>` in `<location>` with `Standard_DS1_v2` size"
- "Spin up a VM with `Standard_B2s` size and no public IP in resource group `<resource-group>`"
- "Create a Linux VM named `<vm-name>` in `<location>` with a custom network security group"

[Expand table](#)

Parameter	Required or optional	Description
Admin username	Required	The admin username for the VM. Required for VM creation.
Location	Required	The Azure region or location. Defaults to the resource group's location if you don't specify it.
Resource group	Required	The name of the Azure resource group. This name is a logical container for Azure resources.
VM name	Required	The name of the virtual machine.
Admin password	Optional	The admin password for Windows VMs or when the SSH key isn't provided for Linux VMs.
Image	Optional	The OS image to use. Can be URN (publisher:offer:SKU:version) or an alias like <code>Ubuntu2404</code> or <code>Win2022Datacenter</code> . Defaults to Ubuntu 24.04 LTS.
Network security group	Optional	Name of the network security group to use or create.
No public IP	Optional	Don't create or assign a public IP address.
OS disk size GB	Optional	OS disk size in GB. Defaults based on image requirements.
OS disk type	Optional	OS disk type: <code>Premium_LRS</code> , <code>StandardSSD_LRS</code> , <code>Standard_LRS</code> . Defaults based on VM size.
OS type	Optional	The Operating System type of the disk. Accepted values: <code>Linux</code> , <code>Windows</code> .
Public IP address	Optional	Name of the public IP address to use or create.

Parameter	Required or optional	Description
Source address prefix	Optional	Source IP address range for NSG inbound rules (for example, <code>203.0.113.0/24</code> or a specific IP). Defaults to <code>*</code> (any source).
SSH public key	Optional	SSH public key for Linux VMs. Can be the key content or path to a file.
Subnet	Optional	Name of the subnet within the virtual network.
Virtual network	Optional	Name of an existing virtual network to use. If you don't specify it, the command creates a new one.
VM size	Optional	The VM size (for example, <code>Standard_D2s_v3</code> or <code>Standard_B2s</code>). Defaults to <code>Standard_DS1_v2</code> if you don't specify it.
Zone	Optional	Availability zone into which to provision the resource.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Virtual machine: delete

Delete an Azure virtual machine permanently. This operation is irreversible and the VM data is lost. Use the `Force deletion` parameter to force-delete a VM that is in a running or failed state.

Example prompts include:

- "Delete VM 'test-vm-01' in resource group 'dev-rg'."
- "Remove virtual machine 'staging-web' from resource group 'staging-rg'."
- "Force delete VM 'stuck-vm' in resource group 'prod-rg'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group.
VM name	Required	The name of the virtual machine to delete.
Force deletion	Optional	Force delete the resource even if it's in a running or failed state.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Virtual machine: list or get

List or get Azure Virtual Machines (VMs) in a subscription or resource group. This command returns VM details, including the name, location, size, provisioning state, OS type, and instance view with runtime status and power state.

Example prompts include:

- "List all virtual machines in my subscription."

- "Show me all VMs in my subscription."
- "What virtual machines do I have?"
- "List virtual machines in resource group `resource-group-name`."
- "Show me VMs in resource group `resource-group-name`."
- "What VMs are in resource group `resource-group-name`?"
- "Get details for virtual machine `vm-name` in resource group `resource-group-name`."
- "Show me virtual machine `vm-name` in resource group `resource-group-name`."
- "What are the details of VM `vm-name` in resource group `resource-group-name`?"
- "Get virtual machine `vm-name` with instance view in resource group `resource-group-name`."
- "Show me VM `vm-name` with runtime status in resource group `resource-group-name`."
- "What is the power state of virtual machine `vm-name` in resource group `resource-group-name`?"
- "Get VM `vm-name` status and provisioning state in resource group `resource-group-name`."
- "Show me the current status of VM `vm-name`."

[Expand table](#)

Parameter	Required or optional	Description
Instance view	Optional	Include instance view details (only available when retrieving a specific VM).
VM name	Optional	The name of the virtual machine.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Virtual machine: update

Update, modify, or reconfigure an existing Azure virtual machine (VM). You can resize a VM, update tags, configure boot diagnostics, or change user data. You might need to deallocate the VM before resizing it to certain sizes.

Here are some example prompts for using this tool:

- "Add license type `Windows_Server` to VM `<vm-name>` in resource group `<resource-group-name>`"
- "Update user data for VM `<vm-name>` in resource group `<resource-group-name>`"
- "Resize VM `<vm-name>` in resource group `<resource-group-name>` to `Standard_B2s`"
- "Enable boot diagnostics for VM `<vm-name>` in resource group `<resource-group-name>`"

[Expand table](#)

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This name is a logical container for Azure resources.
VM name	Required	The name of the virtual machine.
Boot diagnostics	Optional	Enable or disable boot diagnostics: <code>true</code> or <code>false</code> .

Parameter	Required or optional	Description
License type	Optional	License type for Azure Hybrid Benefit: <code>Windows_Server</code> , <code>Windows_Client</code> , <code>RHEL_BYOS</code> , <code>SLES_BYOS</code> , or <code>None</code> to disable.
Tags	Optional	Space-separated tags in <code>key=value</code> format. Use <code>''</code> to clear existing tags.
User data	Optional	Base64-encoded user data for the VM. Use to update custom data scripts.
VM size	Optional	The VM size (for example, <code>Standard_D2s_v3</code> , <code>Standard_B2s</code>). Defaults to <code>Standard_DS1_v2</code> if not specified.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Virtual machine scale set: create

Create, deploy, or provision an Azure Virtual Machine Scale Set (VMSS) for running multiple identical VM instances. This tool helps you deploy workloads that require horizontal scaling, load balancing, or high availability across instances. The default configuration creates two instances of size `Standard_DS1_v2` running Ubuntu 24.04 LTS.

Create a scale set by specifying the `resource_group`, `VMSS name`, and `admin username`, along with other optional settings. Here are some example commands:

- "Create a virtual machine scale set named `my-vmss` in resource group `my-rg`."
- "Create a VMSS with four instances in `my-rg`."
- "Deploy a scale set with a Manual upgrade policy and two instances in `my-rg`."
- "Create a Linux VMSS with SSH public key from `'~/ssh/id_rsa.pub'` in `my-rg`."

 Expand table

Parameter	Required or optional	Description
Admin username	Required	The admin username for the VM. Required for VM creation.
Location	Required	The Azure region or location. Defaults to the resource group's location if you don't specify it.
Resource group	Required	The name of the Azure resource group. This name is a logical container for Azure resources.
Virtual machine scale set (VMSS) name	Required	The name of the virtual machine scale set.
Admin password	Optional	The admin password for Windows VMs or when an SSH key isn't provided for Linux VMs.
Image	Optional	The OS image to use. Can be a URN (publisher:offer:SKU:version) or alias like <code>Ubuntu2404</code> , <code>Win2022Datacenter</code> . Defaults to Ubuntu 24.04 LTS.
Instance count	Optional	Number of VM instances in the scale set. Default is 2.
Os disk size gb	Optional	OS disk size in GB. Defaults based on image requirements.

Parameter	Required or optional	Description
Os disk type	Optional	OS disk type: <code>Premium_LRS</code> , <code>StandardSSD_LRS</code> , <code>Standard_LRS</code> . Defaults based on VM size.
Os type	Optional	The Operating System type of the disk. Accepted values: Linux, Windows.
Ssh public key	Optional	SSH public key for Linux VMs. Can be the key content or path to a file.
Subnet	Optional	Name of the subnet within the virtual network.
Upgrade policy	Optional	Upgrade policy mode: <code>Automatic</code> , <code>Manual</code> , or <code>Rolling</code> . Default is <code>Manual</code> .
Virtual network	Optional	Name of an existing virtual network to use. If you don't specify it, the tool creates a new one.
VM size	Optional	The VM size (for example, <code>Standard_D2s_v3</code> , <code>Standard_B2s</code>). Defaults to <code>Standard_DS1_v2</code> if not specified.
Zone	Optional	Availability zone into which to provision the resource.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Virtual machine scale set: delete

Delete an Azure Virtual Machine Scale Set and all its VM instances permanently. This operation is irreversible. Use the `Force deletion` parameter to force-delete a scale set that is in a running or failed state.

Example prompts include:

- "Delete scale set 'web-frontend-vmss' in resource group 'prod-rg'."
- "Remove VMSS 'test-scaleset' from resource group 'dev-rg'."
- "Force delete virtual machine scale set 'stuck-vmss' in resource group 'staging-rg'."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group.
VMSS name	Required	The name of the virtual machine scale set to delete.
Force deletion	Optional	Force delete the resource even if it's in a running or failed state.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Virtual machine scale set: list or get

List or get Azure Virtual Machine Scale Sets (VMSS) and their instances in a subscription or resource group. This tool returns scale set details, including name, location, SKU, capacity, upgrade policy, and individual VM instance information.

Example prompts include:

- "List all virtual machine scale sets in my subscription."
- "List virtual machine scale sets in resource group `<resource-group-name>`."
- "What scale sets are in resource group `<resource-group-name>`?"
- "Get details for virtual machine scale set `<vmss-name>` in resource group `<resource-group-name>`."
- "Show me VMSS `<vmss-name>` in resource group `<resource-group-name>`."
- "Show me instance `<instance-id>` of VMSS `<vmss-name>` in resource group `<resource-group-name>`."
- "What is the status of instance `<instance-id>` in scale set `<vmss-name>`?"

 Expand table

Parameter	Required or optional	Description
Instance ID	Optional	The instance ID of the virtual machine in the scale set.
Virtual machine scale set (VMSS) name	Optional	The name of the virtual machine scale set.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Virtual machine scale set: update

Update, modify, or reconfigure an existing Azure Virtual Machine Scale Set (VMSS). You can scale the instance count, resize VMs, change the upgrade policy, or update tags on a scale set. Some changes require `update-instances` to roll out to existing VMs. This tool doesn't create a new VMSS. Use `VMSS create` instead. To update a single VM, use `VM update`.

Example prompts include:

- "Update the capacity of VMSS `myScaleSet` to 15."
- "Enable overprovisioning on the scale set `myScaleSet`."
- "Change the VM size to `Standard_D4s_v3` for `myScaleSet`."
- "Clear existing tags on scale set `myScaleSet` in resource group `myResourceGroup`."

 Expand table

Parameter	Required or optional	Description
Resource group	Required	The name of the Azure resource group. This name is a logical container for Azure resources.
Virtual machine scale set (VMSS) name	Required	The name of the virtual machine scale set.
Capacity	Optional	Number of VM instances (capacity) in the scale set.

Parameter	Required or optional	Description
Enable auto os upgrade	Optional	Enable automatic OS image upgrades. Requires health probes or the Application Health extension.
Overprovision	Optional	Enable or disable overprovisioning. When enabled, Azure provisions more VMs than requested and deletes extra VMs after deployment.
Scale in policy	Optional	Scale-in policy to determine which VMs to remove: <code>Default</code> , <code>NewestVM</code> , or <code>OldestVM</code> .
Tags	Optional	Space-separated tags in <code>key=value</code> format. Use <code>''</code> to clear existing tags.
Upgrade policy	Optional	Upgrade policy mode: <code>Automatic</code> , <code>Manual</code> , or <code>Rolling</code> . Default is <code>Manual</code> .
VM size	Optional	The VM size (for example, <code>Standard_D2s_v3</code> , <code>Standard_B2s</code>). Defaults to <code>Standard_DS1_v2</code> if not specified.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local
Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Virtual Machines documentation](#)

Last updated on 04/13/2026

Azure MCP Server tools for Azure Resource Group

The Azure MCP Server lets you manage resource groups and the resources they contain, including: listing resource groups in a subscription and listing resources within a group, with natural language prompts.

Azure Resource Group is the Azure service for organizing and managing related resources as a single lifecycle and access boundary. For more information, see [Azure Resource Group documentation](#).

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

List resource groups

List all resource groups in a subscription. The tool returns resource group names and IDs as a JSON array, which you can use to inventory or audit resources and to drive follow-up automation.

Example prompts include:

- "List resource groups in my subscription."
- "Show my resource groups."
- "Display resource groups in my subscription."

[Tool annotation hints](#):

Destructive: ✗ | Idempotent: ✔ | Open World: ✗ | Read Only: ✔ | Secret: ✗ | Local Required: ✗

List group resources

List all resources in a resource group. The tool retrieves each resource's name, ID, type, and location from Azure Resource Manager and returns a JSON object with a `resources` array containing those fields.

Example prompts include:

- "List all resources in resource group 'my-rg'."
- "Show me what resources are in resource group 'webapp-dev'."
- "What resources exist in resource group 'rg-production'?"

 Expand table

Parameter	Required or optional	Description
Resource group name	Required	The name of the Azure resource group. A resource group is a logical container for Azure resources.

Tool annotation hints:

Destructive:  | Idempotent:  | Open World:  | Read Only:  | Secret:  | Local Required: 

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Resource Group documentation](#)

 **Note:** This article contains content created with AI. [Learn more](#)

Last updated on 03/27/2026

Azure MCP Server tools for Azure Subscriptions

The Azure Model Context Protocol (MCP) Server lets you list and identify Azure subscriptions with natural language prompts.

[Azure Subscriptions](#) provide a way to organize and manage access to Azure resources. Subscriptions are the foundation for resource management, billing, and access control in Azure.

ⓘ Note

Tool parameters: The Azure MCP Server tools define parameters for data they need to complete tasks. Some of these parameters are specific to each tool and are documented below. Other parameters are global and shared by all tools. For more information, see [Tool parameters](#).

List subscriptions

This tool lists all Azure subscriptions for the current account. For each subscription, it returns the subscription ID, display name, state, tenant ID, and whether it's the default subscription. The `isDefault` field indicates the user's default subscription as resolved from the Azure CLI profile (configured via [az account set](#)) or, if not set there, from the `AZURE_SUBSCRIPTION_ID` environment variable.

Example prompts include:

- "Show me all of my subscriptions."
- "List all subscriptions starting with 'northeast'."
- "Which subscription is my default?"

[Tool annotation hints](#):

Destructive: ❌ | Idempotent: ✅ | Open World: ❌ | Read Only: ✅ | Secret: ❌ | Local Required: ❌

Related content

- [What are the Azure MCP Server tools?](#)
- [Get started using Azure MCP Server](#)
- [Azure Subscriptions](#)

ⓘ **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 03/27/2026

What is the Azure MCP Server?

The Azure MCP Server enables AI agents and clients to interact with Azure resources using natural language commands. This article explains its features, benefits, and how it helps developers build and deploy apps to Azure.

It implements the [Model Context Protocol \(MCP\)](#) and supports a wide range of tools, languages, and frameworks to help you build and deploy apps to Azure.

Key features

- **MCP support:** The Azure MCP Server implements the Model Context Protocol, making it compatible with MCP clients such as GitHub Copilot agent mode, the OpenAI Agents SDK, and Semantic Kernel.
- **Entra ID authentication:** The server uses Entra ID through the Azure Identity library, following Azure authentication best practices.
- **Service and tool integration:** The server supports Azure services and tools, including the Azure CLI, Azure Developer CLI (azd), and a broad set of Azure resources.

Supported code editors and tools

You can connect to the Azure MCP Server from popular code editors and tools, including:

- [Visual Studio Code](#)
- [Visual Studio](#)
- [Eclipse](#)
- [Cursor](#)
- [Windsurf](#)
- [IntelliJ](#)
- [Cline](#)

Supported languages and frameworks

The Azure MCP Server supports multiple languages and frameworks, such as:

- [Python](#)
- [.NET](#)

Concepts

The [Model Context Protocol \(MCP\)](#) is an open protocol designed to manage how language models interact with external tools, memory, and context in a safe, structured, and stateful way. MCP defines a client-server architecture with several components:

- **Hosts:** Apps that use MCP clients to connect to and consume data from MCP servers.
- **Clients:** Components of MCP hosts that manage connections and retrieve data from MCP servers.
- **Servers:** Programs that provide features like data resources, tools for performing actions, and prompts to guide interactions.

For example, **Visual Studio Code** is considered a host, and GitHub Copilot agent mode in **Visual Studio Code** acts as an MCP client that connects to MCP servers. You can also build custom intelligent apps that host their own MCP client to connect to MCP servers.

The Azure MCP Server implements a set of [tools](#) per the Model Context Protocol. AI agents and other types of clients use these tools to interact with Azure resources.

How-to guides

You can find step-by-step instructions for common tasks, including:

- [Connect GitHub Copilot coding agent to Azure MCP Server](#)
- [Deploy a self-hosted Azure MCP Server \(Microsoft Foundry\)](#)
- [Deploy a self-hosted Azure MCP Server \(Copilot Studio\)](#)

Tools and best practices

The Azure MCP Server offers a wide range of tools for Azure development. For best practices and tool reference, see [Tools overview](#).

Scenarios for using the Azure MCP Server

The most common scenario is connecting to the Azure MCP Server from an existing client, such as GitHub Copilot agent mode in **Visual Studio Code** or a custom intelligent app. The client can use all available [tools](#) to access and interact with Azure resources using natural language. For example, you can use GitHub Copilot agent mode with the Azure MCP Server to list Azure storage accounts or run KQL queries on Azure databases. To learn how to connect to Azure MCP Server from an existing client, see [Get started using the Azure MCP Server](#).

In advanced scenarios, you might create your own MCP servers to offer custom tools, resources, and prompts for specific tasks involving Azure resources. If you're building an MCP server that needs to connect with Azure, you can use the Azure MCP Server tools from your MCP server.

The MCP server uses your Azure user credentials or managed identity to ensure authorized access. Access is secured through Azure Role-Based Access Control (RBAC), providing fine-grained permissions for approved users. The local MCP server is intended strictly for developer use within your organization. Don't use these tools for external applications or scenarios outside the approved development environment.

Related content

- [Get started using the Azure MCP Server](#)
- [Azure MCP Server tools](#)
- [Model Context Protocol documentation](#) 
- [Azure MCP Server repository](#) 

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 02/19/2026

What are the Azure MCP Server tools?

The Azure Model Context Protocol (MCP) Server exposes many tools you can use from an existing `client` to interact with Azure services through natural language prompts. For example, you can use the Azure MCP Server to interact with Azure resources conversationally from GitHub Copilot agent mode in Visual Studio Code or other AI agents with commands like these:

- "Show me all my resource groups"
- "List blobs in my storage container named 'documents'"
- "What's the value of the 'ConnectionString' key in my app configuration?"
- "Query my log analytics workspace for errors in the last hour"
- "Show me all my Cosmos DB databases"

Developer credentials and security

The MCP server uses your Azure user credentials or managed identity to ensure authorized access. Access is secured through Azure Role-Based Access Control (RBAC), providing fine-grained permissions for approved users. The local MCP server is intended strictly for developer use within your organization. Don't use these tools for external applications or scenarios outside the approved development environment.

Azure MCP Server start parameters

The `azmcp` server supports the following options for server start parameters:

 Expand table

Option	Required or optional	Description
Debug	Optional	Enable debug mode with verbose logging to <code>stderr</code> . Default: <code>false</code> .
Enable insecure transports	Optional	Enable insecure transport. Default: <code>false</code> .

Option	Required or optional	Description
Disable user confirmation (Not recommended)	Optional	Disable user confirmation (elicitation) before allowing high risk commands to run, such as returning secrets (passwords) from KeyVault. When enabled, tools that handle secrets, credentials, or sensitive data will execute without user confirmation. This removes an important security layer designed to prevent unauthorized access to sensitive information. Only use this option in trusted, automated environments where user interaction isn't possible. Never use this option in production environments or when handling untrusted input. Default: <code>false</code> .
Mode	Optional	Server mode: <code>namespace</code> (default), <code>consolidated</code> , <code>all</code> , or <code>single</code> . Default: <code>namespace</code> .
Namespace	Optional	The Azure service namespaces to expose on the MCP server (for example, <code>storage</code> , <code>keyvault</code> , <code>cosmos</code>). Default: all namespaces.
Read only	Optional	Whether the MCP server should be read-only. If true, no write operations are allowed. Default: <code>false</code> .
Tool	Optional	Expose specific tools by name (for example, <code>azmcp_storage_account_get</code>). It automatically switches to <code>all</code> mode. Default: all tools.
Transport	Optional	Transport mechanism to use for Azure MCP Server. Default: <code>stdio</code> .

Tool parameters

The Azure MCP Server tools define parameters for the data they need to complete tasks. For example, these parameters might include the subscription ID, an account name, or a resource group.

You might include the data for these parameters in the prompt you use to call a tool, or the previous conversation context might establish the data. If the conversation context provides the data, the Azure MCP Server can use that information without requiring you to repeat it in every prompt. This context creates a more natural conversational experience while still ensuring all necessary data is available for the tools.

The tools reference articles document the parameters specific to each tool. All of the tools also share the following global parameters.

Parameter	Description
Subscription	Azure subscription ID or name for target resources. This parameter identifies the Azure subscription that contains the resources you want to manage. You can use either the subscription GUID or the display name. If not specified, the Azure MCP Server resolves the default subscription from your Azure CLI profile (set via az account set) or the <code>AZURE_SUBSCRIPTION_ID</code> environment variable. Required for most operations unless a default is configured.
Resource group	The name of the Azure resource group. This is a logical container for Azure resources that helps organize and manage related resources together. Required for most resource-specific operations.
Tenant Id	Azure tenant ID for authentication. This parameter specifies the Microsoft Entra ID tenant to authenticate against. Can be either the GUID identifier or the display name of your Entra ID tenant. Optional - uses default tenant if not specified.
Authentication method	Authentication method to use for Azure operations. Options include <code>credential</code> (Azure CLI/managed identity), <code>key</code> (access key), or <code>connectionString</code> . Default is <code>credential</code> , which uses Azure CLI authentication or managed identity.
Maximum retries	Maximum number of retry attempts for failed operations before giving up. Controls how many times the system attempts to retry a failed request. Default is 3 retries.
Retry delay	Initial delay in seconds between retry attempts. For exponential backoff, this value is used as the base delay that gets multiplied on each retry. Default is 2 seconds.
Retry delay maximum	Maximum delay in seconds between retries, regardless of the retry strategy. This parameter caps the delay time to prevent excessively long waits. Default is 10 seconds.
Retry mode	Retry strategy to use when operations fail. <code>fixed</code> uses consistent delays between retries, while <code>exponential</code> increases the delay between each attempt. Default is <code>exponential</code> for better handling of temporary issues.
Retry network timeout	Network operation timeout in seconds. When operations take longer than this timeout, they are canceled and might be retried if retries are enabled. Default is 100 seconds.

Example prompts include:

- **Set subscription:** "Use subscription 'my-subscription-id' for all operations"
- **Use tenant ID:** "Authenticate using tenant ID 'my-tenant-id'"
- **Set authentication method:** "Use 'credential' authentication for this session"
- **Configure retries:** "Set maximum retries to 5 with a 3-second delay"
- **Set retry mode:** "Use 'fixed' retry mode with a maximum delay of 5 seconds"
- **Set network timeout:** "Set network timeout to 120 seconds for all operations"
- **Configure retry parameters:** "Use exponential retry mode with a maximum of 4 retries and a delay of 2 seconds"

Tool annotations for Azure MCP Server

Tool annotations [↗](#) are hints that provide additional information about the characteristics of each tool. The following table describes the possible hints that can be associated with a tool.

[↗](#) Expand table

Name	Description
Destructive	Indicates whether the tool can make destructive updates to its environment. If true, the tool might delete or modify existing resources. If false, the tool only adds new resources without removing or altering existing ones.
Idempotent	Specifies if repeated calls with the same arguments produce the same result without side effects. If true, multiple executions with the same arguments produce the same result. If false, repeated executions might have additional effects or yield different results.
Open world	Defines whether the tool might interact with an "open world" of external entities. If true, the tool might interact with an unpredictable or dynamic set of entities (for example, web search). If false, the tool's domain of interaction is closed and well-defined (for example, memory access).
Read only	Indicates if the tool performs only read operations. If true, there's no change of state of the environment. If false, the tool might make modifications to its environment.
Secret	Microsoft proprietary annotation that indicates if the tool's response might contain sensitive data requiring sanitization. If true, the response might include secrets, credentials, or keys that should be sanitized before forwarding to the LLM or logging. If false, the response doesn't contain sensitive information.
Local required	Microsoft proprietary annotation to indicate if the tool requires local execution or resources. If true, the tool is only available when the Azure MCP server runs in Local (STDIO) mode. If false, the tool is available in both local and remote server modes.

User confirmation for sensitive data

Tools that handle sensitive data, such as secrets, require user consent before execution through a security mechanism called **elicitation**. When you use tools that access sensitive information, the MCP client prompts you to confirm the operation before proceeding.

Elicitation (user confirmation) Security Feature:

Elicitation prompts appear when tools might expose sensitive information like:

- Key Vault secrets
- Connection strings and passwords

- Certificate private keys
- Other confidential data

These prompts protect against unauthorized access to sensitive information. You can bypass elicitation only in automated scenarios.

Available tools

Azure MCP Server provides the following tools for Azure services and Azure-related functionality.

Best practices

 Expand table

Product, tool, or service	Namespace	Description
Azure best practices	<code>get azure bestpractices get</code>	Get guidance on Azure Functions development, deployment, and Azure SDK usage.
Azure AI best practices for app development	<code>get azure bestpractices ai app</code>	Get best practices and code generation guidance for building AI applications in Azure. Use it when you need recommendations on writing code for AI agents, chatbots, workflows, or any AI/LLM features. Additionally, this command provides guidance for code generation on Microsoft Foundry for application development.
Terraform best practices for Azure	<code>azureterraformbestpractices</code>	Get guidance on implementing Terraform for Azure resources.

AI and Machine Learning

 Expand table

Product, tool, or service	Namespace	Description
Microsoft Foundry	<code>foundry</code>	Work with Foundry models, deployments, and endpoints.
Azure AI Search	<code>search</code>	Manage Azure AI Search resources, including search services, indexes, and queries.

Product, tool, or service	Namespace	Description
Azure Speech in Foundry Tools	speech	Manage Speech resources such as speech-to-text and text-to-speech services.

Analytics

[Expand table](#)

Product, tool, or service	Namespace	Description
Azure App Lens	applens	Diagnose and analyze application performance issues using Azure App Lens.
Azure Data Explorer	kusto	Work with Azure Data Explorer clusters, databases, tables, and queries.
Azure Event Hubs	eventhubs	Manage Azure Event Hubs namespaces and event hubs.

Compute

[Expand table](#)

Product, tool, or service	Namespace	Description
Azure App Service	appservice	Manage web apps, database connections, diagnostics, deployments, and application settings for Azure App Service.
Azure Functions	functionapp	List Azure Functions.
Azure Kubernetes Service	aks	List Azure Kubernetes Service clusters.
Azure Service Fabric	servicefabric	Manage Service Fabric managed clusters, including node details and restarts.
Azure Compute	compute	Get details for Azure virtual machines, virtual machine scale sets, and managed disks.

Containers

[Expand table](#)

Product, tool, or service	Namespace	Description
Azure Container Registry	acr	List Azure Container Registry instances.
Azure Functions	functionapp	List Azure Functions.
Azure Kubernetes Service	aks	List Azure Kubernetes Service clusters.
Azure Service Fabric	servicefabric	Manage Service Fabric managed clusters, including node details and restarts.

Databases

 Expand table

Product, tool, or service	Namespace	Description
Azure Cosmos DB	cosmos	Work with Azure Cosmos DB accounts, databases, containers, and documents.
Azure Database for MySQL	mysql	Manage Azure Database for MySQL servers, databases, and tables.
Azure Database for PostgreSQL	postgres	Manage Azure Database for PostgreSQL servers, databases, and tables.
Azure Redis	redis	Create and list Azure Redis resources, including Managed Redis and Cache for Redis.
Azure SQL	sql	Work with Azure SQL Database servers, databases, firewall rules, and elastic pools.

Developer tools

 Expand table

Product, tool, or service	Namespace	Description
Azure App Configuration	appconfig	Manage centralized application settings and feature flags.
Azure Application Insights	applicationinsights	List Application Insights resources.

Product, tool, or service	Namespace	Description
Azure CLI	extension	Find Azure CLI commands and get installation instructions for CLI tools.
Azure Developer CLI	extension	Get installation instructions for Azure Developer CLI (azd), learn usage for deployment scenarios, and access azd MCP tools when azd is installed locally
Azure Load Testing	loadtesting	Create, run, and see load testing.

DevOps

 Expand table

Product, tool, or service	Namespace	Description
Azure Bicep schema	bicepschema	Retrieve Bicep schemas for Azure resources to use in Infrastructure as Code templates.
Azure Deploy	deploy	Deploy and manage Azure resources using templates and scripts.
Azure Developer CLI	extension	Get installation instructions for Azure Developer CLI (azd), learn usage for deployment scenarios, and access azd MCP tools when azd is installed locally
Azure Managed Grafana	grafana	List Azure Managed Grafana workspaces.
Azure Monitor	monitor	Query Azure Monitor logs and metrics.
Azure Workbooks	workbooks	Create, manage, and update Azure Workbooks for data visualization and reporting.

Hybrid and multicloud

 Expand table

Product, tool, or service	Namespace	Description
Azure Database for PostgreSQL	postgres	Manage Azure Database for PostgreSQL servers, databases, and tables.
Azure SQL	sql	Work with Azure SQL Database servers, databases, firewall rules, and elastic pools.

Identity

 Expand table

Product, tool, or service	Namespace	Description
Azure RBAC	role	View and manage Azure role-based access control assignments.

Integration

 Expand table

Product, tool, or service	Namespace	Description
Azure Event Grid	eventgrid	Manage Azure Event Grid resources, including topics and subscriptions.
Azure Native ISV	datadog	Work with Azure Native ISV services, including Datadog integration for monitoring and observability.
Azure Service Bus	servicebus	Manage Azure Service Bus resources, including queues, topics, and peek at messages.

Internet of Things (IoT)

 Expand table

Product, tool, or service	Namespace	Description
Azure Cosmos DB	cosmos	Work with Azure Cosmos DB accounts, databases, containers, and documents.
Azure Event Grid	eventgrid	Manage Azure Event Grid resources, including topics and subscriptions.
Azure Functions	functionapp	List Azure Functions.

Management and governance

 Expand table

Product, tool, or service	Namespace	Description
Azure Advisor	<code>advisor</code>	Get recommendations to optimize Azure resources and improve performance.
Azure Cloud Architect	<code>cloudarchitect</code>	Design cloud systems by gathering requirements through guided questions and recommending optimal solutions.
Azure Managed Grafana	<code>grafana</code>	List Azure Managed Grafana workspaces.
Azure Migrate	<code>azuremigrate</code>	Get step-by-step guidance for changing a Platform Landing Zone and make or download platform landing zone setups for Azure Migrate projects.
Azure Monitor	<code>monitor</code>	Query Azure Monitor logs and metrics.
Azure Policy	<code>policy</code>	Manage Azure Policy assignments, definitions, and initiatives.
Azure Pricing	<code>pricing</code>	Get Azure retail pricing, cost estimates, and billing details for services, SKUs, and regions.
Azure Quick Review CLI	<code>extension</code>	Generate compliance and security reports for Azure resources.
Azure Quotas	<code>quota</code>	Manage Azure resource quotas and limits.
Azure Resource Health	<code>resourcehealth</code>	Check availability status, view health events, and track service-impacting issues for Azure resources.
Azure Well-Architected Framework	<code>wellarchitectedframework</code>	Get architectural best practices, design patterns, and recommendations for Azure services based on the five Well-Architected Framework pillars.
Azure Workbooks	<code>workbooks</code>	Create, manage, and update Azure Workbooks for data visualization and reporting.
Resource Groups	<code>group</code>	List Azure resource groups and list resources within a group.
Subscription	<code>subscription</code>	List Azure subscriptions.

Messaging

 Expand table

Product, tool, or service	Namespace	Description
Azure Event Grid	eventgrid	Manage Azure Event Grid resources, including topics and subscriptions.
Azure Event Hubs	eventhubs	Manage Azure Event Hubs namespaces and event hubs.
Azure Service Bus	servicebus	Manage Azure Service Bus resources, including queues, topics, and peek at messages.

Mobile

 Expand table

Product, tool, or service	Namespace	Description
Foundry	foundry	Work with Foundry models, deployments, and endpoints.
Azure AI Search	search	Manage Azure AI Search resources, including search services, indexes, and queries.
Azure App Configuration	appconfig	Manage centralized application settings and feature flags.
Azure Communication Services	communication	Send SMS and email messages using Azure Communication Services.

Security

 Expand table

Product, tool, or service	Namespace	Description
Azure App Configuration	appconfig	Manage centralized application settings and feature flags.
Azure Confidential Ledger	confidentialledger	Manage Azure Confidential Ledger resources, including ledgers and transactions.
Azure Key Vault	keyvault	List and create keys, secrets, certificates in Azure Key Vault.

Storage

 Expand table

Product, tool, or service	Namespace	Description
Azure Confidential Ledger	confidentialledger	Manage Azure Confidential Ledger resources, including ledgers and transactions.
Azure File Sync	storagesync	Manage Azure file sync services, register servers, create sync groups, configure cloud endpoints, and set up server endpoints for Azure File Sync.
Azure Files	fileshares	Manage Azure file shares, including creating, updating, deleting, and snapshotting file shares.
Azure Managed Lustre	managedlustre	Manage Azure Managed Lustre file systems, including auto-import and auto-export jobs.
Azure Storage	storage	Manage Azure Storage accounts, containers, blobs, and tables.
Azure Storage	storage	List Azure Storage accounts, containers, blobs, and tables.

Virtual desktop infrastructure (VDI)

 Expand table

Product, tool, or service	Namespace	Description
Azure Virtual Desktop	virtualdesktop	Manage Azure Virtual Desktop host pools, session hosts, and user sessions.

Web

 Expand table

Product, tool, or service	Namespace	Description
Azure AI Search	search	Manage Azure AI Search resources, including search services, indexes, and queries.
Azure App Configuration	appconfig	Manage centralized application settings and feature flags.
Azure App Service	appservice	Manage web apps, database connections, diagnostics, deployments, and application settings for Azure App Service.

Product, tool, or service	Namespace	Description
Azure Communication Services	communication	Send SMS and email messages using Azure Communication Services.
Azure Functions	functionapp	List Azure Functions.
Azure SignalR	signalr	Manage Azure SignalR resources and runtimes.

Other

 Expand table

Product, tool, or service	Namespace	Description
Azure Marketplace	marketplace	Discover Azure Marketplace products and offers.
Azure MCP tool	No namespace	Discover and manage available Azure MCP Server tools.

Related content

- [What is the Azure MCP Server?](#)
- [Get started using Azure MCP Server](#)
- [Azure MCP Server repository](#) 

 **Note:** The author created this article with assistance from AI. [Learn more](#)

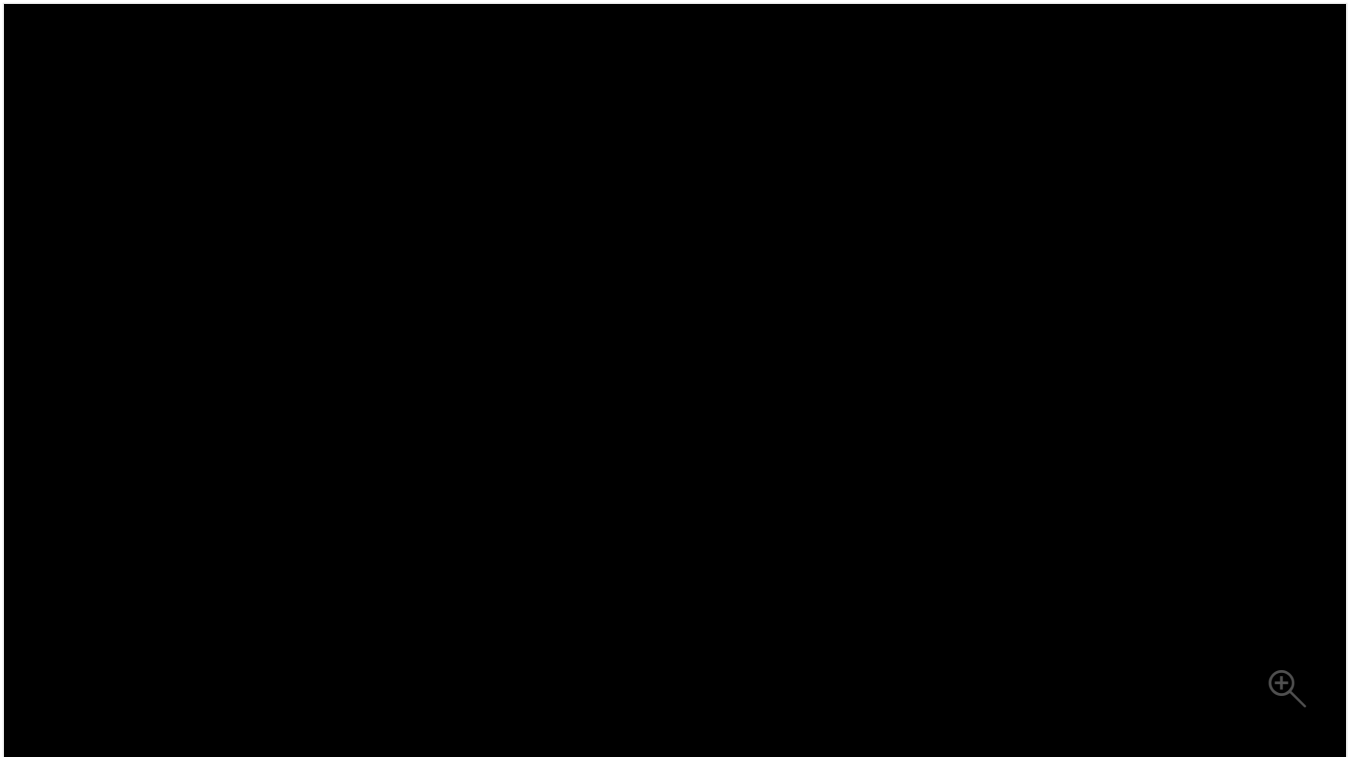
Last updated on 03/11/2026

What is Azure Skills?

Azure Skills are agent skills that extend your AI coding assistant with Azure-specific domain knowledge and specialized workflows. They give your assistant the ability to manage resources, deploy applications, and monitor services directly from your development environment.

Azure Skills in Visual Studio Code

The following demonstration shows Azure Skills in action inside Visual Studio Code. A developer uses natural language in the Copilot chat panel to interact with Azure services — no portal, no CLI commands, no context switching.



Azure Skills are agent skills that extend Azure-specific domain knowledge and specialized workflows for your coding agent. Azure Skills gives your AI assistant the ability to manage resources, deploy applications, and monitor services directly from your development environment.

Work with Azure without switching between tools, context windows, or documentation tabs. Ask your AI assistant to build, validate, and deploy, and it handles the Azure operations for you. Azure Skills uses the [Azure MCP Server](#) to provide your AI assistant with tools to interact with 40+ Azure services. Skills layer high-level workflows on top of those tools.

Traditional Azure workflows require context-switching between your editor, the Azure portal, and documentation while learning CLI commands and running validation steps manually. **Azure Skills** eliminates this friction. Your AI assistant becomes a full Azure development partner, understanding your application architecture and executing Azure operations at your direction.

The prepare, validate, and deploy workflow

Azure Skills follow a three-step workflow designed to prevent errors and ensure safe deployments.

When you ask your AI assistant to prepare an application for Azure, it:

1. Analyzes your codebase
2. Creates a detailed deployment plan
3. Generates infrastructure-as-code
4. Validates the setup
5. Deploys your app to Azure

All without you leaving your editor.

 Expand table

Step	Skill	What happens
Plan	azure-prepare	Your assistant analyzes your app, creates <code>.azure/plan.md</code> with deployment strategy, and waits for your approval before proceeding.
Check	azure-validate	Validates the plan before deployment. Runs configuration checks, permission verification, and infrastructure validation.
Deploy	azure-deploy	Executes the deployment. Runs provisioning, infrastructure deployment, and application setup.

This structured approach keeps deployments safe and auditable. You always review the plan before anything happens in Azure.

Related content

- [Install and configure Azure Skills](#) — Detailed setup instructions and authentication options.
- [Get started with Azure Skills](#) — Hands-on walkthrough for your first deployment.

- [Azure Skills reference](#) — Full documentation of all skills and their capabilities.
 - [Azure MCP Server](#) — Technical documentation for the underlying Azure integration.
-

Last updated on 04/17/2026

Install and configure Azure Skills

This article shows you how to install Azure Skills, authenticate to your Azure account, and verify the installation. After setup, you can manage Azure resources from your chat interface.

Prerequisites

Make sure you have:

- **Node.js Long Term Support (LTS):** [Download from nodejs.org](#) or use a version manager like [nvm](#).
- **Azure account:** [Create a free account](#) if you don't have one.
- **An AI assistant:** [GitHub Copilot CLI](#), Claude Code, or another compatible platform.
- **Azure CLI** (recommended for local development): Simplifies authentication. [Install Azure CLI](#). Environment variables or managed identity can be used as alternatives in CI/CD pipelines.

ⓘ Note

Azure Skills uses the same authentication as [Azure MCP Server](#). If you already authenticated for Azure MCP Server, Azure Skills uses those credentials.

Authenticate to Azure

Azure Skills needs credentials to access your Azure resources. Choose the method that fits your use case.

Azure CLI (recommended)

This method is the easiest for local development and testing.

1. Install Azure CLI:

- Visit the [Azure CLI installation guide](#).
- Or use a package manager: `brew install azure-cli` (macOS), `apt-get install azure-cli` (Linux).

2. Sign in to Azure:

Bash

```
az login
```

A browser window opens. Sign in by using your Azure account credentials.

3. Verify authentication:

Bash

```
az account show
```

You see your subscription details in the terminal. Azure Skills detects this authentication.

Install Azure Skills

Choose the installation method for your AI assistant.

GitHub Copilot CLI

Add the marketplace (first time only):

```
/plugin marketplace add microsoft/azure-skills
```

Install the plugin:

```
/plugin install azure@azure-skills
```

Update the plugin:

```
/plugin update azure@azure-skills
```

For the full list of supported hosts and installation options, see the [Azure Skills repository](#).

Verify installation

After installation, confirm Azure Skills is ready to use.

1. List installed plugins:

Bash

```
/plugin list
```

The output includes `azure@azure-skills` and available skills.

2. Test with a quick command:

Bash

```
/ask List my Azure subscriptions
```

Your AI assistant queries your Azure account and displays your subscriptions.

Try Azure Skills

Now that you installed and verified Azure Skills, try these prompts in your AI assistant's chat window:

Prepare a deployment plan for your project:

prompt

```
Analyze my project structure and prepare a deployment plan for deploying to Azure.  
What infrastructure do I need and how should I set it up?
```

Expected response: The assistant analyzes your codebase and generates infrastructure-as-code templates. It creates a deployment plan at `.azure/plan.md` and asks for your approval.

Diagnose an Azure resource health issue:

prompt

```
I'm getting errors from my Azure App Service. What's wrong and how do I fix it?
```

Expected response: The assistant inspects your app service, checks logs, and reviews recent deployments. It identifies issues and provides remediation steps.

Set up monitoring and observability:

prompt

Configure monitoring and alerting for my application. Which Application Insights features should I enable? What metrics should I track?

Expected response: The assistant proposes a monitoring strategy and generates instrumentation code. It configures Azure Alerts for critical metrics tailored to your app stack.

Supported AI assistants

You can use Azure Skills across these platforms:

- **GitHub Copilot CLI:** Full integration with GitHub Copilot tools
- **Claude Code:** Through the Model Context Protocol (MCP)
- **Other MCP-compatible AI assistants:** Any tool that supports MCP

For a complete list of tools that support Azure integration, see [Azure MCP Server supported tools](#).

Configuration options

Telemetry

By default, Azure Skills collect usage telemetry to improve the service.

To disable telemetry:

Bash

```
export AZURE_MCP_COLLECT_TELEMETRY=false
```

Troubleshooting

"Authentication failed" error

Problem: Your AI assistant can't authenticate to Azure.

Solutions:

- **Azure CLI:** Run `az login` again and ensure you're authenticated.

- **Environment variables:** Verify `AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, and `AZURE_CLIENT_SECRET` are set correctly.
- **Managed identity:** Confirm your Azure resource has an assigned managed identity.

"Plugin not found" error

Problem: The Azure Skills plugin isn't installed or recognized.

Solutions:

- To register the marketplace, run `/plugin marketplace add microsoft/azure-skills` .
- To install the plugin, run `/plugin install azure@azure-skills`.
- Restart your AI assistant.
- Verify Node.js is installed.

"Insufficient permissions" error

Problem: Your Azure account doesn't have the required permissions.

Solutions:

- Check your [Azure role-based access control \(RBAC\) role assignments](#).
- Request more roles from your Azure administrator.
- Use a different subscription where you have higher permissions.

"Invalid subscription" error

Problem: Azure Skills can't find or access your specified subscription.

Solutions:

- To see available subscriptions, run `az account list`.
- Verify the subscription ID or name is correct.
- Check that your credentials have access to the subscription.

Related content

- [Get started with Azure Skills](#)
- [Overview of Azure Skills](#)
- [GitHub Copilot for Azure](#)
- [Azure MCP Server get started guide](#)

Last updated on 04/17/2026

Get started with Azure Skills

In this quickstart, you prepare, validate, and deploy an application to Azure by using Azure Skills.

Prerequisites

- Azure Skills installed ([Install and configure Azure Skills](#))
- Azure CLI authenticated (`az login` completed successfully)
- GitHub Copilot CLI or Claude Code ready to use
- A sample application (or create a Node.js app)

Scenario

You have a Node.js application. You want to deploy it to Azure with:

- Web application hosting (Azure App Service)
- A storage account for application data
- Monitoring with Application Insights

Prepare your application

In your AI assistant, go to your project directory and ask:

text

Prepare my application for Azure deployment

The `azure-prepare` skill:

1. Analyzes your codebase
2. Identifies technology stack (Node.js, npm, and so on)
3. Creates `.azure/plan.md` with a deployment strategy
4. Generates infrastructure as code
5. Waits for your approval

Review the generated plan

Open `.azure/plan.md` and review:

- **Project Information**—Application name and deployment mode
- **Requirements**—Classification and scale (small, medium, large)
- **Components**—Technologies detected
- **Deployment Strategy**—Technology used to deploy your application (Azure Developer CLI, Bicep, Terraform, or Azure CLI).
- **Architecture**—Azure services selected
- **Implementation Plan**—Step-by-step tasks

Example plan content:

YAML

```
# Azure Deployment Plan

## Project Information
- Application: my-app
- Mode: NEW

## Requirements
- Classification: Web Application
- Scale: Small
- Environment: Production

## Components
- Runtime: Node.js 18+
- Package Manager: NPM
- Application Type: Express web server

## Recipe
- Type: AZD (Azure Developer CLI)

## Azure Services
- Azure App Service (web app hosting)
- Azure Storage Account (data)
- Application Insights (monitoring)

## Status: Awaiting Approval
```

Approve the plan

If the plan looks correct, tell your AI assistant:

text

Approve this plan and proceed to validation

The skill updates the plan status to **Approved** and moves to the next step.

Tip

If the skill doesn't recognize your project type, make sure you're in the project root directory with a recognizable project file (`package.json`, `requirements.txt`, `.csproj`, or similar).

Validate the deployment plan

Your AI assistant runs the `azure-validate` skill to check:

- Azure CLI access and permissions
- Bicep or Terraform template syntax, if applicable
- Azure subscription and region availability
- Service quota limits
- Required permissions for your account

Validation finishes and records proof of all checks in the plan. The plan status updates to `Validated`.

Review validation results

Check `.azure/plan.md` for the **Validation Proof** section, which shows:

- Commands executed
- Timestamp
- Results (passed or failed)

Example:

YAML

```
## Validation Proof
- Command: azd provision --preview
- Timestamp: 2026-03-16T14:22:00Z
- Result: ✓ All validation checks passed
```

If validation fails, review errors and ask your AI assistant to fix issues:

text

Fix the validation errors and try again



Tip

If validation fails, check that your Azure account has the required permissions and that the resources specified in the plan are available in your selected region.

Deploy to Azure

When your plan is validated, tell your AI assistant:

text

Deploy my application to Azure

The `azure-deploy` skill:

1. Confirms plan status is `Validated`
2. Provisions Azure resources (storage, app service, monitoring)
3. Deploys your application code
4. Configures application settings
5. Provides your application endpoint

Deployment typically takes 3-5 minutes.

View your deployed application

After successful deployment, your AI assistant provides:

- Application URL (for App Service)
- Storage account name and access keys
- Application Insights instrumentation key

Example:

Output

Deployment complete!

Your app is live at: <https://my-app-abcd1234.azurewebsites.net>

Resources deployed:

- App Service: my-app-prod
- Storage Account: mystorageabcd1234
- Application Insights: my-app-insights

Monitor your app: <https://portal.azure.com/...>

To verify that your app is running, visit your application URL in a browser.

Tip

If deployment fails, check the error output for permission or quota issues. Run `az account show` to verify you're authenticated to the correct subscription.

Verify your deployment

Test your application:

1. **Visit your URL**—Open the application URL in a browser.
2. **Check monitoring**—View logs in Application Insights.
3. **Test functionality**—Use key features of your app.

Ask your AI assistant for monitoring status:

text

Show me the application logs and performance metrics

Your AI assistant queries Application Insights and displays recent activity, errors, and performance data.

Update and redeploy

If you make code changes, redeploy easily:

1. **Update your code** in your editor.
2. **Ask your AI assistant:**

text

Update the deployment with my latest changes

3. The skill runs `azure-prepare` to check for changes, then `azure-deploy` to update your resources.

Updated resources reuse existing infrastructure. Only changed components redeploy.

Clean up resources

When you no longer need your application, delete Azure resources to avoid charges:

text

Delete all Azure resources for this application

Your AI assistant:

1. Lists resources to be deleted (for your confirmation)
2. Deletes the resource group and all contents
3. Confirms cleanup complete

Example:

Output

Resources to delete:
- Resource Group: my-app-rg
- All contained resources

Are you sure? (yes/no)

Type `yes` to confirm deletion.

Troubleshooting

Deployment fails with authentication error

Problem: Your AI assistant can't authenticate to Azure.

Solution: Reauthenticate by using `az login` and try again.

Plan validation fails

Problem: Azure Skills reports validation errors.

Solution: Ask your AI assistant to review and fix issues:

text

Why did validation fail? Fix the errors.

Application not accessible after deployment

Problem: The provided URL returns an error or times out.

Solution:

1. Verify the URL is correct.
2. Wait 1-2 minutes for DNS propagation.
3. Check Application Insights logs for errors.

text

Show me recent errors in Application Insights

Next steps

- [Azure Model Context Protocol \(MCP\) Server documentation](#)—Deeper technical details

Last updated on 04/17/2026

Azure skill for App Insights instrumentation

Guidance for instrumenting webapps with Azure Application Insights. Provides telemetry patterns, SDK setup, and configuration references.

Skill: `appinsights-instrumentation` | [Source code](#)

What it provides

This skill provides GitHub Copilot with specialized knowledge. Guidance for instrumenting webapps with Azure Application Insights. Provides telemetry patterns, SDK setup, and configuration references.

Prerequisites

- **Azure subscription:** [Create a free account](#) if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **PowerShell** (v7.4+): Install with `winget install Microsoft.PowerShell`.
- **Azure CLI with Bicep** (v2.60.0+): [Install](#), sign in with `az login`, then run `az bicep install`.

When to use this skill

Use this skill when you need to:

- Work with App Insights SDK, telemetry patterns, Application Insights guidance, and instrumentation examples
- Work with Apm best practices

Example prompts

Try these prompts to activate this skill:

- "Instrument my webapp to send telemetry to App Insights"
- "How do I instrument my app with Azure App Insights?"
- "Add AppInsights instrumentation to my web application"
- "Add App Insights instrumentation to my Node.js app"
- "Configure Application Insights for my Python webapp"

- "Set up telemetry monitoring in Azure"
- "Instrument my application to send data to App Insights"
- "Add observability to my Azure web application"
- "How to enable App Insights autoinstrumentation?"
- "Configure telemetry for my Azure App Service"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) 

Last updated on 04/17/2026

Azure skill for AI Gateway

Configure Azure API Management as an AI Gateway for AI models, Model Context Protocol (MCP) tools, and agents.

Skill: `azure-aigateway` | [Source code](#)

What it provides

This skill provides GitHub Copilot with specialized knowledge. Configure Azure API Management as an AI Gateway for AI models, MCP tools, and agents.

Prerequisites

- **Azure subscription:** [Create a free account](#) if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.

When to use this skill

Use this skill when you need to:

- Work with semantic caching, token limit, content safety, and load balancing
- Work with AI model governance, MCP rate limiting, and jailbreak detection
- Add Azure OpenAI back end
- Add AI Foundry model
- Test AI gateway in Azure
- Work with LLM policies
- Configure AI back end in Azure
- Work with token metrics, AI cost control, convert API to MCP, and import OpenAPI to gateway

Example prompts

Try these prompts to activate this skill:

- "Set up an AI Gateway for my Azure OpenAI models"

- "Configure Azure API Management as a gateway for my AI models"
- "Add a gateway to my MCP server"
- "Set up APIM for my AI workloads"
- "Add rate limiting to my model requests"
- "Limit tokens for my AI API"
- "How do I ratelimit my MCP server?"
- "Enable semantic caching for my AI API"
- "Set up semantic cache for Azure OpenAI in APIM"
- "Add content safety to my AI endpoint"

Related content

- [Azure MCP Server overview](#)

Last updated on 04/17/2026

Azure skill for AI Services

Skill: `azure-ai` | [Source code](#)

What it provides

This skill provides GitHub Copilot with specialized knowledge about Azure AI Services services and workflows in Azure.

Azure services knowledge

 Expand table

Service	When to use
AI Search	Full-text, vector, hybrid search
Speech	Speech-to-text, text-to-speech
OpenAI	GPT models, embeddings, DALL-E
Document Intelligence	Form extraction, OCR

Prerequisites

- **Azure subscription:** [Create a free account](#) if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.

When to use this skill

Use this skill when you need to:

- Query and manage Azure AI Search resources.
- Perform full-text, vector, hybrid, and semantic search queries.
- Execute vector, hybrid, and semantic search queries.
- Convert speech to text and text to speech using Azure Cognitive Services.

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
 - [Skill source code](#) 
-

Last updated on 04/17/2026

Azure skill for cloud migration

Assess and migrate cross-cloud workloads to Azure with migration reports and code conversion guidance. Supports AWS, GCP, and other providers.

Skill: `azure-cloud-migrate` | [Source code](#) 

What it provides

This skill provides GitHub Copilot with specialized knowledge. Assess and migrate cross-cloud workloads to Azure with migration reports and code conversion guidance. Supports AWS, GCP, and other providers.

Prerequisites

- **Azure subscription:** [Create a free account](#)  if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#) , Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.

When to use this skill

Use this skill when you need to:

- Migrate AWS Lambda functions to Azure Functions.
- Migrate GCP Cloud Run services to Azure Container Apps.
- Plan and execute migrations of AWS or GCP infrastructure to Azure.
- Perform readiness assessments for Lambda function or Cloud Run migrations.
- Convert AWS serverless architectures to Azure Compute services.
- Generate migration readiness reports.
- Execute migrations from AWS to Azure Cloud.
- Execute migrations from Google Cloud Platform to Azure.
- Plan migrations across multiple cloud providers to Azure.

Example prompts

Try these prompts to activate this skill:

- "How do I migrate my AWS Lambda functions to Azure Functions?"
- "I want to migrate from AWS to Azure"
- "Can you do a Lambda migration assessment for my project?"
- "Convert my serverless functions to Azure"
- "Generate a migration readiness report for my Lambda functions"
- "Help me migrate code to Azure Functions"
- "Assess my AWS Lambda project for Azure migration"
- "I need to move my Lambda workloads to Azure Functions"
- "Migrate my GCP Cloud Run services to Azure Container Apps"
- "How do I move from Cloud Run to Azure?"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) 

Last updated on 04/17/2026

Azure skill for compliance

Run Azure compliance and security audits with `azqr` plus Key Vault expiration checks. Covers best-practice assessment, resource review, policy/compliance validation, and security posture checks.

Skill: `azure-compliance` | [Source code](#)

What it provides

This skill provides GitHub Copilot with specialized knowledge. Run Azure compliance and security audits with `azqr` plus Key Vault expiration checks. Covers best-practice assessment, resource review, policy/compliance validation, and security posture checks.

Prerequisites

- **Azure subscription:** [Create a free account](#) if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.
- **Azure Key Vault** (optional): Required only for key, secret, and certificate expiration audits.

When to use this skill

Use this skill when you need to:

- Work with compliance scan and security audit
- Before running `azqr` (compliance cli tool)
- Work with Azure best practices, Key Vault expiration check, expired certificates, and expiring secrets
- Work with orphaned resources and compliance assessment

Example prompts

Try these prompts to activate this skill:

- "Run `azqr` to check Azure compliance"
- "Check my Azure subscription for compliance issues"

- "Perform compliance assessment using Azure Quick Review"
- "Assess my Azure resources against best practices"
- "Review my Azure security posture"
- "Run compliance scan on my Azure subscription"
- "Identify orphaned resources in Azure"
- "Find resources that don't comply with best practices"
- "Show me expired certificates in my Key Vault"
- "Check what secrets are expiring in the next 30 days"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) 

Last updated on 04/17/2026

Azure skill for compute

Azure virtual machine (VM) and Virtual machine scale set (VMSS) router for recommendations, pricing, autoscale, orchestration, and connectivity troubleshooting. This skill can answer questions about autoscaling and orchestration but isn't meant to help execute those tasks.

Skill: `azure-compute` | [Source code](#)

What it provides

This skill provides GitHub Copilot with specialized knowledge. Azure VM and Virtual machine scale set (VMSS) router for recommendations, pricing, autoscale, orchestration, and connectivity troubleshooting.

Prerequisites

- **Azure subscription** (required for troubleshooting actual VMs/VMSS): [Create a free account](#) if you don't have one. Not required for general questions, recommendations, or pricing information.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **Azure CLI** (v2.60.0+) (required for pulling diagnostics from actual VMs/VMSS): [Install](#) and sign in with `az login`. Not required for general questions, recommendations, or pricing information.

When to use this skill

Use this skill when you need to:

- Create and manage Azure virtual machines (VMs) and virtual machine scale sets (VMSS).
- Configure autoscaling for virtual machine scale sets.
- Select appropriate VM SKUs for your workload (server, web hosting, burstable compute, or lightweight).
- Optimize VM selection for different workloads, including development/test scenarios and back-end services.
- Work with autoscale, load balancer, Flexible orchestration, and Uniform orchestration
- Work with cost estimate, Linux, black screen, and reset password

- Troubleshoot virtual machine connectivity issues.

Example prompts

Try these prompts to activate this skill:

- "Help me choose a VM for my workload"
- "What VM size should I use for a web server?"
- "Compare Azure VM families for machine learning workloads"
- "How much will a Standard D4s v3 VM cost?"
- "Help me set up autoscale for my VM scale set"
- "I can't connect to my VM via RDP, help me troubleshoot"
- "How do I reset the password on my Azure VM?"
- "What's the difference between Flexible and Uniform orchestration?"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) 

Last updated on 04/17/2026

Azure skill for cost optimization

Analyze and optimize your Azure spending with historical cost queries, spending forecasts, and actionable cost-reduction recommendations.

Skill: `azure-cost` | [Source code](#)

What it provides

This skill gives GitHub Copilot access to Azure Cost Management APIs and optimization best practices. It can analyze your current spending, project future costs, and suggest specific ways to reduce waste.

Prerequisites

- **Azure subscription:** [Create a free account](#) if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.
- **Azure roles:** Your account must have the [Cost Management Reader](#), [Monitoring Reader](#), and [Reader](#) roles on the target subscription or resource group.
- **Azure Kubernetes Service:** An AKS cluster for container orchestration.

Related tools

[Expand table](#)

Tool	Command	Purpose
<code>azure__documentation</code>	Search Azure documentation	Search for relevant Azure documentation by keyword
<code>azure__extension_cli_generate</code>	Generate Azure CLI commands	Generate Azure CLI commands from a natural-language description
<code>azure__get_azure_bestpractices</code>	Get Azure best practices	Retrieve best practices for a specific optimization scenario
<code>azure__extension_azqr</code>	Run Azure Quick Review	Run a compliance scan against a subscription or resource group

Tool	Command	Purpose
azure__aks	Azure Kubernetes Service operations	Manage Azure Kubernetes Service clusters and workloads

When to use this skill

Use this skill when you need to:

- Analyze costs of your existing Azure resources and infrastructure.
- Review your Azure bill, costs broken down by service, and costs per resource.
- Compare monthly cost summaries, identify cost trends, and pinpoint top cost drivers.
- Calculate amortized costs and actual spending.
- Forecast future spending and project end-of-month costs.
- Plan budget forecasts based on historical data.
- Identify and implement cost optimization strategies.
- Find opportunities to reduce cloud spending.
- Discover cost-saving recommendations tailored to your infrastructure.

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) ↗

Last updated on 04/17/2026

Azure skill for deploy

Execute Azure deployments for already-prepared applications that have existing `.azure/deployment-plan.md` and infrastructure files.

Skill: `azure-deploy` | [Source code](#) ↗

What it provides

This skill enables GitHub Copilot to execute production deployments using your prepared infrastructure-as-code files. You get automated provisioning, reliable updates to deployed resources, and safe infrastructure deployments without manual CLI commands.

Prerequisites

- **Prepared deployment plan:** Run the [azure-prepare](#) skill first to generate your `.azure/deployment-plan.md` and infrastructure files. Then run the [azure-validate](#) skill to verify readiness.
- **Azure subscription:** [Create a free account](#) ↗ if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#) ↗, Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.

When to use this skill

Use this skill when you need to:

- Provision and deploy resources to Azure using the `azd up` command
- Update deployed resources using the `azd deploy` command
- Execute deployments with Azure Developer CLI or your infrastructure-as-code tool
- Work with push to production, push to cloud, go live, and ship it
- Work with bicep deploy, terraform apply, publish to Azure, and launch on Azure

Example prompts

Try these prompts to activate this skill:

- "Execute deployment to Azure production"

- "Deploy and provision my Azure infrastructure"
- "Push my deploy to Azure production"
- "Ship and deploy my Azure app"
- "Run the Azure deployment now"
- "Deploy my Azure Functions app to cloud using the Azure Developer CLI."
- "Deploy my serverless function app to Azure"
- "Deploy Azure Functions to production"
- "Deploy my app and verify the role-based access control (RBAC) roles are assigned correctly"
- "Run deployment and check live role assignments on Azure"

Deployment workflow

This skill is the final step in the deployment workflow:

1. [azure-prepare](#) — generates infrastructure files and `.azure/deployment-plan.md`
2. [azure-validate](#) — validates the deployment plan and infrastructure before deploying
3. [azure-deploy](#) (this skill) — executes the deployment

Related content

- [Azure skill for prepare](#)
- [Azure skill for validate](#)
- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) ↗

Last updated on 04/17/2026

Azure skill for diagnostics

Debug Azure production issues on Azure using AppLens, Azure Monitor, resource health, and safe triage.

Skill: `azure-diagnostics` | [Source code](#)

What it provides

This skill provides GitHub Copilot with specialized knowledge. Debug Azure production issues on Azure using AppLens, Azure Monitor, resource health, and safe triage.

Prerequisites

- **Azure subscription:** [Create a free account](#) if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.
- **Azure compute resources:** This skill supports Azure Container Apps, Azure Functions, Azure Kubernetes Service (AKS), and other Azure compute services.

When to use this skill

Use this skill when you need to:

- Debug production issues in Azure
- Troubleshoot container apps in Azure
- Troubleshoot functions in Azure
- Troubleshoot AKS in Azure
- Work with kubectl can't connect, kube-system/CoreDNS failures, pod pending, and crashloop
- Work with node not ready
- Upgrade failures in Azure
- Analyze logs in Azure
- Work with insights, image pull failures, cold start issues, and health probe failures
- Work with resource health and root cause of errors

Example prompts

Try these prompts to activate this skill:

- "Debug my Azure Container App"
- "Troubleshoot production issues in my container app"
- "Diagnose errors in my Azure service"
- "Help me troubleshoot container apps on Azure"
- "Analyze logs with KQL for my app"
- "How do I analyze application logs?"
- "View application logs for my container"
- "Fix image pull failures in Container Apps"
- "My container app has image pull errors"
- "Resolve cold start issues"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) ↗

Last updated on 04/17/2026

Azure skill for enterprise infrastructure planning

Architect and provision enterprise Azure infrastructure from workload descriptions. For cloud architects and platform engineers planning networking, identity, security, compliance, and multi-resource topologies with waf alignment. Generates Bicep or Terraform directly (no `azd`).

Skill: `azure-enterprise-infra-planner` | [Source code](#) 

What it provides

This skill provides GitHub Copilot with specialized knowledge. Architect and provision enterprise Azure infrastructure from workload descriptions. For cloud architects and platform engineers planning networking, identity, security, compliance, and multi-resource topologies with waf alignment. Generates Bicep or Terraform directly (no `azd`).

Prerequisites

- **Azure subscription:** [Create a free account](#)  if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#) , Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.

When to use this skill

Use this skill when you need to:

- Design enterprise Azure infrastructure from high-level requirements.
- Architect Azure landing zones with security, governance, and cost controls.
- Design hub-and-spoke network topologies for multi-region Azure deployments.
- Plan multi-region disaster recovery topologies.
- Configure virtual network security with firewalls and private endpoints.
- Deploy Bicep templates at subscription scope (for infrastructure-only workflows; use `azure-prepare` for application-centric deployments).

Example prompts

Try these prompts to activate this skill:

- "Deploy a geo-redundant backup solution for on-premises SQL servers using Azure Backup, configure encryption-at-rest, and automate monthly DR tests."
- "Deploy 3-tier architecture with hardened OS images, virtual machine (VM) backups scheduled daily, and application-level redundancy for the business logic tier."
- "Configure a site recovery plan for disaster failover from East to West Azure region, replicate major VM workloads, and automate DNS failbacks."
- "Provision a jumpbox VM for secure management, establish NSGs for each tier, and connect tiers using internal Azure Load Balancer."
- "Spin up Linux VMs for each tier using Terraform, automate patch management through Azure Automation, and log traffic between subnets for compliance."
- "Deploy three distinct VM scale sets for a legacy app, route incoming HTTP/S through Application Gateway with Web Application Firewall (WAF), and encrypt all data disks."
- "Set up Azure Backup for critical VM workloads, create a long-term retention policy for compliance, and test backup restores quarterly."
- "Deploy disaster recovery for VMware VMs using Azure Site Recovery, configure runbooks for smooth failover, and maintain compliance audit trails."

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) ↗

Last updated on 04/17/2026

Azure skill for Hosted Copilot SDK

Build and deploy GitHub Copilot SDK apps to Azure.

Skill: `azure-hosted-copilot-sdk` | [Source code](#) 

What it provides

This skill provides GitHub Copilot with specialized knowledge. Build and deploy GitHub Copilot SDK apps to Azure.

Note

This skill automatically activates when your codebase contains `@github/copilot-sdk` in `package.json` or `CopilotClient` in source files. When detected, this skill becomes the entry point for deploy, modify, and add-feature workflows instead of the general-purpose `azure-prepare` skill.

Prerequisites

- **Azure subscription:** [Create a free account](#)  if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#) , Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.

When to use this skill

Use this skill when you need to:

- Build custom copilot applications on Azure infrastructure.
- Create and configure copilot applications.
- Use the GitHub Copilot SDK to scaffold and build copilot-powered applications.
- Deploy your copilot application to Azure infrastructure.
- Host applications on Azure using Azure OpenAI models or bring your own model (BYOM).
- Integrate your own large language model (LLM) with the Copilot SDK.
- Configure Azure OpenAI models, managed identity authentication, and the Copilot SDK service.

- Build chat applications using the Copilot SDK service template and the `CopilotClient` library.
- Implement session management and message handling using the GitHub Models API.

Example prompts

Try these prompts to activate this skill:

- "Build a Copilot SDK app and deploy it"
- "Create a new copilot SDK service"
- "Scaffold a copilot-powered app on Azure"
- "Build with the GitHub Copilot SDK and host it"
- "Build a Copilot SDK app with my own Azure model"
- "Create a copilot app using my Azure OpenAI model"
- "Set up a copilot service with `BYOM` and `DefaultAzureCredential`"
- "Build a copilot app that uses a self-hosted model on Azure"
- "Deploy a copilot SDK app with my own endpoint"
- "Create a copilot app and bring your own model from Azure OpenAI"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) ↗

Last updated on 04/17/2026

Azure skill for Kubernetes

Plan, create, and configure production-ready Azure Kubernetes Service (AKS) clusters. Covers Day-0 checklist, SKU selection (Automatic vs Standard), networking options (private API server, Azure cni Overlay, egress configuration), security, and operations (autoscaling, upgrade strategy, cost analysis).

Skill: `azure-kubernetes` | [Source code](#)

What it provides

This skill gives GitHub Copilot deep expertise in AKS cluster planning and configuration. It helps you make critical Day-0 decisions — networking topology, API server access, and pod IP model — that are difficult to change after cluster creation. Specifically, it provides:

- **SKU selection guidance:** Recommends AKS Automatic vs Standard based on your control and customization needs.
- **Networking configuration:** Advises on Azure CNI Overlay vs VNet-routable CNI, egress strategies (Static Egress Gateway, UDR + Firewall), ingress options (App Routing, Istio, Application Gateway for Containers), and DNS settings.
- **Security best practices:** Guides Microsoft Entra ID integration, Workload Identity for pods, Key Vault secrets via CSI Driver, Azure Policy with Deployment Safeguards, and image signing.
- **Operations and scaling:** Configures autoscaling (KEDA, Node Auto Provisioning), upgrade strategies (maintenance windows, Fleet Manager), and observability (Managed Prometheus, Container Insights, Grafana).
- **Optimization:** Recommends rightsizing, Spot node pools, Cluster AutoScaler (CAS), and Vertical Pod Autoscaler (VPA).

Related tools

[Expand table](#)

Tool	Command	Purpose
<code>mcp_azure_mcp_aks</code>	AKS Model Context Protocol (MCP) entry point used to discover the exact AKS-specific tools exposed by the client	Discover the callable AKS tool first, then use that tool's parameters

Prerequisites

- **Azure subscription:** [Create a free account](#) if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.
- **Azure Key Vault:** A key vault for secrets and certificate management.
- **Azure Kubernetes Service:** An AKS cluster for container orchestration.

When to use this skill

Use this skill when you need to:

- Create AKS environment in Azure
- Provision AKS environment in Azure
- Enable AKS observability in Azure
- Design AKS networking in Azure
- Choose AKS SKU in Azure
- Secure AKS in Azure

Suggested workflow

- **AKS Automatic (default):** Best for most production workloads, provides a curated experience with pre-configured best practices for security, reliability, and performance. Use unless you have specific custom requirements for networking, autoscaling, or node pool configurations not supported by Node Auto-Provisioning (NAP).
- **AKS Standard:** Use if you need full control over environment configuration, which requires additional overhead to set up and manage.
- **Azure CNI Overlay (recommended):** Uses private IP addresses for pods (not routable from the virtual network), scales to large environments, and works well for most production workloads.
- **Azure CNI (VNet-routable):** pod IPs directly from VNet (pod subnet or node subnet), use when pods must be directly addressable from VNet or on-prem
- **Azure CNI powered by Cilium (recommended):** eBPF (extended Berkeley Packet Filter)-based for high-performance packet processing, network policies, and observability
- **Static Egress Gateway** for stable, predictable outbound IPs
- For restricted egress: user-defined route (UDR) + Azure Firewall or network virtual appliance (NVA)

- **App Routing add-on with Gateway API** - recommended default for HTTP/HTTPS workloads
- **Istio service mesh with Gateway API** - for advanced traffic management, mTLS, canary releases
- **Application Gateway for Containers** - for L7 load balancing with Web Application Firewall (WAF) integration
- Enable **LocalDNS** on all node pools for reliable, performant DNS resolution
- Use **Microsoft Entra ID** everywhere (control plane, Workload Identity for pods, node access). Avoid static credentials.
- Azure Key Vault through **Secrets Store CSI Driver** for secrets
- Enable **Azure Policy + Deployment Safeguards**
- Enable **Encryption at rest** for etcd/API server; **in-transit** for node-to-node
- Allow only signed, policy-approved images (Azure Policy + Ratify), prefer **Azure Container Registry**
- **Isolation**: Use namespaces, network policies, scoped logging
- Use Managed Prometheus and Container Insights with Grafana for AKS observability (logs + metrics).
- Enable Diagnostic Settings to collect control plane logs and audit logs in a Log Analytics workspace for security monitoring and troubleshooting.
- For other monitoring and troubleshooting tools, use features like the Agent CLI for AKS, Application Insights, Resource Health Center, AppLens detectors, and Azure Advisors.
- Configure **Maintenance Windows** for controlled upgrade timing
- Enable **automatic upgrades** for control plane and node OS to stay up-to-date with security patches and Kubernetes versions
- Consider **Long-Term Support (LTS) versions** for enterprise stability (2-year support) by upgrading your AKS environment to the Premium tier
- **Fleet upgrades**: Use **AKS Fleet Manager** for staged rollout across test to production environments
- Use **Ephemeral OS disks** (`--node-osdisk-type Ephemeral`) for faster node startup
- Select **Azure Linux** as node OS (smaller footprint, faster boot)
- Enable **KEDA (Kubernetes Event-Driven Autoscaling)** for event-driven autoscaling beyond Horizontal Pod Autoscaler (HPA)
- **Dedicated system node pool**: At least 2 nodes, tainted for system workloads only (`CriticalAddonsOnly`)
- Enable **Node Auto Provisioning (NAP)** on all pools for cost savings and responsive scaling
- Use **latest generation SKUs (v5/v6)** for host-level optimizations

- **Avoid B-series VMs** - burstable SKUs cause performance/reliability issues
- Use SKUs with **at least 4 vCPUs** for production workloads
- Set **topology spread constraints** to distribute pods across hosts/zones per SLO
- Deploy across **3 Availability Zones** (`--zones 1 2 3`)
- Use **Standard tier** for zone-redundant control plane + 99.95% SLA for API server availability
- Enable **Microsoft Defender for Containers** for runtime protection
- Configure **PodDisruptionBudgets** for all production workloads
- Use **topology spread constraints** to ensure pod distribution across failure domains
- Use **Spot node pools** for batch/interruptible workloads (up to 90% savings)
- **Stop/Start development and test clusters** to reduce costs: `az aks stop` and `az aks start`
- Consider **Reserved Instances** or **Savings Plans** for steady-state workloads

Example prompts

Try these prompts to activate this skill:

- "Help me create an AKS cluster"
- "I need to set up a new Kubernetes cluster on Azure"
- "Create a production-ready AKS cluster with best practices"
- "How do I provision an AKS cluster for my team?"
- "What networking options should I choose for AKS?"
- "AKS Day-0 checklist"
- "Plan AKS configuration for production"
- "Design AKS networking with private API server"
- "What's the difference between AKS Automatic and Standard?"
- "Should I use AKS Automatic or Standard SKU?"

Related content

- [Azure MCP Server overview](#)

Azure skill for Kusto (Data Explorer)

This skill queries and analyzes data in Azure Data Explorer (also called Kusto or ADX) using Kusto Query Language (KQL) to support log analytics, telemetry monitoring, and time-series analysis.

Skill: `azure-kusto` | [Source code](#)

What it provides

This skill provides GitHub Copilot with specialized knowledge. This skill queries and analyzes data in Azure Data Explorer (also called Kusto or ADX) using Kusto Query Language (KQL) to support log analytics, telemetry monitoring, and time-series analysis.

Prerequisites

- **Azure subscription:** [Create a free account](#) if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.

When to use this skill

Use this skill when you need to:

- Write KQL queries to analyze logs, telemetry data, and time-series information in Azure Data Explorer clusters.
- Perform log analytics, analyze time-series data from IoT devices, and detect anomalies.

Example prompts

Try these prompts to activate this skill:

- "Query my Kusto database for events in the last hour"
- "Write a KQL query to analyze telemetry data from my ADX cluster"
- "Show me the schema for tables in my Azure Data Explorer cluster"
- "Analyze IoT sensor data for anomalies in the last 24 hours"
- "Create a time series chart of application logs from my Kusto database"

- "What tables are available in my Azure Data Explorer cluster?"
- "Aggregate request latency metrics by service using KQL"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) 

Last updated on 04/17/2026

Azure skill for messaging

Troubleshoot and resolve issues with Azure Messaging SDKs for Event Hubs and Service Bus. Covers connection failures, authentication errors, message processing issues, and SDK configuration problems.

Skill: `azure-messaging` | [Source code](#)

What it provides

This skill provides GitHub Copilot with specialized knowledge. Troubleshoot and resolve issues with Azure Messaging SDKs for Event Hubs and Service Bus. Covers connection failures, authentication errors, message processing issues, and SDK configuration problems.

Prerequisites

- **Azure subscription:** [Create a free account](#) if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.

Related tools

[Expand table](#)

Tool	Command	Purpose
<code>mcp_azure_mcp_eventhubs</code>	<code>Namespace/hub ops</code>	List namespaces, hubs, consumer groups
<code>mcp_azure_mcp_servicebus</code>	<code>Queue/topic ops</code>	List namespaces, queues, topics, subscriptions
<code>mcp_azure_mcp_monitor</code>	<code>logs_query</code>	Query diagnostic logs with KQL
<code>mcp_azure_mcp_resourcehealth</code>	<code>get</code>	Check service health status
<code>mcp_azure_mcp_documentation</code>	<code>Doc search</code>	Search Microsoft Learn for troubleshooting docs

When to use this skill

Use this skill when you need to:

- Troubleshoot Event Hubs SDK errors, Service Bus SDK issues, and messaging connection failures (including AMQP protocol errors).
- Resolve event processor host issues, message lock expiration, and lock renewal failures.
- Work with lock renewal batch, send timeout, receiver disconnected, and SDK troubleshooting
- Debug Azure Messaging SDK problems affecting Event Hubs consumers, Service Bus queues, and topic subscriptions.
- Enable and configure logging for Event Hubs.
- Set up logging for Service Bus and configure SDK logging in Python, Java, and JavaScript.
- Configure .NET Service Bus clients and manage Event Hubs checkpoints.
- Event hub not receiving messages
- Handle dead-letter messages, resolve batch processing locks, and troubleshoot session timeouts.
- Address inactive connections, link detachment, slow reconnection, and session errors.

Example prompts

Try these prompts to activate this skill:

- "event hub SDK error in my Python app"
- "my event hub consumer isn't receiving messages"
- "event hub checkpoint store failing"
- "eventhub python connection timeout"
- "eventhub javascript client disconnects"
- "service bus SDK issue with message lock lost"
- "service bus queue issue with dead letter"
- "servicebus java send timeout"
- "servicebus dotnet receiver disconnected"
- "service bus message lock expired during batch processing"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) ↗

Azure skill for prepare

Prepare Azure apps for deployment (infra Bicep/Terraform, azure.yaml, Dockerfiles).

Skill: `azure-prepare` | [Source code](#) 

What it provides

This skill provides GitHub Copilot with specialized knowledge. Prepare Azure apps for deployment (infra Bicep/Terraform, azure.yaml, Dockerfiles).

Prerequisites

- **Azure subscription:** [Create a free account](#)  if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#) , Claude Code, or another [compatible MCP client](#).
- **Azure CLI with Bicep (v2.60.0+):** [Install](#), sign in with `az login`, then run `az bicep install`.
- [Terraform](#)  (v1.5+).

When to use this skill

Use this skill when you need to:

- Create app in Azure
- Build web app in Azure
- Create API in Azure
- Create serverless HTTP API
- Create front end in Azure
- Create back end in Azure
- Build a service in Azure
- Work with modernize application
- Update application in Azure
- Add authentication in Azure

Example prompts

Try these prompts to activate this skill:

- "Create a dad joke generator and deploy to Azure"
- "Build a web app and host it on Azure"
- "I want to deploy my application to Azure"
- "Set up Azure infrastructure for my project"
- "Prepare my app for Azure deployment"
- "Create an API and run it on Azure"
- "Migrate my application to Azure"
- "Configure Azure hosting for my app"
- "Create a serverless HTTP API using Azure Functions and deploy to Azure"
- "Create an event-driven function app to process messages and deploy to Azure Functions"

Deployment workflow

This skill is the first step in the deployment workflow:

1. **azure-prepare** (this skill) — generates infrastructure files and `.azure/deployment-plan.md`
2. **azure-validate** — validates the deployment plan and infrastructure before deploying
3. **azure-deploy** — executes the deployment

Related content

- [Azure skill for validate](#)
- [Azure skill for deploy](#)
- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) [↗](#)

Last updated on 04/17/2026

Azure skill for quotas

Check/manage Azure quotas and usage across providers. For deployment planning, capacity validation, region selection.

Skill: `azure-quotas` | [Source code](#) ↗

What it provides

This skill provides GitHub Copilot with specialized knowledge. Check/manage Azure quotas and usage across providers. For deployment planning, capacity validation, region selection.

Prerequisites

- **Azure subscription and region:** [Create a free account](#) ↗ if you don't have one. Both a subscription and a target region are needed to check quotas.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#) ↗, Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.

When to use this skill

Use this skill when you need to:

- Check quotas in Azure
- Work with service limits and current usage
- Request quota increase in Azure
- Work with quota exceeded
- Validate capacity in Azure
- Work with regional availability
- Provisioning limits in Azure
- Work with vCPU limit

Example prompts

Try these prompts to activate this skill:

- "How do I check my Azure quota limits?"

- "What are the service limits for my Azure subscription?"
- "Check current usage for my compute quota"
- "I need to request a quota increase for VMs in East US"
- "My deployment failed with a quota exceeded error"
- "How do I validate deployment capacity before provisioning?"
- "Help me select a region based on quota availability"
- "Compare quotas across regions for Standard_D4s_v3"
- "What is the provisioning limit for public IP addresses?"
- "Check regional capacity for Container Apps"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) ↗

Last updated on 04/17/2026

Azure skill for role-based access control (RBAC)

Helps users find the right Azure RBAC role for an identity with least privilege access, then generate CLI commands and Bicep code to assign it. Also provides guidance on permissions required to grant roles.

Skill: `azure-rbac` | [Source code](#) 

What it provides

This skill gives GitHub Copilot expertise in Azure role-based access control so it can guide you through secure role assignments. Specifically, it provides:

- **Least-privilege role recommendations:** Searches built-in Azure roles to find the minimal role definition that matches the permissions your identity needs.
- **Custom role definitions:** When no built-in role fits, generates a custom role definition scoped to exactly the permissions you require.
- **Role assignment commands:** Produces ready-to-run Azure CLI commands and Bicep code snippets for assigning roles to managed identities, service principals, or users.
- **Granting permissions guidance:** Explains what permissions you need (such as `Microsoft.Authorization/roleAssignments/write`) to assign roles, and which built-in roles like User Access Administrator or Owner provide them.

Prerequisites

- **Azure subscription:** [Create a free account](#)  if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#) , Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.
- **Azure role:** Your account must have `Microsoft.Authorization/roleAssignments/write` permission, such as [User Access Administrator](#) or [Owner](#).

When to use this skill

Use this skill when you need to:

- Work with bicep for role assignment, least privilege role, RBAC role for, and role to read blobs
- Work with role for managed identity and custom role definition
- Assign role to identity
- Work with permissions to assign roles

Example prompts

Try these prompts to activate this skill:

- "What Azure RBAC role should I assign to my managed identity?"
- "Which Azure role gives least privilege access to read blobs from storage?"
- "What role do I need for my identity to access Azure Key Vault secrets?"
- "Help me find the right Azure role for container registry access"
- "I need a custom role definition for my Azure storage account"
- "What Azure role should I use to give my function app access to Service Bus?"
- "Assign an Azure RBAC role to my identity for Cosmos DB read access"
- "What is the least privilege role for reading from a storage queue?"
- "I need to assign a role to my app service managed identity for database access"
- "Generate Bicep code for assigning a role to my function app"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) ↗

Last updated on 04/17/2026

Azure skill for resource lookup

List, find, and show Azure resources across subscriptions or resource groups. Handles prompts like "list websites", "list virtual machines", "list my VMs", "show storage accounts", "find container apps", and "what resources do I have".

Skill: `azure-resource-lookup` | [Source code](#)

What it provides

This skill provides GitHub Copilot with specialized knowledge. List, find, and show Azure resources across subscriptions or resource groups. Handles prompts like "list websites", "list virtual machines", "list my VMs", "show storage accounts", "find container apps", and "what resources do I have".

Prerequisites

- **Azure subscription:** [Create a free account](#) if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.
- **Azure Key Vault:** A key vault for secrets and certificate management.
- **Azure Storage:** A storage account for blob, file, queue, or table data.
- **Azure Kubernetes Service:** An AKS cluster for container orchestration.
- **Azure Cosmos DB:** A Cosmos DB account for NoSQL data.

Related tools

[Expand table](#)

Tool	Command	Purpose
<code>extension_cli_generate</code>	<code>Generate az graph query commands</code>	Primary tool - generate ARG queries from user intent
<code>mcp_azure_mcp_subscription_list</code>	<code>List available subscriptions</code>	Discover subscription scope before querying
<code>mcp_azure_mcp_group_list</code>	<code>List resource groups</code>	Narrow query scope

When to use this skill

Use this skill when you need to:

- Work with resource inventory
- Find resources by tag
- Work with tag analysis
- Orphaned resource discovery (not for cost analysis)
- Work with unattached disks, count resources by type, and cross-subscription lookup
- And Azure Resource Graph queries

Example prompts

Try these prompts to activate this skill:

- "List the websites in my subscription"
- "Show me the websites in my resource group"
- "List all virtual machines in my subscription"
- "Show me all VMs in resource group 'my-rg'"
- "List my Azure storage accounts"
- "List all my Azure Container Registries"
- "List the container apps in my subscription"
- "Show me the container apps in my resource group"
- "What resources do I have across all my subscriptions?"
- "Show me all my Azure resources"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) ↗

Last updated on 04/17/2026

Azure skill for resource visualizer

Analyze Azure resource groups and generate detailed Mermaid architecture diagrams showing the relationships between individual resources.

Skill: `azure-resource-visualizer` | [Source code](#) ↗

What it provides

This skill provides GitHub Copilot with specialized knowledge. Analyze Azure resource groups and generate detailed Mermaid architecture diagrams showing the relationships between individual resources.

Prerequisites

- **Azure subscription:** [Create a free account](#) ↗ if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#) ↗, Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.

When to use this skill

Use this skill when you need to:

- Create architecture diagram in Azure
- Visualize Azure resources
- Work with generate Mermaid diagram
- Analyze resource group in Azure
- Work with diagram my resources, architecture visualization, resource topology, and map Azure infrastructure

Example prompts

Try these prompts to activate this skill:

- "Create an architecture diagram for my Azure resource group"
- "Generate a Mermaid diagram of my resource group"
- "Visualize my Azure resources"

- "Visualize the architecture of my Azure resources"
- "Architecture visualization for my Azure infrastructure"
- "Show me the relationships between my Azure resources"
- "Show resource relationships"
- "How are my Azure resources connected?"
- "Analyze my resource group"
- "Analyze resource group architecture"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) 

Last updated on 04/17/2026

Azure skill for storage

Azure Storage Services including Blob Storage, File Shares, Queue Storage, Table Storage, and Data Lake. Provides object storage, SMB file shares, async messaging, NoSQL key-value, and big data analytics capabilities. Includes access tiers (hot, cool, archive) and lifecycle management.

Skill: `azure-storage` | [Source code](#)

What it provides

This skill provides GitHub Copilot with specialized knowledge. Azure Storage Services including Blob Storage, File Shares, Queue Storage, Table Storage, and Data Lake. Provides object storage, SMB file shares, async messaging, NoSQL key-value, and big data analytics capabilities. Includes access tiers (hot, cool, archive) and lifecycle management.

Prerequisites

- **Azure subscription:** [Create a free account](#) if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.
- **Azure Storage:** A storage account for blob, file, queue, or table data.

Azure services knowledge

 Expand table

Service	When to use
Blob Storage	Objects, files, backups, static content
File Shares	SMB file shares, lift-and-shift
Queue Storage	Async messaging, task queues
Table Storage	NoSQL key-value (consider Cosmos DB)
Data Lake	Big data analytics, hierarchical namespace

When to use this skill

Use this skill when you need to:

- Work with blob storage, file shares, queue storage, and table storage
- Work with data lake
- Upload files in Azure
- Work with download blobs, storage accounts, access tiers, and lifecycle management

Example prompts

Try these prompts to activate this skill:

- "Upload a file to my Azure Blob Storage container"
- "Download a blob from my storage account"
- "List all containers in my Azure Storage account"
- "Set up lifecycle management to move blobs to archive tier"
- "Create a file share in my storage account"
- "What's the difference between hot, cool, and archive access tiers?"
- "Set up a queue storage for async processing"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) ↗

Last updated on 04/17/2026

Azure skill for upgrade

Assess and upgrade Azure workloads between plans, tiers, or SKUs within Azure. Generates assessment reports and automates upgrade steps.

Skill: `azure-upgrade` | [Source code](#) 

What it provides

This skill provides GitHub Copilot with specialized knowledge. Assess and upgrade Azure workloads between plans, tiers, or SKUs within Azure. Generates assessment reports and automates upgrade steps.

Prerequisites

- **Azure subscription:** [Create a free account](#)  if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#) , Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.

When to use this skill

Use this skill when you need to:

- Upgrade Consumption to Flex Consumption
- Upgrade Azure Functions plan
- Migrate hosting plan in Azure
- Upgrade Functions SKU in Azure
- Move to Flex Consumption
- Upgrade Azure service tier
- Work with change hosting plan
- Upgrade function app plan
- Migrate App Service to Container Apps

Example prompts

Try these prompts to activate this skill:

- "Upgrade my function app from Consumption to Flex Consumption"
- "Move my function app to a better plan"
- "Is my function app ready for Flex Consumption?"
- "Automate the steps to upgrade my Functions plan"
- "Upgrade my Azure Functions SKU"
- "Change my function app hosting plan"
- "Migrate my Azure Functions from Consumption to Flex Consumption"
- "Assess my function app for upgrade readiness"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) 

Last updated on 04/17/2026

Azure skill for validate

Pre-deployment validation for Azure readiness. Run deep checks on configuration, infrastructure (Bicep or Terraform), role-based access control (RBAC) role assignments, managed identity permissions, and prerequisites before deploying.

Skill: `azure-validate` | [Source code](#) 

What it provides

This skill provides GitHub Copilot with specialized knowledge. Pre-deployment validation for Azure readiness. Run deep checks on configuration, infrastructure (Bicep or Terraform), RBAC role assignments, managed identity permissions, and prerequisites before deploying.

Prerequisites

- **Azure subscription:** [Create a free account](#)  if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#) , Claude Code, or another [compatible MCP client](#).
- **Azure CLI (v2.60.0+):** [Install](#) and sign in with `az login`.

When to use this skill

Use this skill when you need to:

- Validate my app in Azure
- Check deployment readiness in Azure
- Run preflight checks in Azure
- Verify configuration in Azure
- Check if ready to deploy
- Validate azure.yaml
- Validate Bicep in Azure
- Test before deploying in Azure
- Troubleshoot deployment errors in Azure
- Validate Azure Functions

Example prompts

Try these prompts to activate this skill:

- "Check if my app is ready to deploy to Azure"
- "Validate my azure.yaml configuration"
- "Run preflight checks before Azure deployment"
- "Troubleshoot deployment errors"
- "Verify my infrastructure configuration before deploying"
- "Is my app ready for Azure deployment?"
- "Validate my Bicep configuration"
- "Validate my Bicep template before deploying to Azure"
- "Check my deployment permissions before running `azd up`"
- "Verify my Bicep files are valid before provisioning"

Deployment workflow

This skill is the second step in the deployment workflow:

1. [azure-prepare](#) — generates infrastructure files and `.azure/deployment-plan.md`
2. [azure-validate](#) (this skill) — validates the deployment plan and infrastructure before deploying
3. [azure-deploy](#) — executes the deployment

Related content

- [Azure skill for prepare](#)
- [Azure skill for deploy](#)
- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) ↗

Last updated on 04/17/2026

Azure skill for Entra app registration

Guides Microsoft Entra ID app registration, OAuth 2.0 authentication, and MSAL integration.

Skill: `entra-app-registration` | [Source code](#) 

What it provides

This skill provides GitHub Copilot with specialized knowledge. Guides Microsoft Entra ID app registration, OAuth 2.0 authentication, and MSAL integration.

Prerequisites

- **Azure subscription:** [Create a free account](#)  if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#) , Claude Code, or another [compatible MCP client](#).
- **Azure CLI with Bicep (v2.60.0+):** [Install](#), sign in with `az login`, then run `az bicep install`.
- **Azure Key Vault:** A key vault for secrets and certificate management.

When to use this skill

Use this skill when you need to:

- Create a Microsoft Entra ID app registration for your application.
- Register your application with Microsoft Entra ID.
- Configure OAuth 2.0 authentication flows for your registered application.
- Set up authentication and authorization for your application.
- Add API permissions in Azure
- Work with generate service principal, MSAL example, console app auth, and Entra ID setup
- Work with Microsoft Entra ID authentication

Example prompts

Try these prompts to activate this skill:

- "How do I create an app registration in Azure?"
- "Register a Microsoft Entra ID app for my web application"

- "Configure OAuth authentication for my application"
- "Set up authentication with Microsoft Entra ID"
- "Add API permissions to my Entra app registration"
- "Generate a service principal for Azure authentication"
- "Show me MSAL examples for Microsoft Entra ID authentication."
- "Create a console app with Microsoft Entra ID authentication"
- "Help me set up Entra ID authentication for my app"
- "Configure Microsoft Entra ID OAuth authentication for my API"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) 

Last updated on 04/17/2026

Azure skill for Microsoft Foundry

Deploy, evaluate, and manage Foundry agents end-to-end: Docker build, acr push, hosted/prompt agent create, container start, batch eval, prompt optimization, prompt optimizer workflows, agent.yaml, dataset curation from traces.

Skill: `microsoft-foundry` | [Source code](#)

What it provides

This skill provides GitHub Copilot with specialized knowledge. Deploy, evaluate, and manage Foundry agents end-to-end: Docker build, acr push, hosted/prompt agent create, container start, batch eval, prompt optimization, prompt optimizer workflows, agent.yaml, dataset curation from traces.

Prerequisites

- **Azure subscription:** [Create a free account](#) if you don't have one.
- **AI assistant with Azure Skills:** [GitHub Copilot for Azure](#), Visual Studio Code with [Azure MCP extension](#), Claude Code, or another [compatible MCP client](#).
- **PowerShell** (v7.4+): Install with `winget install Microsoft.PowerShell`.
- **Bash:** A Bash-compatible shell (macOS/Linux terminal or [WSL](#) on Windows).

When to use this skill

Use this skill when you need to:

- Deploy agents to Azure AI Foundry.
- Configure and manage hosted agents.
- Create agents in Azure AI Foundry.
- Invoke and test agents.
- Evaluate agent performance using Azure AI Foundry evaluators.
- Run batch evaluations against your agents.
- Optimize prompts for agent instructions.
- Improve agent prompts and system instructions using the prompt optimizer.
- Refine agent instructions for better performance.
- Optimize system prompts for your agents.

- Manage RBAC permissions, role assignments, managed identities, and service principals for Foundry resources.

Example prompts

Try these prompts to activate this skill:

- "How do I deploy an AI model from Microsoft Foundry catalog?"
- "Build a RAG application with Azure AI Foundry knowledge index"
- "Create an AI agent in Microsoft Foundry with web search"
- "Evaluate agent performance using Foundry evaluators"
- "Optimize my prompt for a Microsoft Foundry agent"
- "Improve my agent instructions in Azure AI Foundry"
- "Use a prompt optimizer on my Foundry system prompt"
- "Set up agent monitoring and continuous evaluation in Foundry"
- "Help me with Microsoft Foundry model deployment"
- "How to use knowledge index for RAG in Azure AI Foundry?"

Related content

- [Azure Model Context Protocol \(MCP\) Server overview](#)
- [Skill source code](#) 

Last updated on 04/17/2026